



Lecture 12: 3D Backbone Cont. + Detection and Segmentation

Li Yi

May 15, 2025

Announcement

- Project released, due June 15
- Lessons from previous years
- Start early

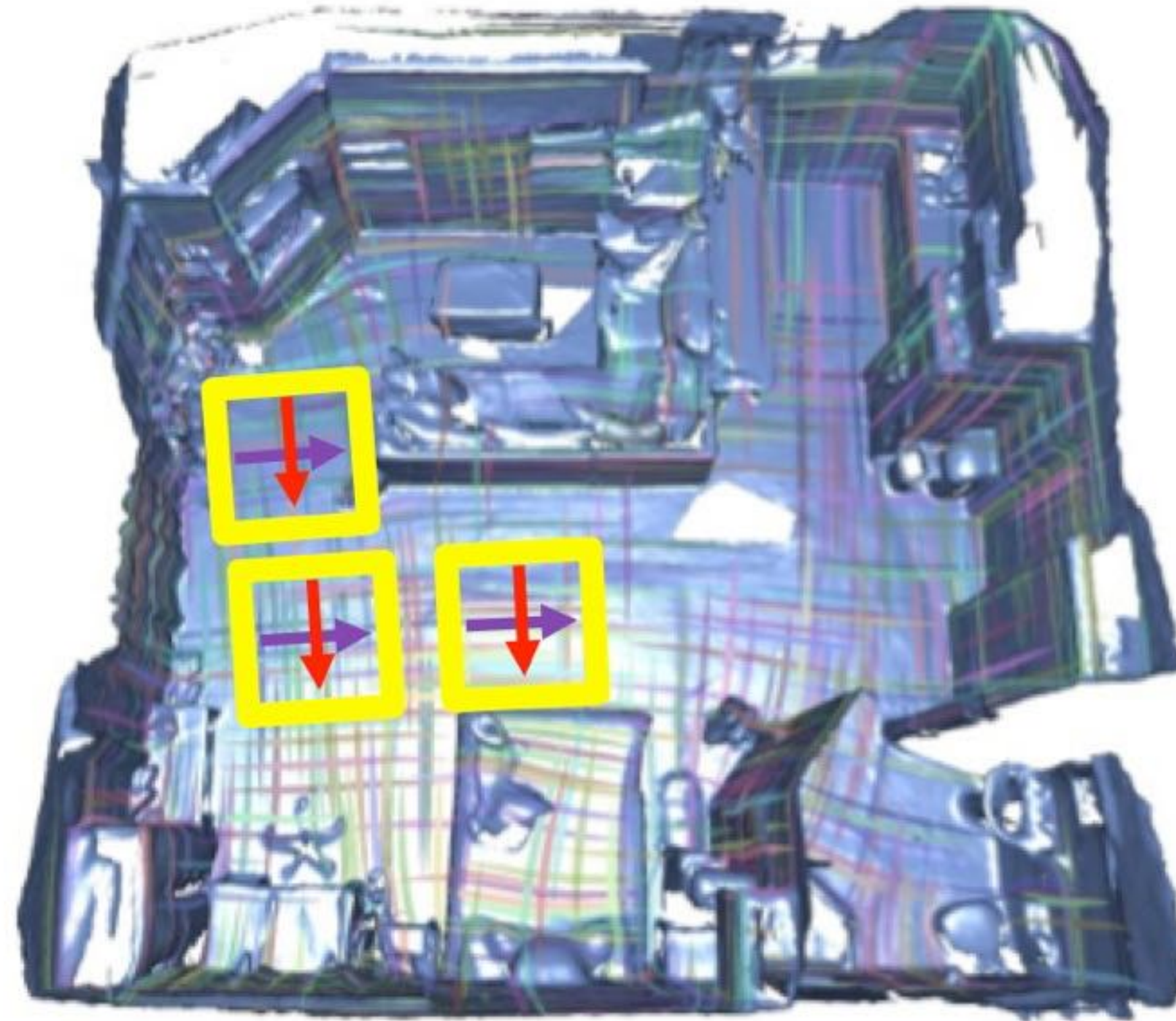
Recap

- Voxel Networks
 - Projection
 - Sparse Convolution
- Point Networks
 - PointNet
 - PointNet++
 - Sampling Issues
- Graph Networks
 - Geodesic CNN
 - TextureNet

3D Backbone Cont.

- Voxel Networks
 - Projection
 - Sparse Convolution
- Point Networks
 - PointNet
 - PointNet++
 - Sampling Issues
- Graph Networks
 - Geodesic CNN
 - TextureNet

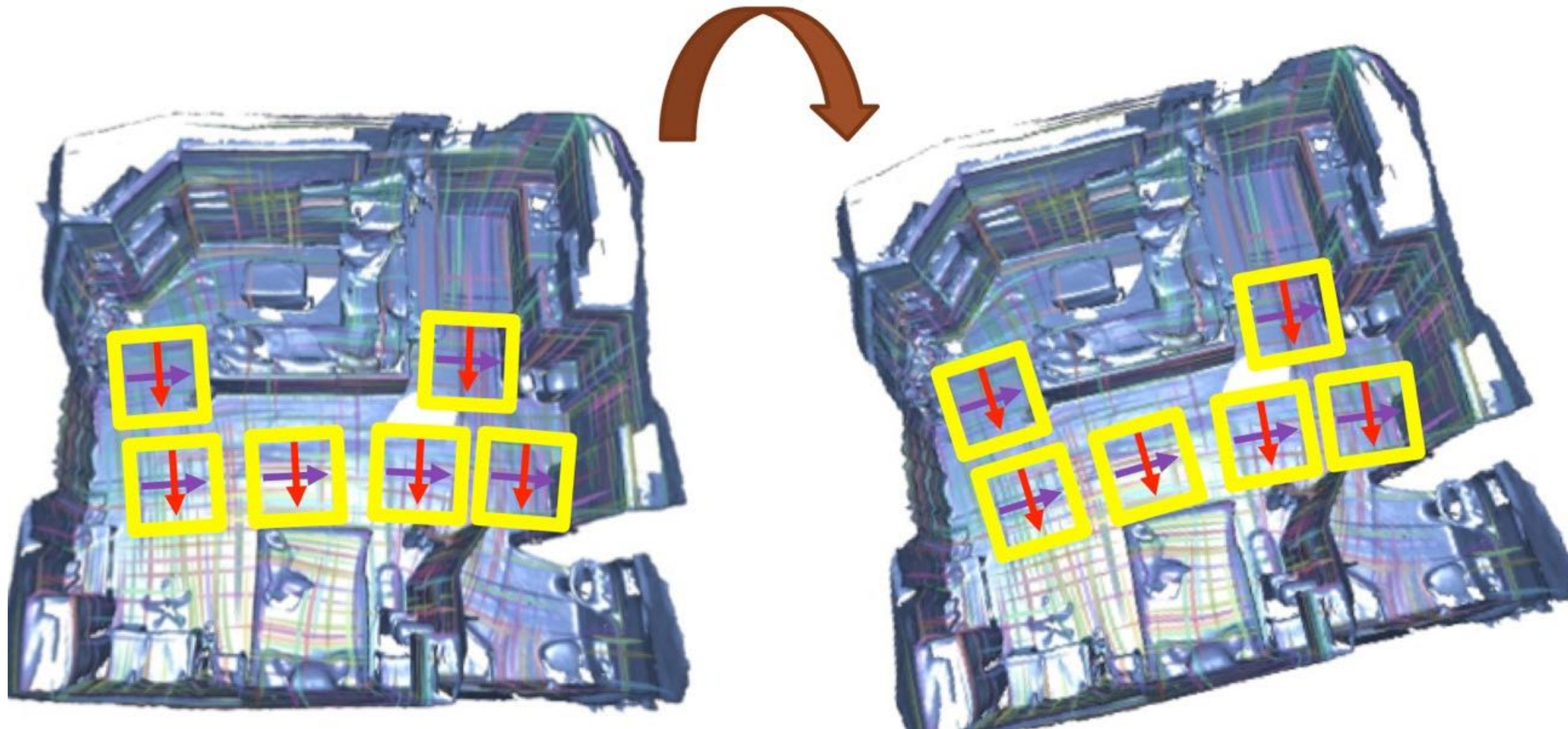
TextureNet



TextureNet: Consistent Local Parametrizations for Learning from High-Resolution Signals on Meshes

Huang et al. CVPR 2019

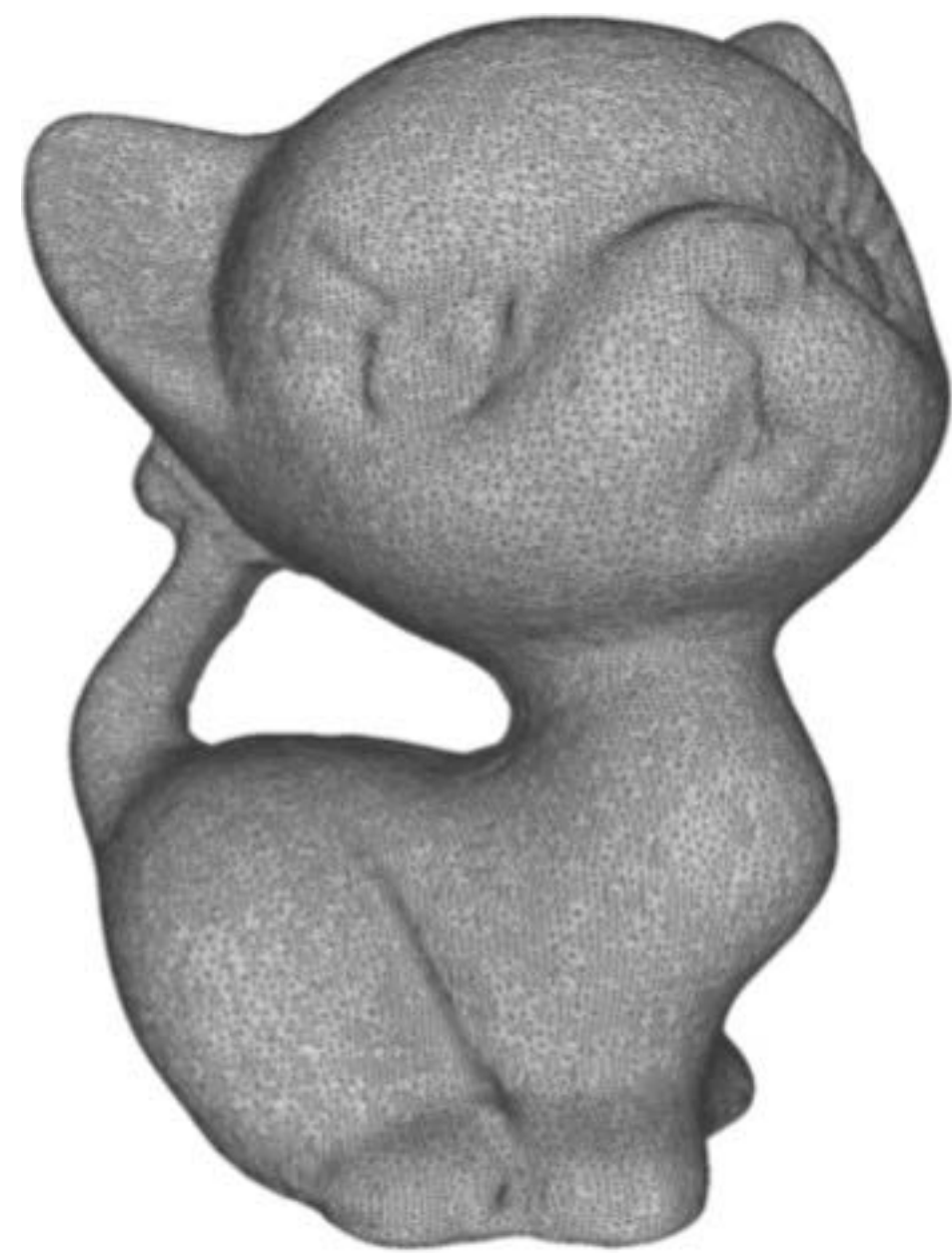
TextureNet



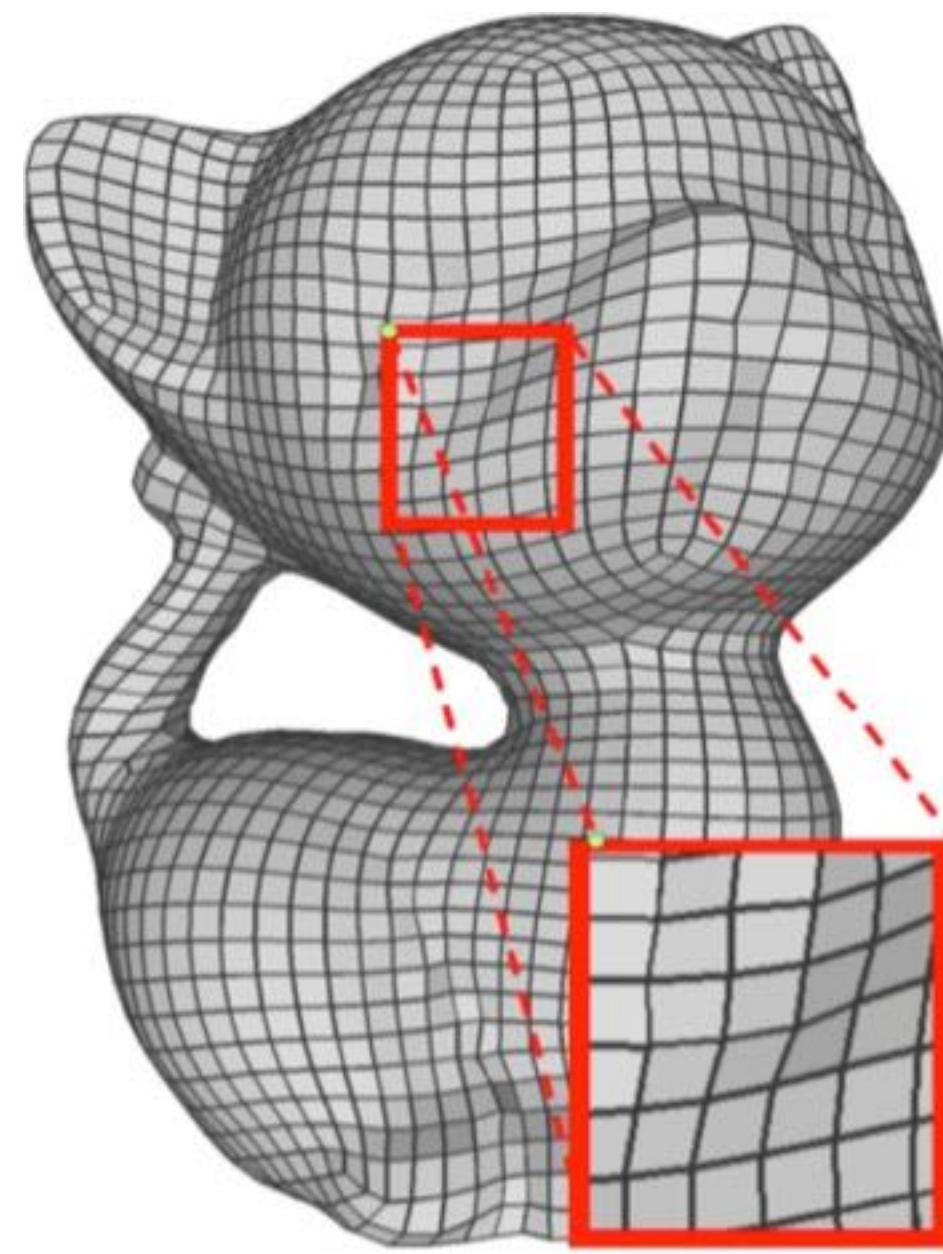
TextureNet: Consistent Local Parametrizations for Learning from High-Resolution Signals on Meshes

Huang et al. CVPR 2019

QuadriFlow Parameterization



Input triangulation



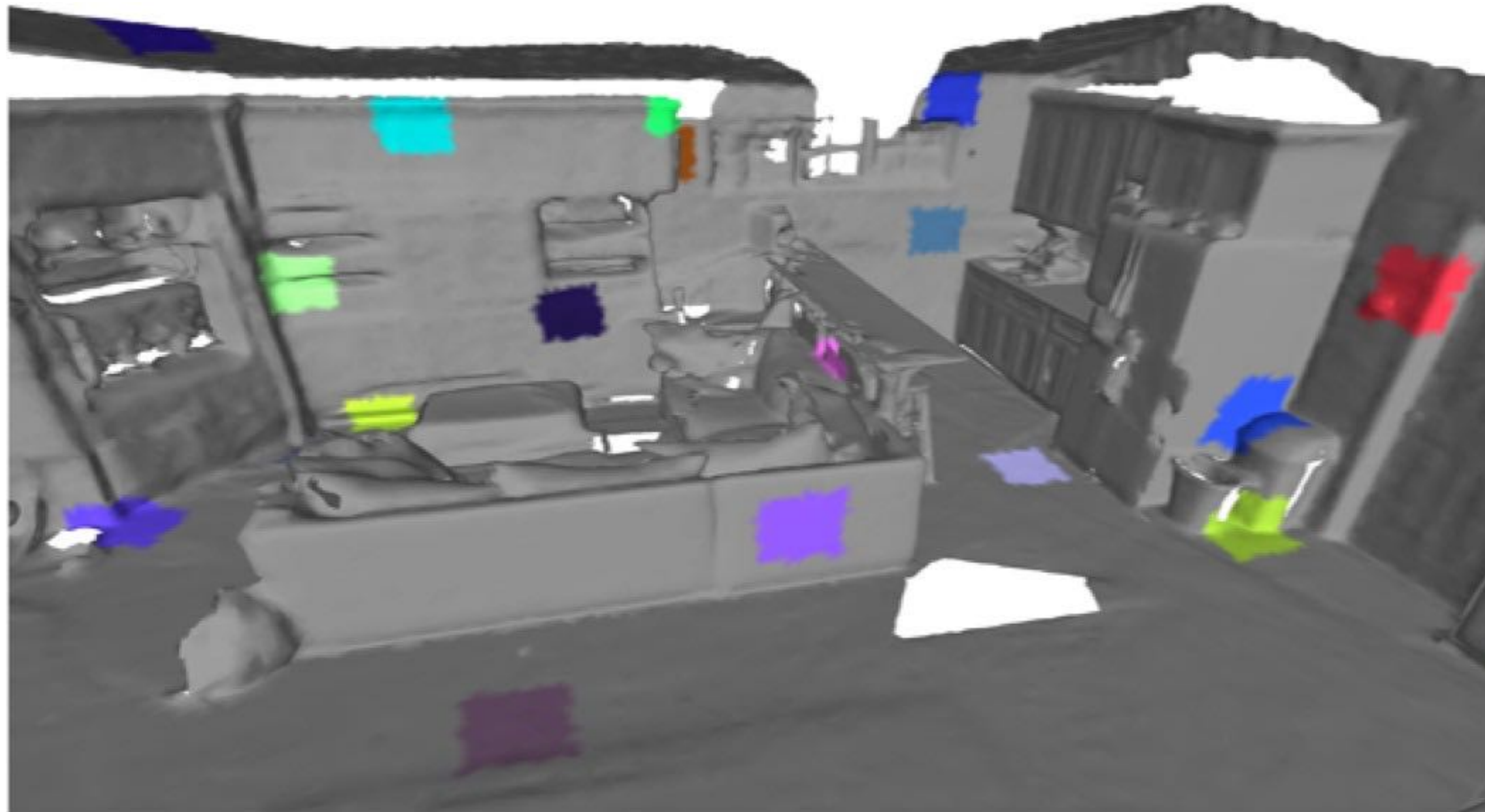
QuadriFlow

- Tangent crosses on vertices
- Smooth
- Canonical alignment

QuadriFlow: A Scalable and Robust Method for Quadrangulation

Huang et al. SGP 2018

TextureNet

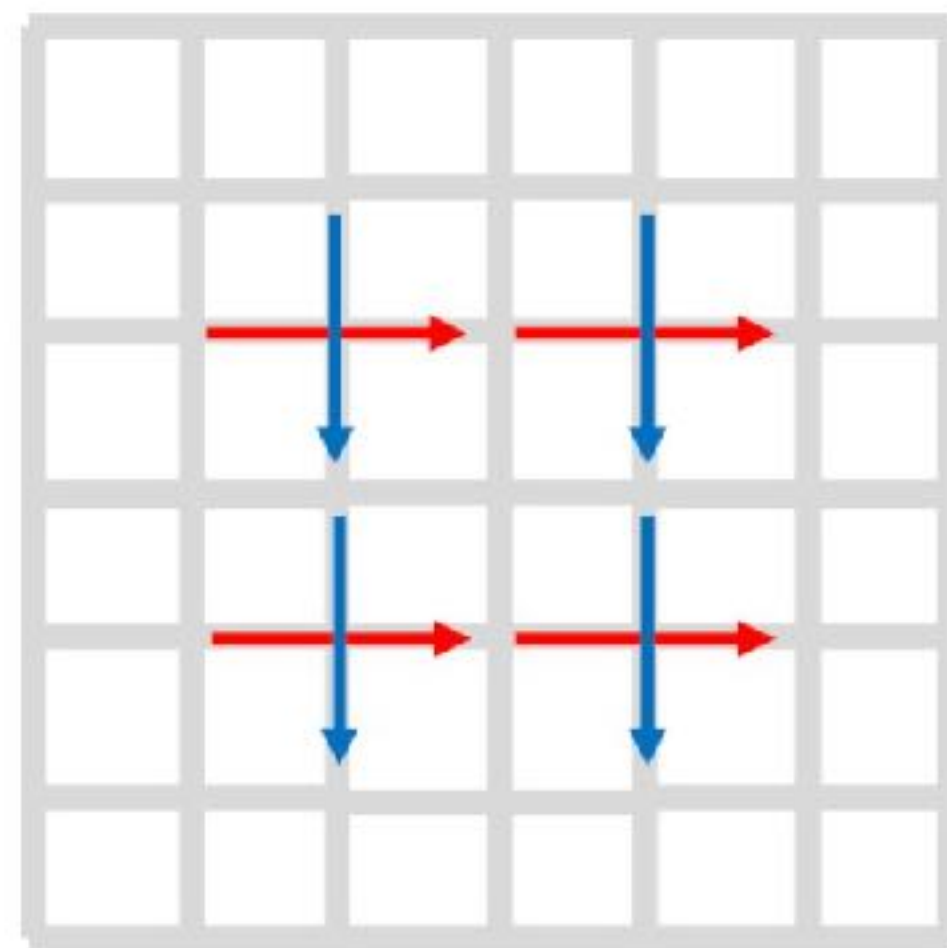
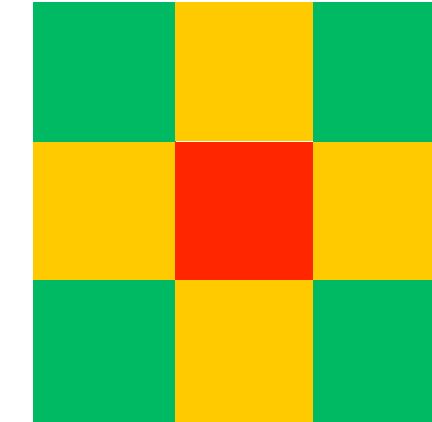
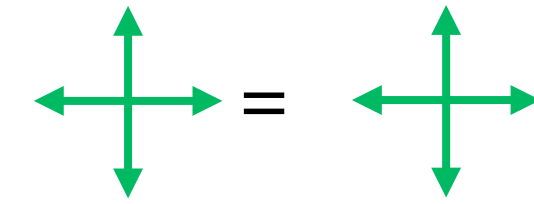
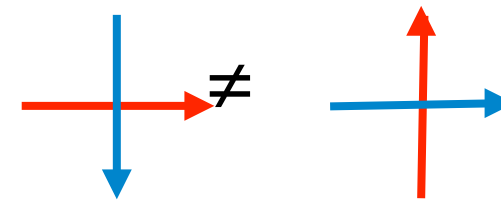


TextureNet: Consistent Local Parametrizations for Learning from High-Resolution Signals on Meshes

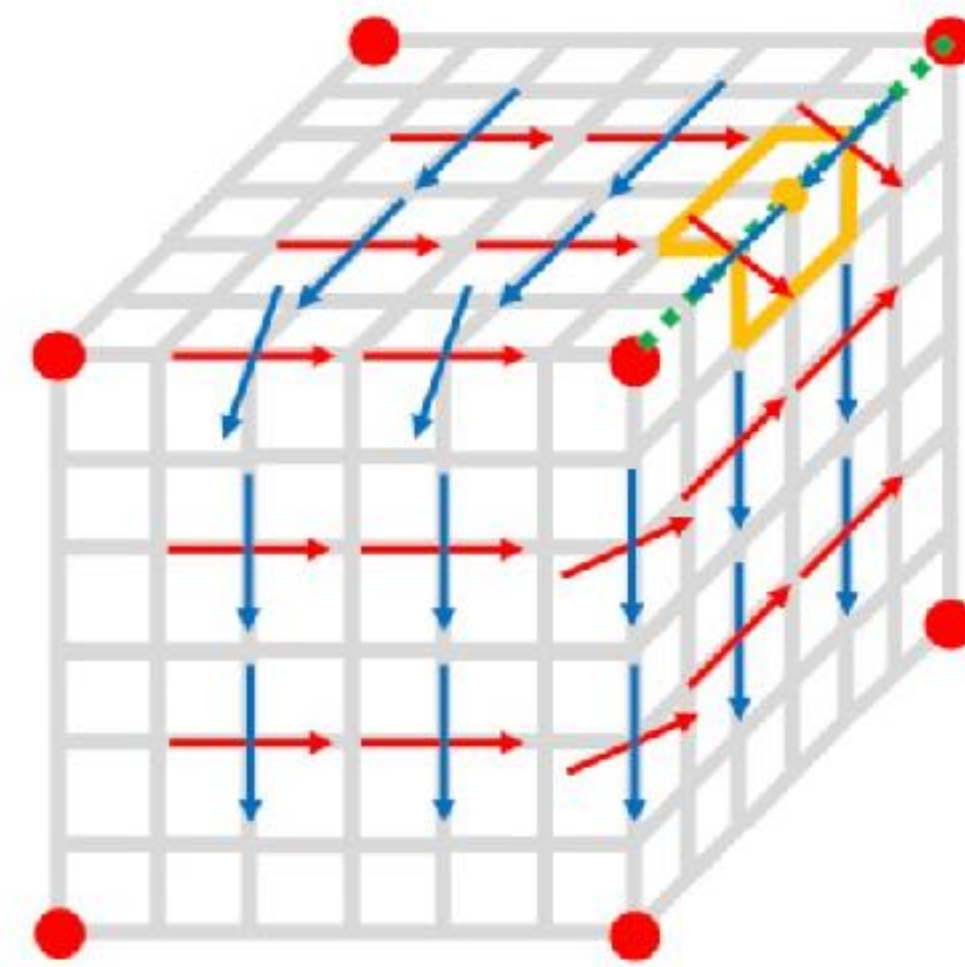
Huang et al. CVPR 2019

TextureNet

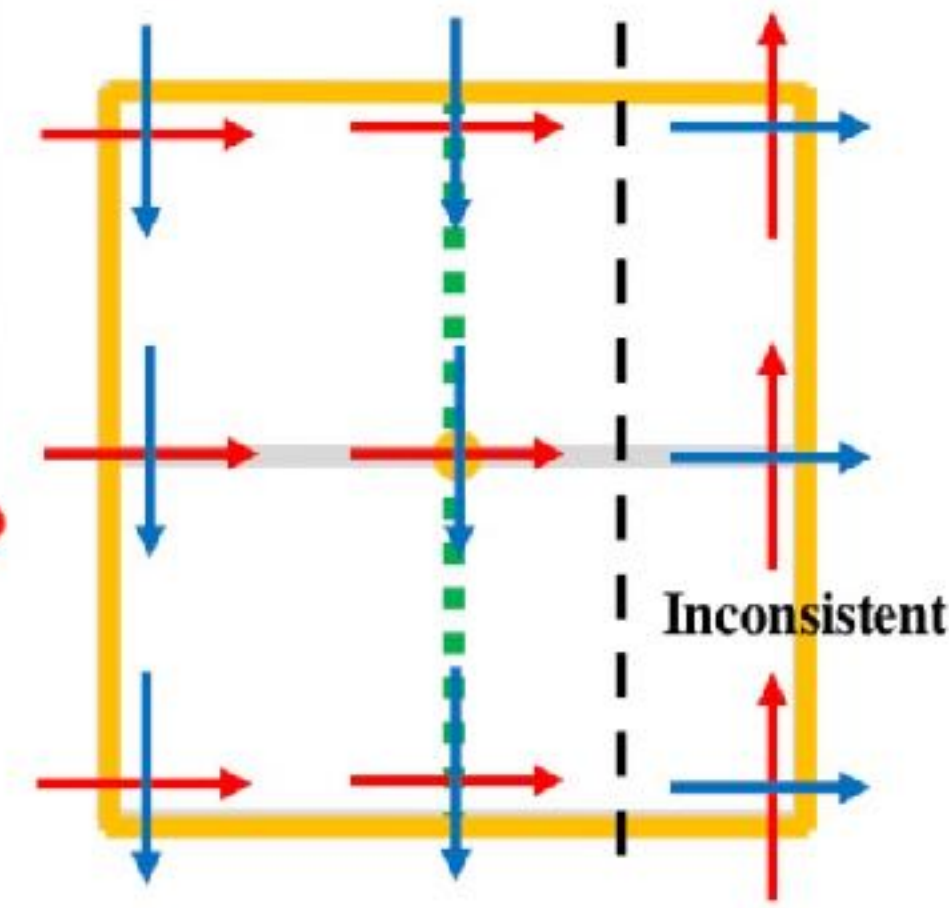
Consistent Orientation



(a) Image Coordinate



(b) 3D
parameterization

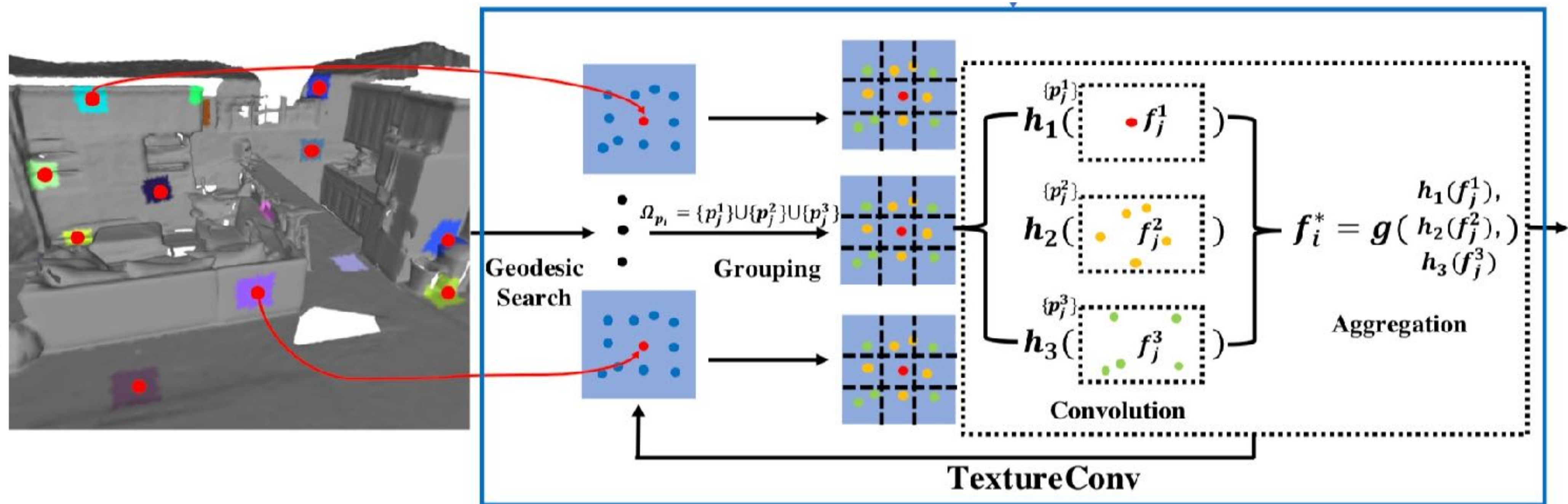


(c) Inconsistent Frame

TextureNet: Consistent Local Parametrizations for Learning from High-Resolution Signals on Meshes

Huang et al. CVPR 2019

Texture Convolution



TextureNet: Consistent Local Parametrizations for Learning from High-Resolution Signals on Meshes

Huang et al. CVPR 2019

TextureNet



3D Textured Model

More Work

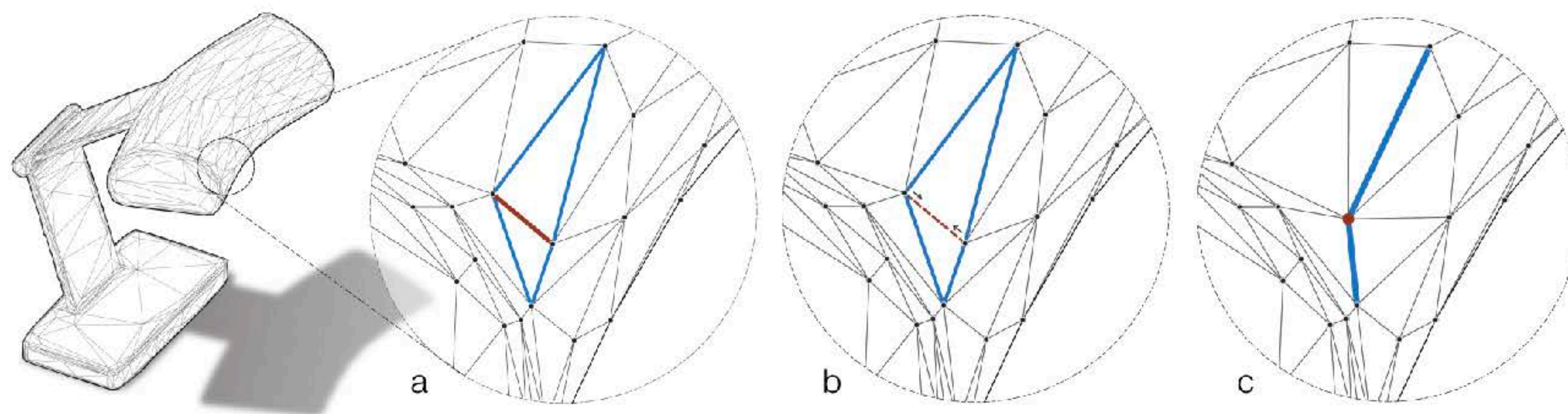
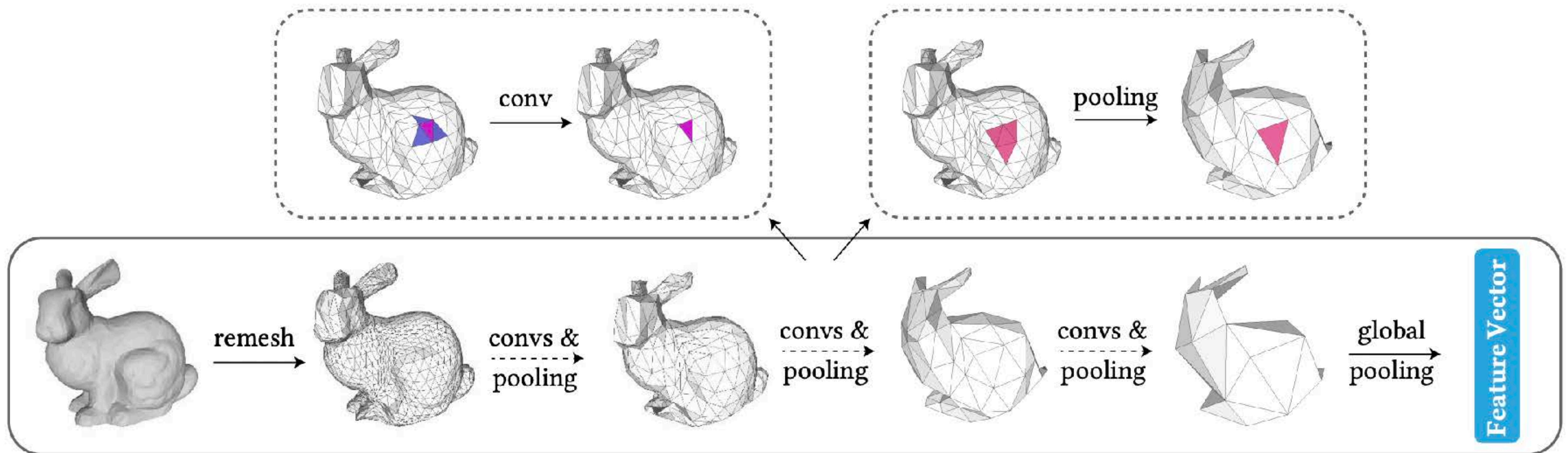


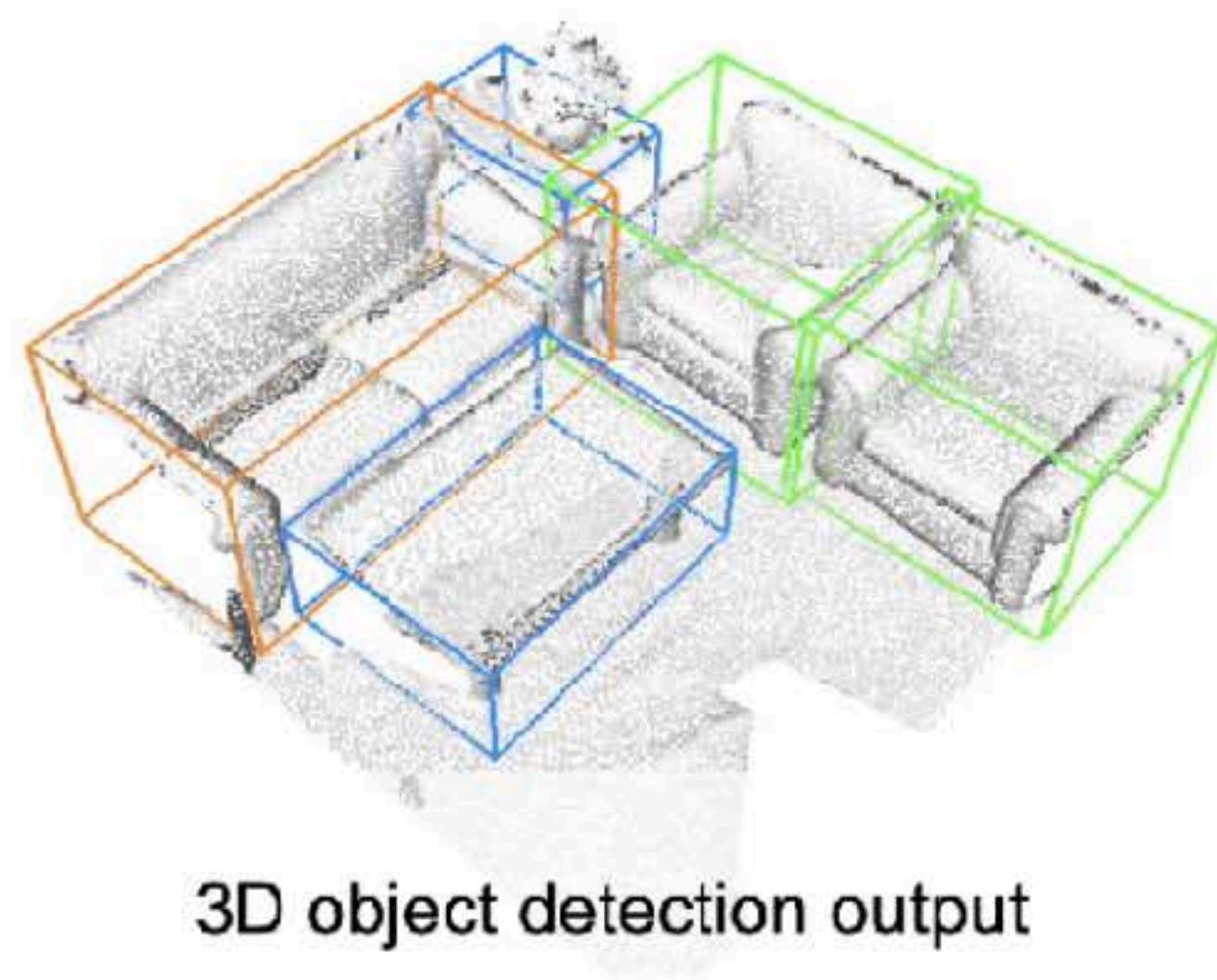
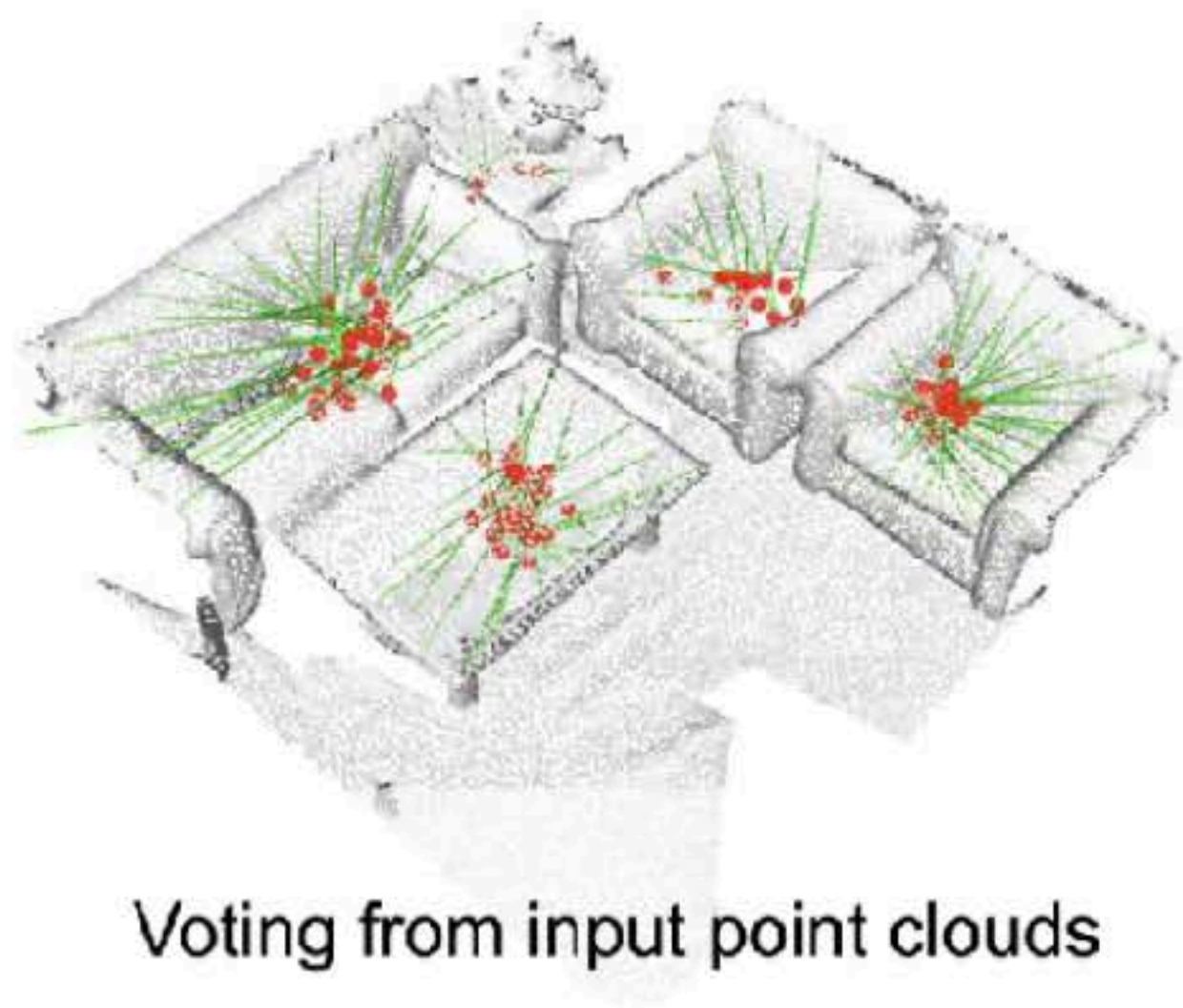
Fig. 2. (a) Features are computed on the edges by applying convolutions with neighborhoods made up of the four edges of the two incident triangles of an edge. The four blue edges are incident to the red edge. The pooling step is shown in (b) and (c). In (b), the red edge is collapsing to a point, and the four incident (blue) edges merge into the two (blue) edges in (c). Note that in a single edge collapse step, five edges are converted into two.

More Work

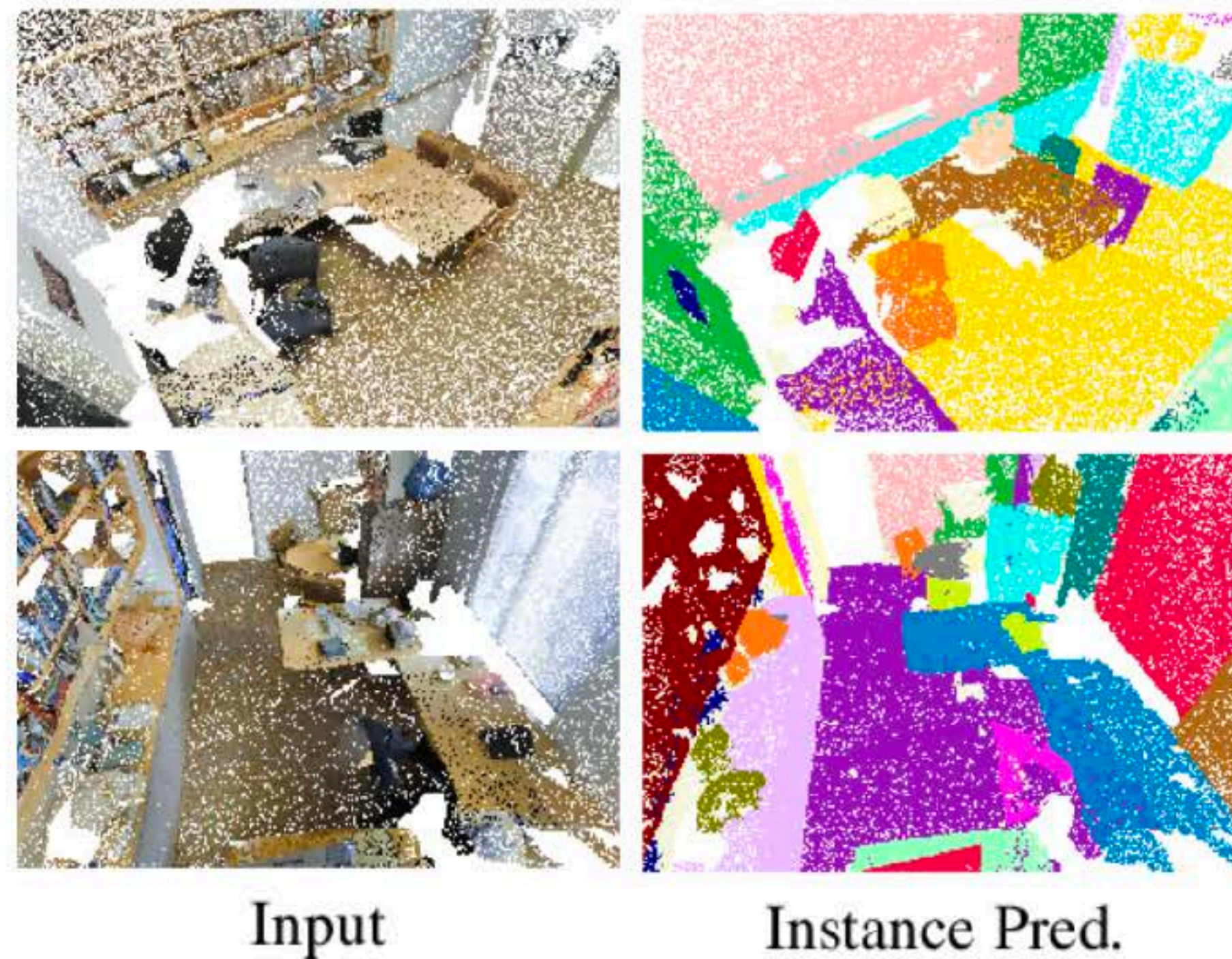


Subdivision-Based Mesh Convolution Networks, Hu et al. 2021

Today's Focus



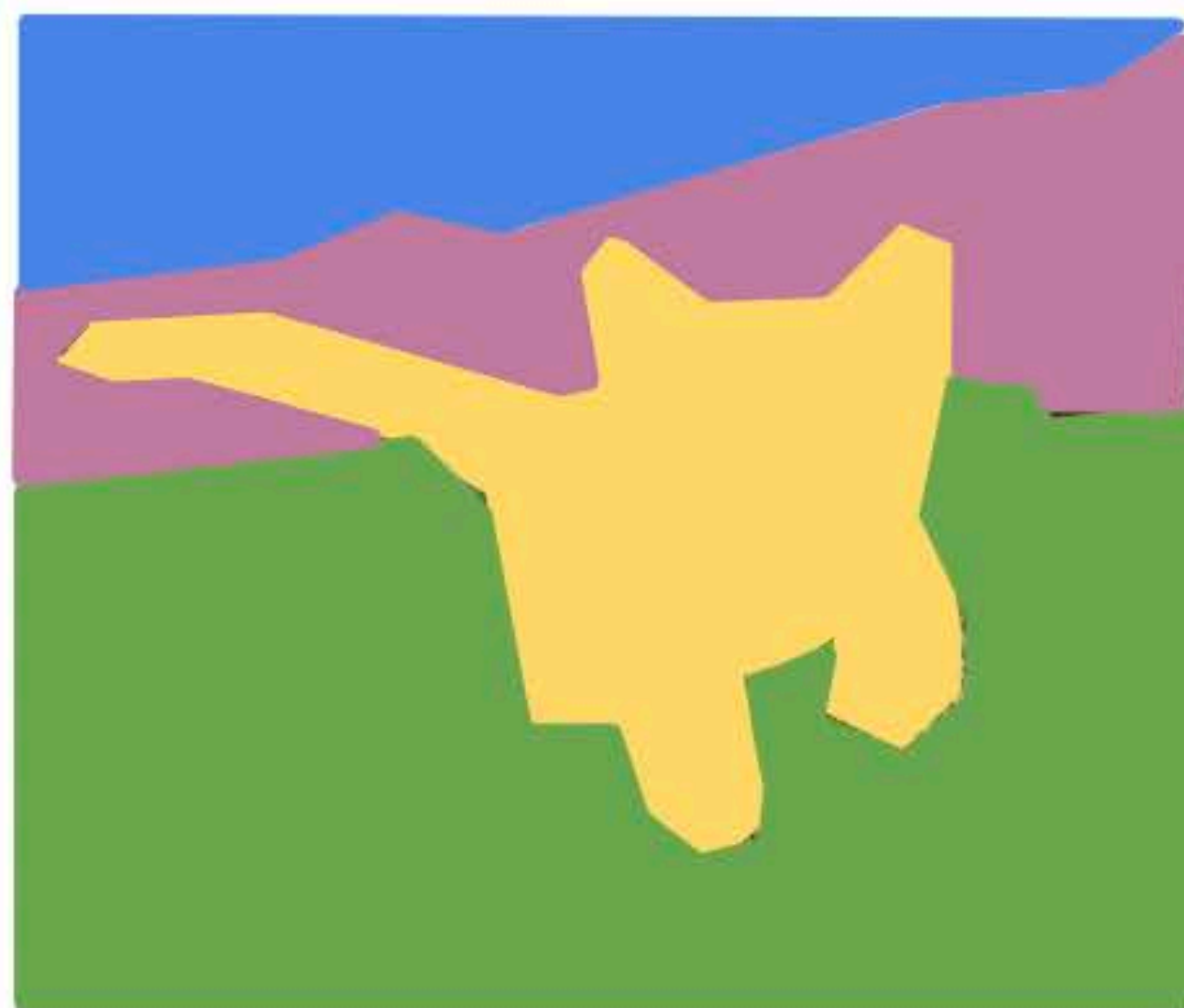
3D Detection



3D Segmentation

Detection and Segmentation Tasks

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



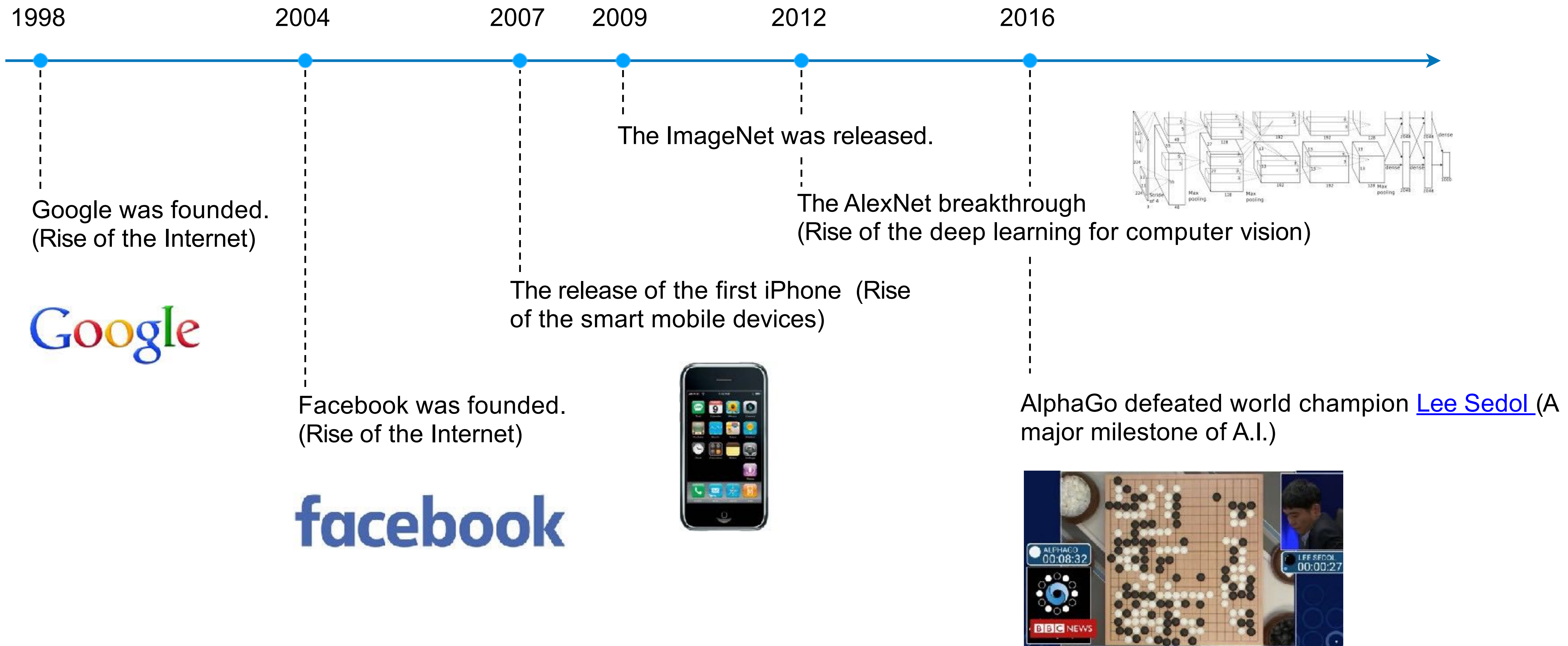
DOG, DOG, CAT

[This image is CC0 public domain](#)

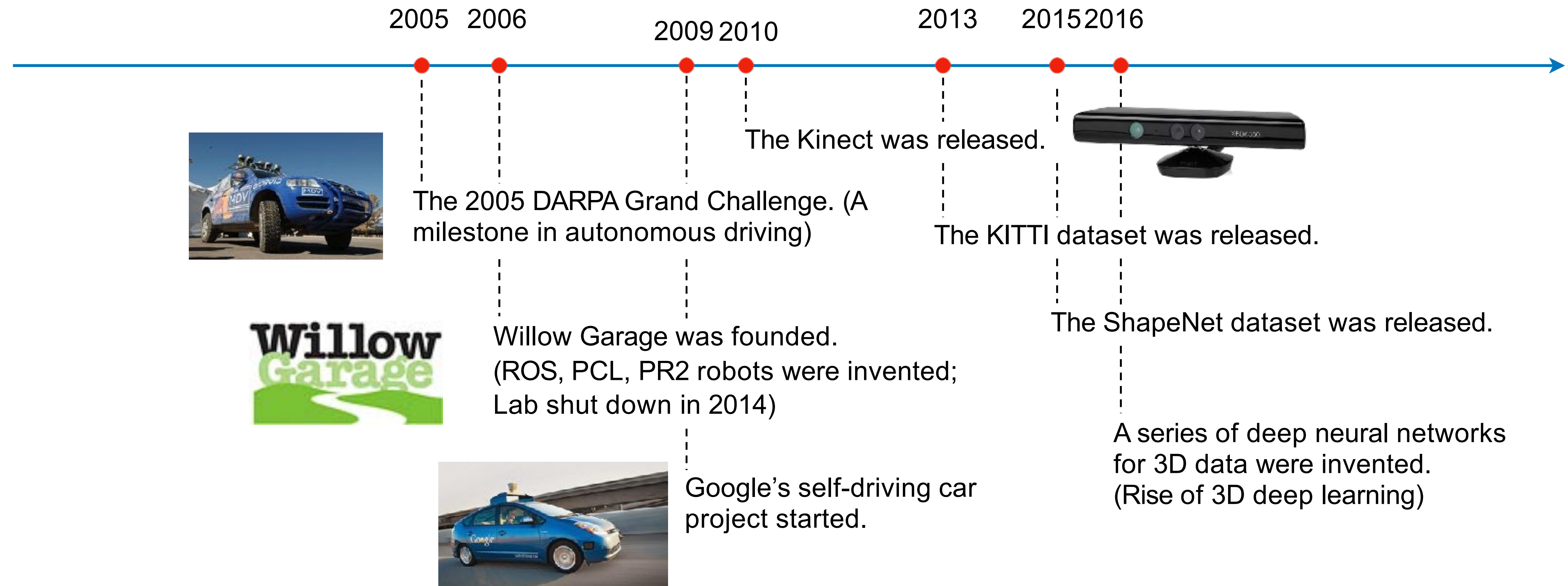
Outline

- 3D Object Detection
 - *Background*
 - The History and Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - Bottom-Up Approaches

The big picture: A.I. applications from the **virtual** world to the **physical** world



The big picture: A.I. applications from the **virtual** world to the **physical** world

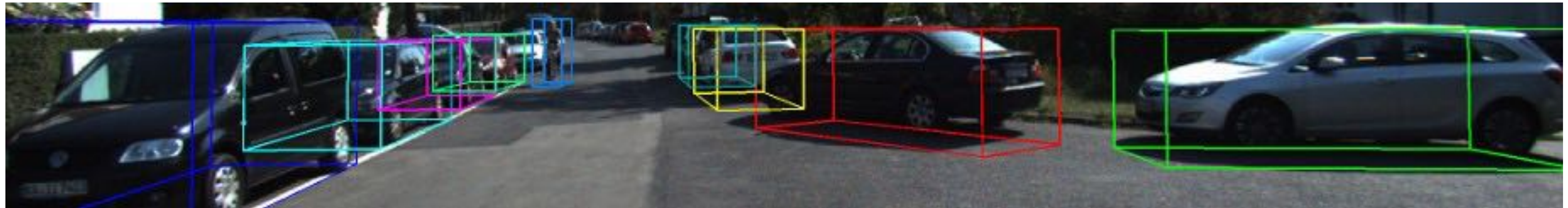


We are yet to reach the time where A.I. is widely applied to the “robots” in the physical world. 3D deep learning and 3D object detection are the core technologies towards that future!

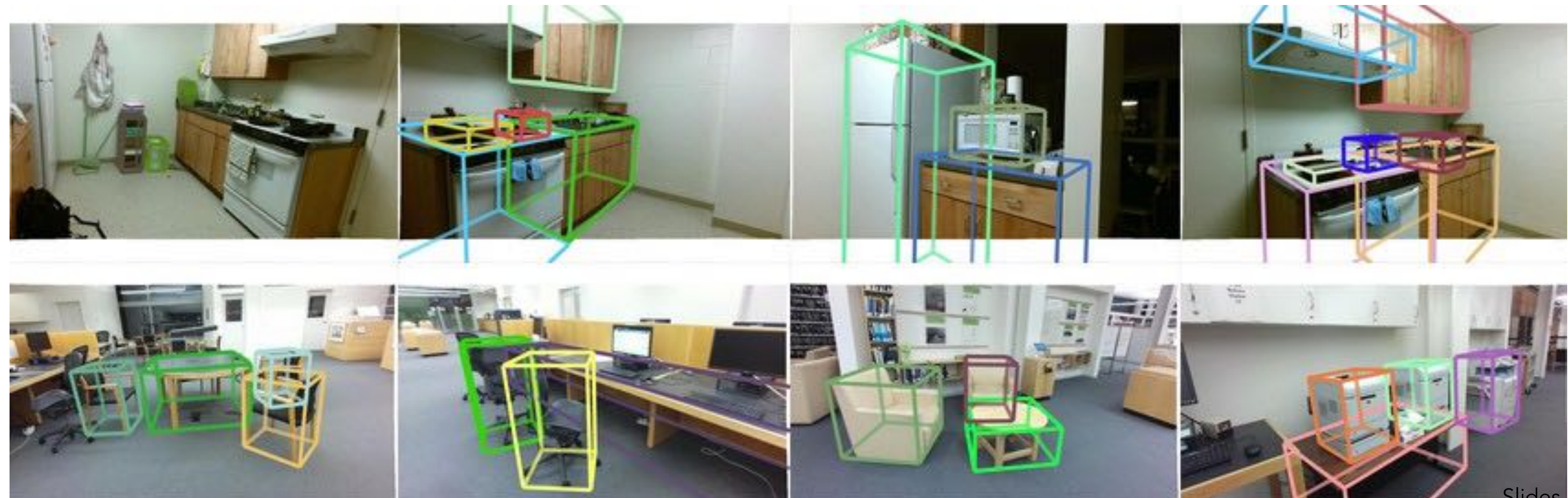
What is 3D object detection?

- **Input:** sensor data of a 3D scene (RGB/depth/radar images).
- **Output:** localization, shape and semantics of the 3D objects in the scene (amodal 3D bounding boxes).

KITTI:



SUN RGB-D:



What is 3D object detection?

- **Input:** sensor data of a 3D scene (RGB/depth/radar images).
- **Output:** localization, shape and semantics of the 3D objects in the scene (amodal 3D bounding boxes).

KITTI:

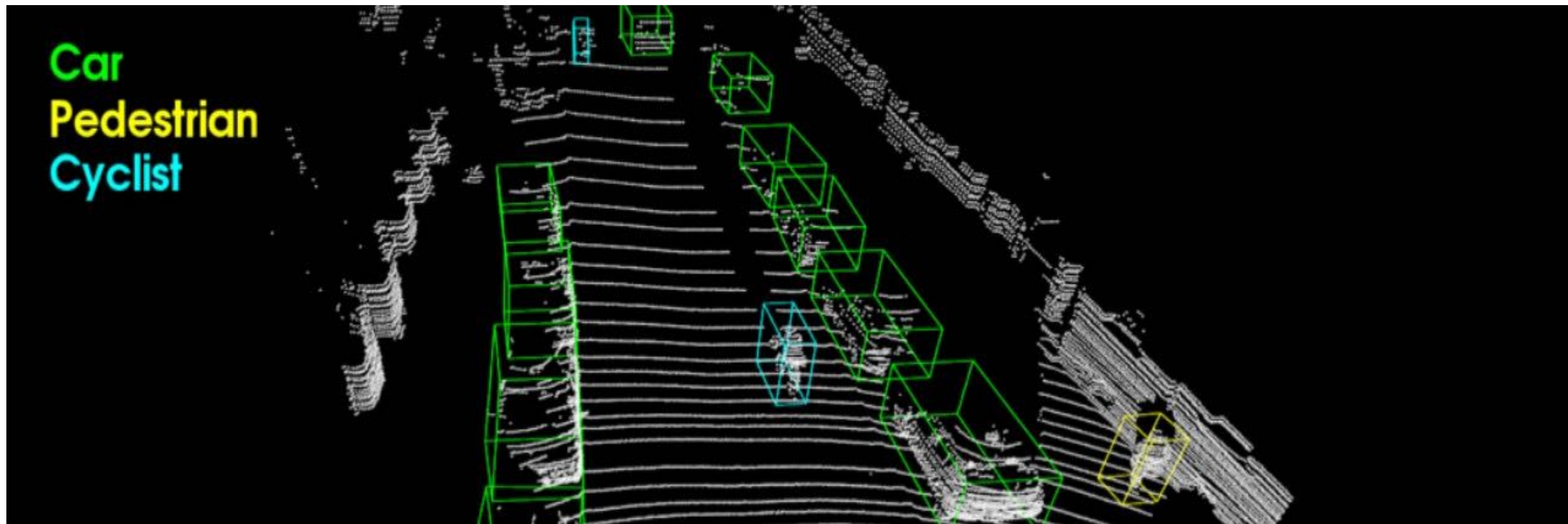


Figure from VoxelNet [Zhou et al. 2018]

What is 3D object detection?

- **Input:** sensor data of a 3D scene. (RGB/depth/radar images).
 - E.g. point clouds (N, 3+C) where the channels are x,y,z and features like color, intensity etc.
- **Output:** localization, shape and semantics of the 3D objects in the scene.
 - E.g. upright 3D amodal, oriented bounding boxes (K, 7) with center xyz, length, width, height, heading; semantic classes (K,) and box scores (K,).

KITTI:

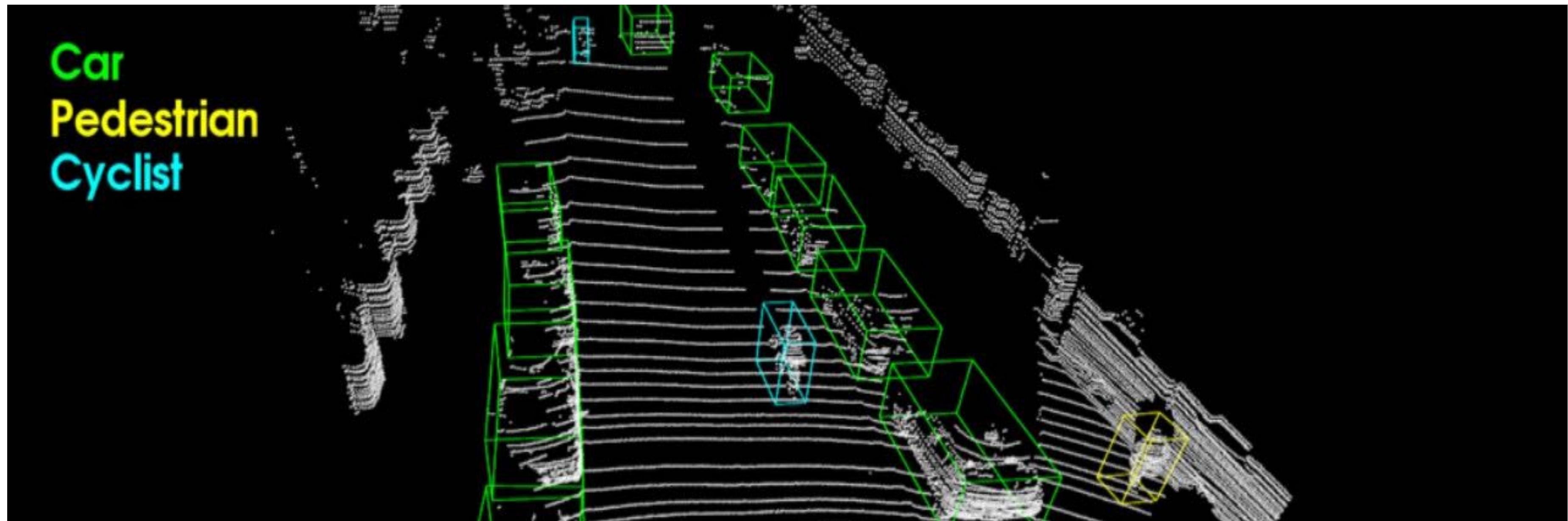


Figure from VoxelNet [Zhou et al. 2018]

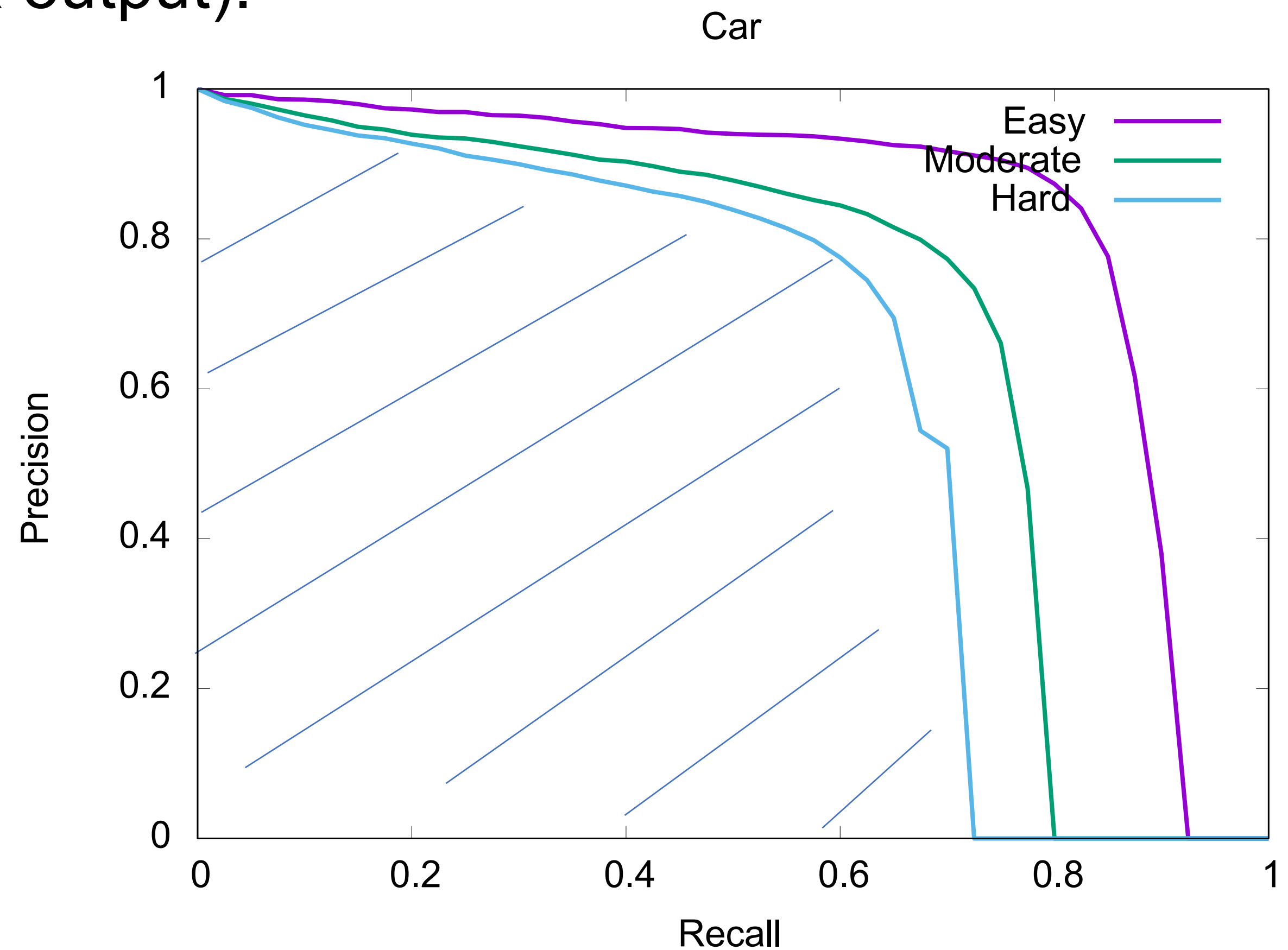
What is 3D object detection?

Evaluation Metric: Average Precision (AP) with a 3D Intersection over Union (IoU) threshold (assuming 3D bounding box output).

For each score threshold, we get an “operation point” on the PR curve — all predictions with scores higher than the threshold are considered “positive” detections while all others with scores lower than the threshold is considered “negative” detections.

By scanning through the score thresholds e.g. from 0 to 1, we get the PR curve.

Average precision is the area under the curve.



A detailed explanation of the Average Precision metric:

<https://jonathan-hui.medium.com/map-mean-average-precision-for-object-detection-45c121a31173>

Outline

- 3D Object Detection
 - Background
 - *The History and Recent Development*
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - Bottom-Up Approaches

The history of 3D object detection

Pre deep learning:

Template-based:

Generalized Hough Voting (2010) [1]

Local/global descriptor+matching+ICP (2012) [2]

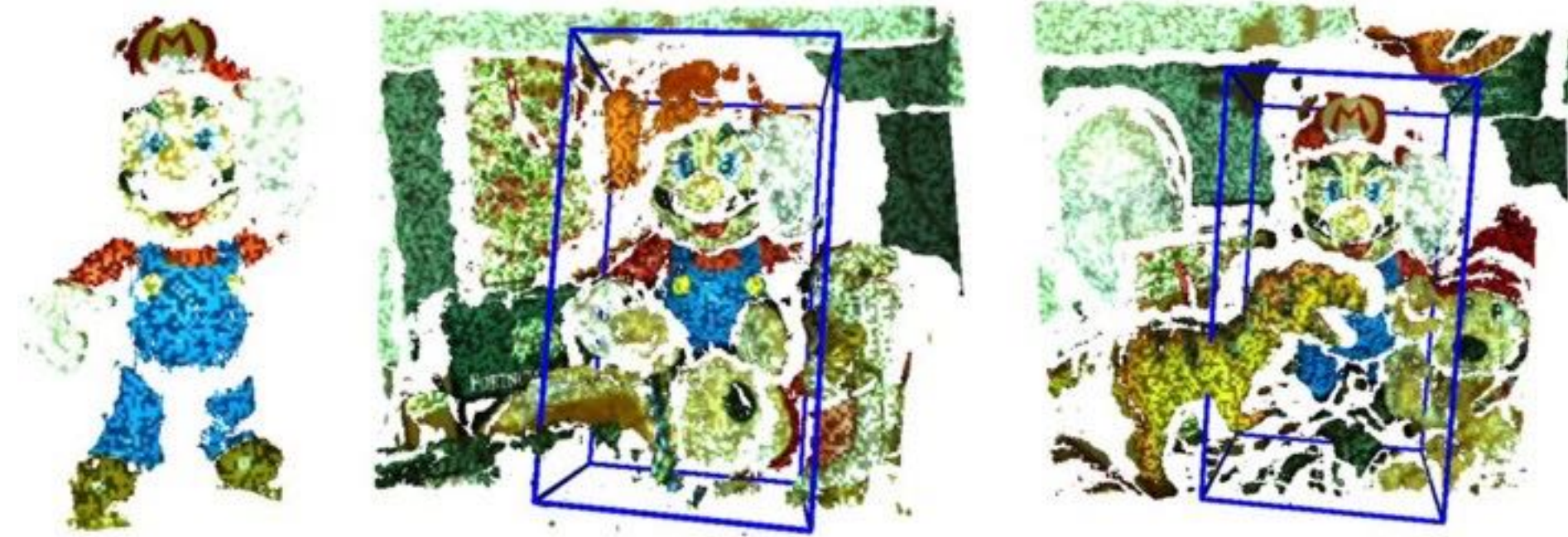


Figure from [1]

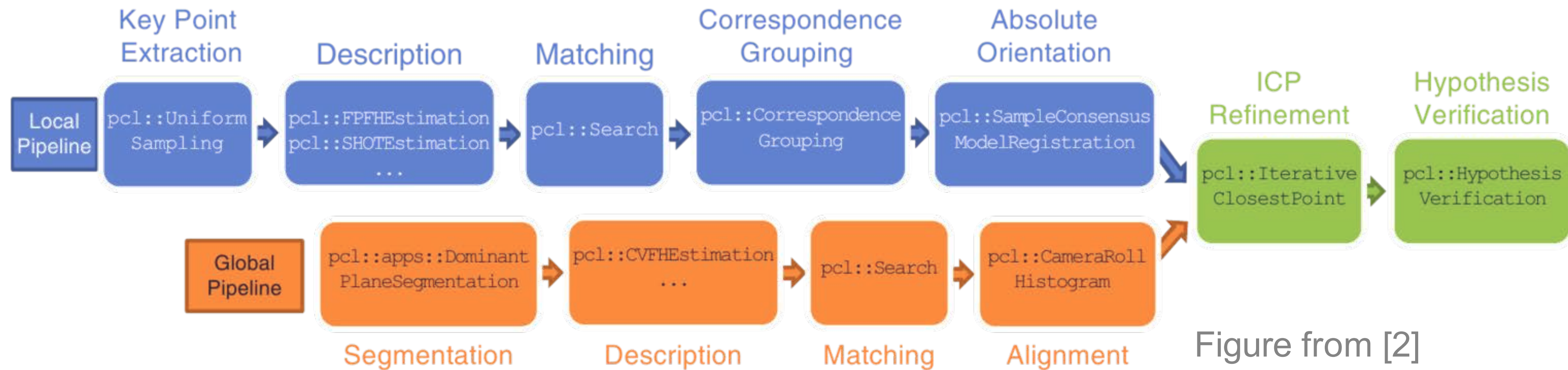


Figure from [2]

The history of 3D object detection

Pre deep learning:

Clustering-based:

Object Discovery in 3D scenes via Shape Analysis (2013) [3]
Data-driven “objectness” score prediction.

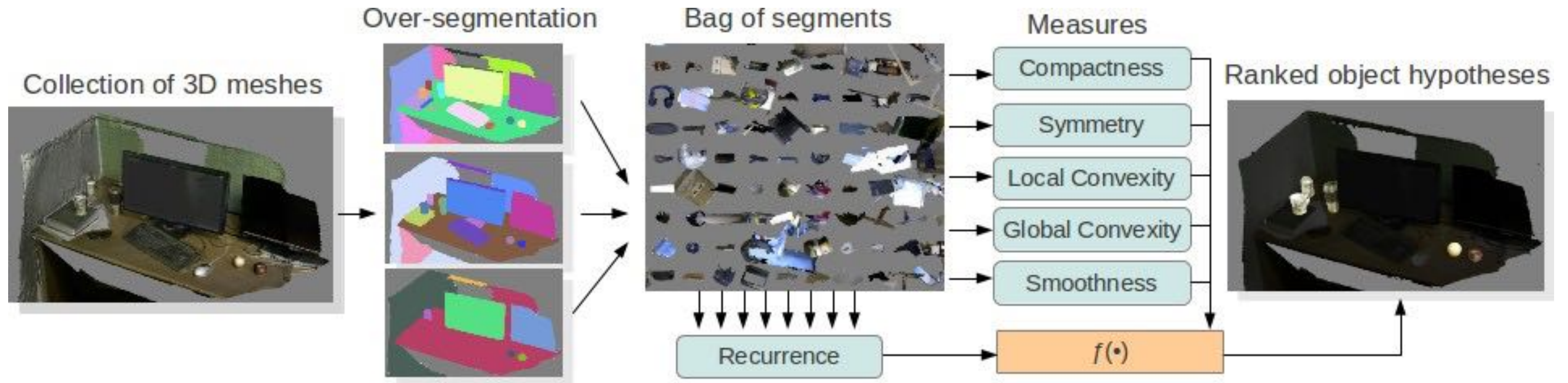


Figure from [3]

The history of 3D object detection

Pre deep learning:

Sliding window based:

Sliding Shapes for 3D Object Detection in Depth Images (2014) [4]

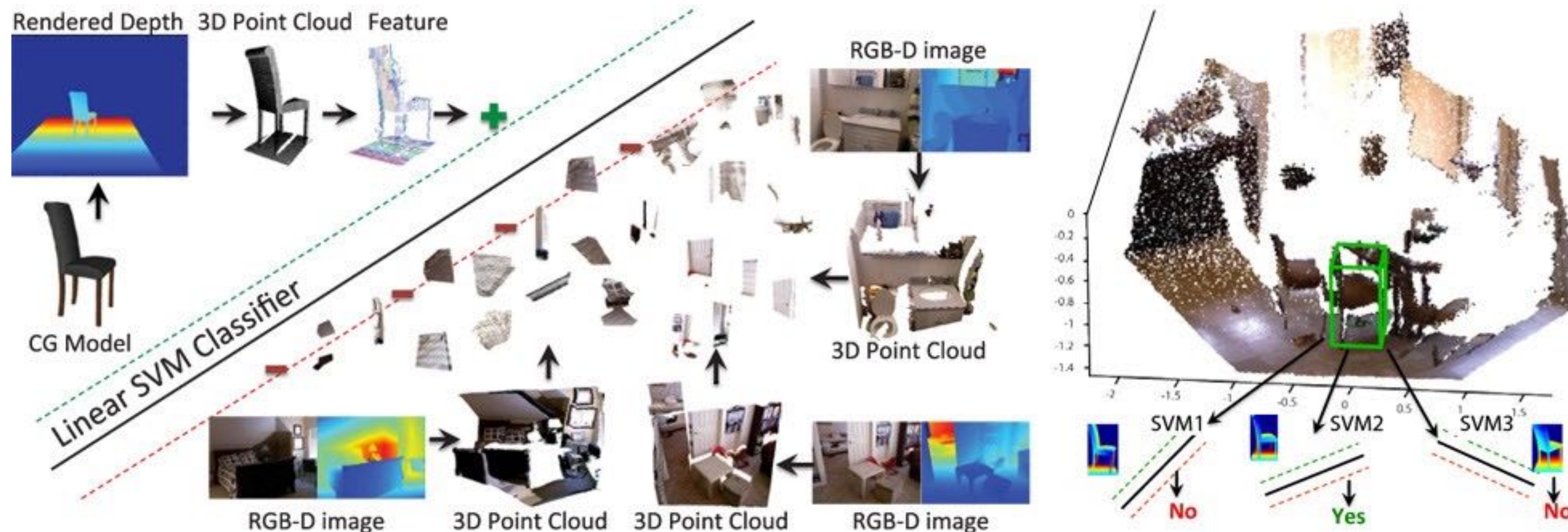


Figure from [4]

The deep learning era of 3d object detection

Three factors prepared us for this phase:

- **The rise of 3D sensors and access to large-scale 3D datasets**
 - Commercial depth cameras, Lidars.
 - KITTI, SUN RGB-D, ShapeNet, ScanNet...
- **The progresses of 2D object detectors**
 - R-CNN (deep nets for classification) -> Fast R-CNN (parallel processing)
-> Faster R-CNN (region proposal network)...
- **The rise of 3D deep learning**
 - A series of novel deep neural network architectures for 3D data (MVCNN, 3DCNNs, PointNet, PointNet++ etc.) has been invented.

The deep learning era of 3d object detection

A first serious attempt of using deep nets for 3d detection:

Deep Sliding Shapes for Amodal 3D Object Detection in RGB-D Images (2016) [5]
- 3D CNNs for Faster-RCNN style region proposal.

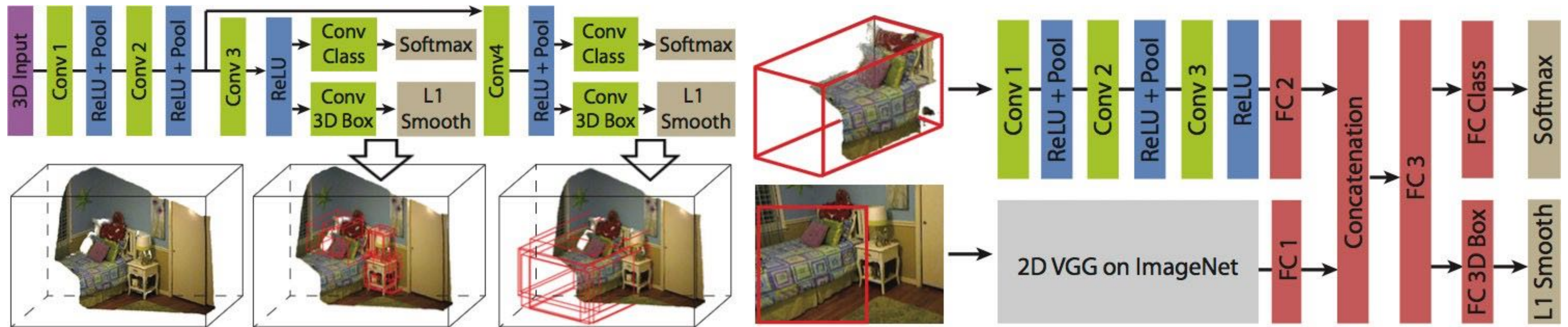


Figure from [5]

Con: 3D CNNs are very costly in both memory and time.

The deep learning era of 3d object detection

Solutions

Image-driven

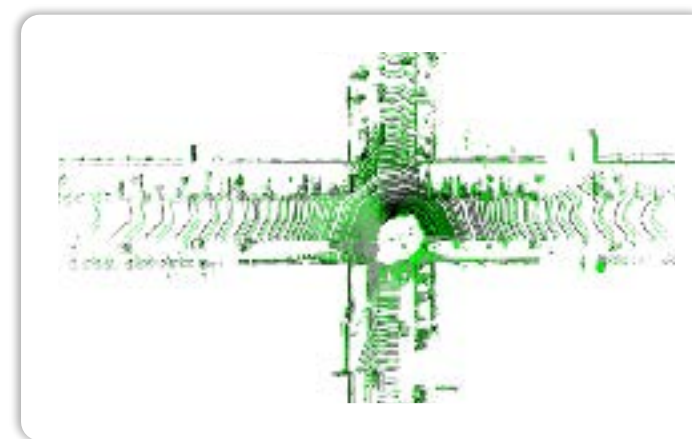
Monocular view detectors
Frustum-based detectors



E.g.: **Frustum PointNets [6]**

Dimension reduction

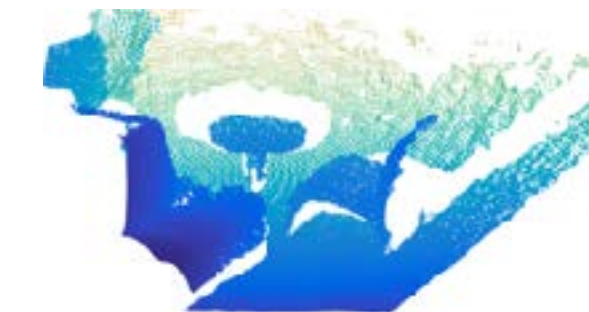
Bird's eye view detectors



PointPillars [7]

Leveraging
Sparsity in 3D

Point set deep nets
Sparse 3D conv, GNNs



VoteNet [8]

Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ *Frustum PointNets*
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - Bottom-Up Approaches

Image-driven 3D object detection

- Key idea: Leverage mature 2D object detectors to propose objects from RGB images.

Monocular or stereo view based

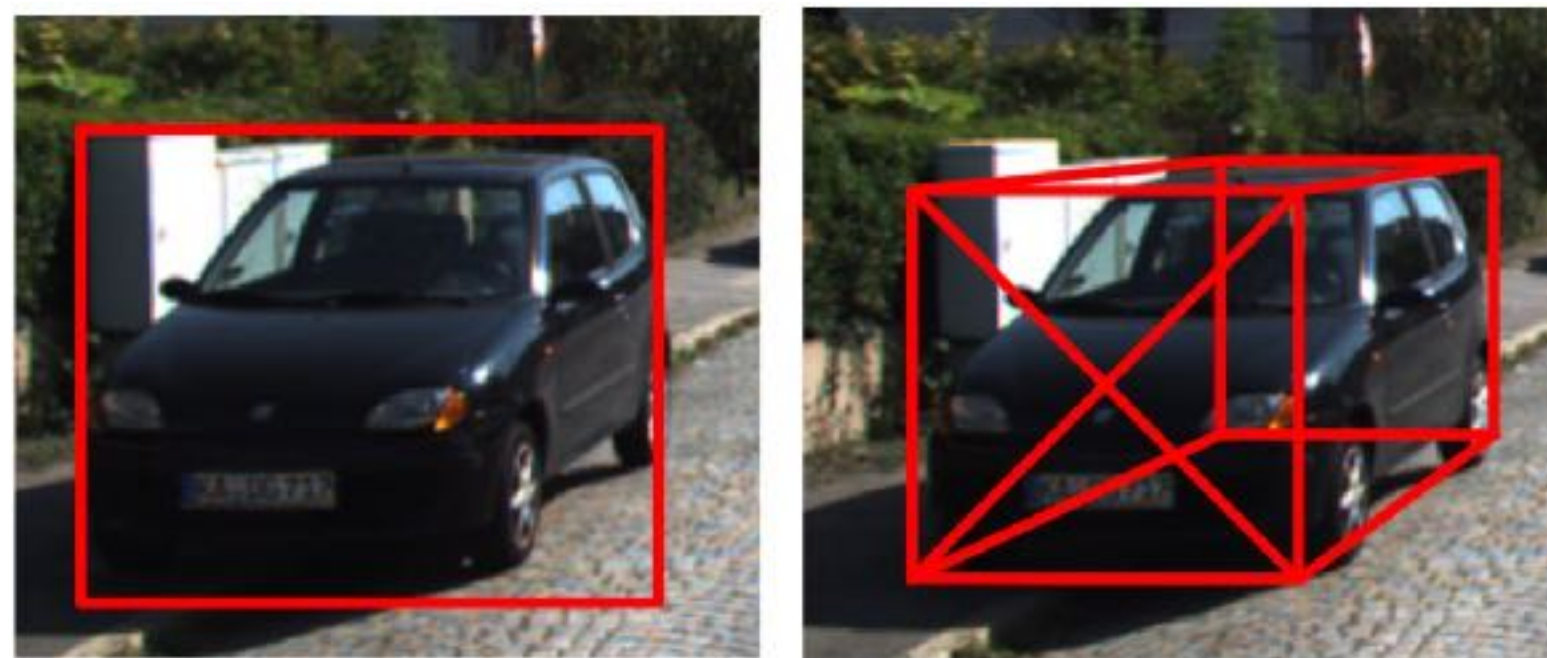
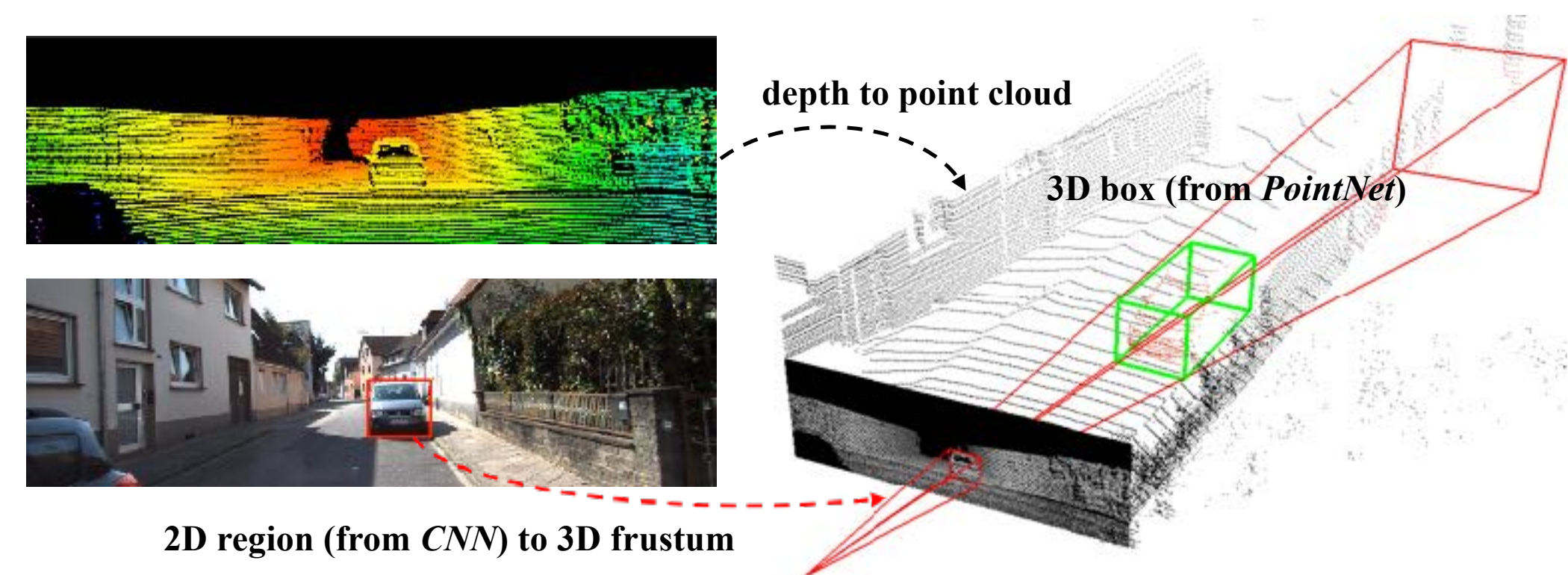


Figure from [9]

3d bounding box estimation using deep learning and geometry (2017) [9]
Pseudo-lidar (2019) [10]
Objects as Points (2019) [11]

RGB-D data based



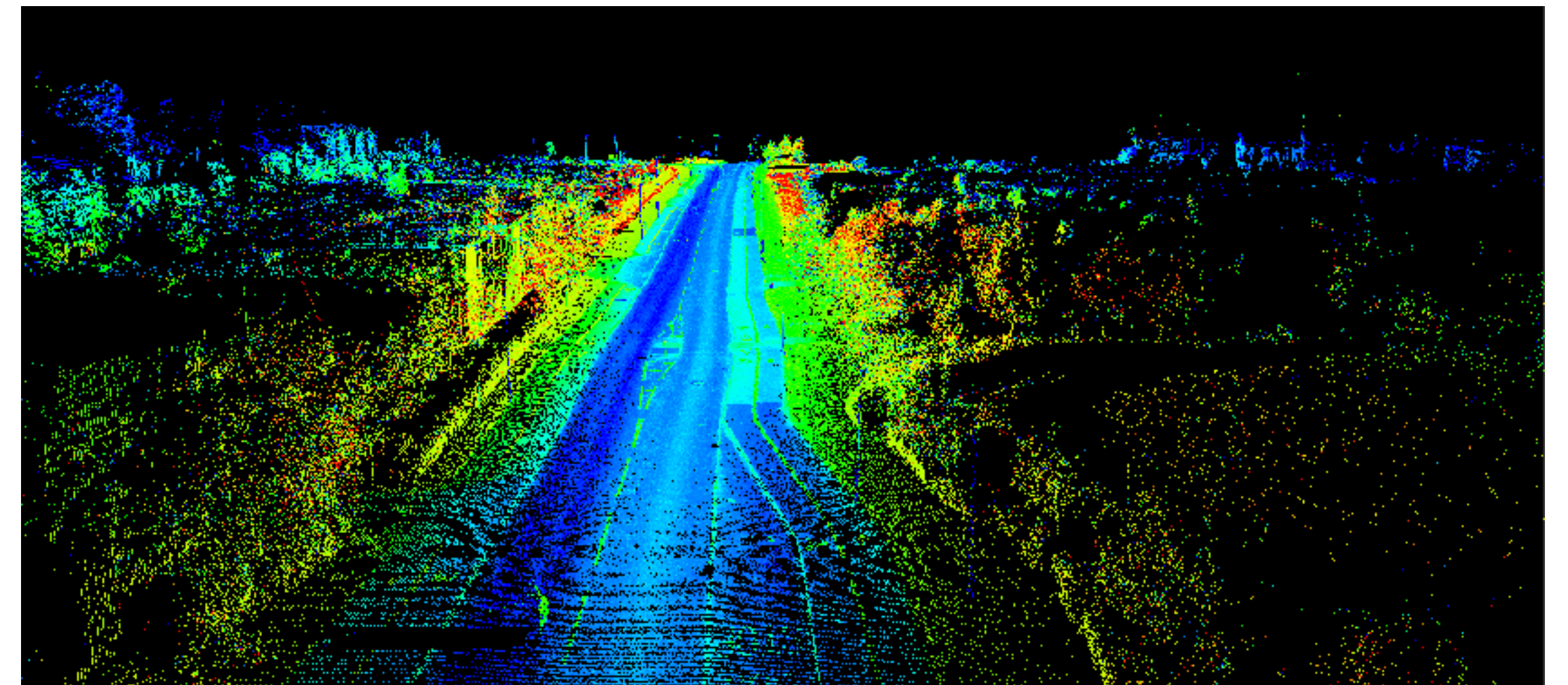
Frustum PointNets [6]

Images and Point Clouds



RGB images

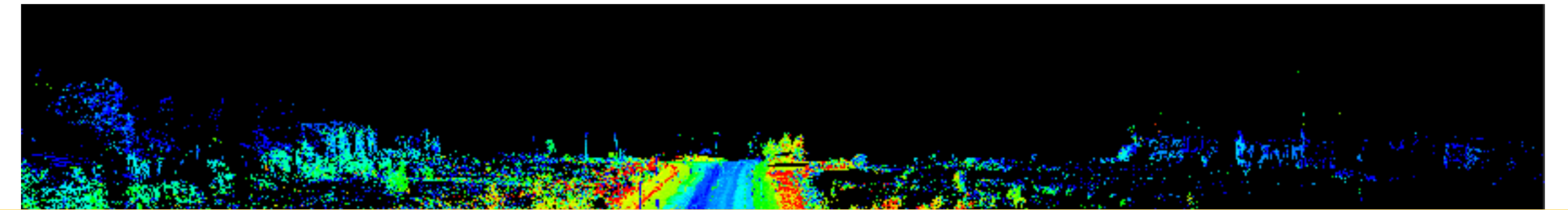
*High
resolution Rich
textures*



Lidar point clouds

*Accurate depth
Accurate 3D geometry*

Images and Point Clouds

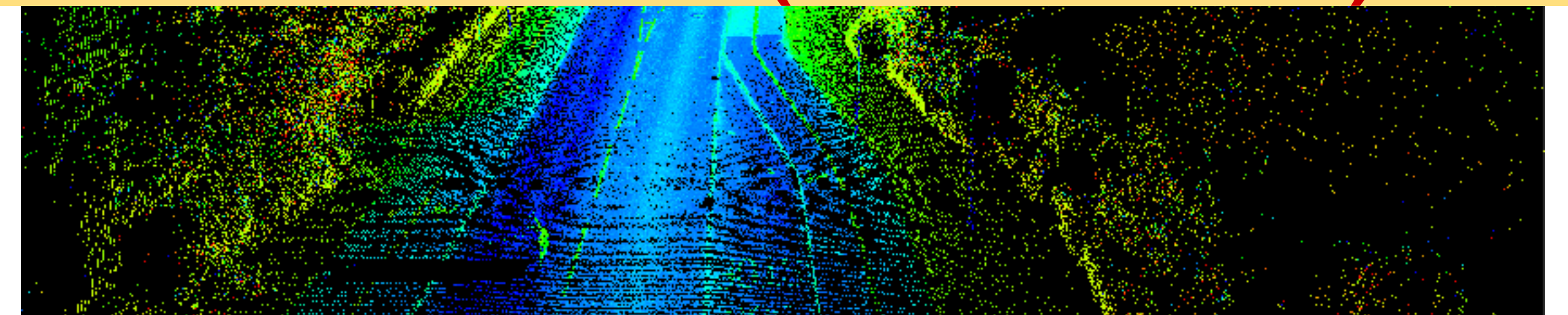


Can we get the best of both worlds (2D & 3D)?



RGB images

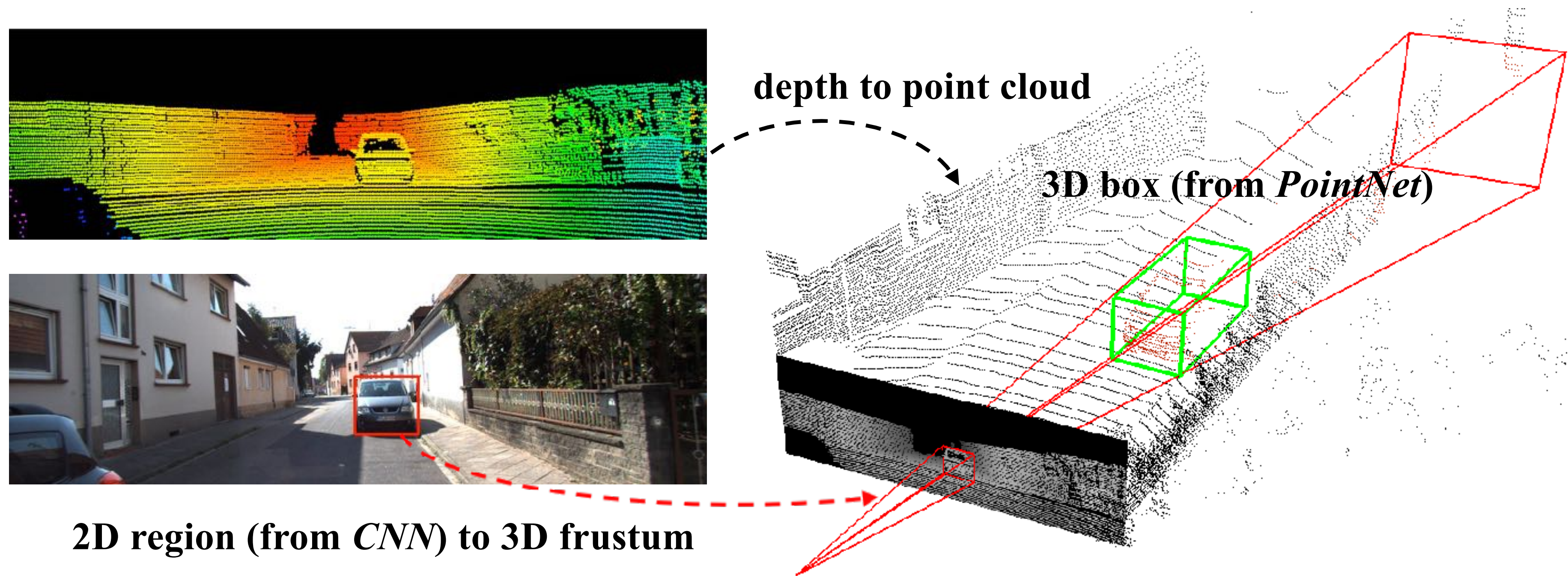
*High
resolution Rich
textures*



Lidar point clouds

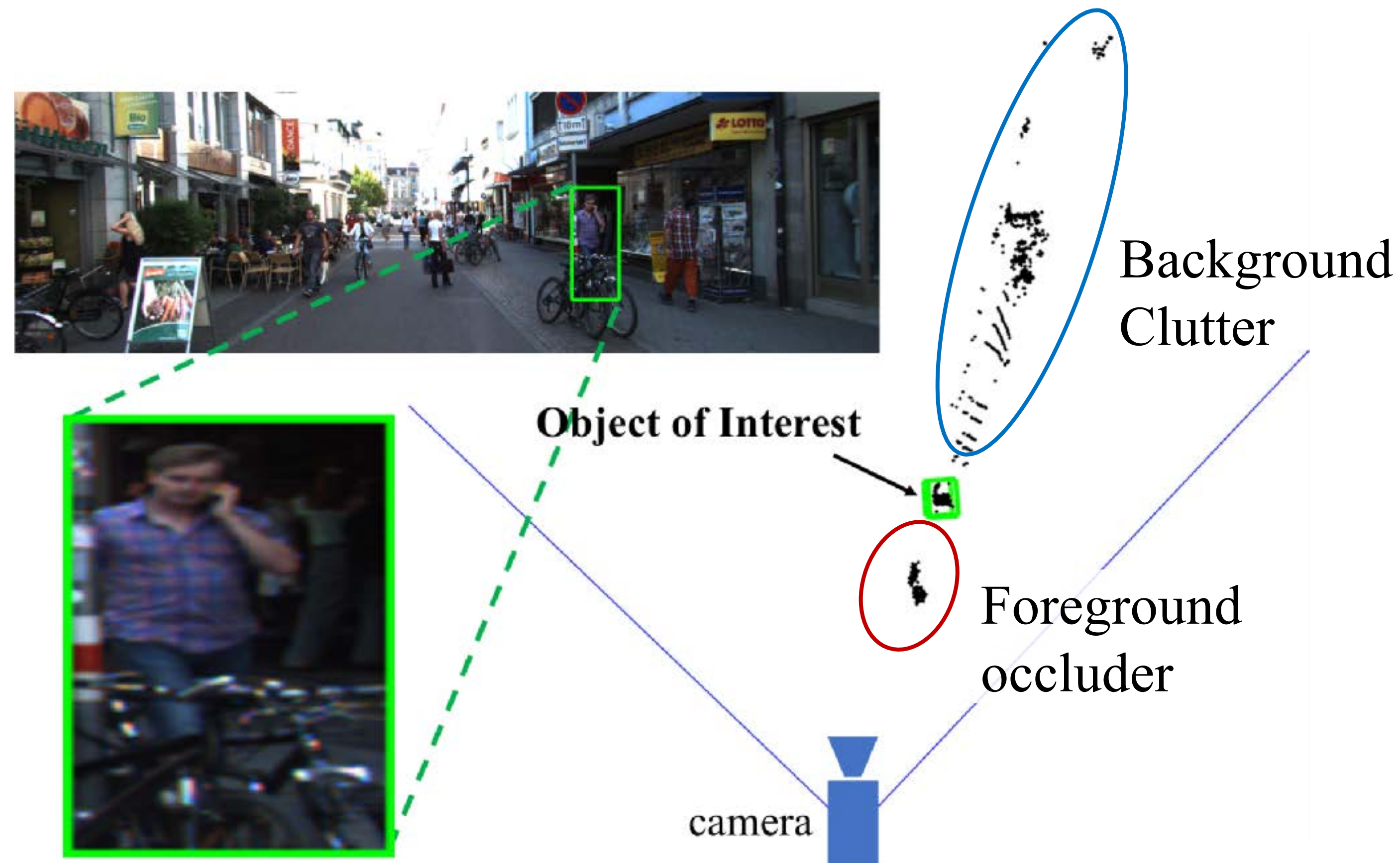
*Accurate depth
Accurate 3D geometry*

Frustum PointNets for 3D Object Detection



- + **Leveraging mature 2D detectors** for region proposal. greatly reducing 3D search space.
- + **3D deep learning** for accurate object localization in frustum point clouds.

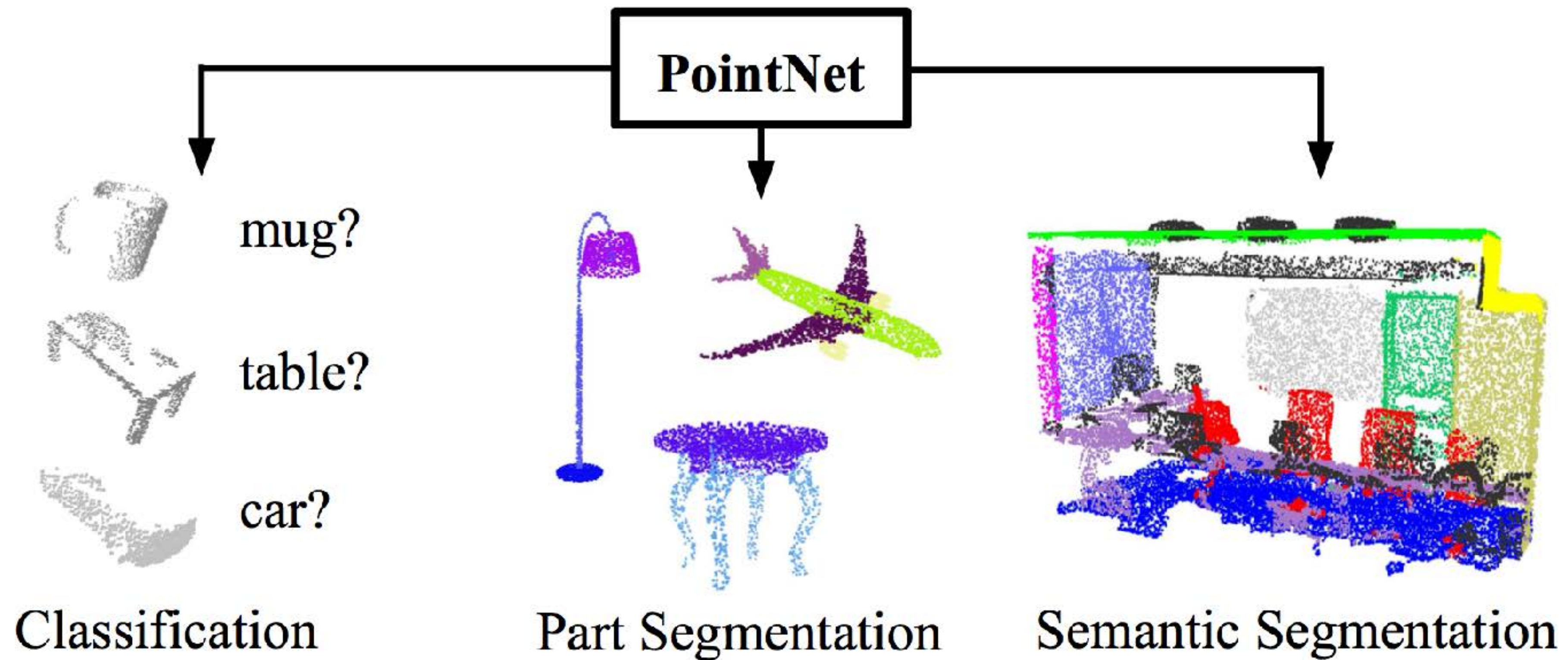
Frustum-based 3D Object Detection: Challenges



- Occlusions and clutters are common in frustum point clouds
- Large range of point depths

Frustum PointNets

Use **PointNets** for **data-driven** object detection in frustums.

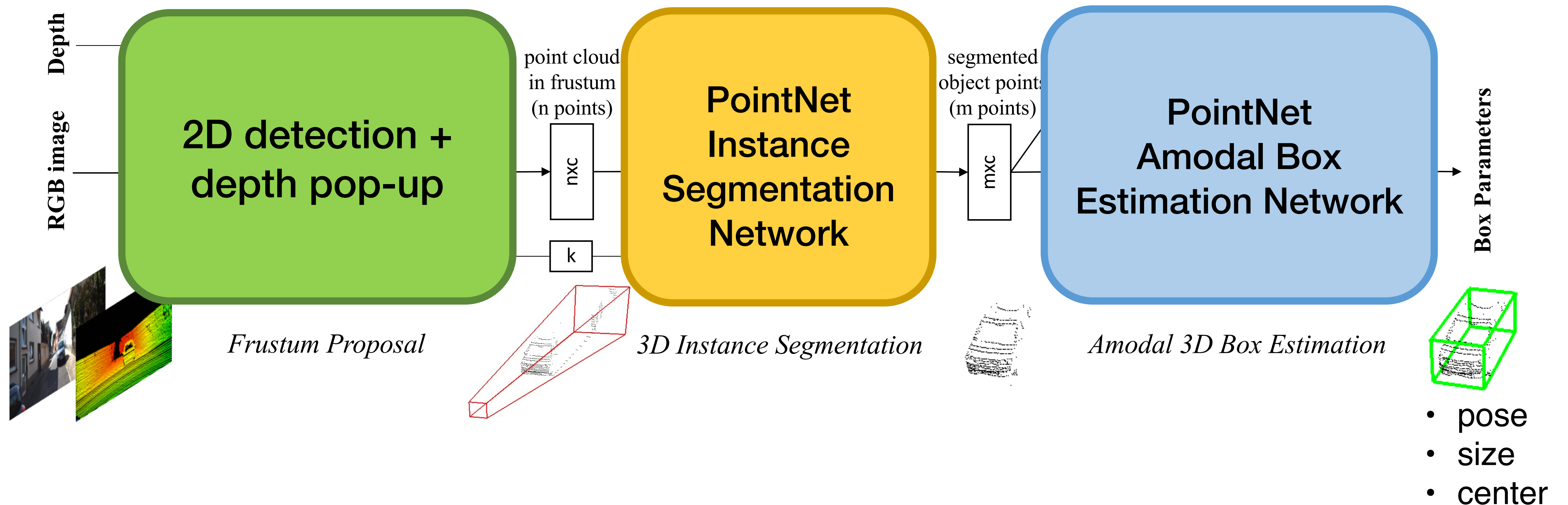


PointNet [Qi et al. CVPR 2017]

PointNet++ [Qi et al. NIPS 2017]

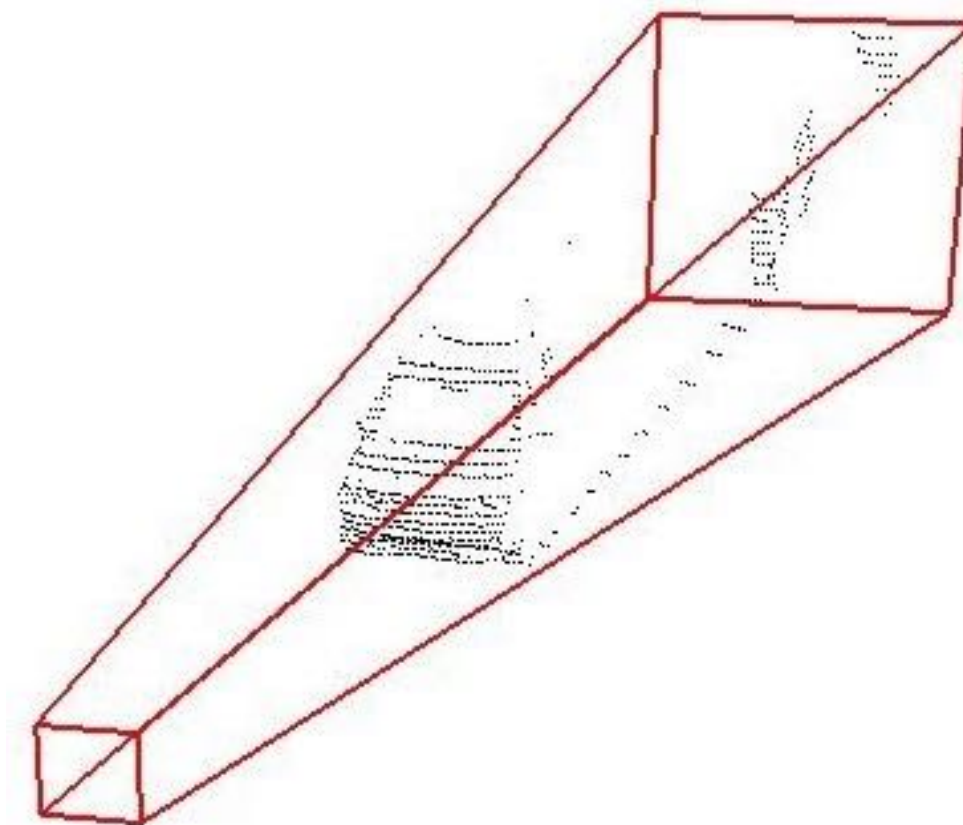
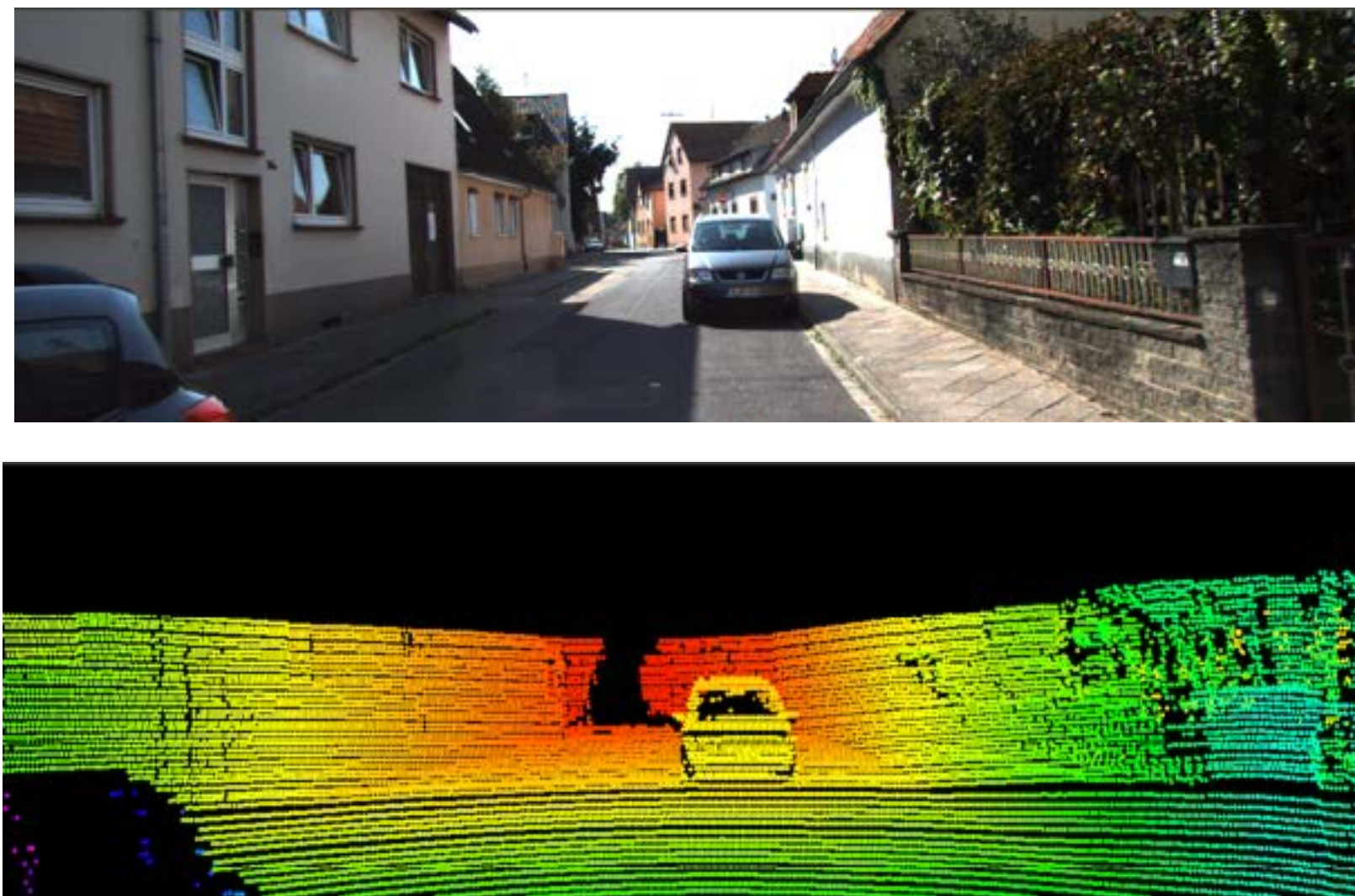
Frustum PointNets

Use **PointNets** for **data-driven** object detection in frustums.



Frustum Proposal

Propose 3D frustums by 2D region proposals in images and depth pop-up

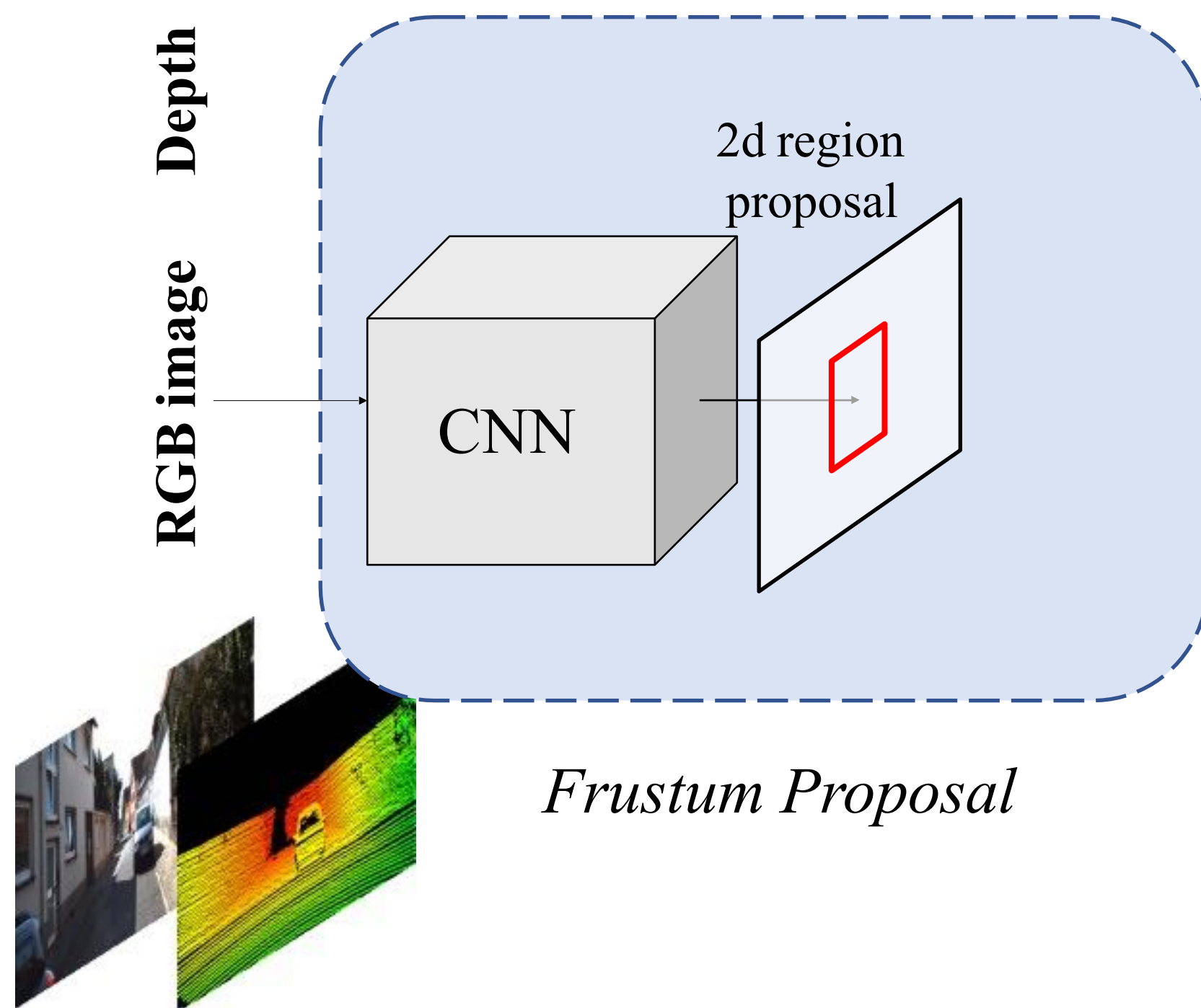


Frustum Proposal

Input: RGB-D data



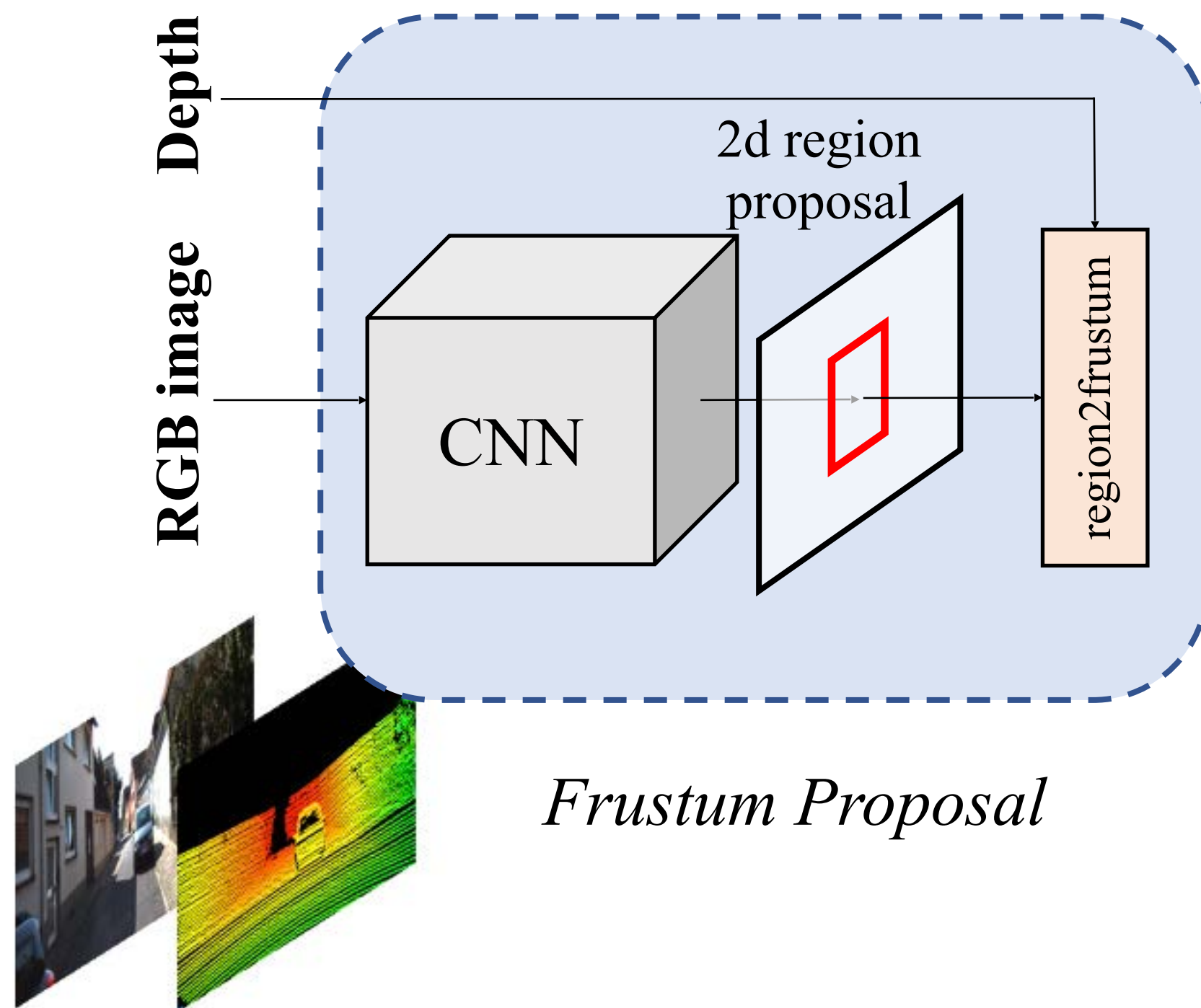
Frustum Proposal



Input: RGB-D data

Image region proposal using a 2D detector on RGB images (high resolution)

Frustum Proposal

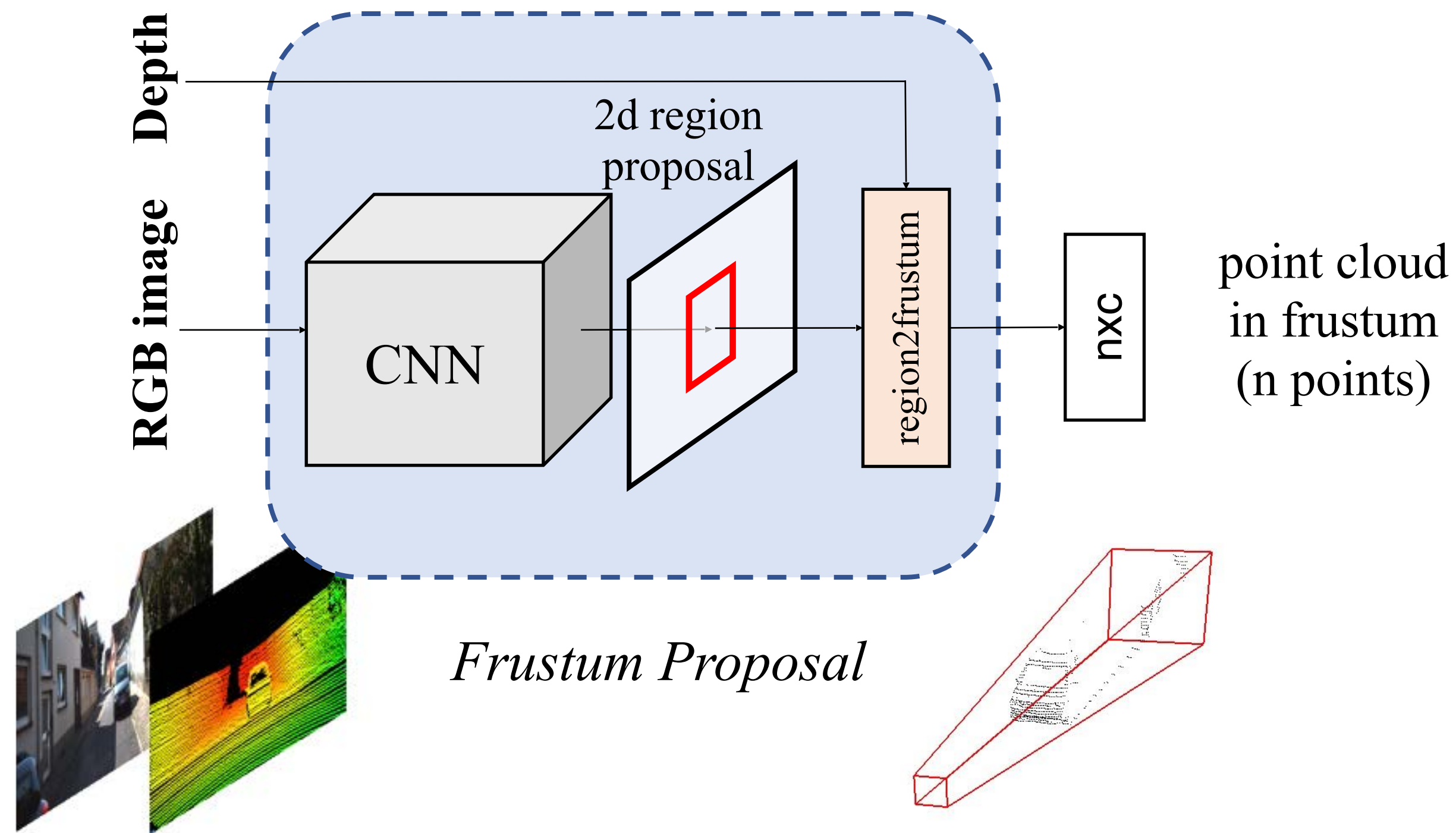


Input: RGB-D data

Image region proposal using a 2D detector on RGB images (high resolution)

Frustum proposal by lifting a 2D region into a 3D frustum.

Frustum Proposal



Input: RGB-D data

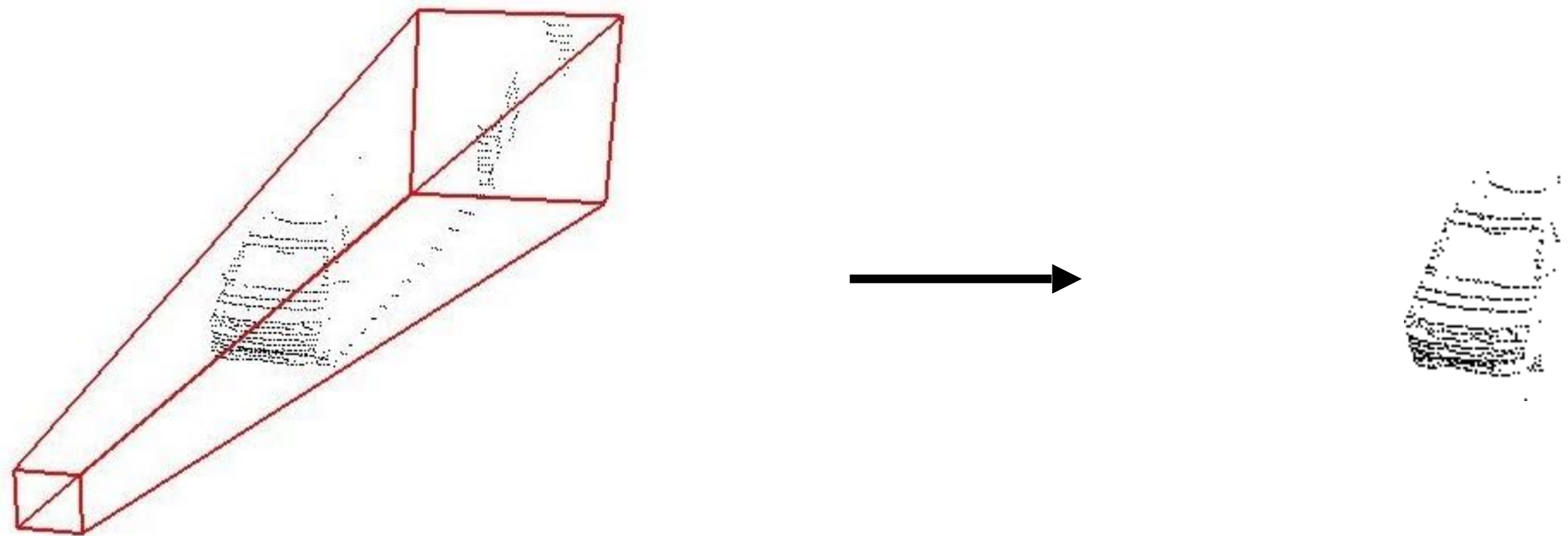
Image region proposal using a 2D detector on RGB images (high resolution)

Frustum proposal by lifting a 2D region into a 3D frustum.

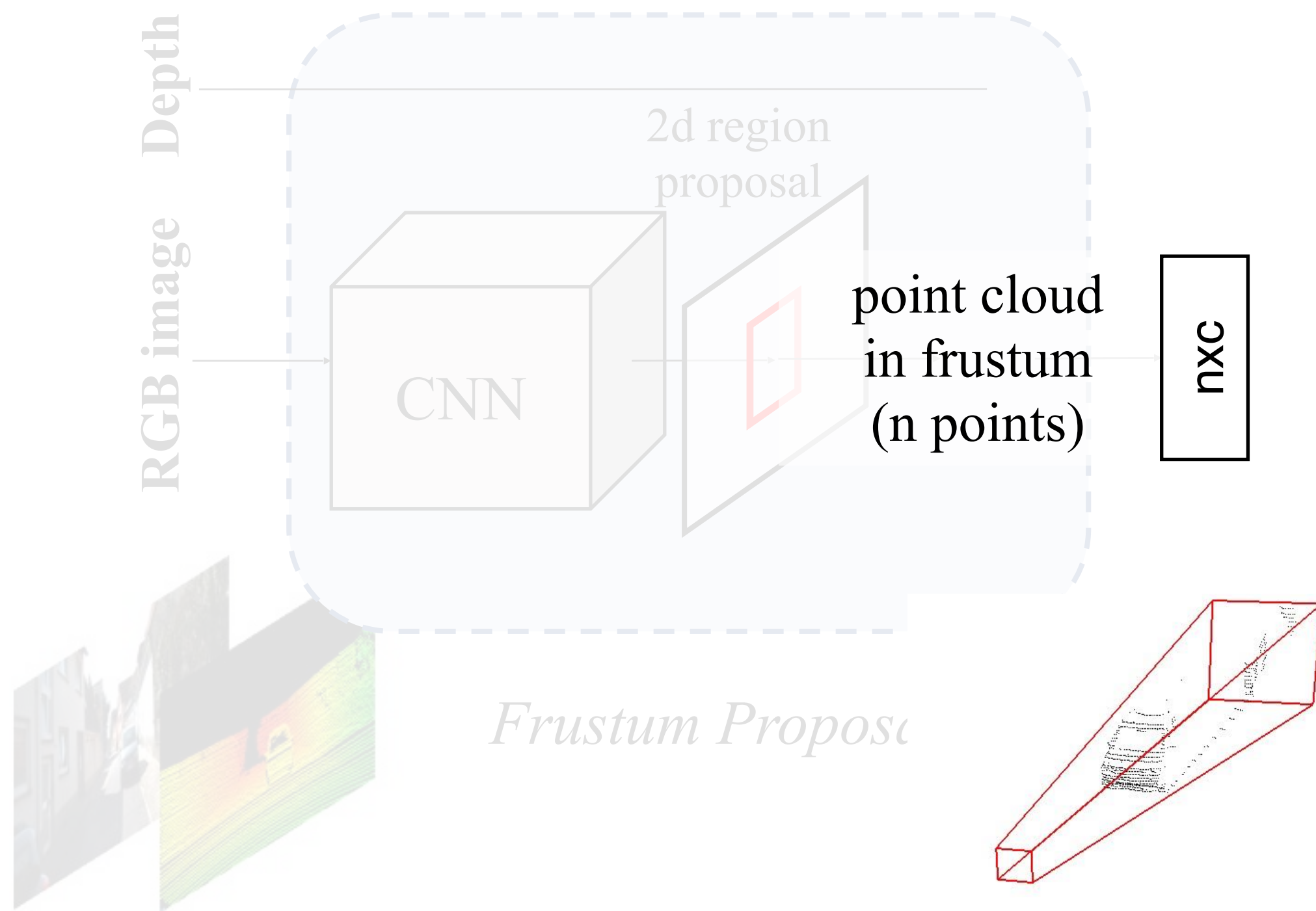
Points in the frustum are extracted and are called a *frustum point cloud*.

3D Instance Segmentation in Frustums

Localize objects in frustums by point cloud segmentation.

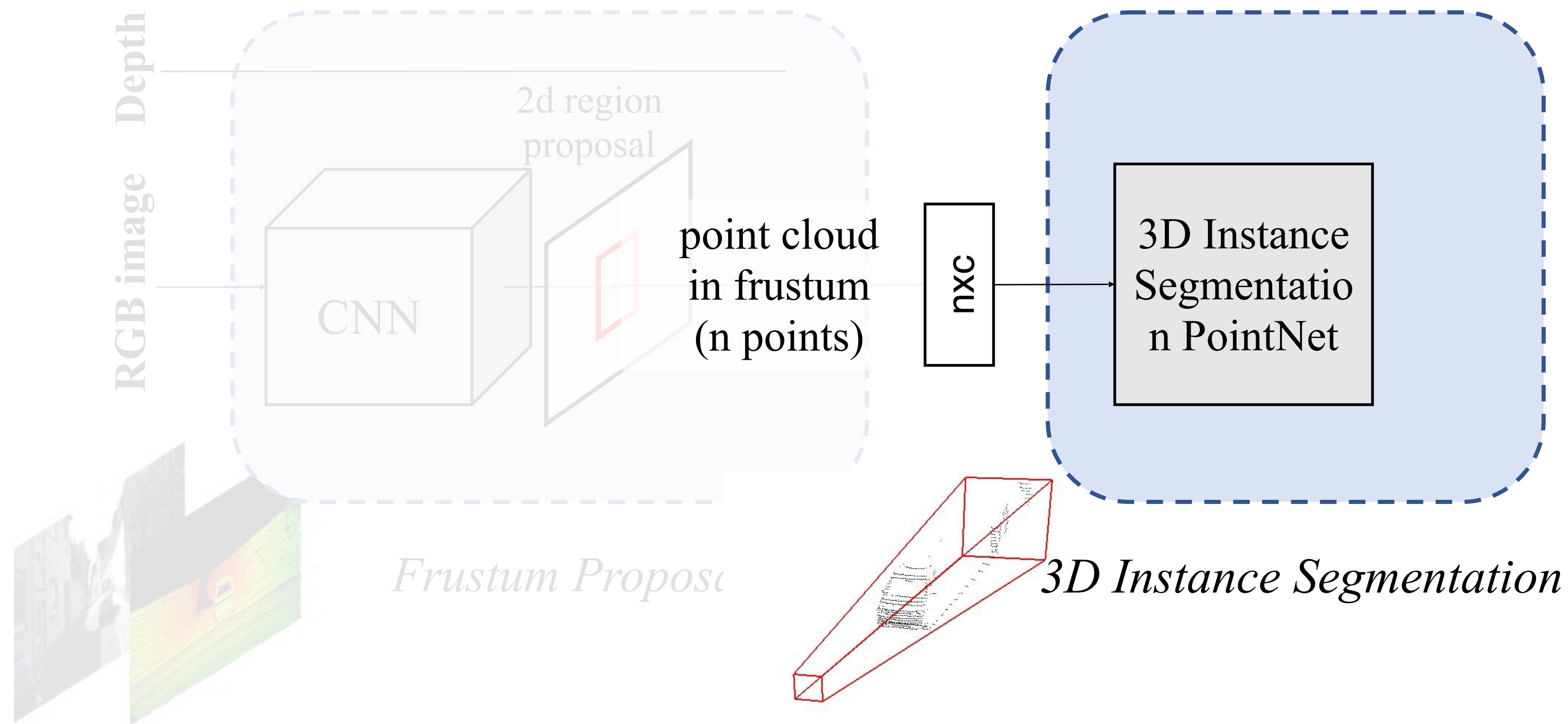


3D Instance Segmentation in Frustums



Input: frustum point cloud

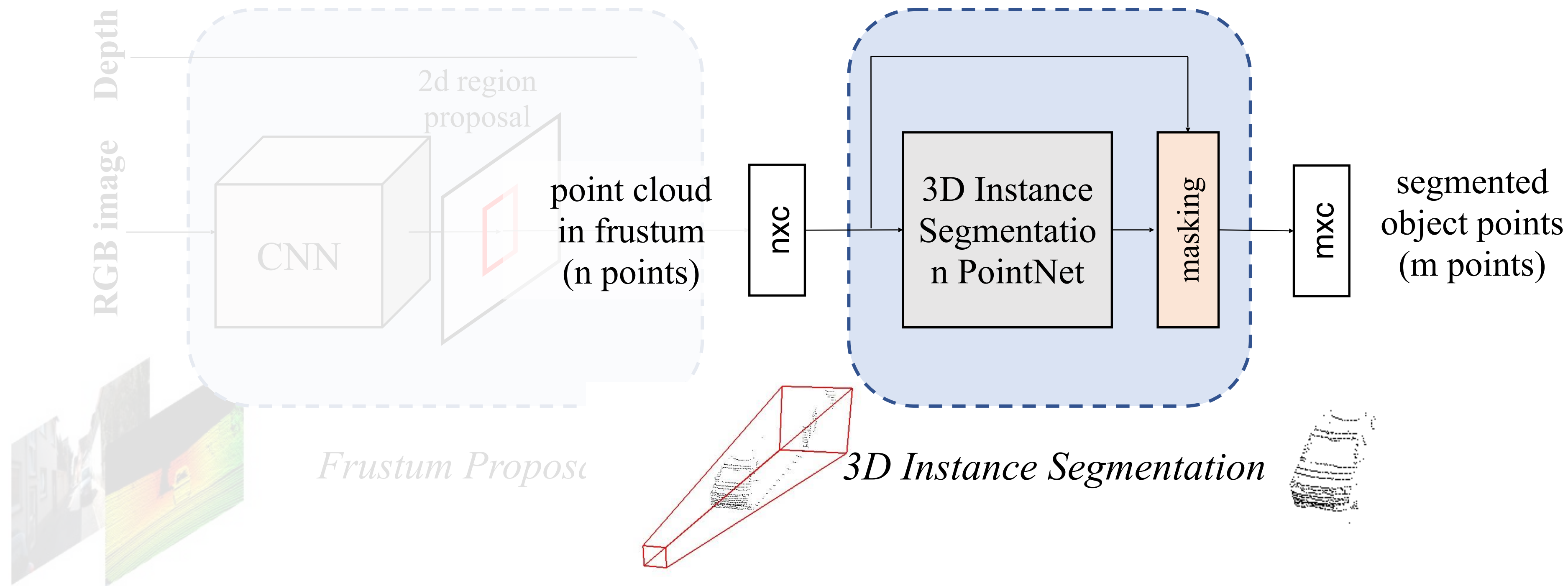
3D Instance Segmentation in Frustums



Input: frustum point cloud

Point cloud binary segmentation with PointNet: object of interest v.s. others

3D Instance Segmentation in Frustums

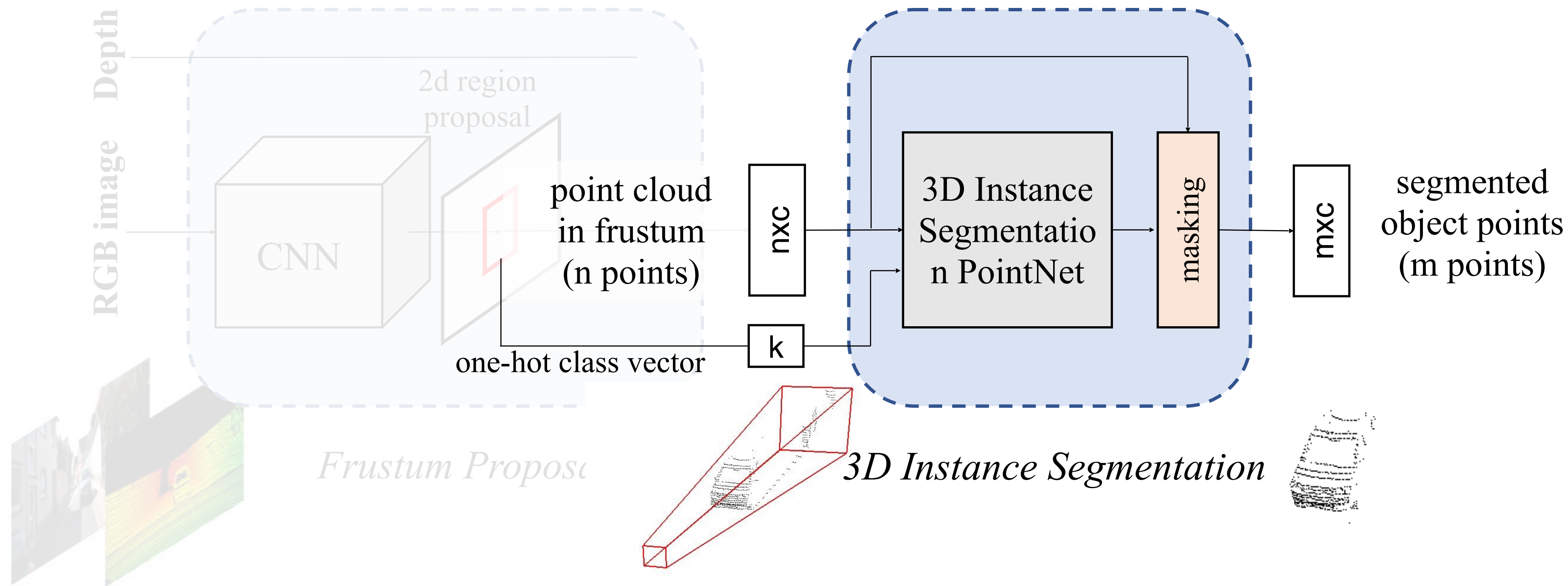


Input: frustum point cloud

Point cloud binary segmentation with PointNet: object of interest v.s. others

Points that are classified as object points are extracted for the next step.

3D Instance Segmentation in Frustums



Input: frustum point cloud

Point cloud binary segmentation with PointNet: object of interest v.s. others

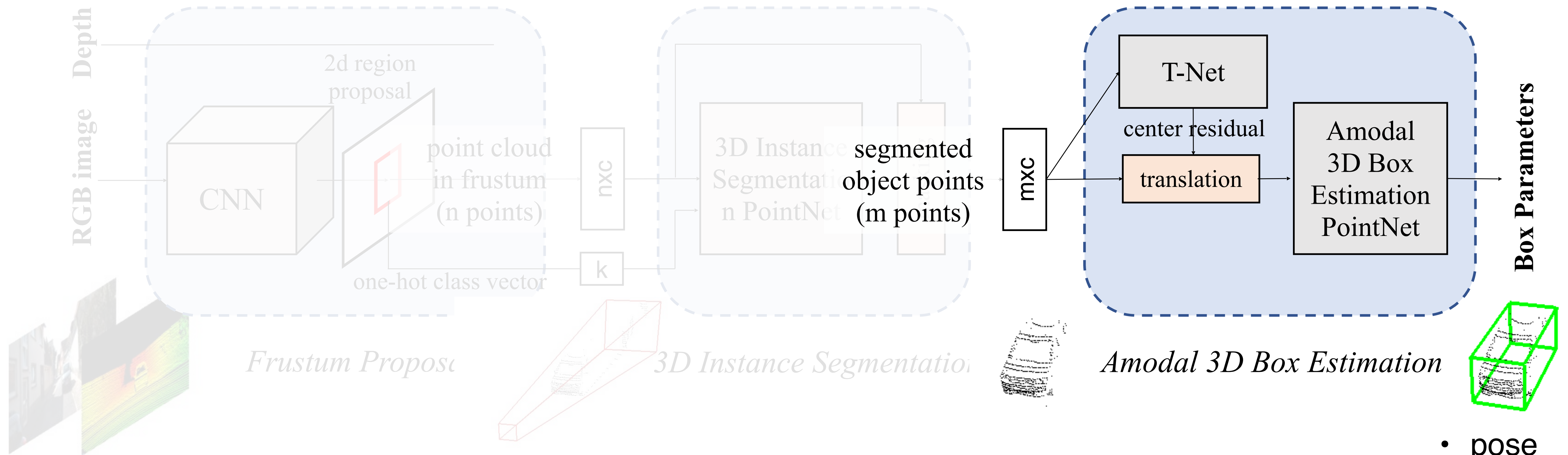
Points that are classified as object points are extracted for the next step.

Amodal 3D Bounding Box Estimation

Estimate 3D bounding boxes from segmented object point clouds.



Amodal 3D Bounding Box Estimation

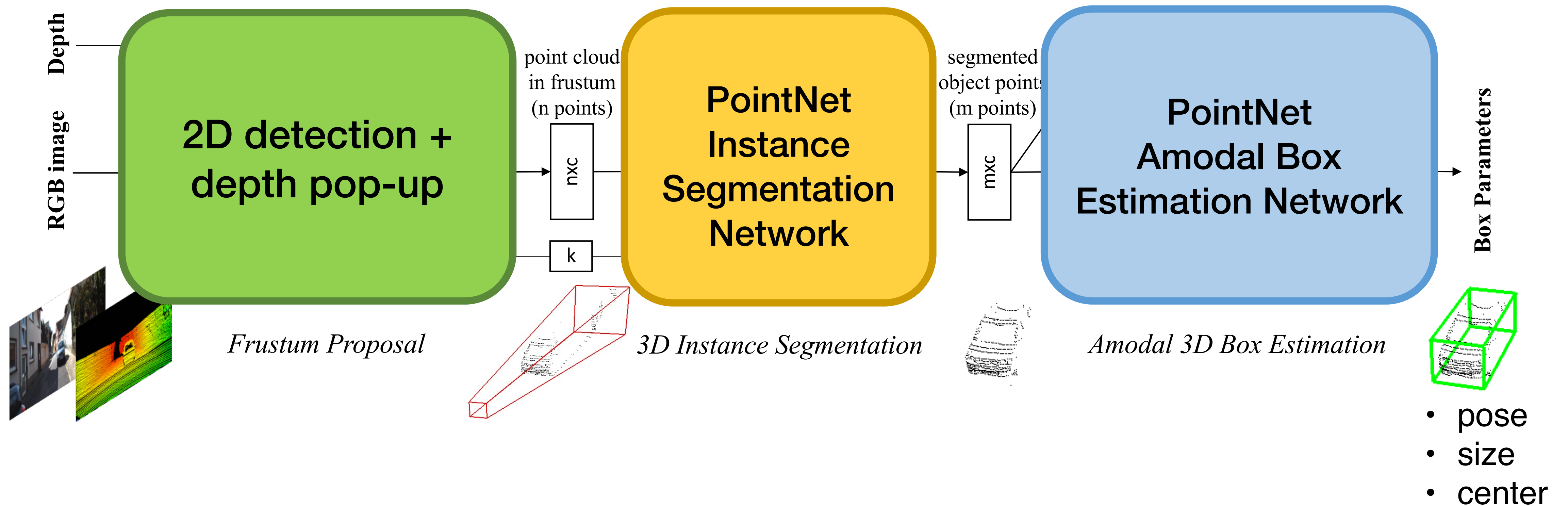


Input: object point cloud

A regression PointNet estimates amodal 3D bounding box for the object

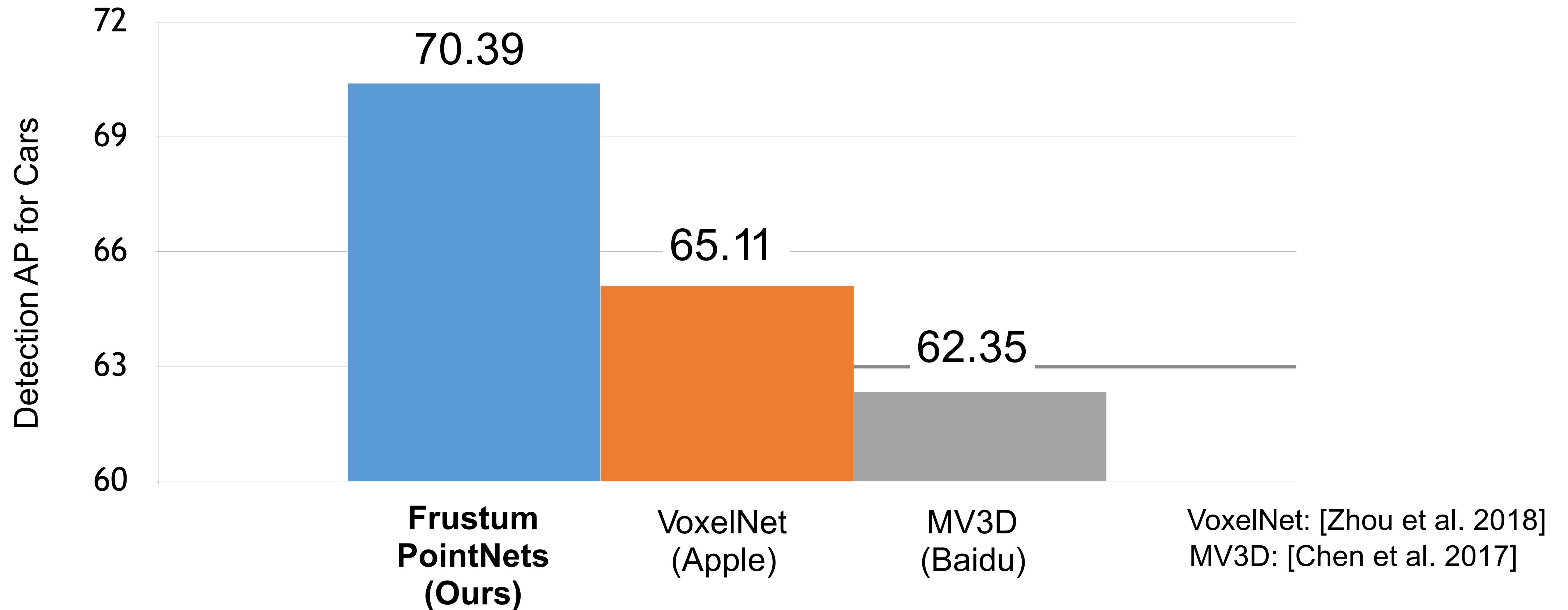
- pose
- size
- center

Frustum PointNets



KITTI Results: Quantitative

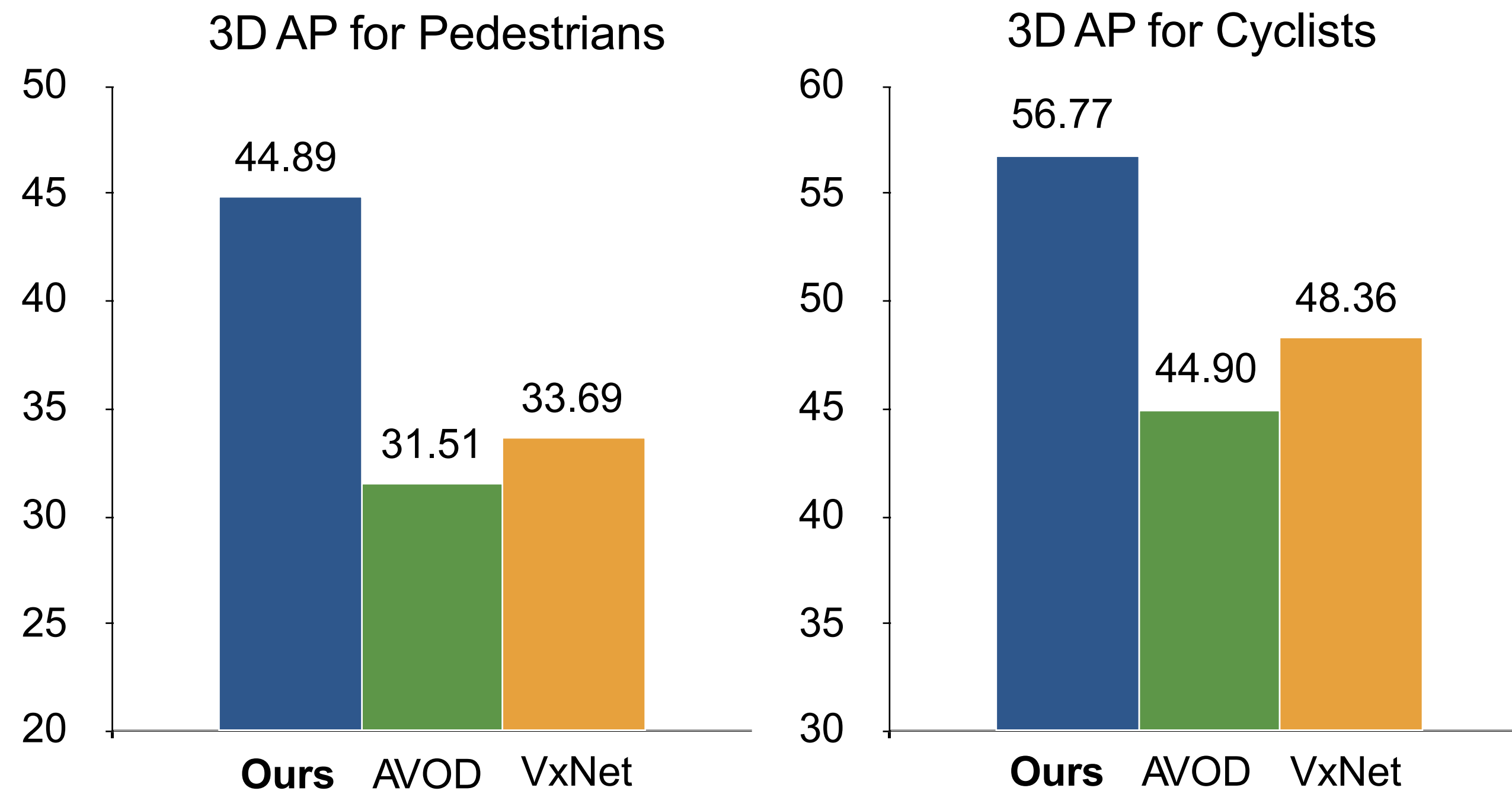
Leading performance on KITTI benchmark
(at the time of publication)



KITTI Results: Quantitative

Leading performance on KITTI benchmark
(at the time of publication)

Especially leading at smaller objects (pedestrians and cyclists) – hard to localize with 3D proposals only.



AVOD: [Ku et al. 2018]
VxNet: [Zhou et al. 2017]

Frustum PointNets: Key to our Success

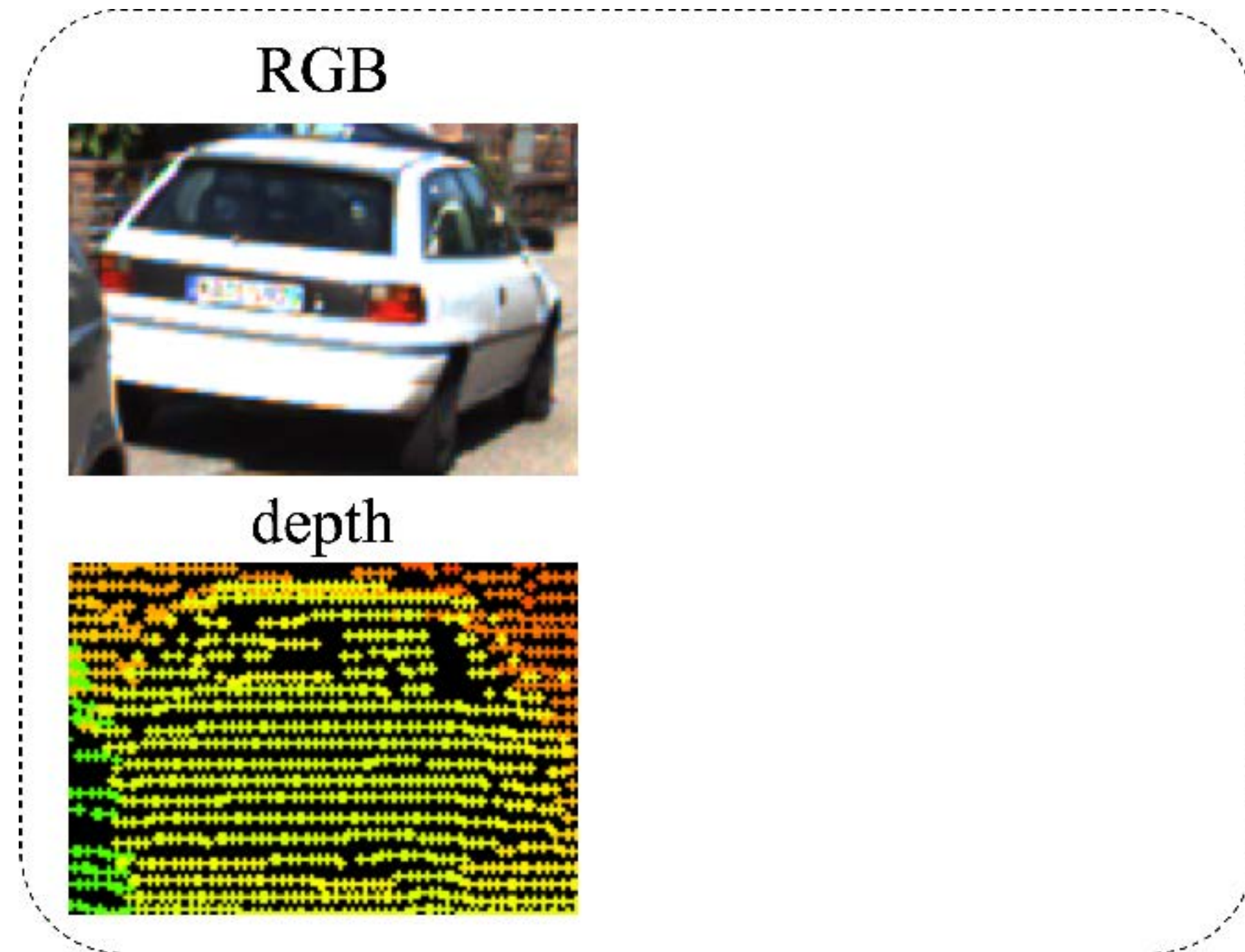
- **Representation** matters — 2D v.s. 3D

Instance segmentation: depth range maps v.s. point clouds.

Frustum PointNets: Key to our Success

- **Representation** matters — 2D v.s. 3D

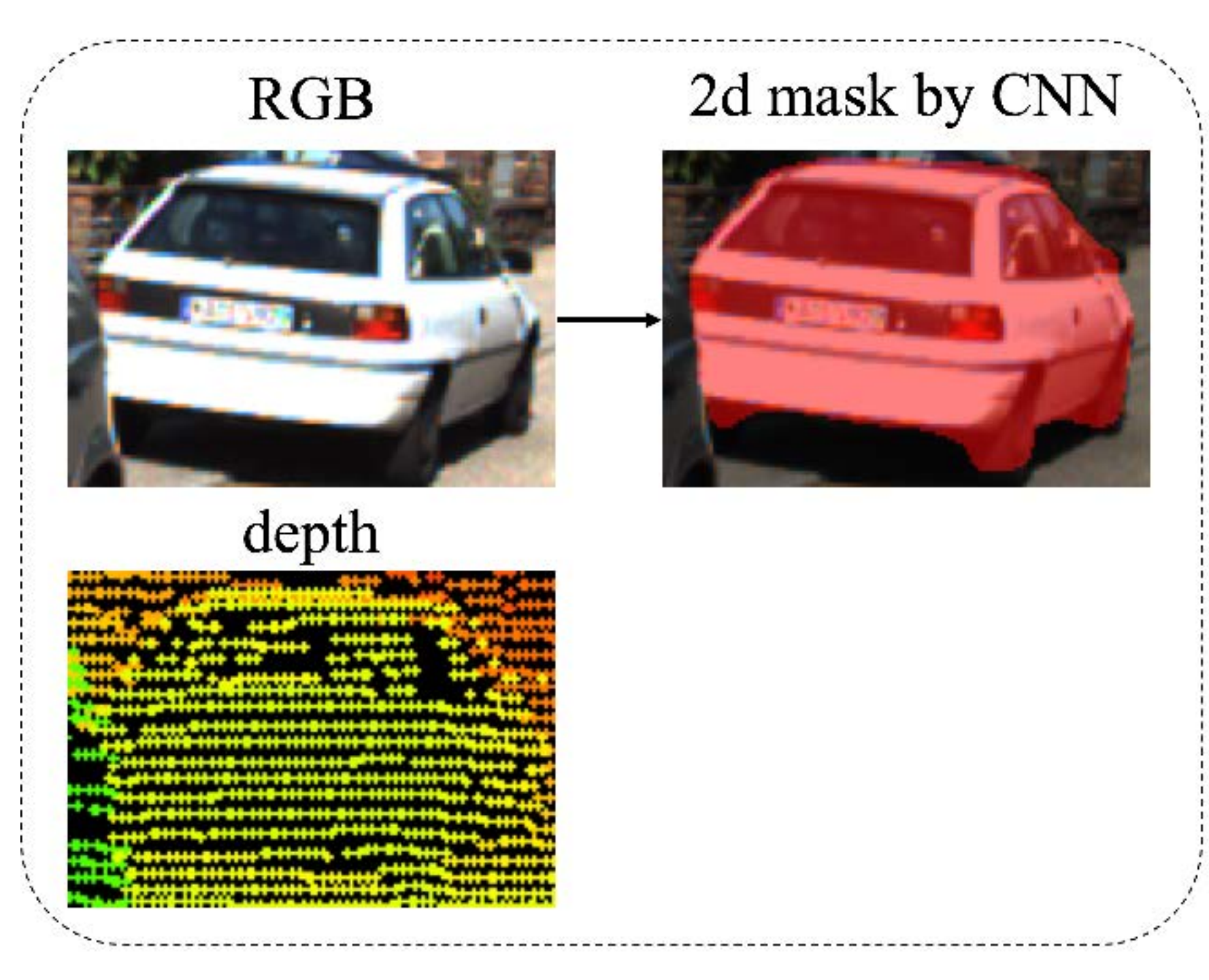
Instance segmentation: depth range maps v.s. point clouds.



Frustum PointNets: Key to our Success

- **Representation** matters — 2D v.s. 3D

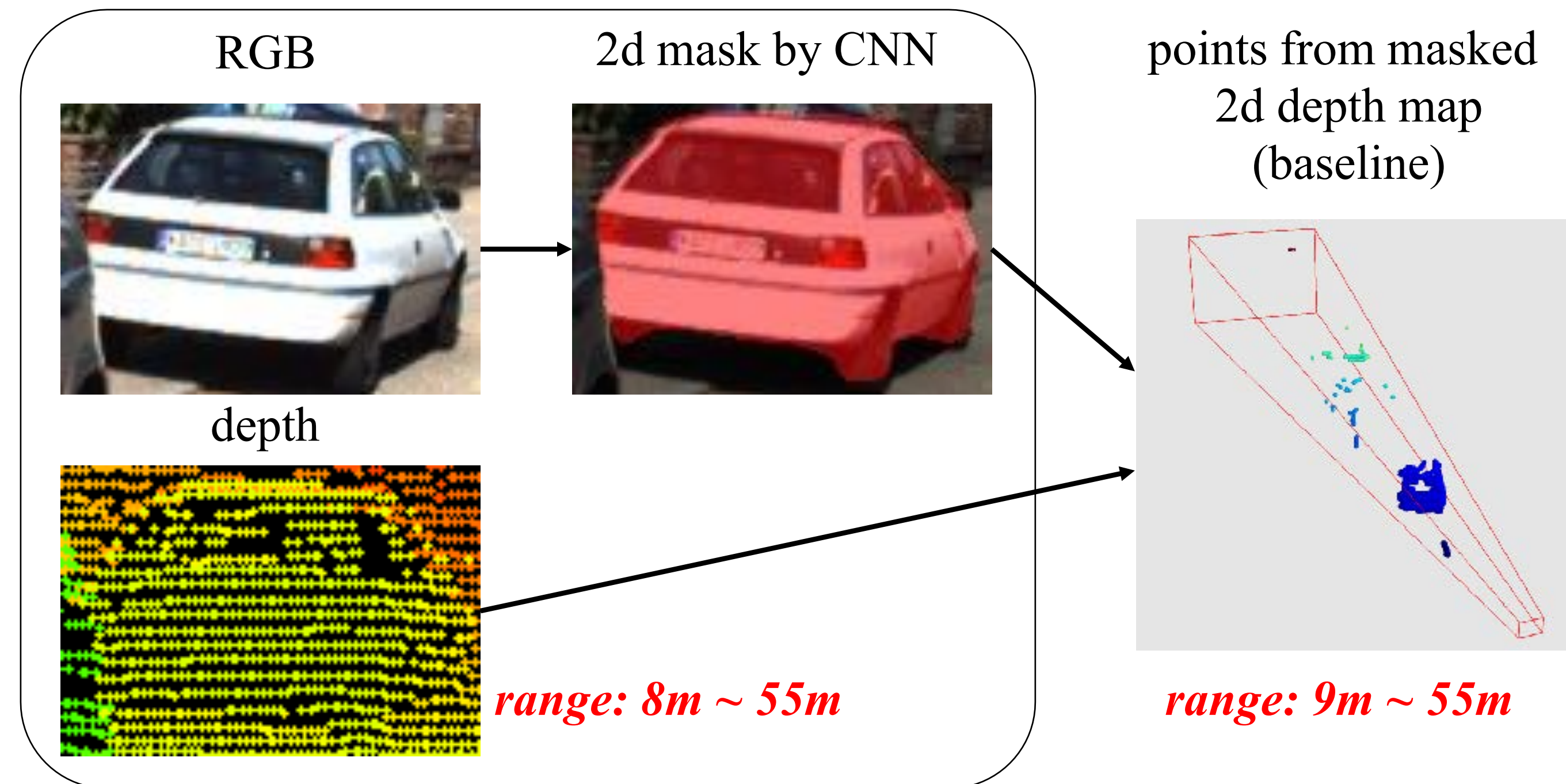
Instance segmentation: depth range maps v.s. point clouds.



Frustum PointNets: Key to our Success

- **Representation matters — 2D v.s. 3D**

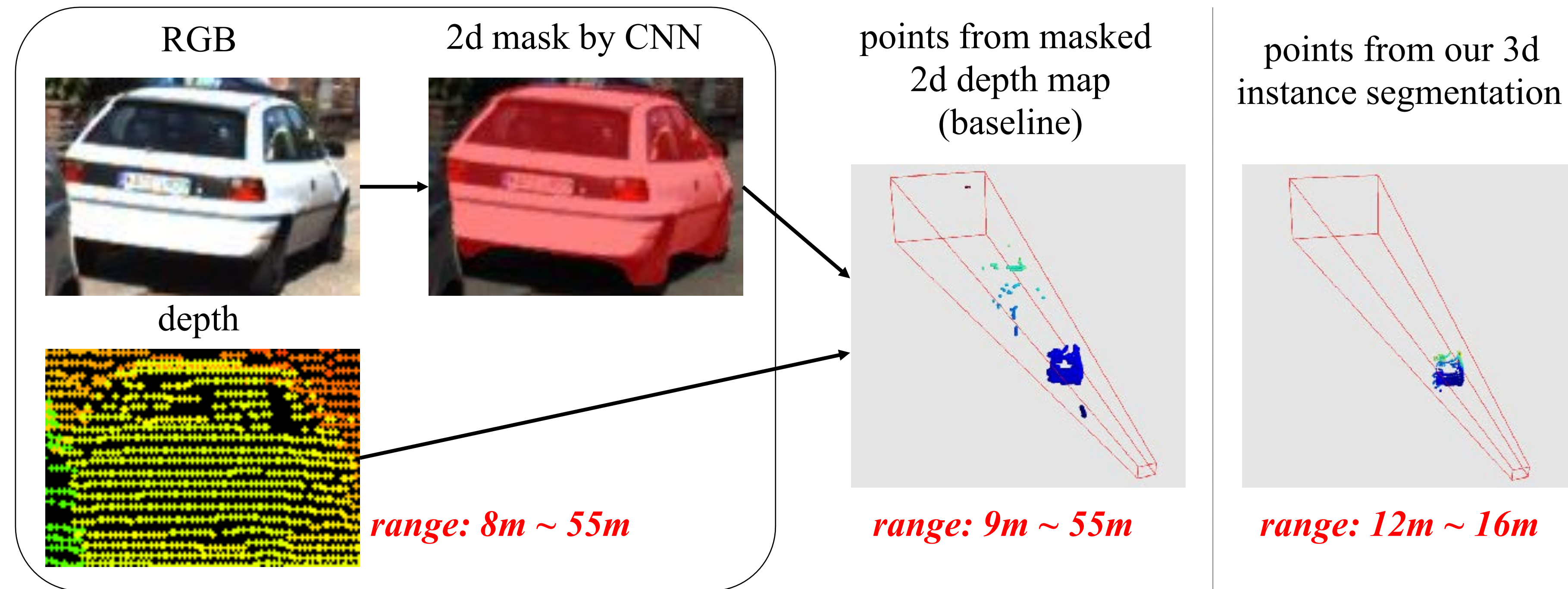
Instance segmentation: depth range maps v.s. point clouds.



Frustum PointNets: Key to our Success

- **Representation** matters — 2D v.s. 3D

Instance segmentation: depth range maps v.s. point clouds.



Frustum PointNets: Key to our Success

- **Representation** matters — 2D v.s. 3D

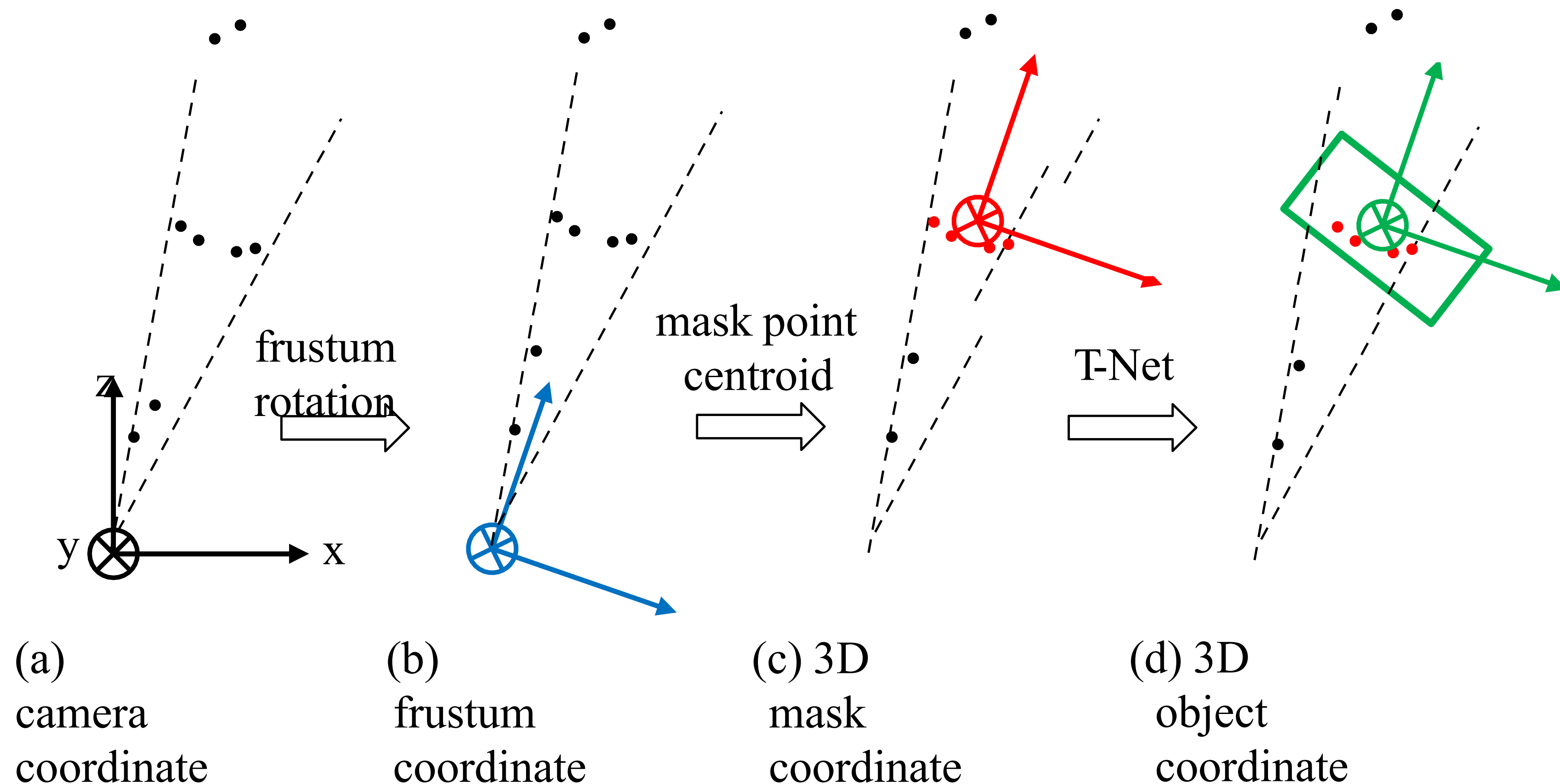
Effects of depth representation

network arch.	mask	depth representation	accuracy
ConvNet	-	image	18.3
ConvNet	2D	image	27.4
PointNet	-	point cloud	33.5
PointNet	2D	point cloud	61.6
PointNet	3D	point cloud	74.3
PointNet	2D+3D	point cloud	70.0

dataset: KITTI; metric: 3D bounding box estimation accuracy (%) under IoU 0.7

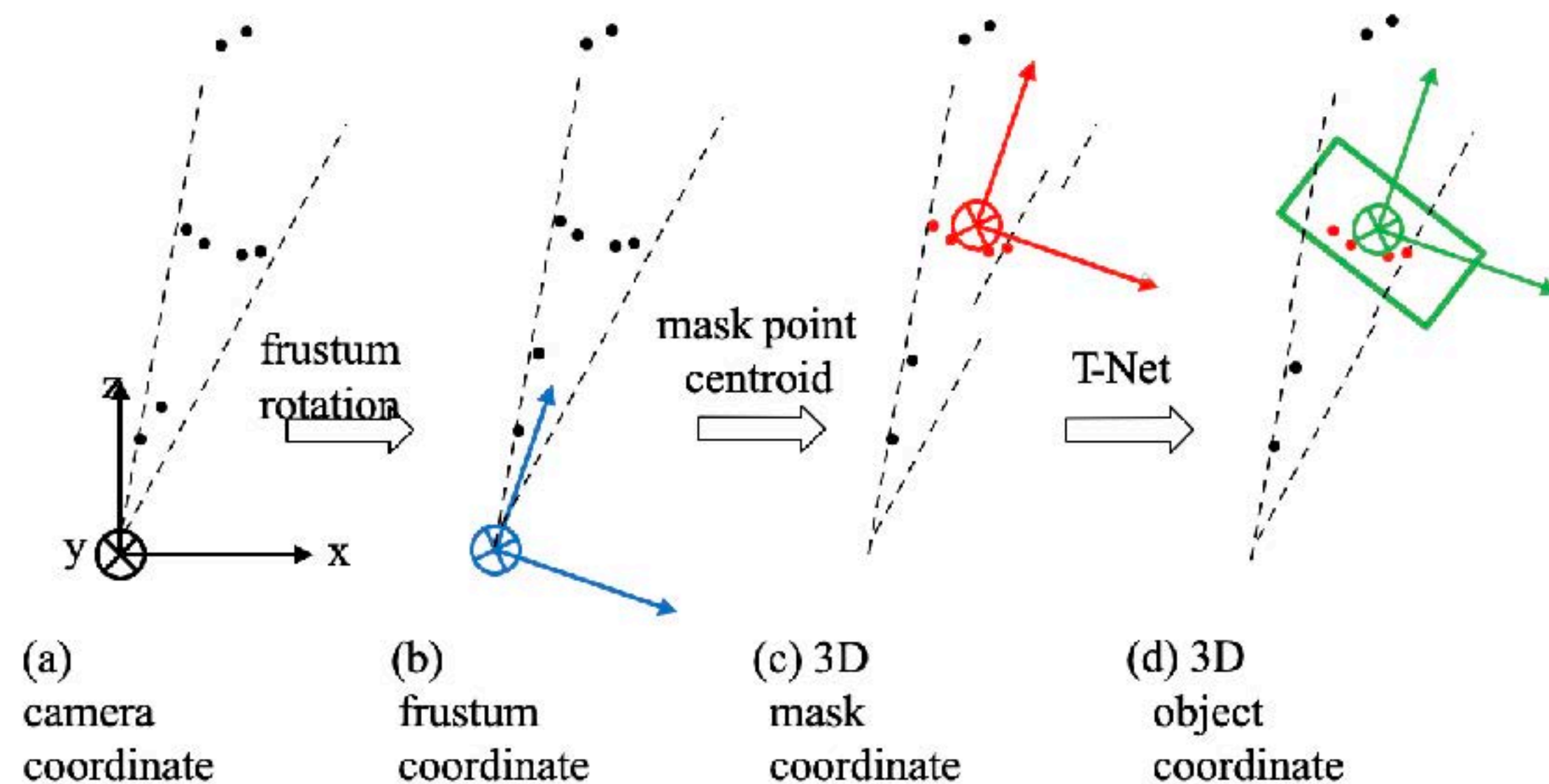
Frustum PointNets: Key to our Success

- **Canonicalize** the problem with coordinate transformations



Frustum PointNets: Key to our Success

- **Canonicalize** the problem with coordinate transformations



frustum rot.	mask centralize	t-net	accuracy
-	-	-	12.5
✓	-	-	48.1
-	✓	-	64.6
✓	✓	-	71.5
✓	✓	✓	74.3

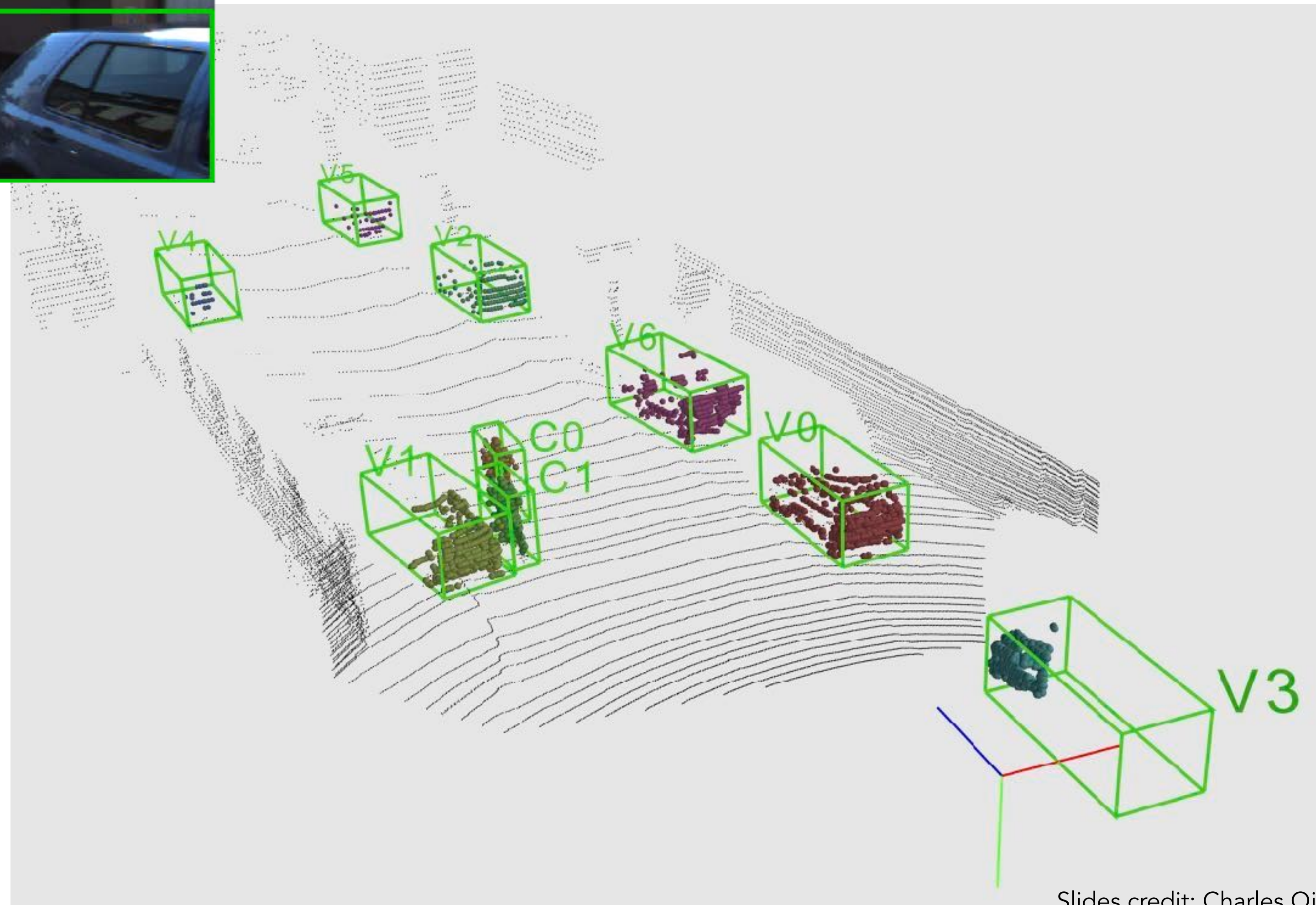
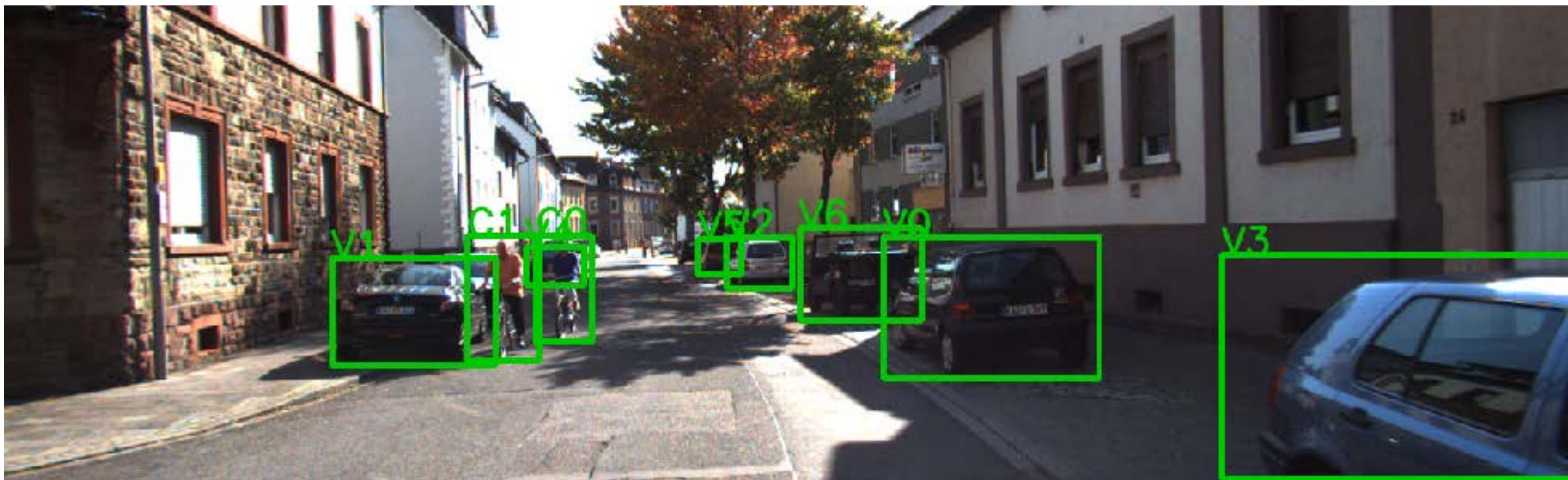
dataset: KITTI; metric: 3D bounding box estimation accuracy (%) under IoU 0.7

Frustum PointNets: Key to our Success

Respect and exploit 3D

- **Representation matters** — using 3D representation and 3D deep learning for the 3D problem.
- **Canonicalize the problem** — exploiting geometric transformations in point clouds.

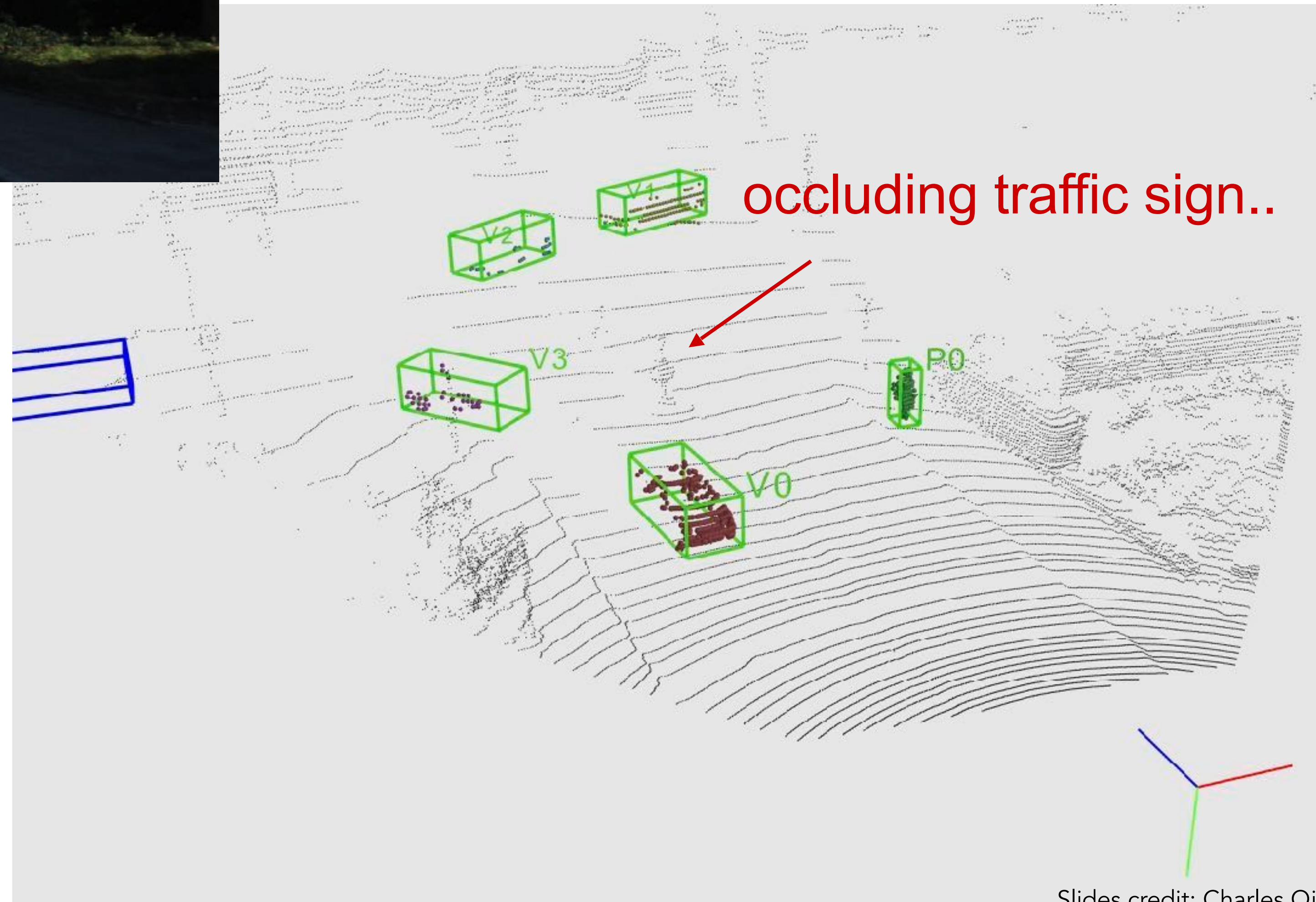
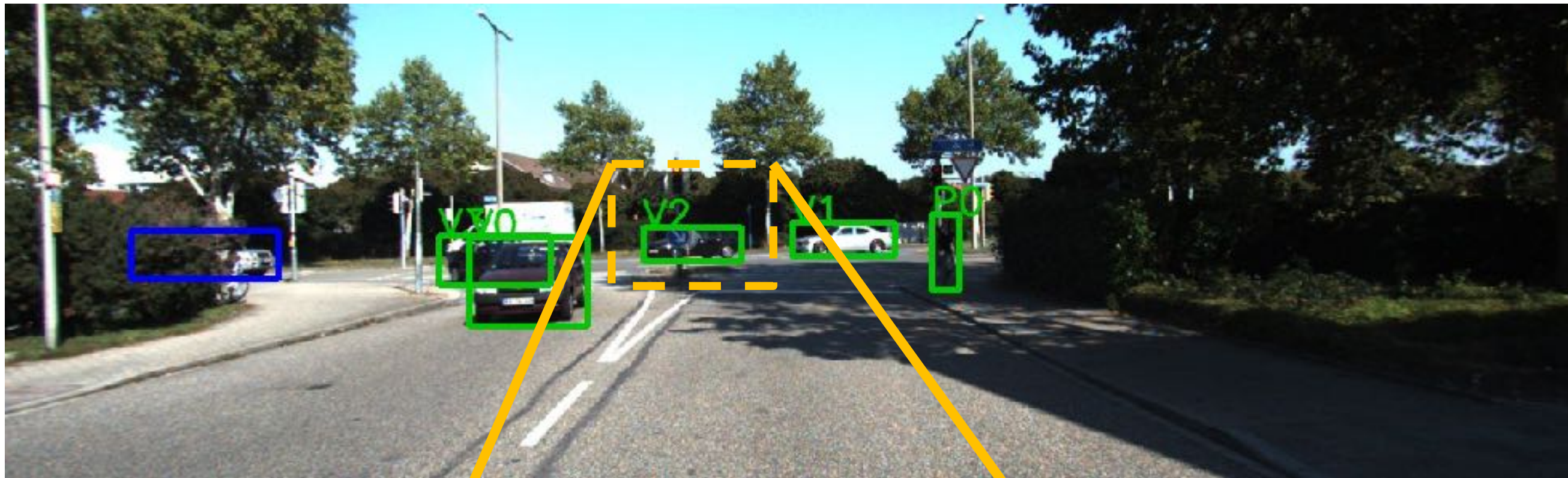
KITTI Results: Qualitative



Remarkable box estimation accuracy even with a dozen of points or with very partial point clouds.

KITTI Results: Qualitative

Correct segmentation in point clouds with heavy occlusion.



KITTI Results: Qualitative

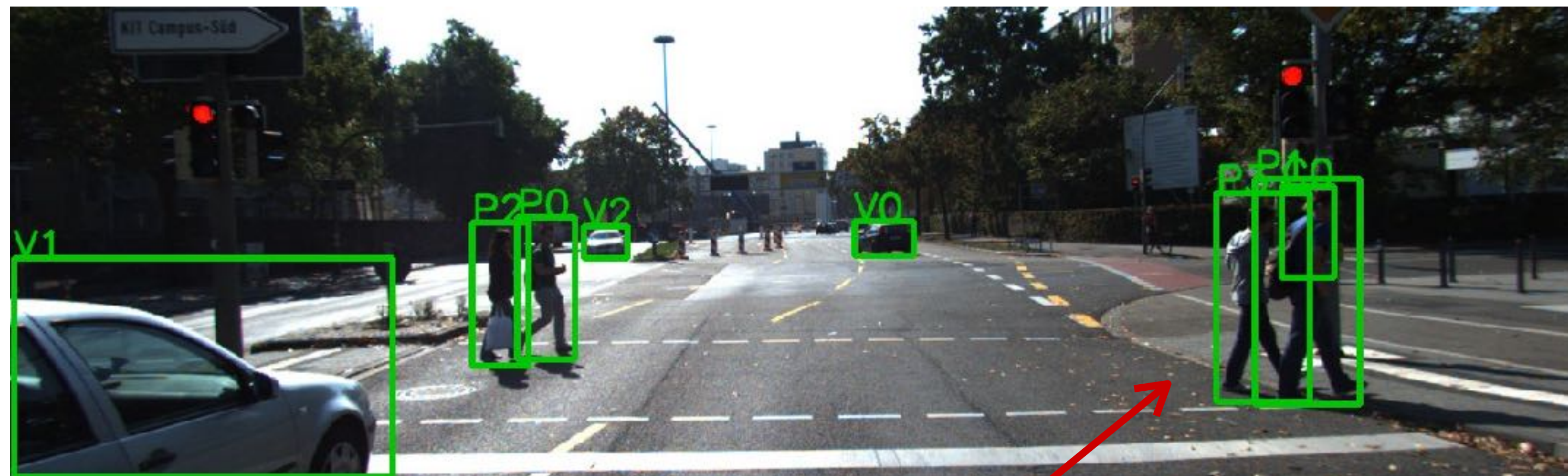


Missing 2D detection results in
no 3D detection

Multiple ways of proposal could
help (e.g. bird's eye view,
multiple 2D proposal networks)

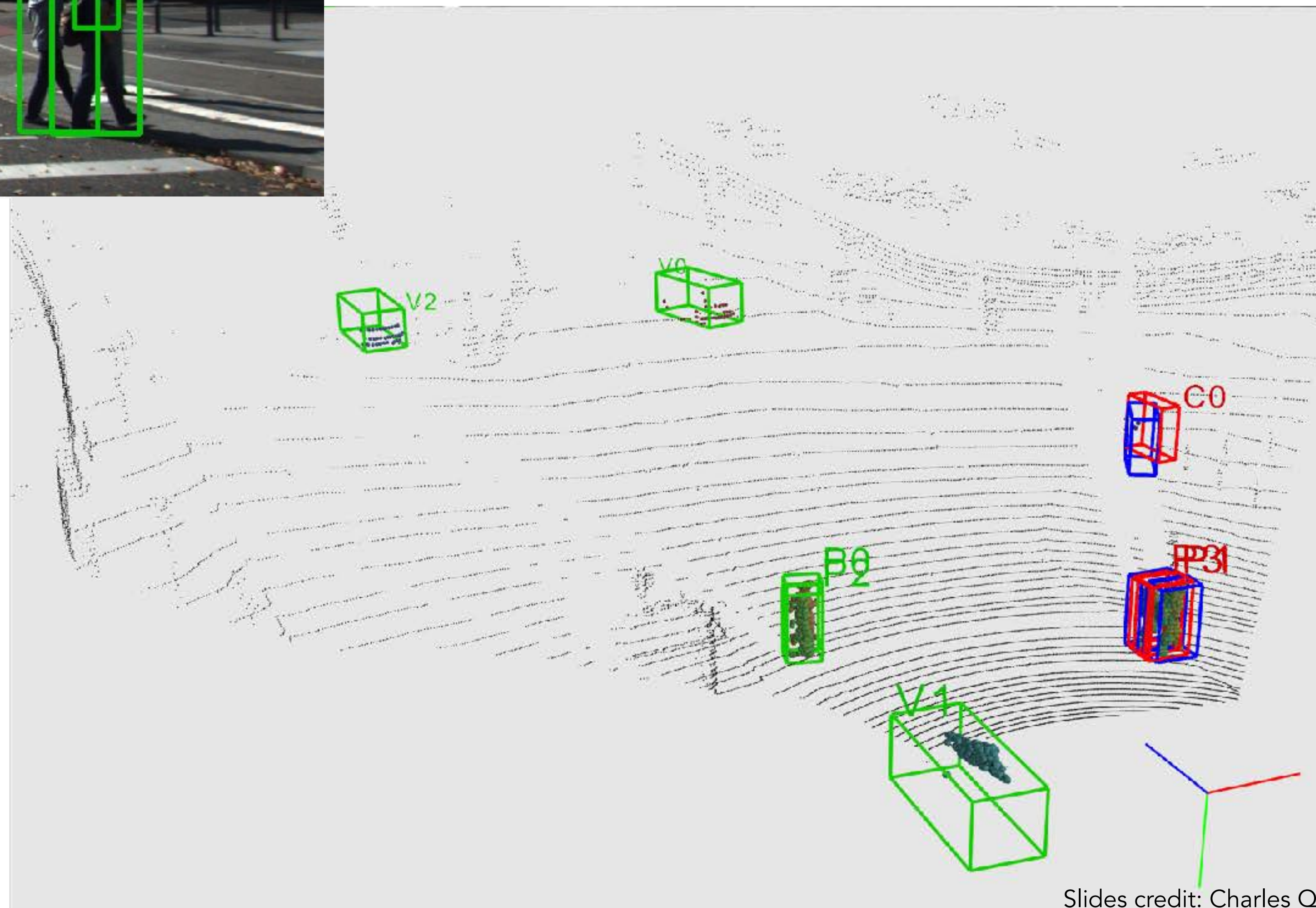


KITTI Results: Qualitative



Very strong occlusion.

Challenging case for instance segmentation (multiple close-by objects in a single frustum)



Limitation of the Frustum PointNets

- Hard dependence on 2D detections: will miss objects due to strong occlusions in 2D views or unfavorable illumination conditions.
- No support of multiple 3D proposals in a frustum.

Solution: *object proposal from 3D point clouds.*
(VoteNet & ImVoteNet)

Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ *PointPillars*
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - Bottom-Up Approaches

Bird's eye view 3D object detector

- Key idea: Converting the 3D learning problem to a 2D learning problem.

Volumetric and Multi-View CNNs for Object Classification on 3D Data (CVPR'16) by Qi et al. [12]

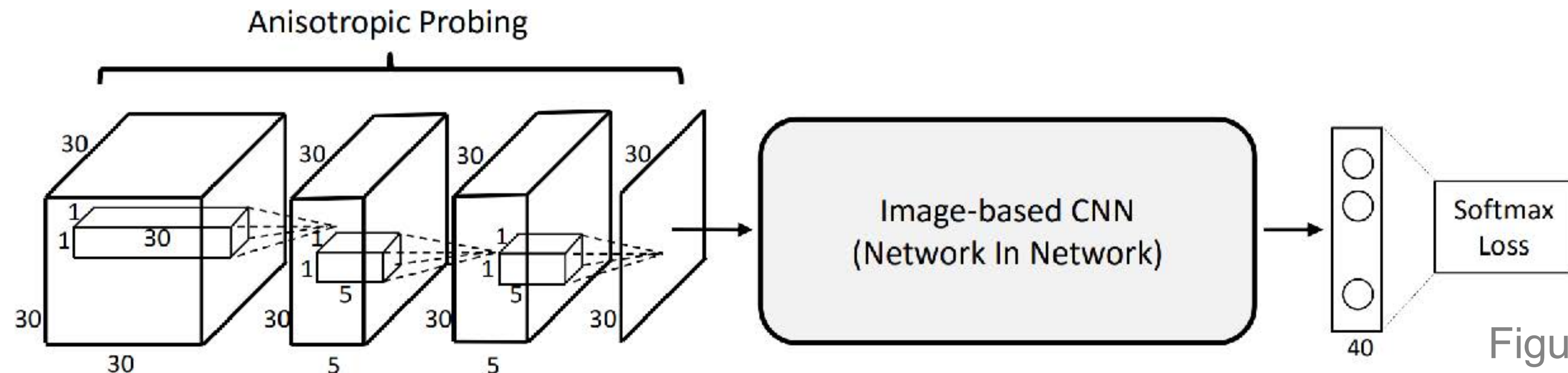


Figure from [12]

3D CNN with Anisotropic Probing kernels.

We use an elongated kernel to convolve the 3D cube and aggregate information to a 2D plane.

Then we use a 2D NIN (NIN-CIFAR10 [23]) to classify the 2D projection of the original 3D shape.

Bird's eye view 3D object detector

- Key idea: Converting the 3D learning problem to a 2D learning problem.

The work that started the KITTI 3D object detection challenge:

Multi-View 3D Object Detection Network for Autonomous Driving (2017) [13]

- Hand designed features are used to convert a 3D scene point cloud to a bird's eye image.

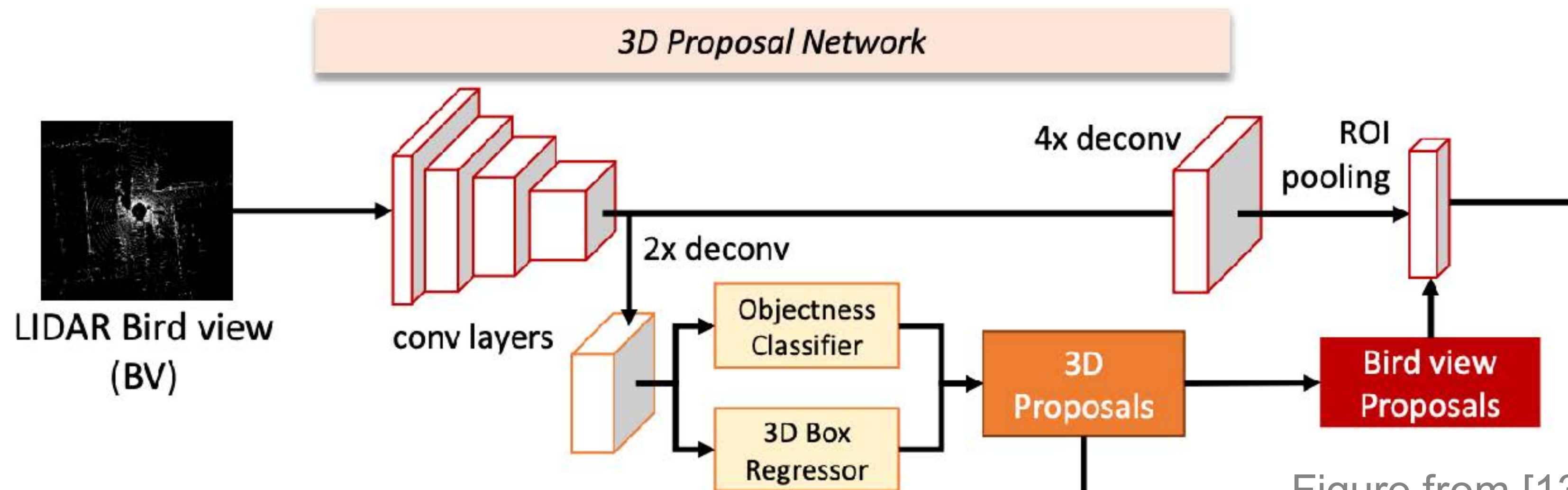


Figure from [13]

Bird's eye view 3D object detector

- From hand designed projection to data-driven projection (with PointNet like architectures).

Voxelnet: End-to-end learning for point cloud based 3d object detection (2018) [14]

Pointpillars: Fast encoders for object detection from point clouds (2019) [7]

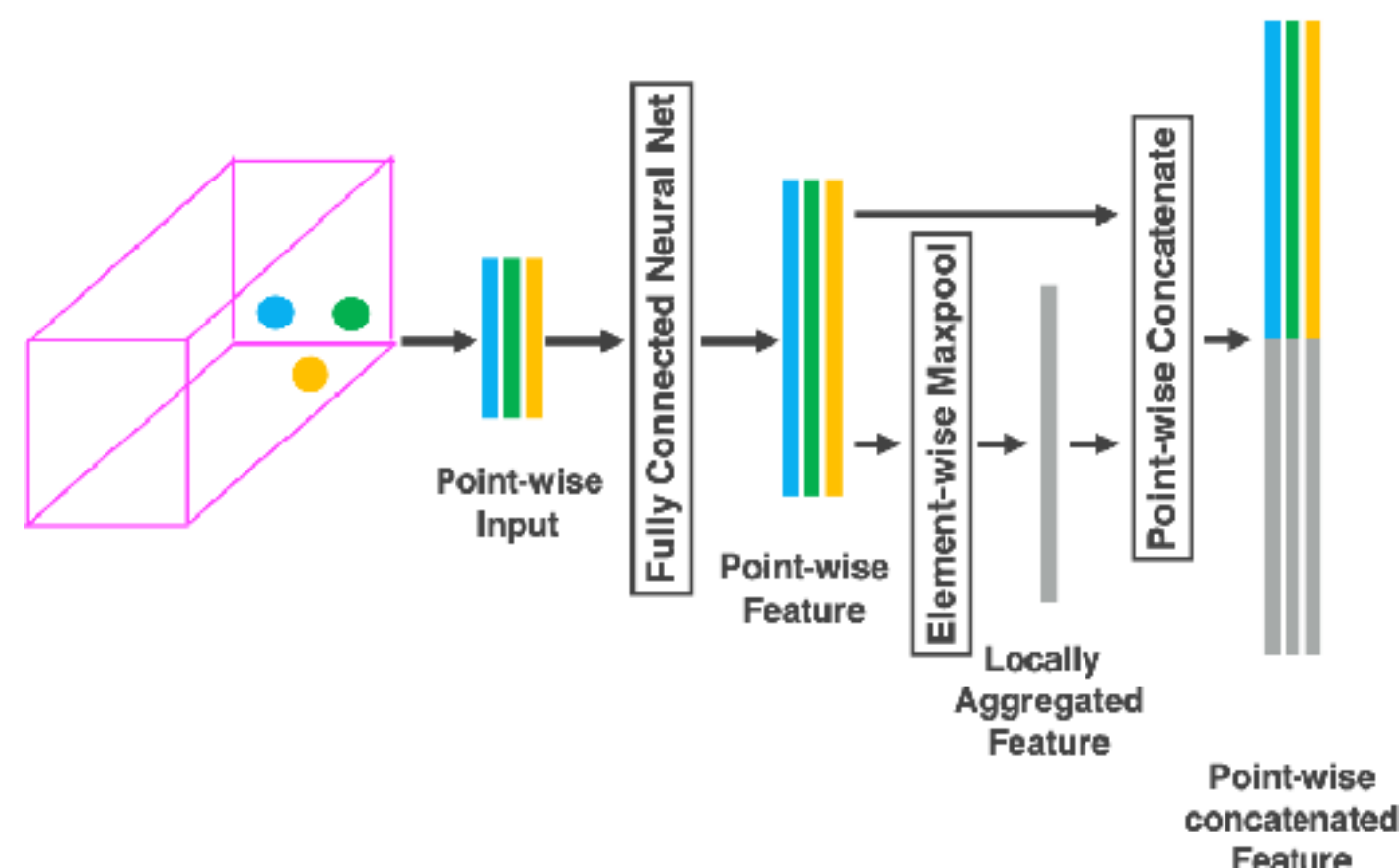


Figure from [14]

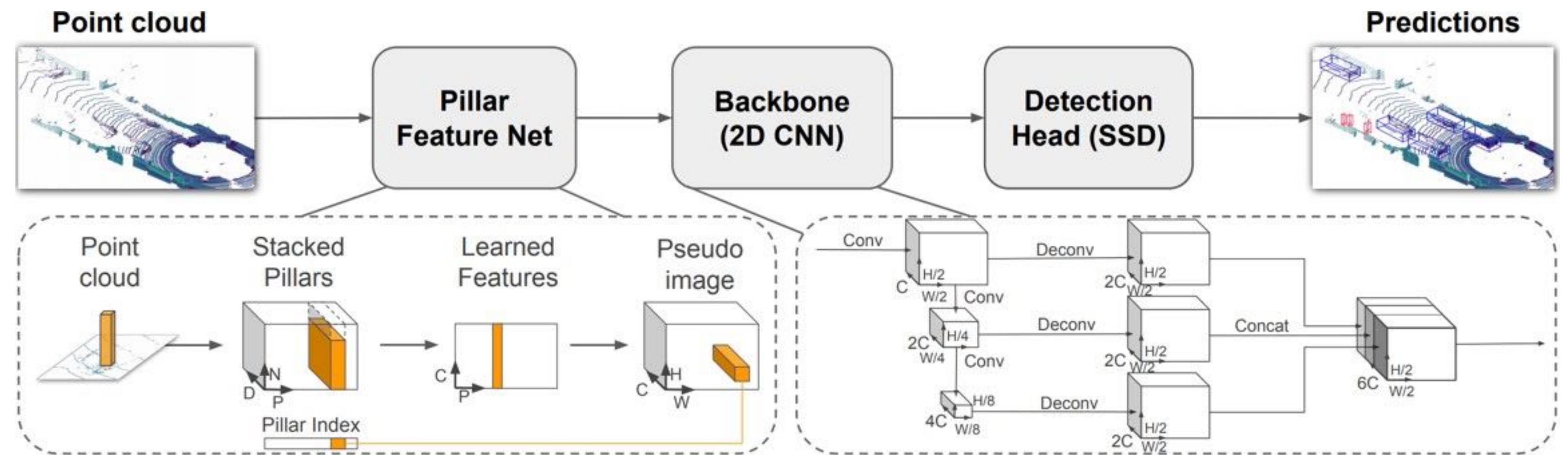
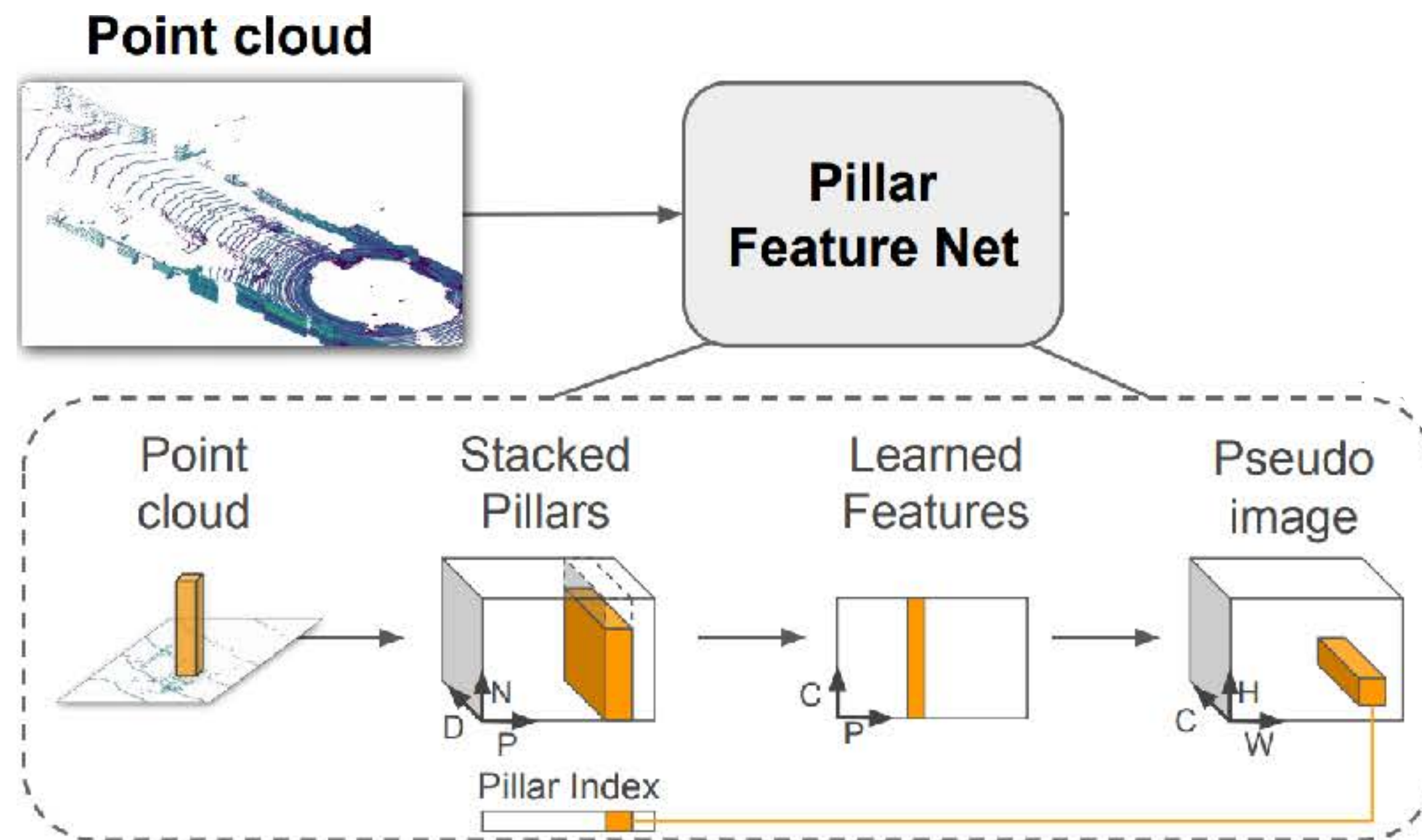


Figure from [7]

PointPillars



1. Pillar encoding:

$B \times P \times N \times D \rightarrow \text{PointNet} \rightarrow B \times P \times C$

2. Scatter the dense features to the top-down view:

Indices: $P \times 3$ (for H, W and B)

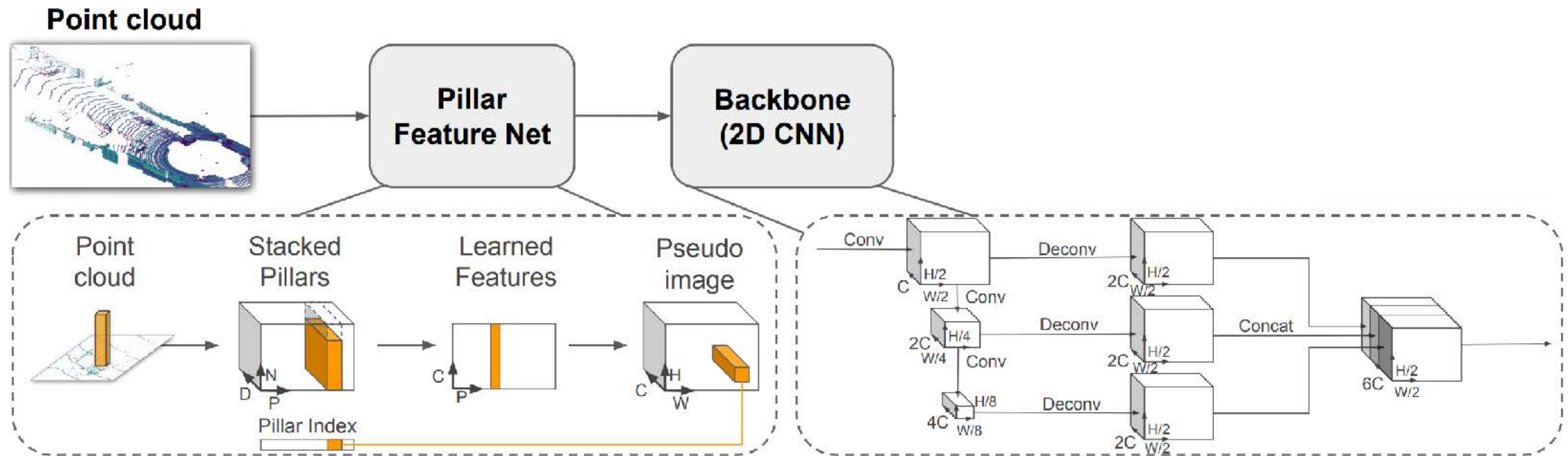
Dense pillar features: $B \times P \times C$
 $\rightarrow B \times H \times W \times C$

Handling sparsity in the top-down image (typically, >90% of the pixels are empty): B: batch size.

P: the maximum number of non-empty pillars per sample (the buffer size). N: the maximum number of points to keep per pillar (the buffer size).

D: point dimension/number of channels.

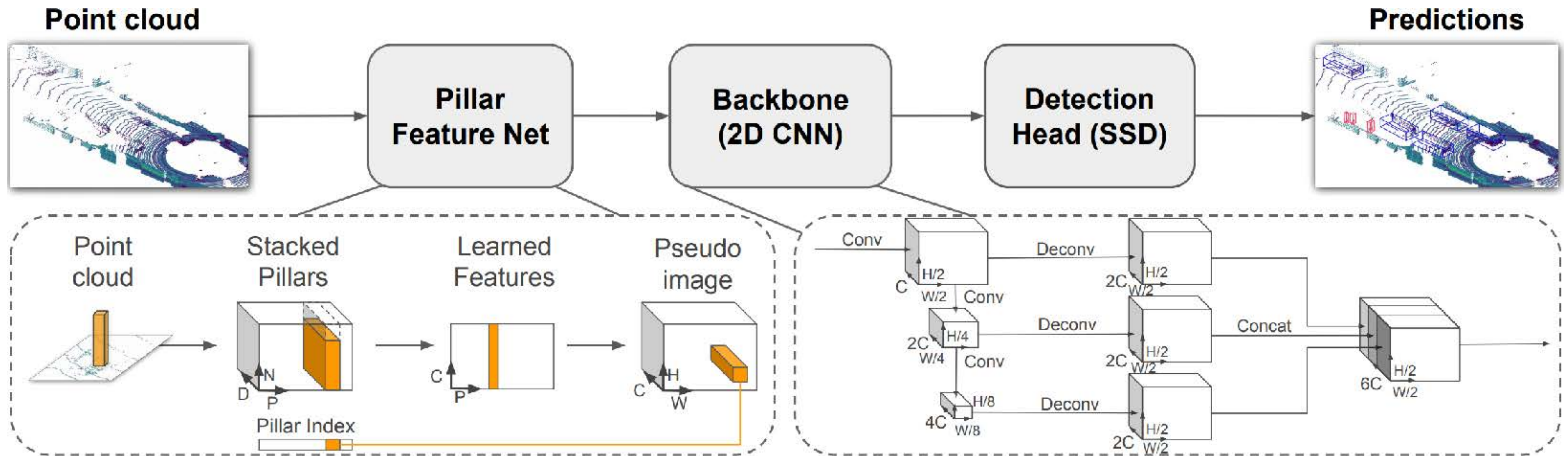
PointPillars



SSD-style backbone 2D CNN

SSD: Single-Stage Detector by Wei Liu et al.

PointPillars



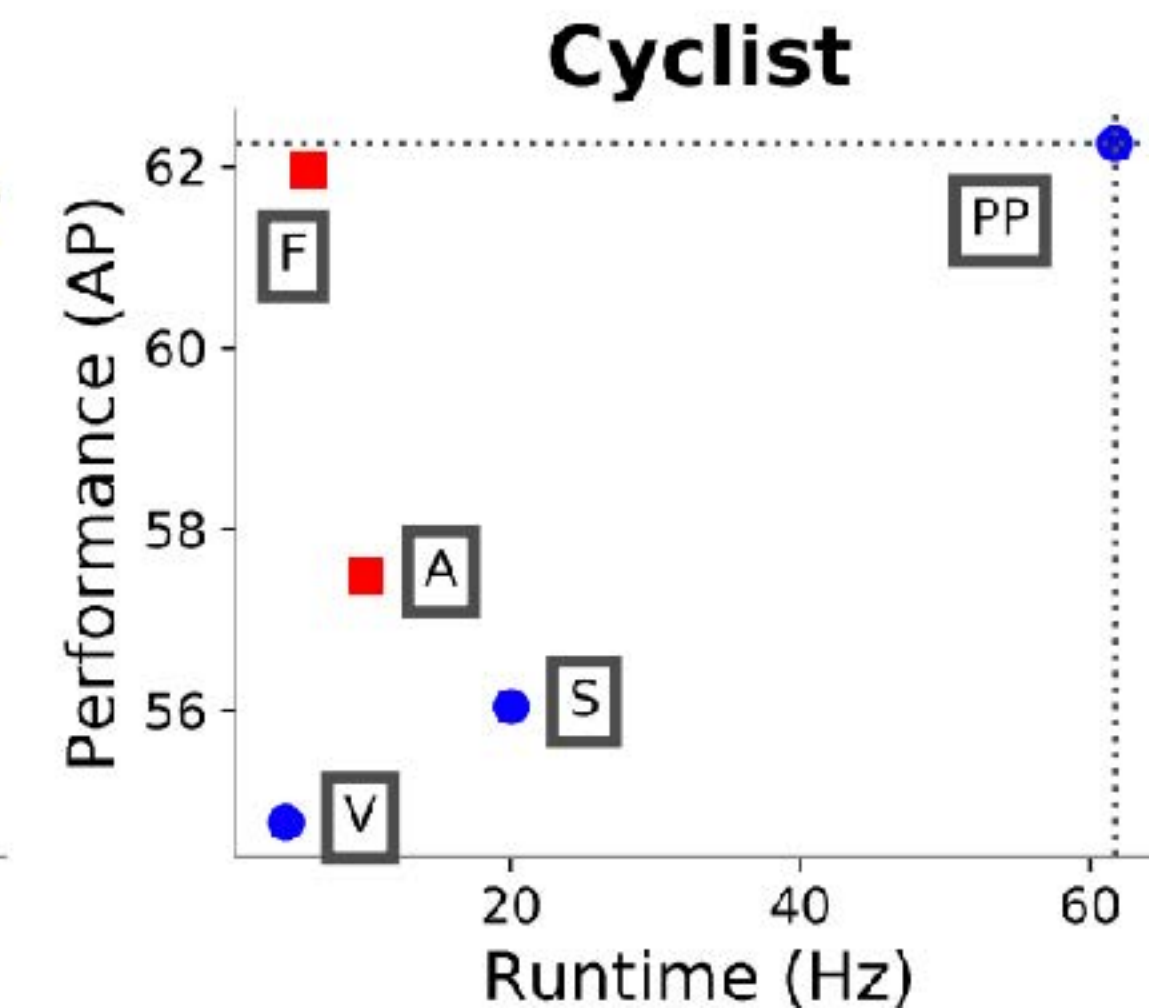
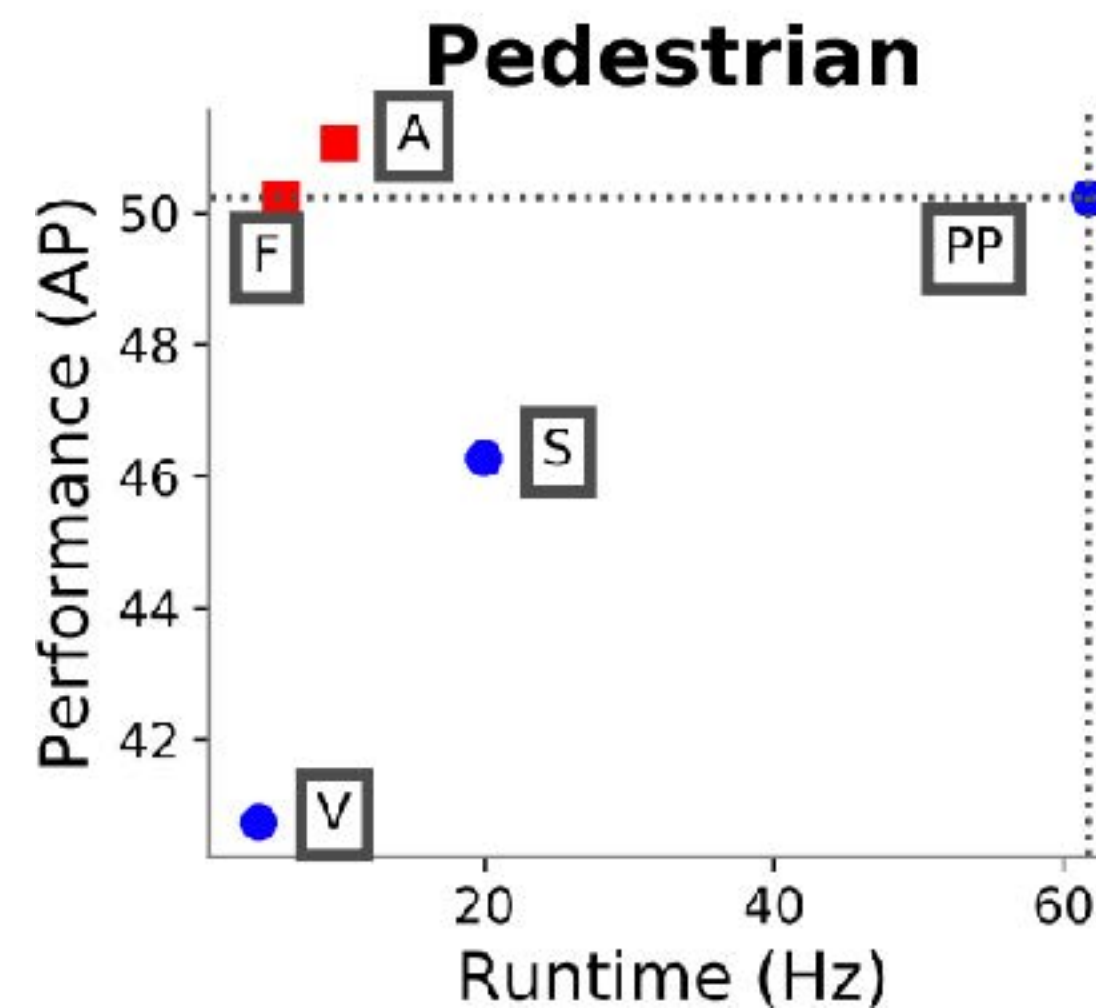
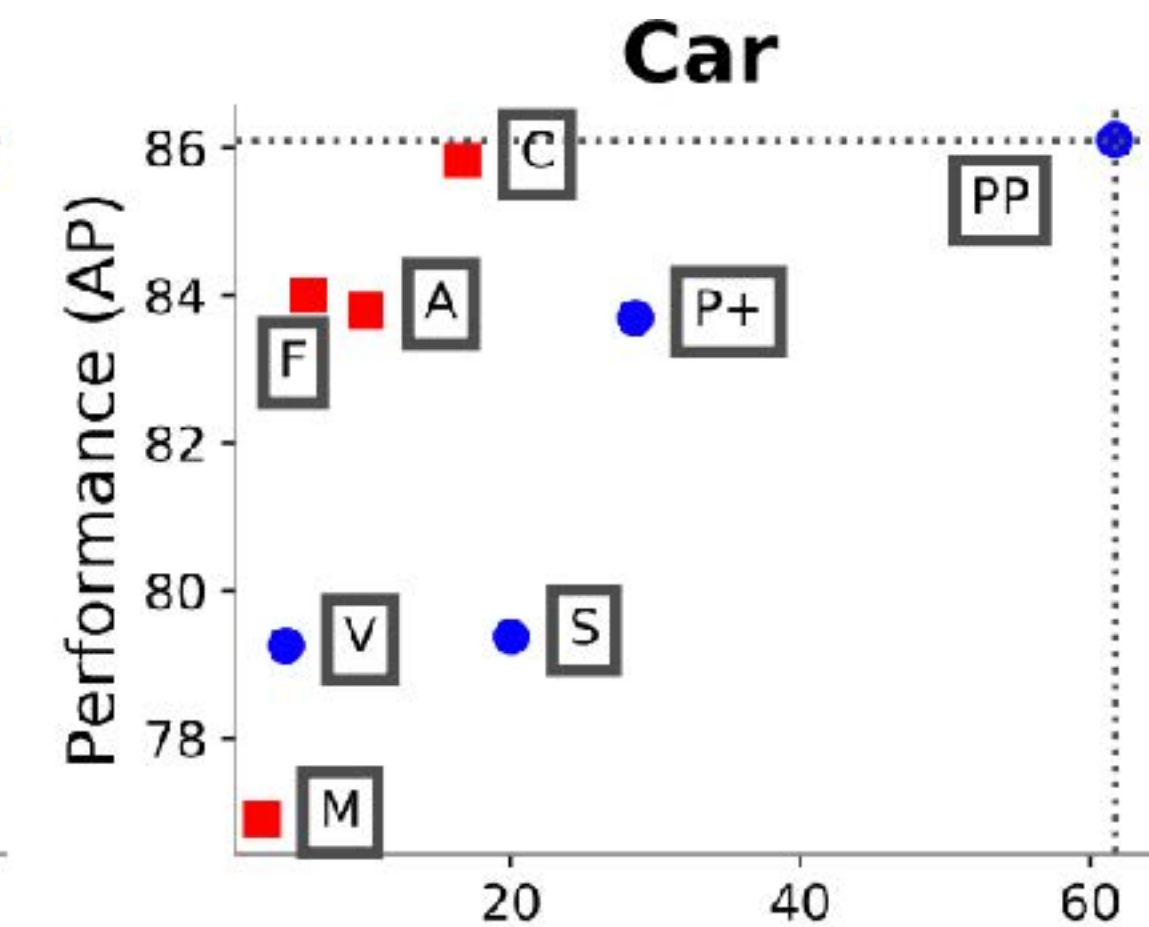
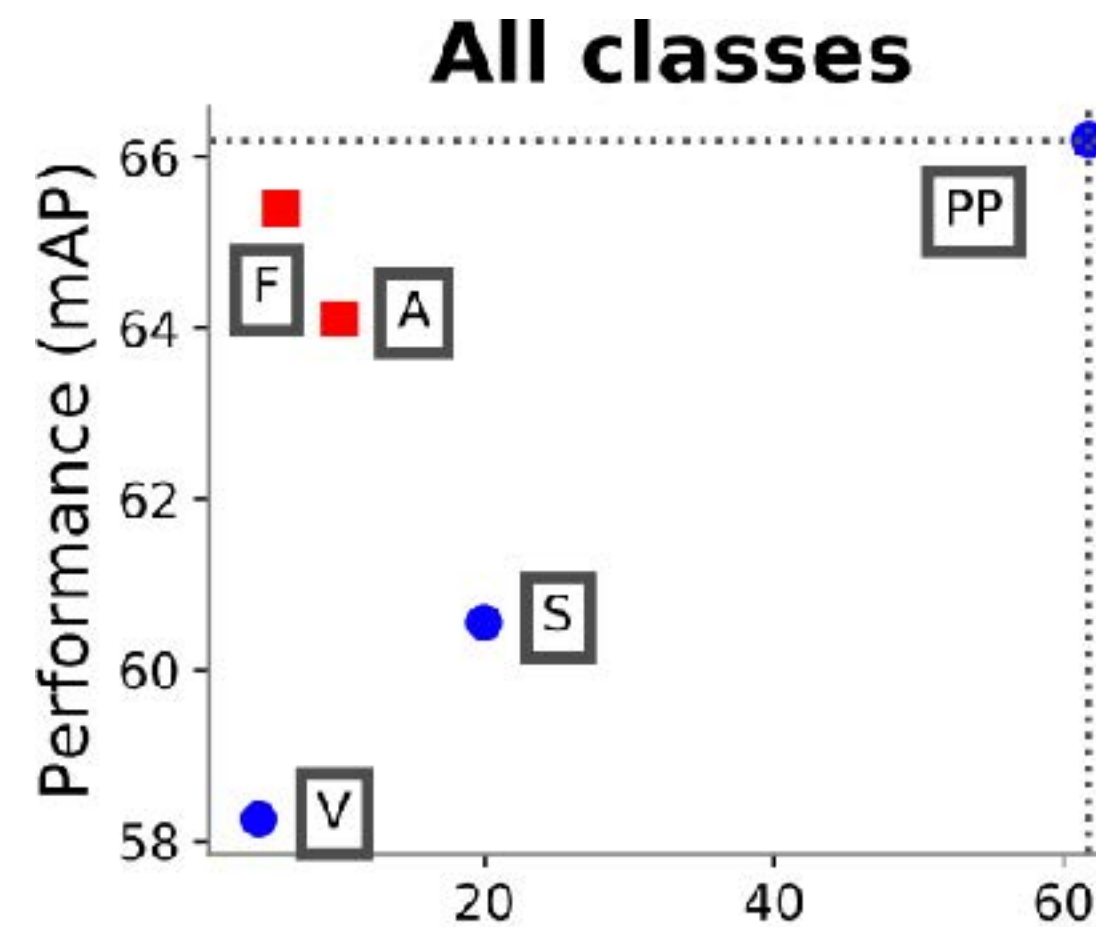
PointPillars

The biggest advantages:

- Inference speed.
- Simplicity.

Weaknesses:

- Assumption of a projection plane (not generalizable to more complex 3d scenes).
- Aggressive compression of the dimension.



Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ ***VoteNet***
- 3D Instance Segmentation
 - Top-Down Approaches
 - Bottom-Up Approaches

Point cloud based 3D object detectors

- Key idea: Use sparsity aware backbone architectures (e.g. PointNet++, Sparse 3D convnet) and design 3D detection frameworks that leverage sparsity.

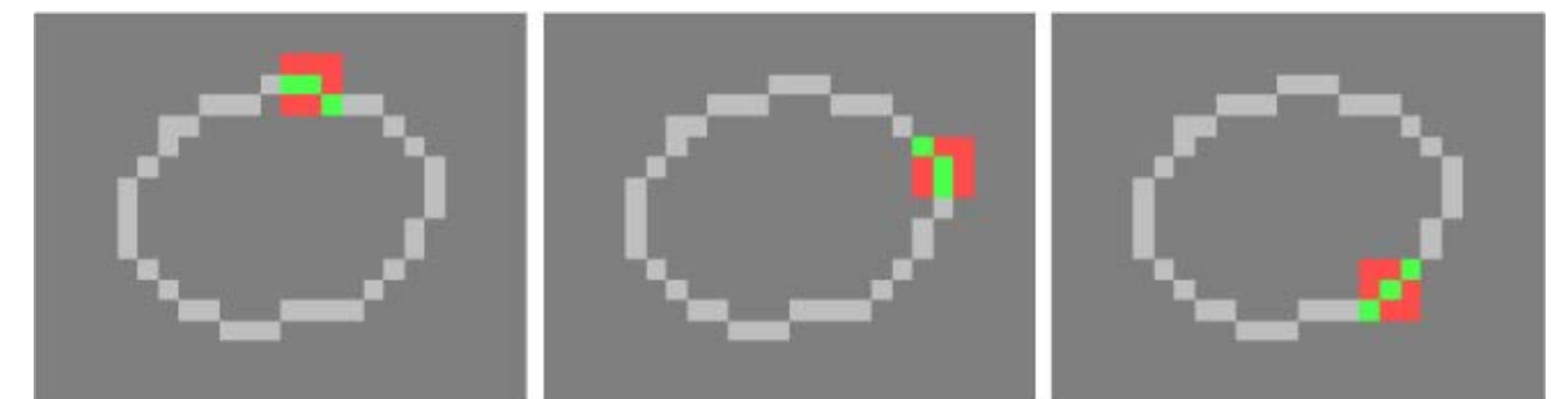
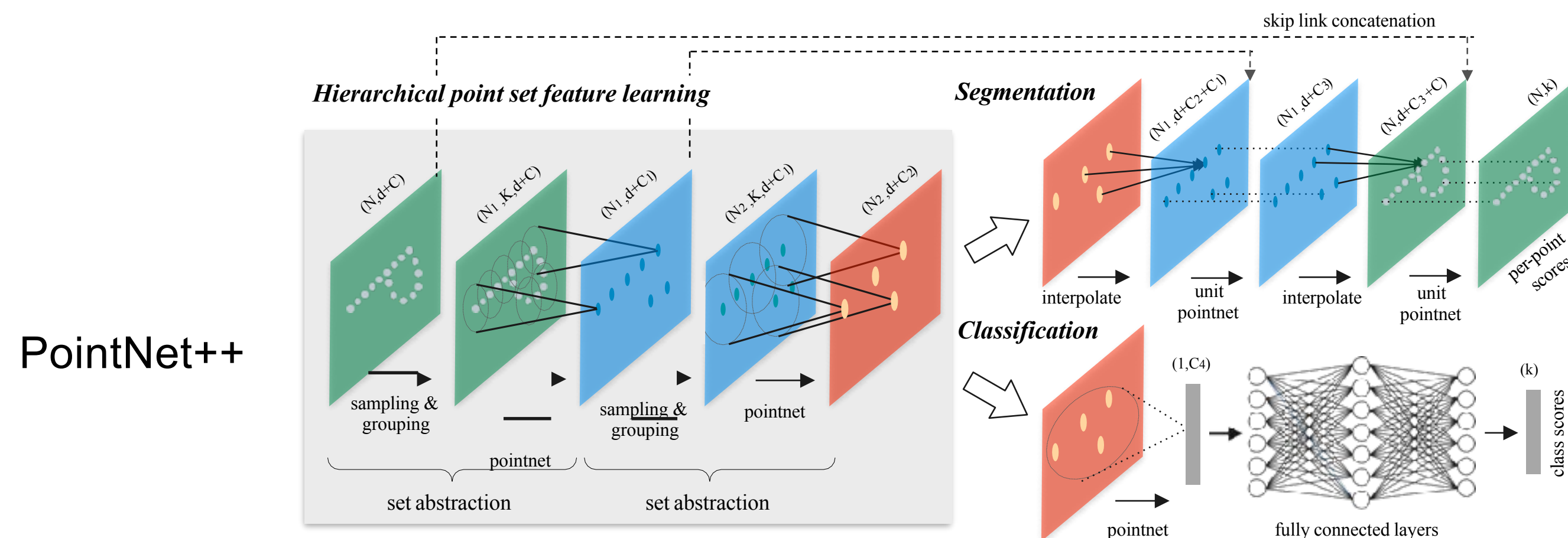
Deep Hough Voting for 3D Object Detection in Point Clouds (2019) [8]

PointRCNN: 3D Object Proposal Generation and Detection from Point Cloud (2019) [15]

STD: Sparse-to-Dense 3D Object Detector for Point Cloud (2019) [16]

3DSSD: Point-based 3D Single Stage Object Detector (2020) [17]

Pv-rcnn: Point-voxel feature set abstraction for 3d object detection (2020) [18]



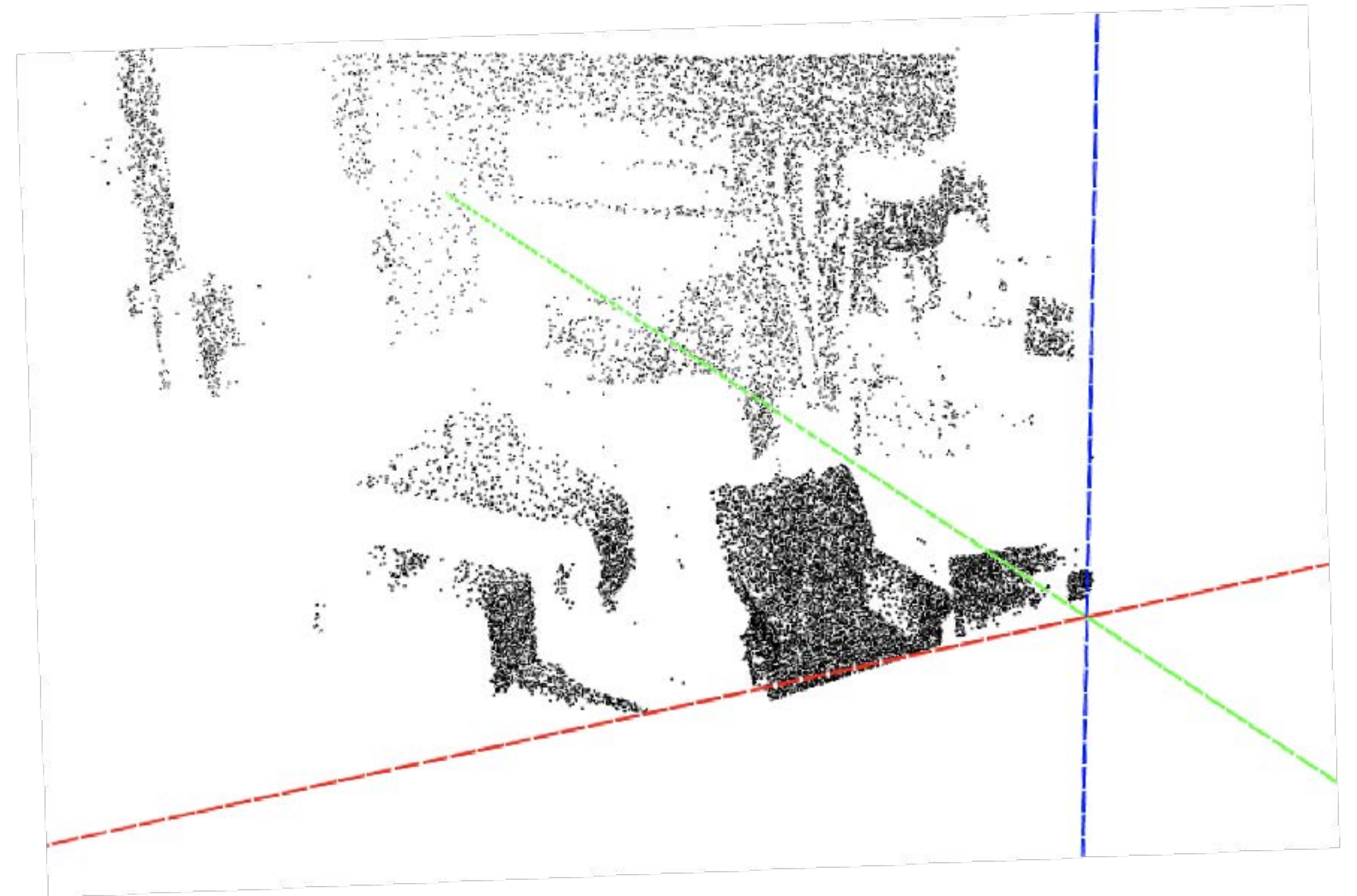
Sub manifold sparse 3d conv

Observation: 2D v.s. 3D

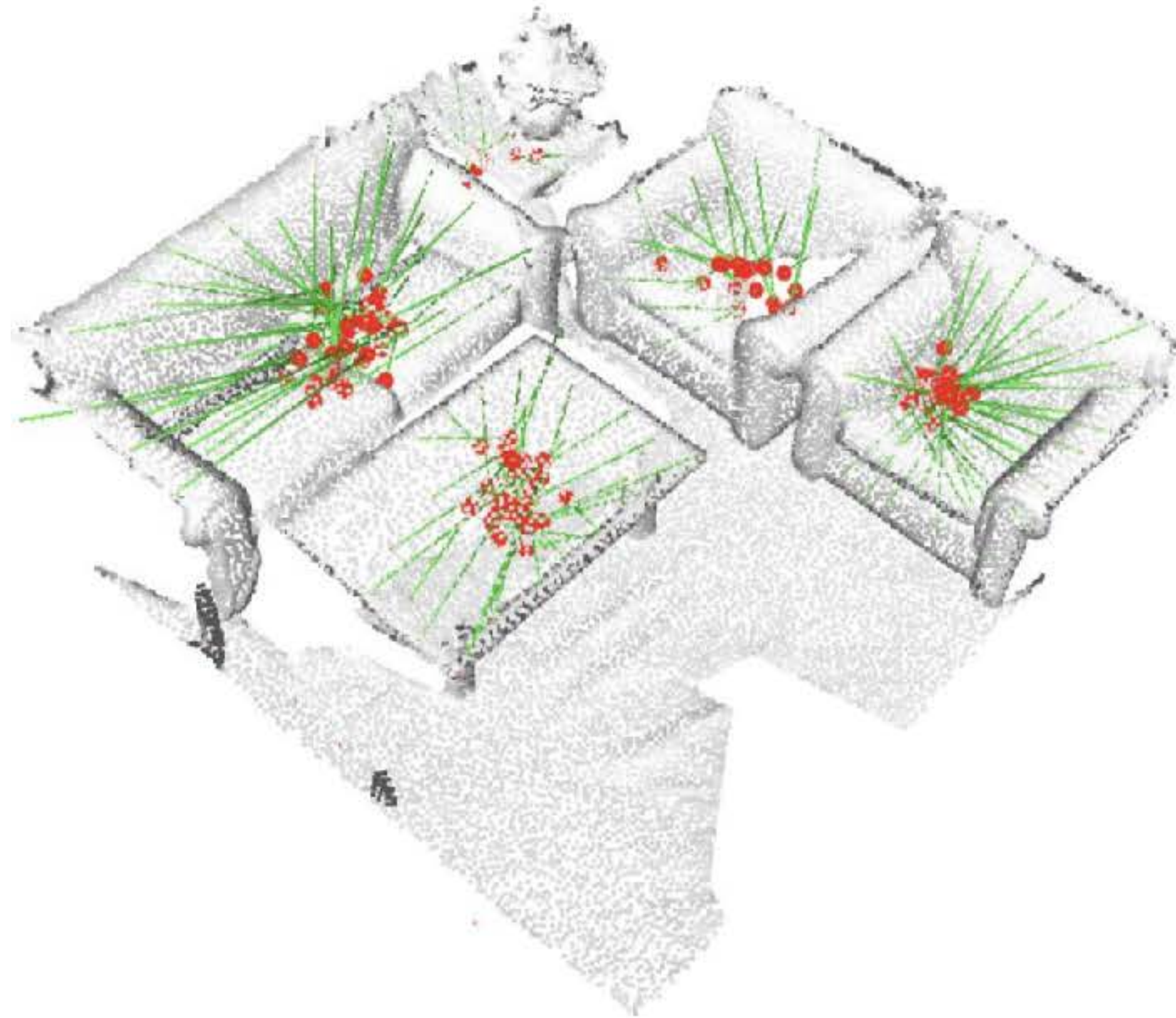
Dense 2D pixel array



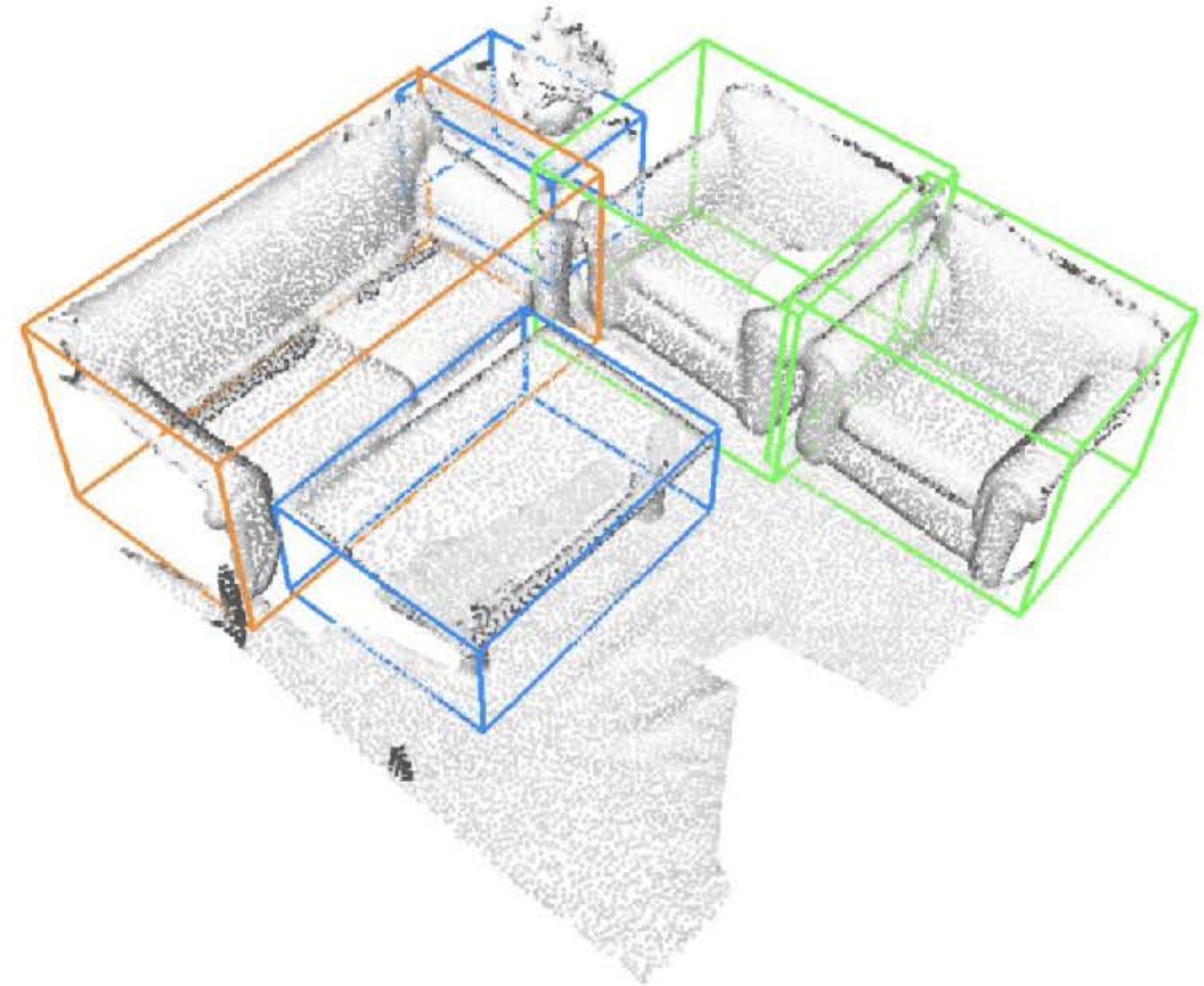
**Sparse 3D points
(only on object surfaces)**



Our solution: Voting



Voting from surface points



Detected 3D bounding boxes

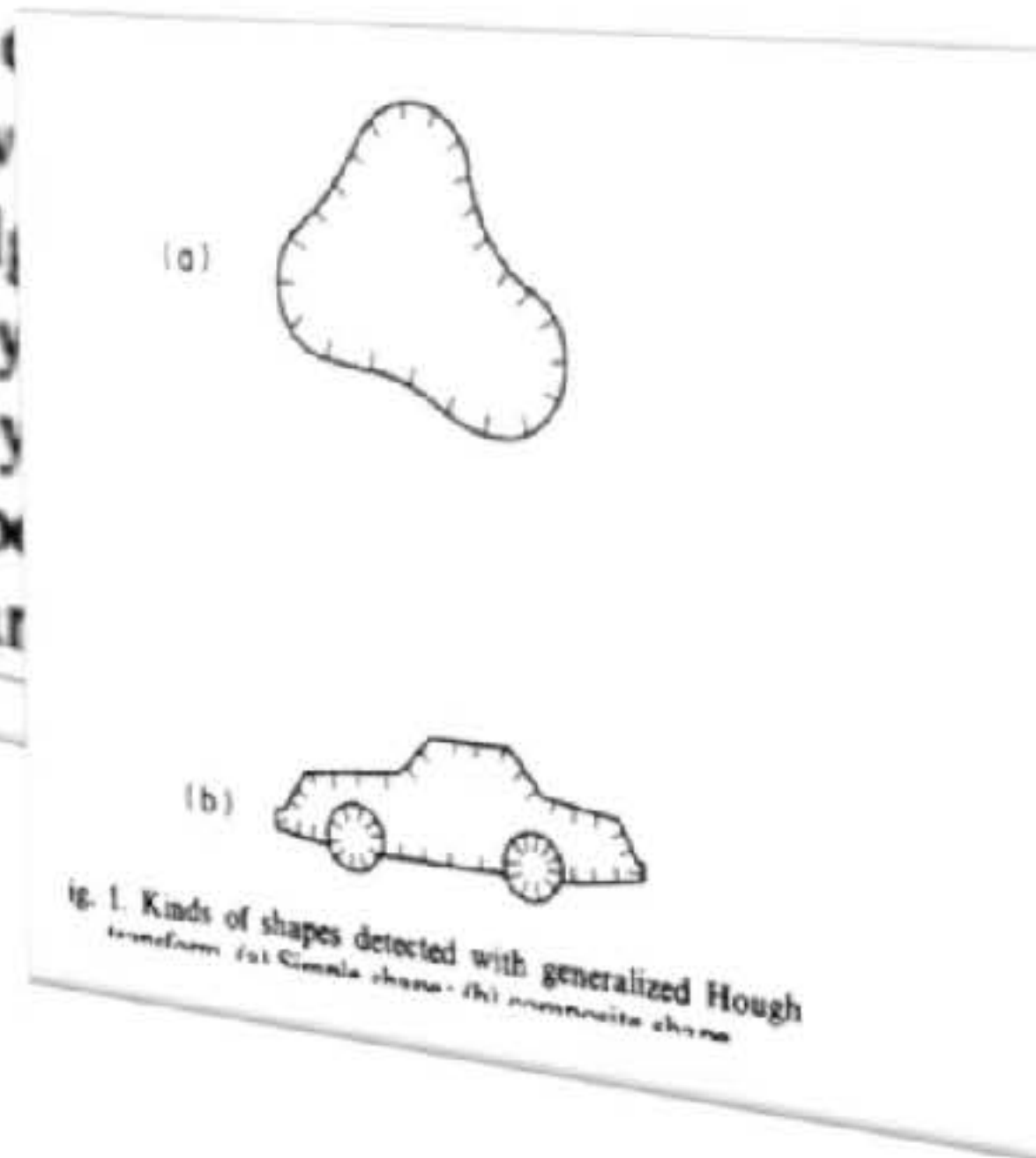
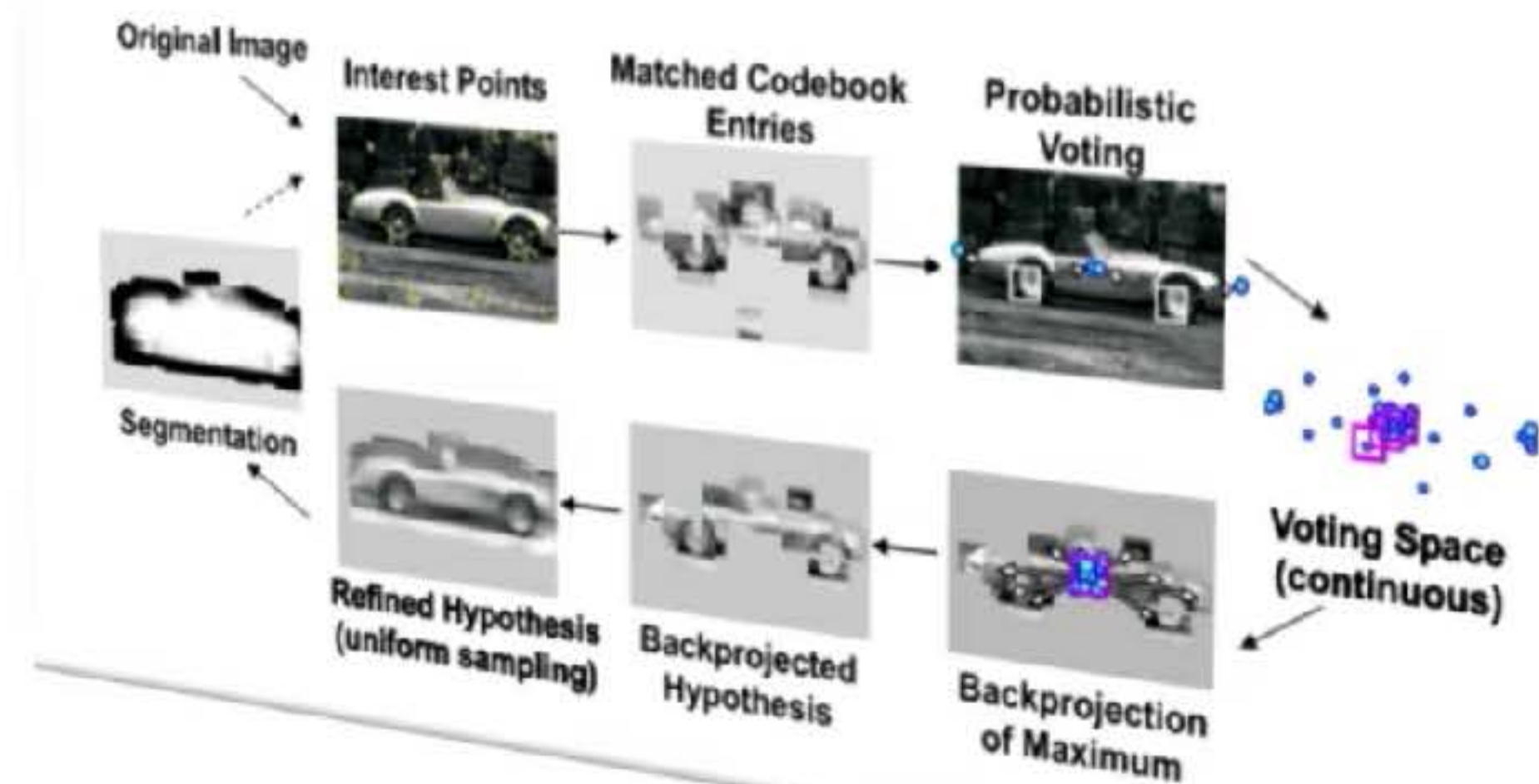
GENERALIZING THE HOUGH TRANSFORM TO DETECT ARBITRARY SHAPES*

D. H. BALLARD

Computer Science Department, University of Rochester, Rochester, NY 14627, U.S.A.

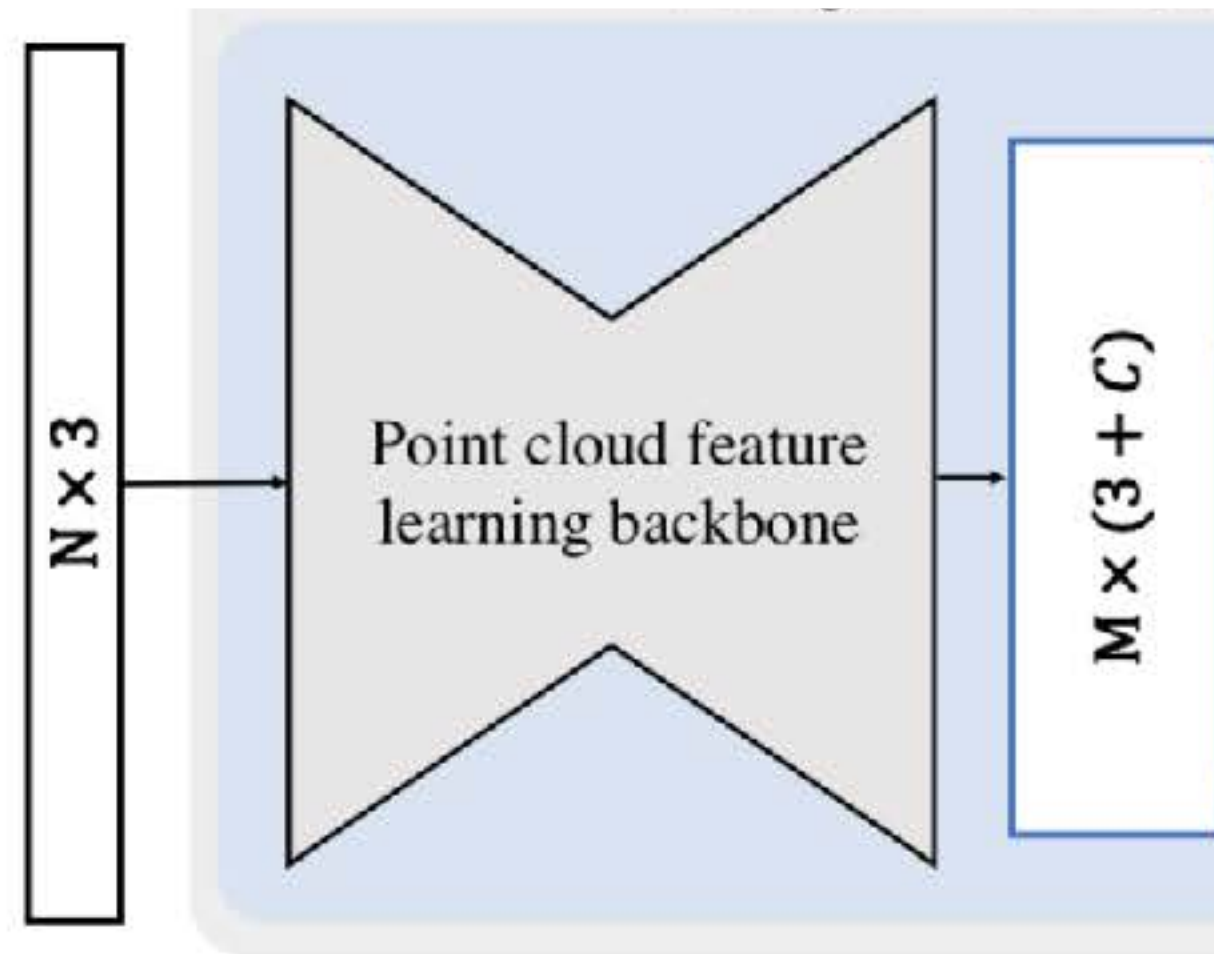
(Received 10 October 1979; in revised form 9 September 1980; received for publication 23 September 1980)

Abstract—The Hough transform is a method for detecting curves by a curve and parameters of that curve.



points on
 $s^{(1,2)}$ and
ralized to
as. ⁽⁶⁾ The
ons. ^(7,8,9)
mapping
es of that

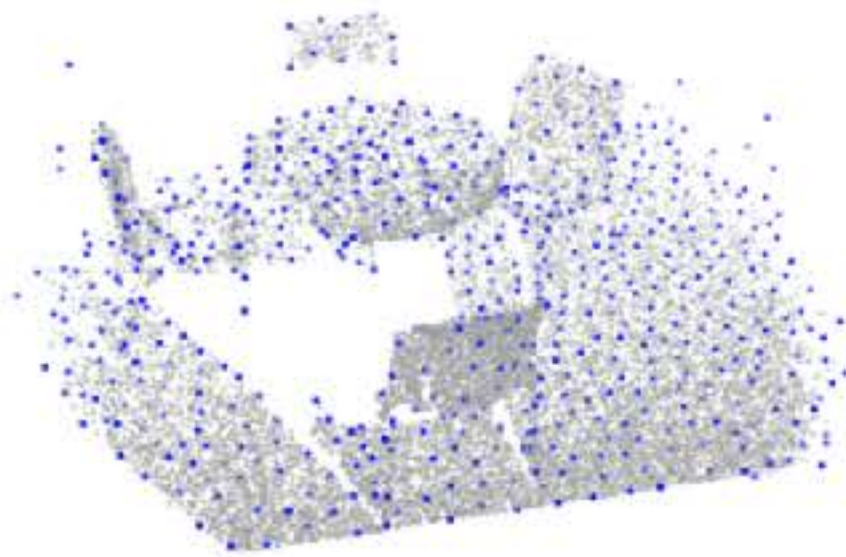
Deep Hough voting: Detection pipeline



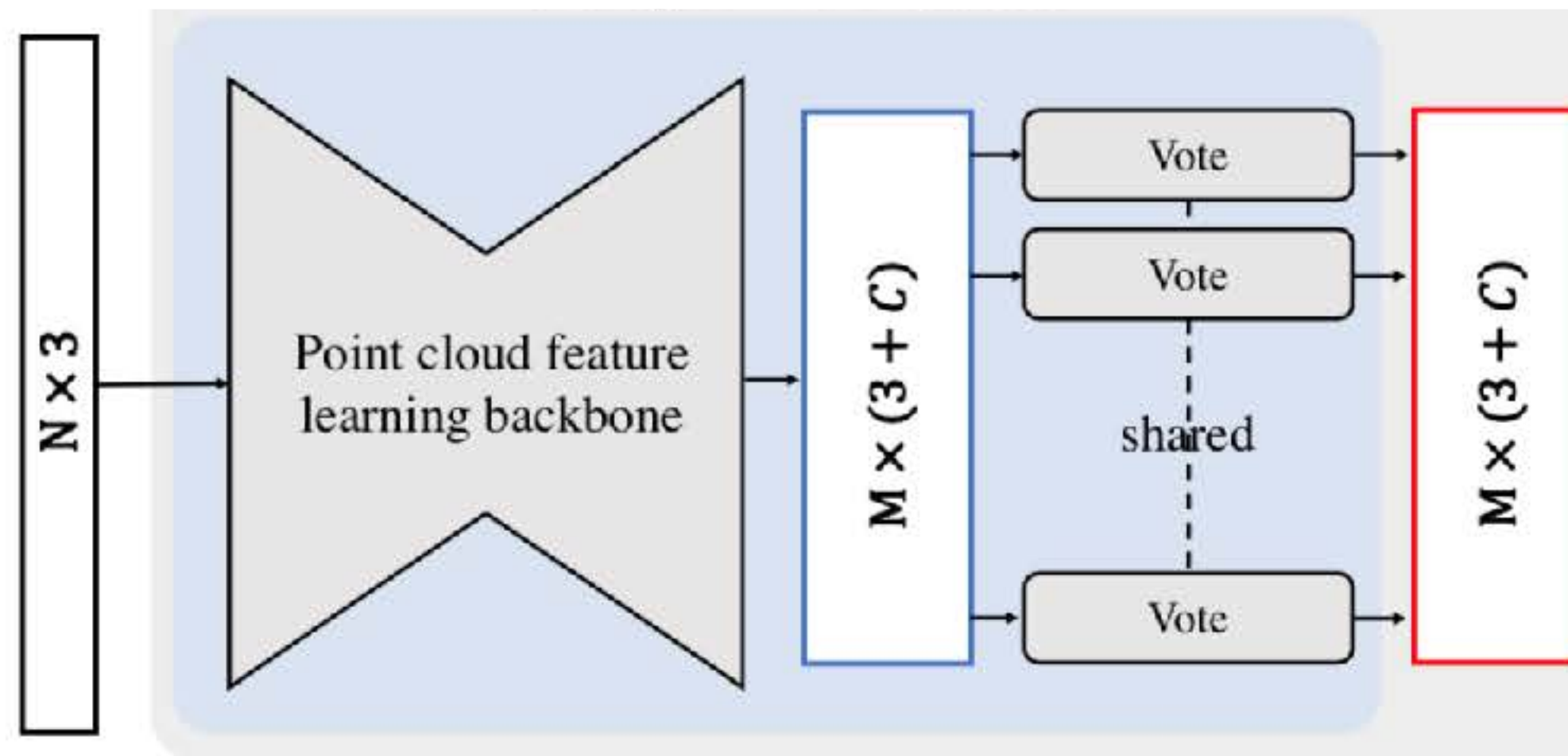
Input:
point cloud



Seeds
(XYZ + feature)



Deep Hough voting: Detection pipeline

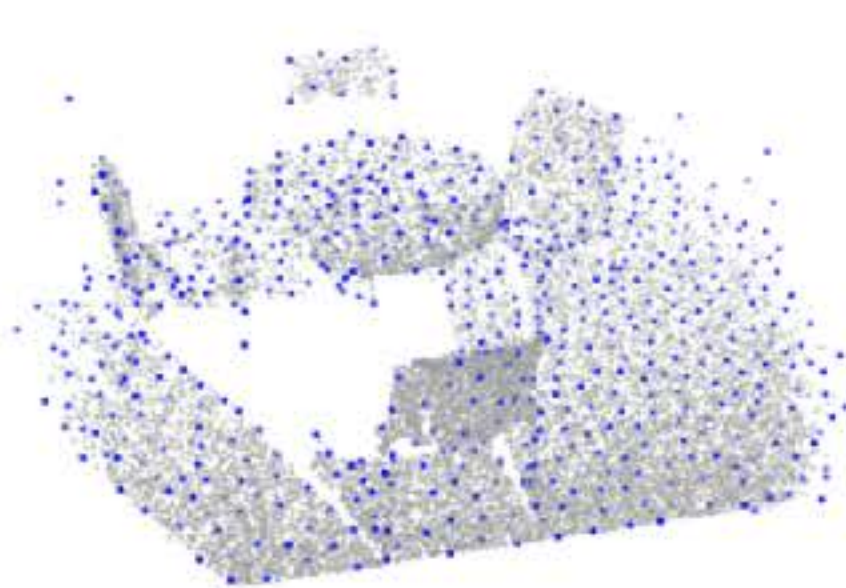


Explicit supervision for the votes (the XYZ translations)

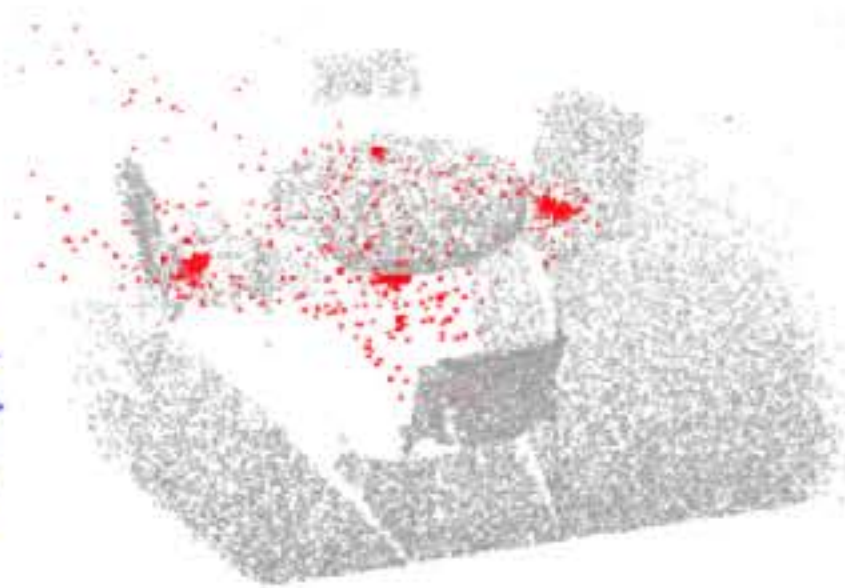
Input:
point cloud



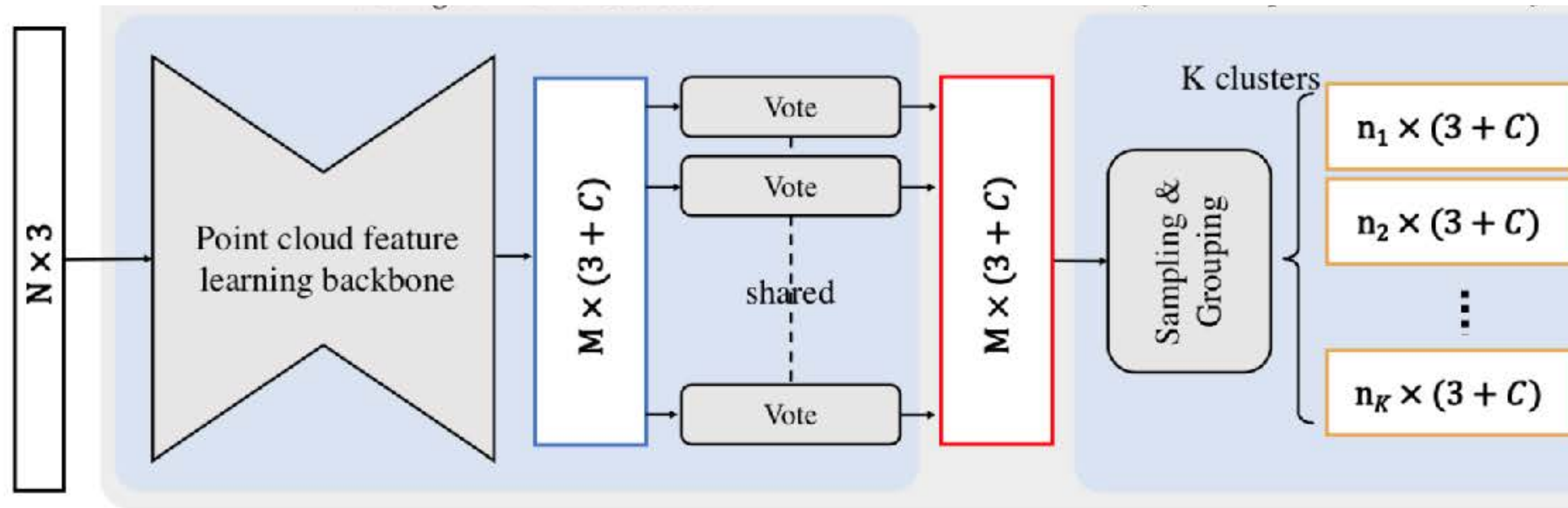
Seeds
(XYZ + feature)



Votes
(XYZ + feature)



Deep Hough voting: Detection pipeline



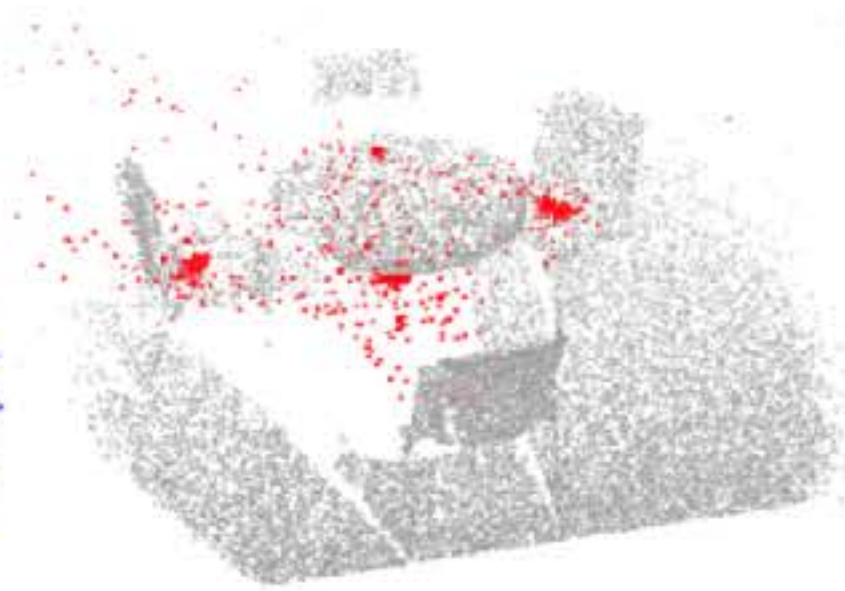
Input:
point cloud



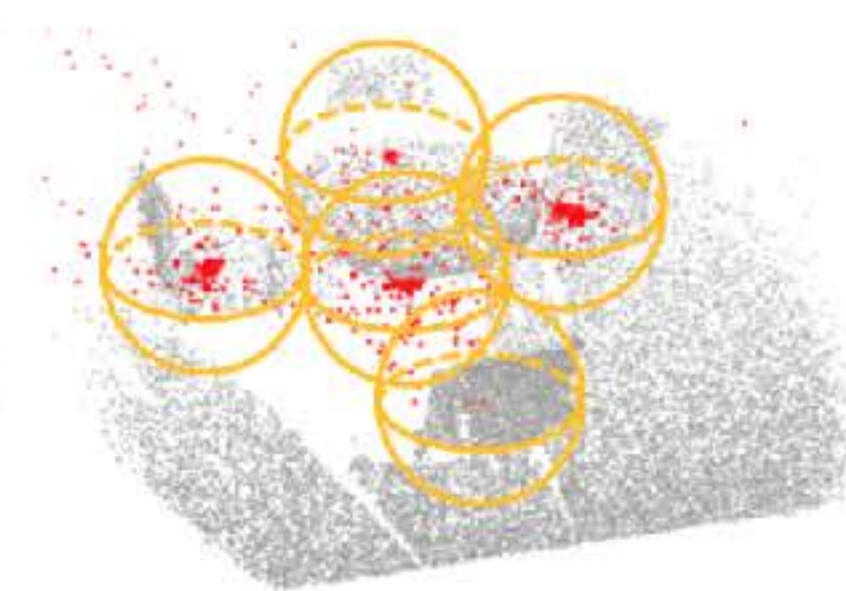
Seeds
(XYZ + feature)



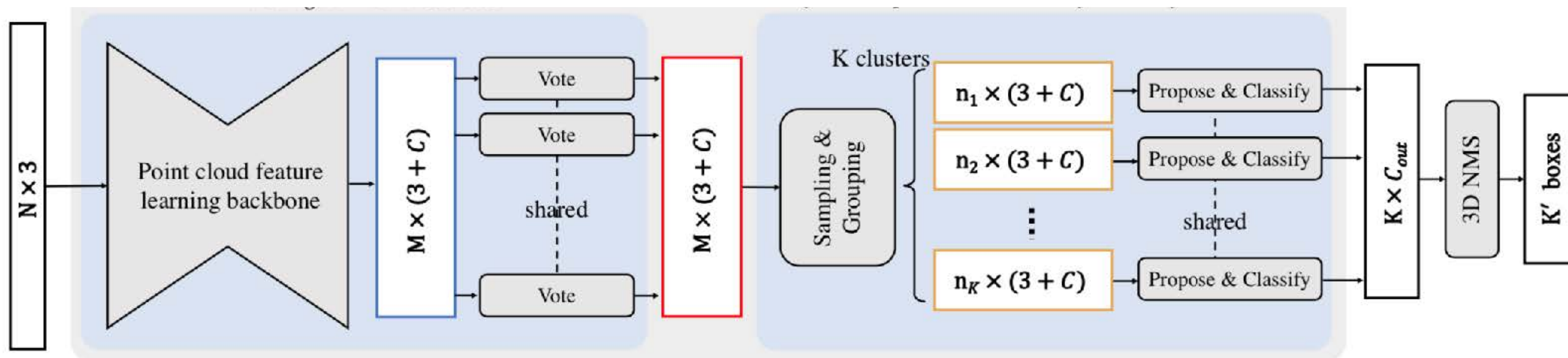
Votes
(XYZ + feature)



Vote clusters



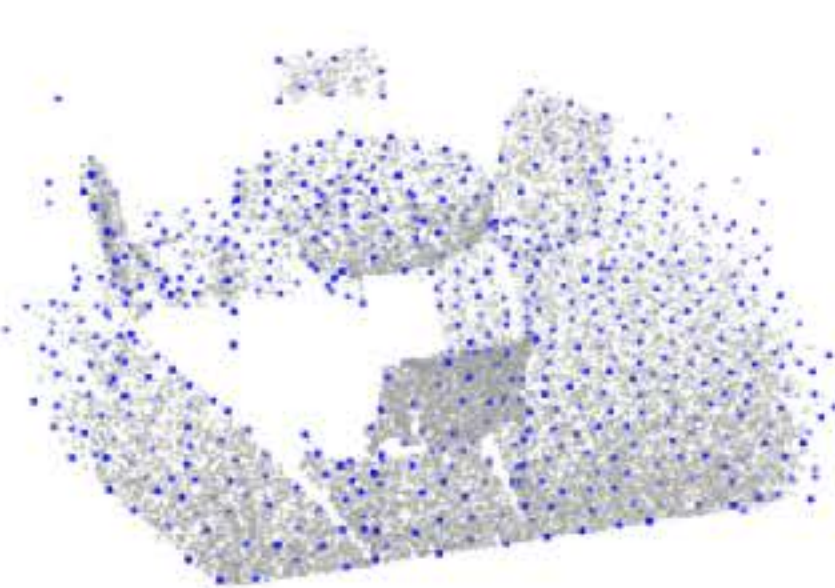
Deep Hough voting: Detection pipeline



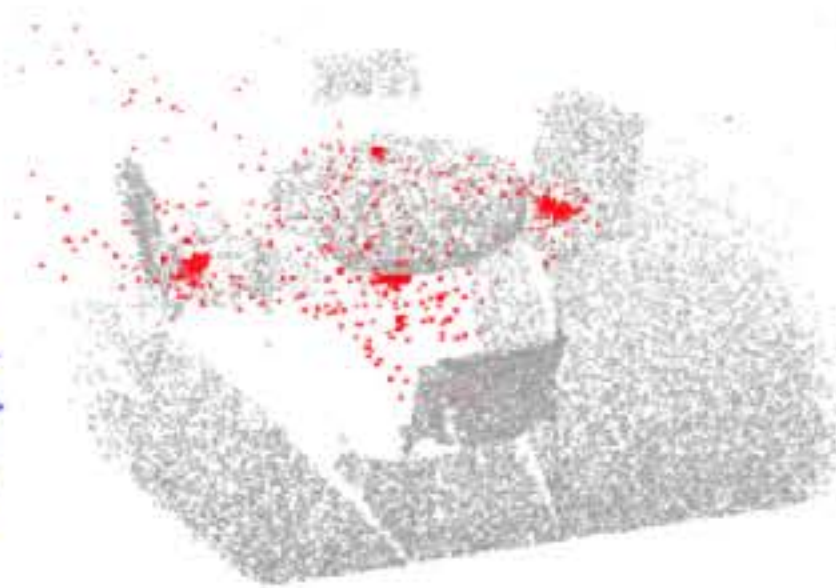
Input:
point cloud



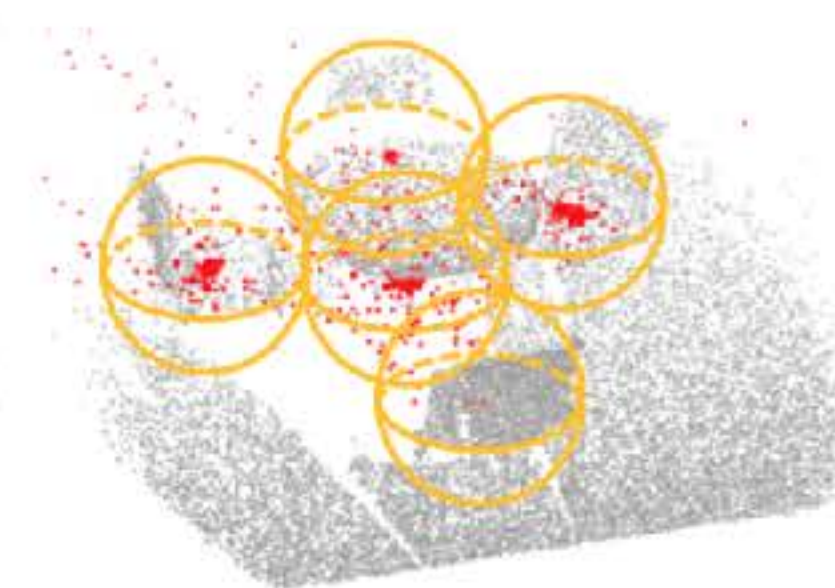
Seeds
(XYZ + feature)



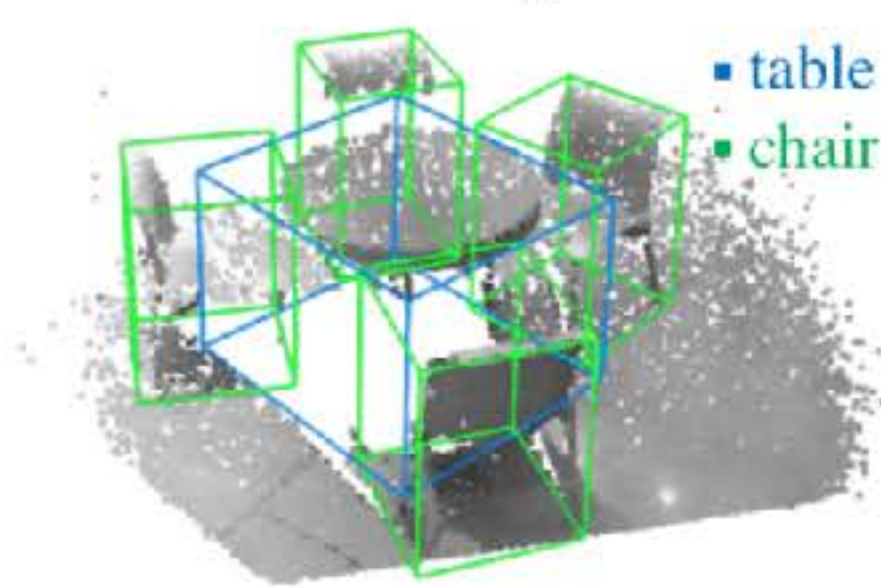
Votes
(XYZ + feature)



Vote clusters

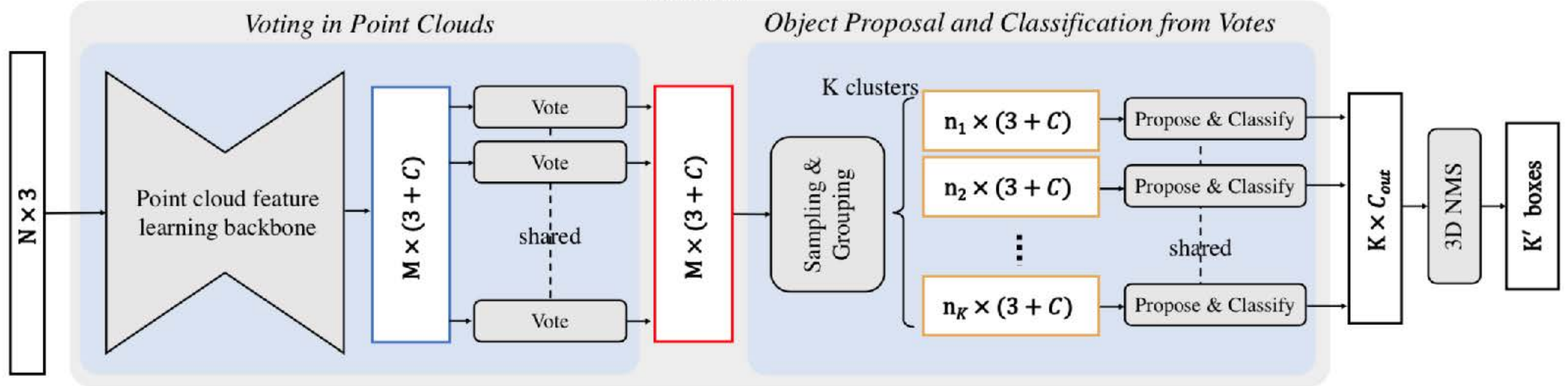


Output:
3D bounding boxes



Deep Hough voting: Detection pipeline

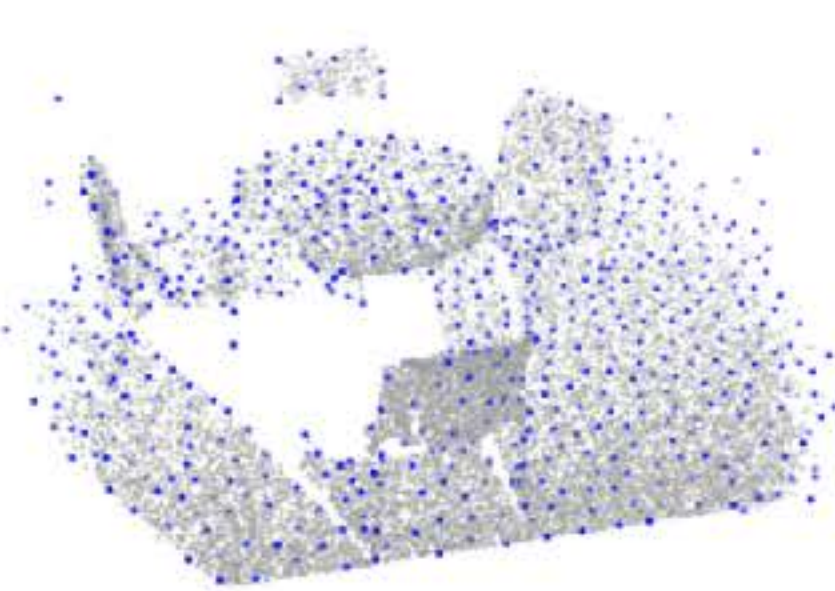
VoteNet



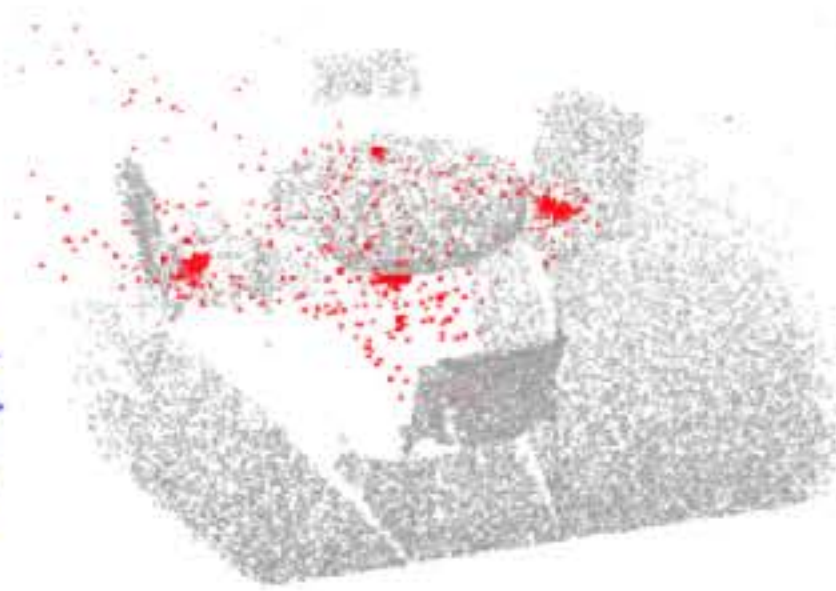
Input:
point cloud



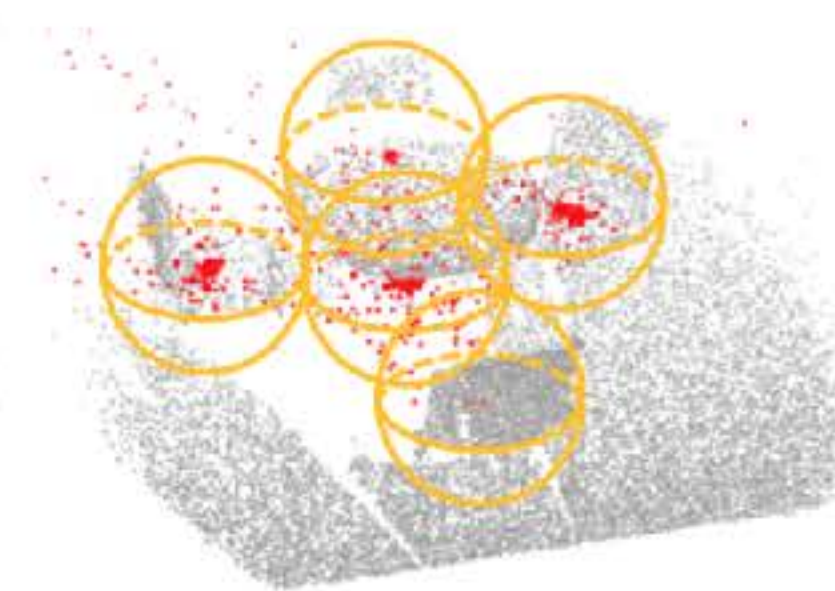
Seeds
(XYZ + feature)



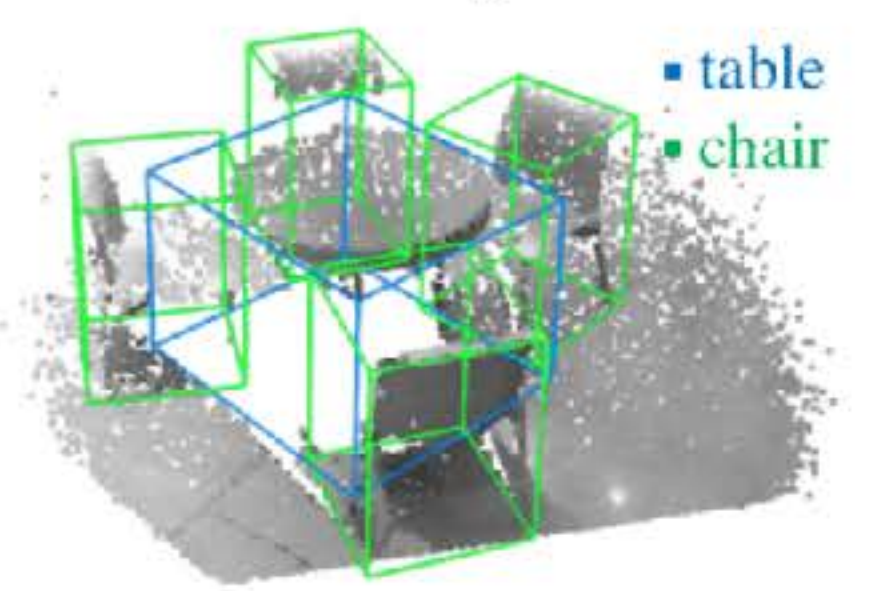
Votes
(XYZ + feature)



Vote clusters



Output:
3D bounding boxes

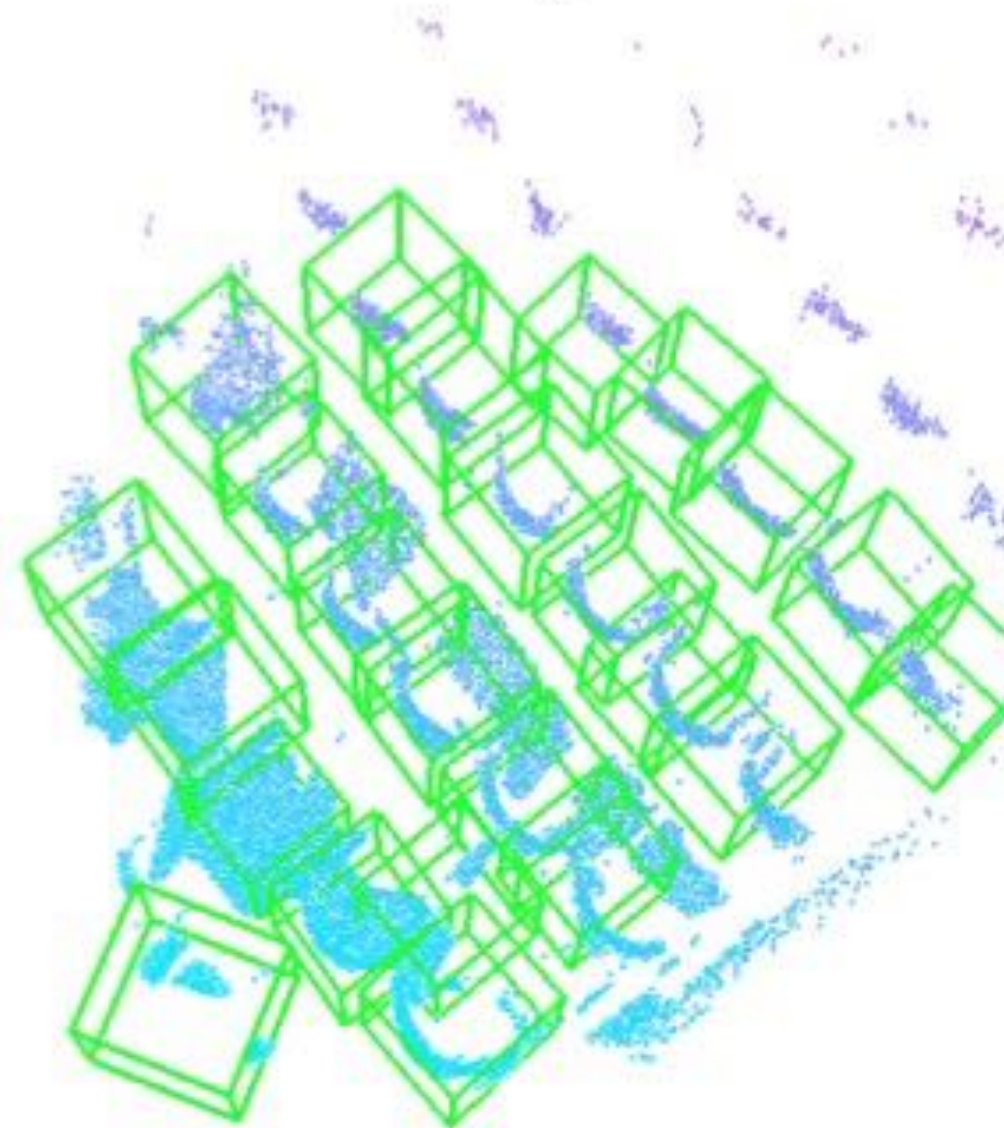


Results: SUN RGB-D (single depth images)

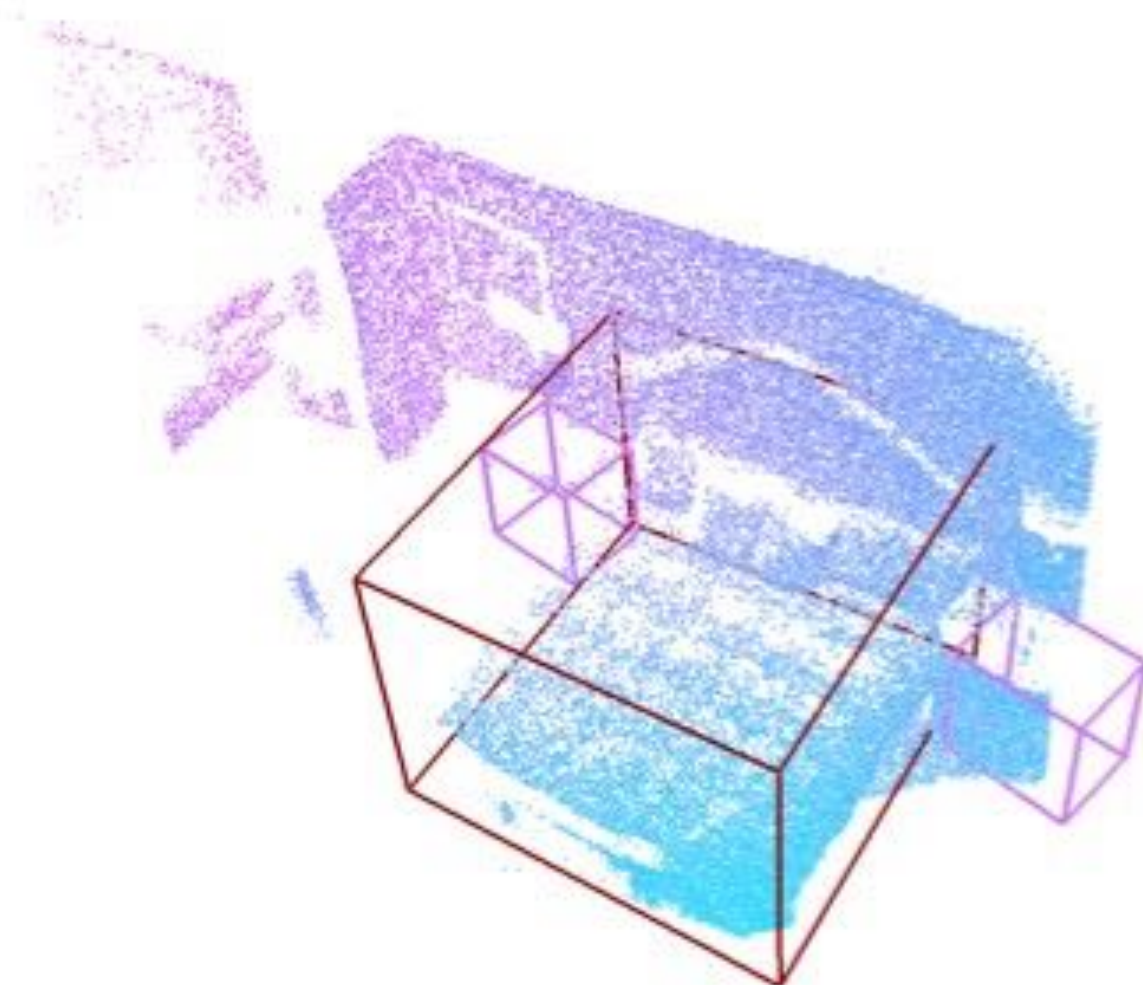
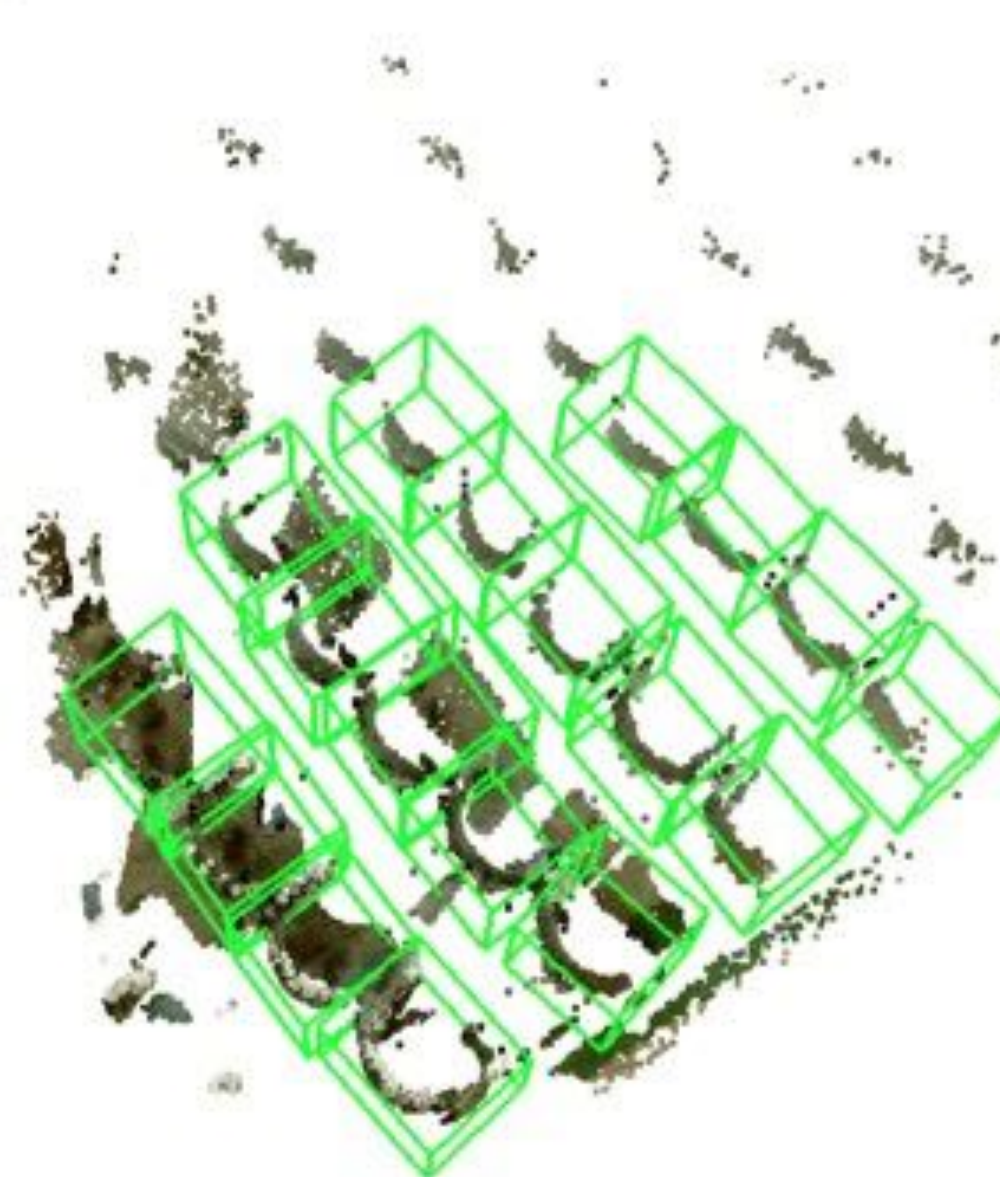
Image of the scene



VoteNet prediction

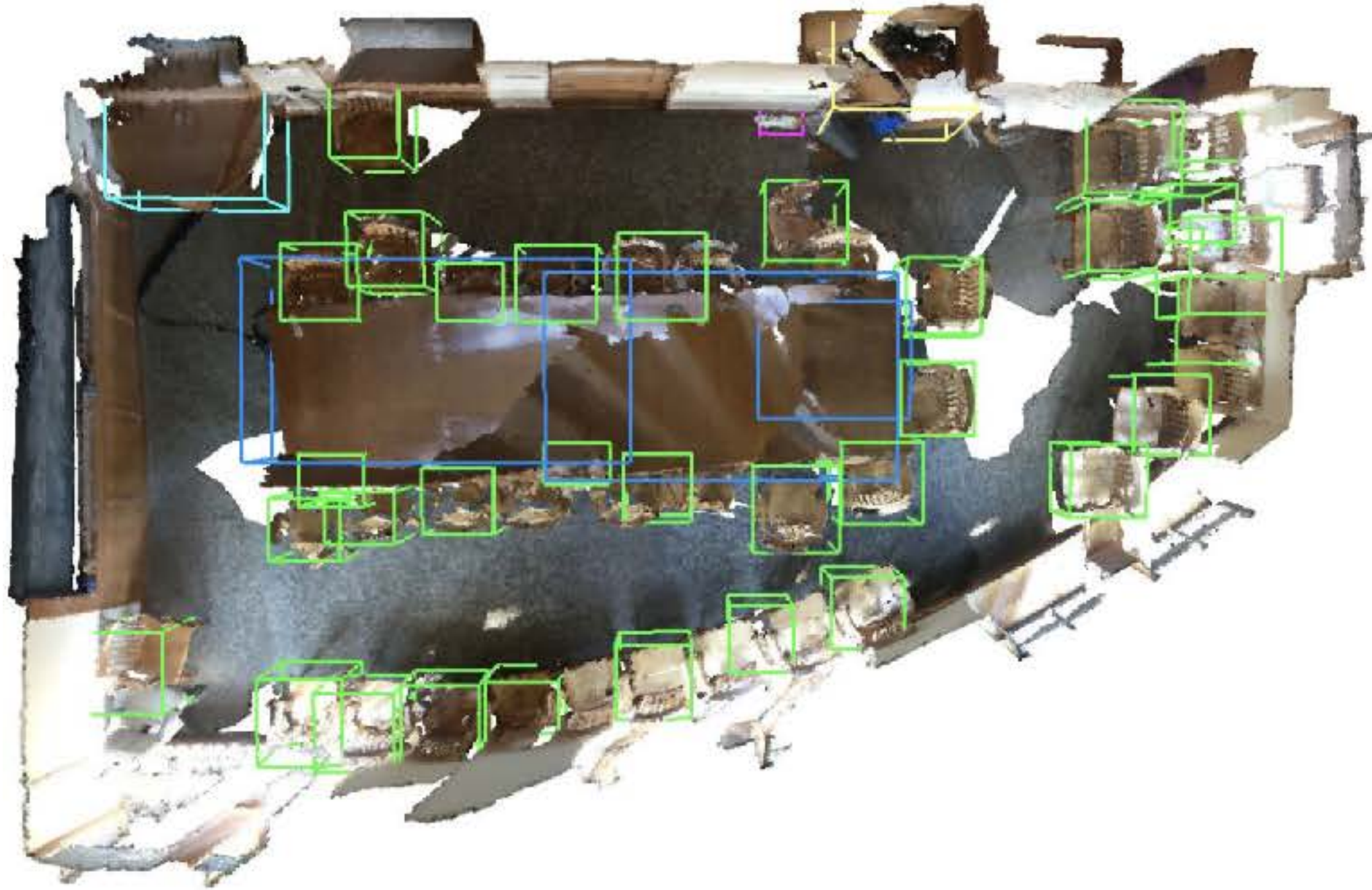


Ground truth

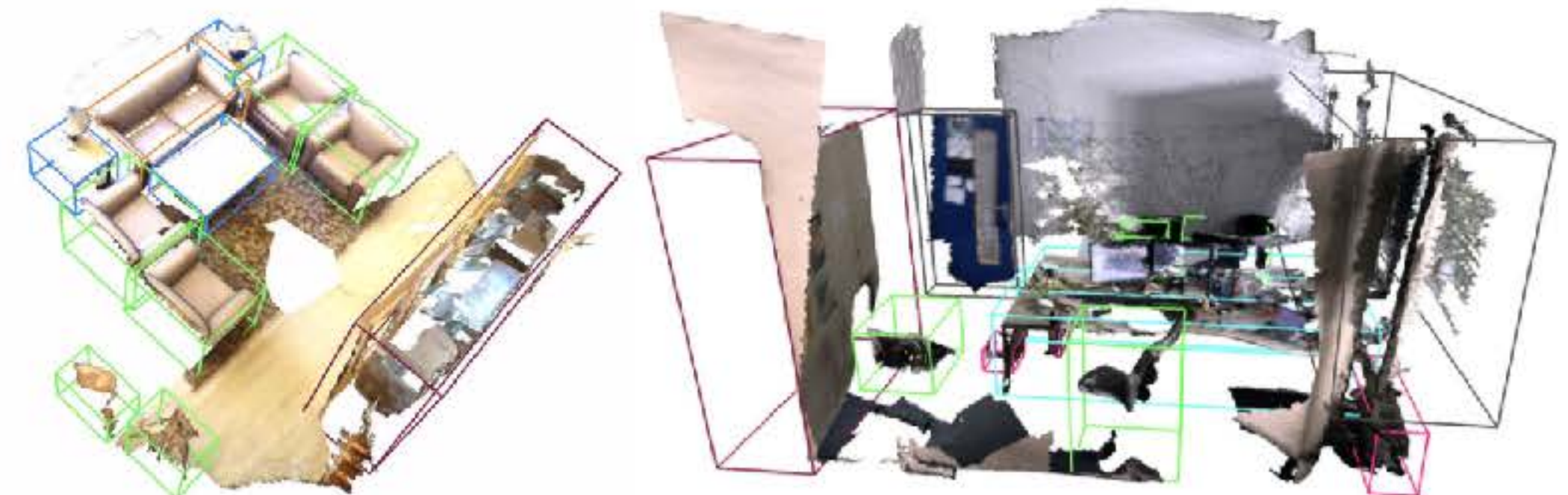
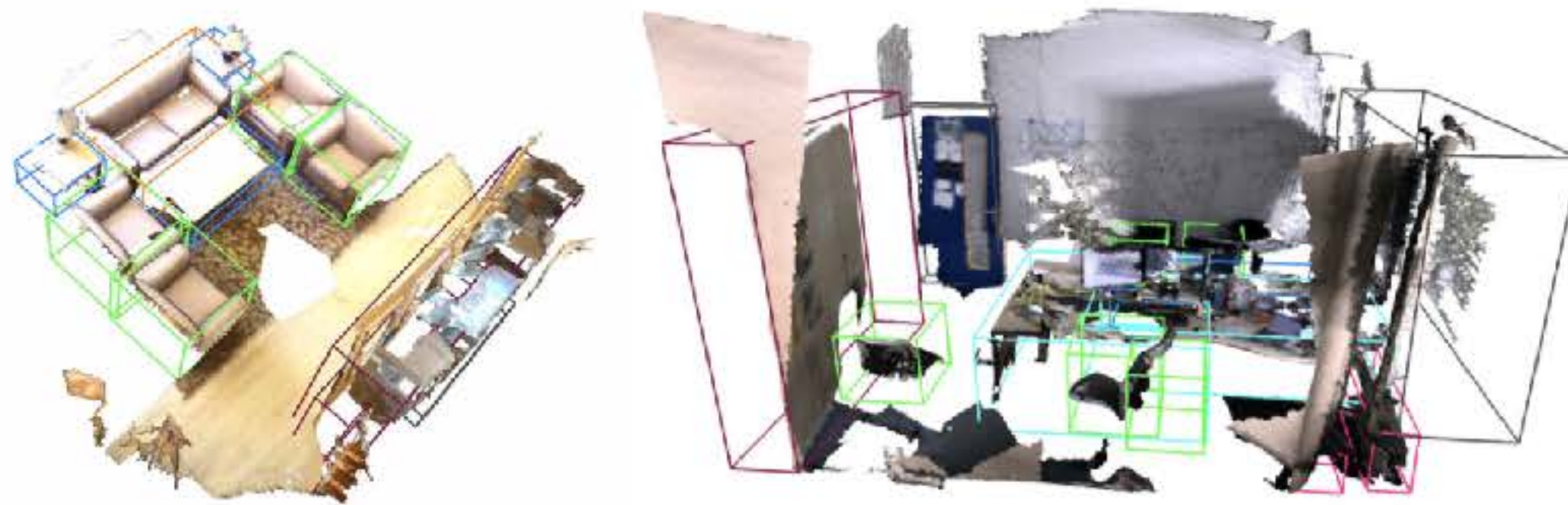
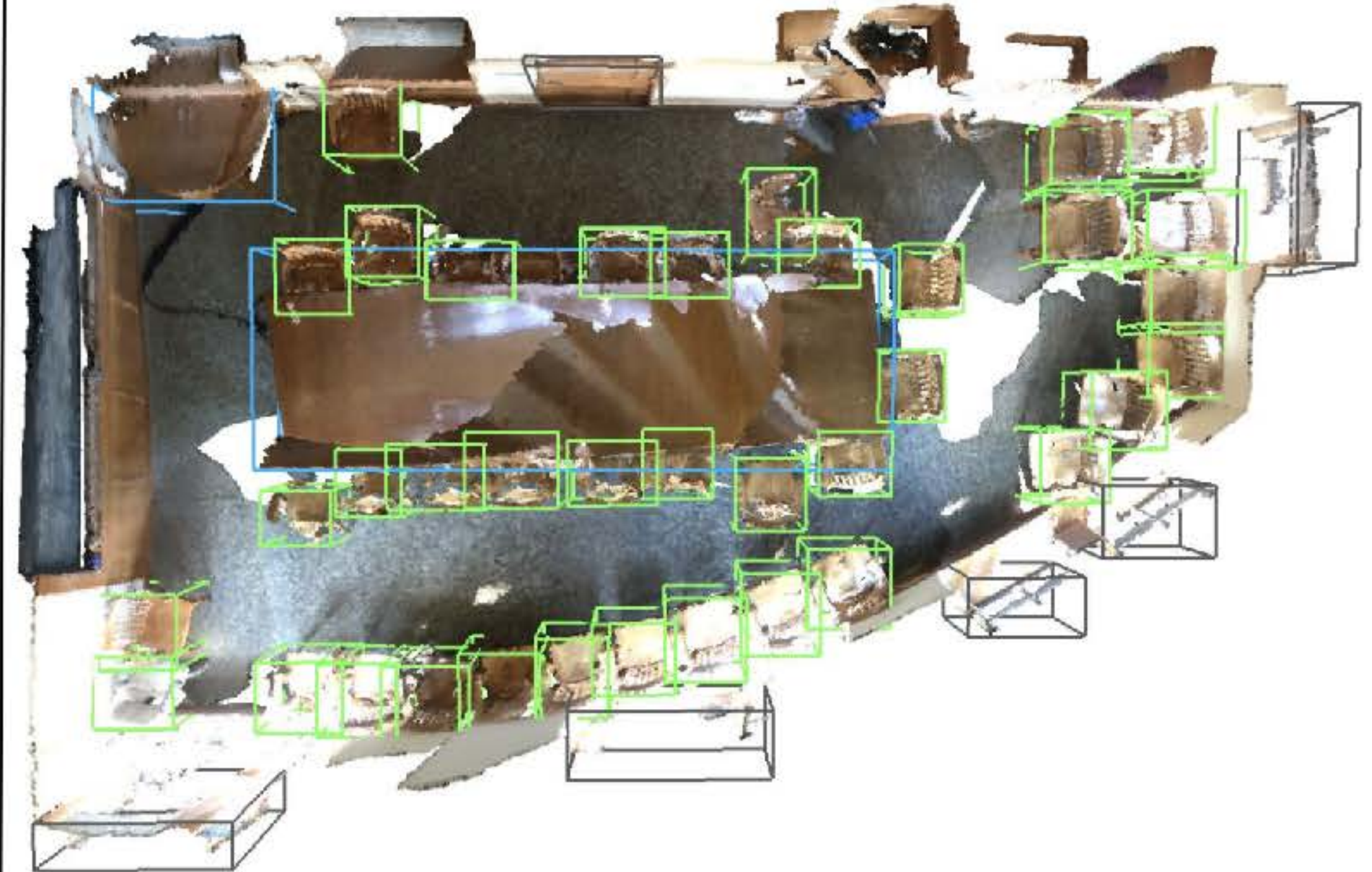


Results: ScanNet (3D reconstructions)

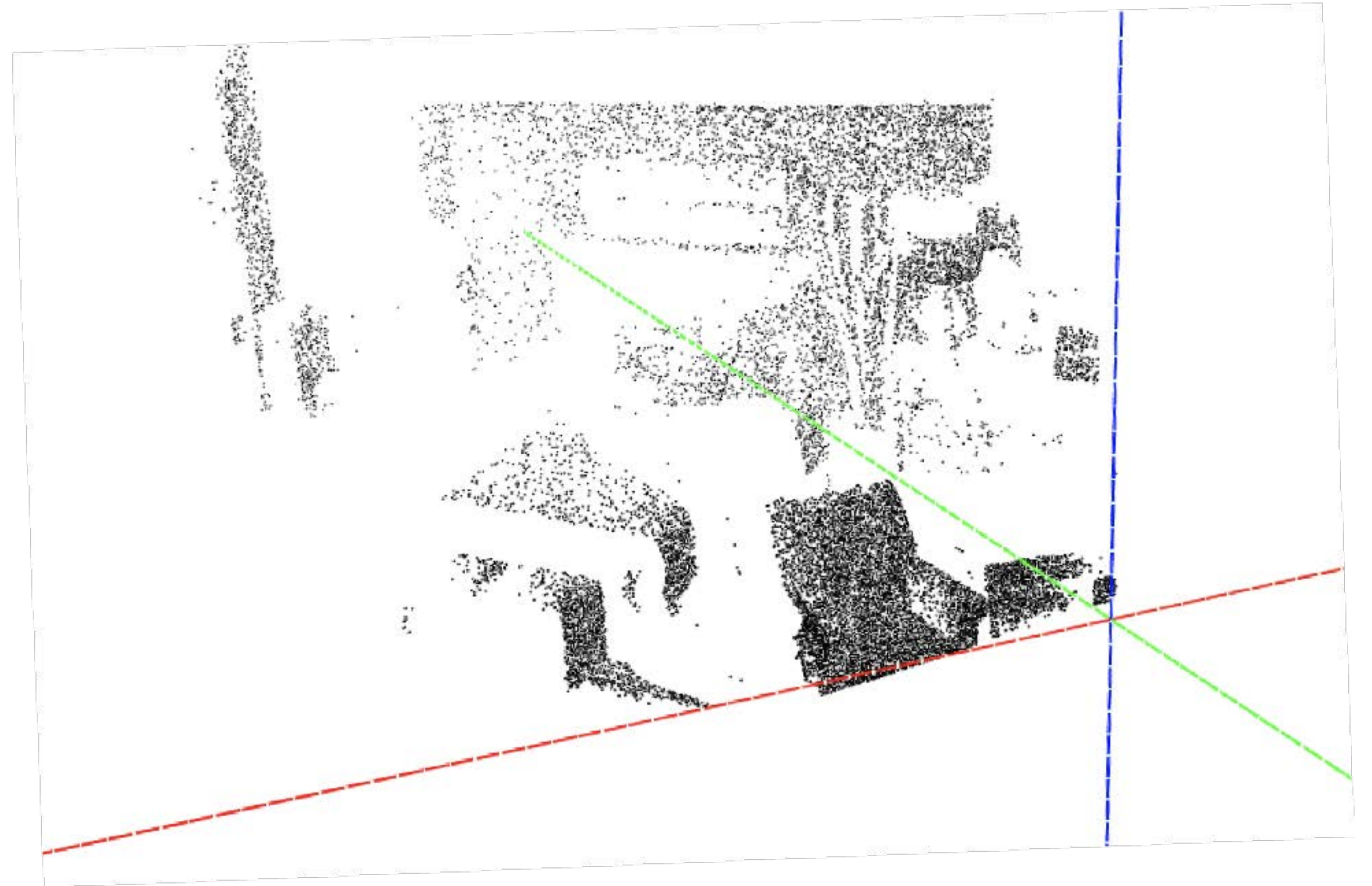
VoteNet prediction



Ground truth



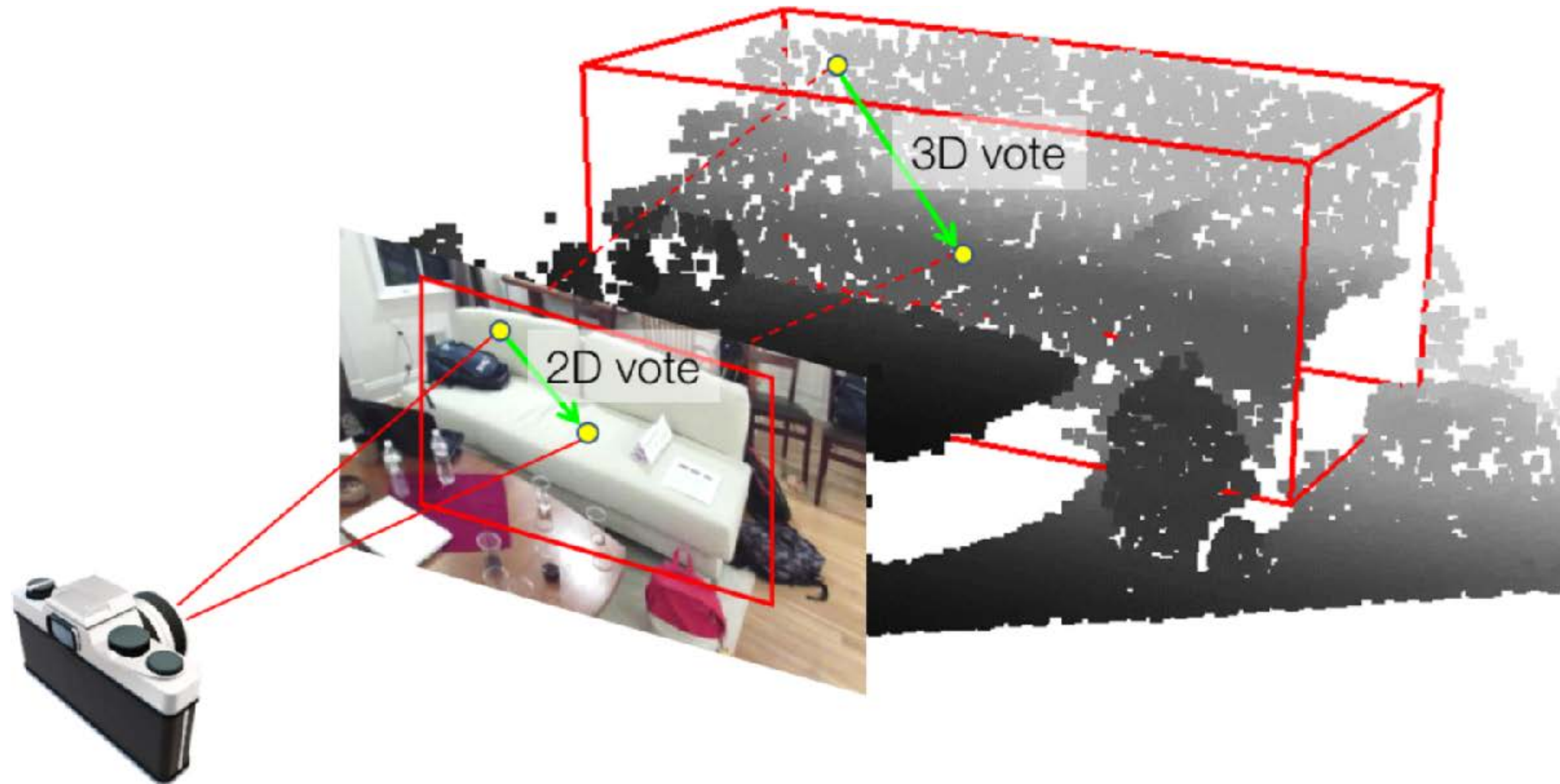
Can images help the VoteNet detection?



*Images are in **high resolution**, have **rich texture**, and can even provide useful geometric cues for object localization & shape/pose estimation.*

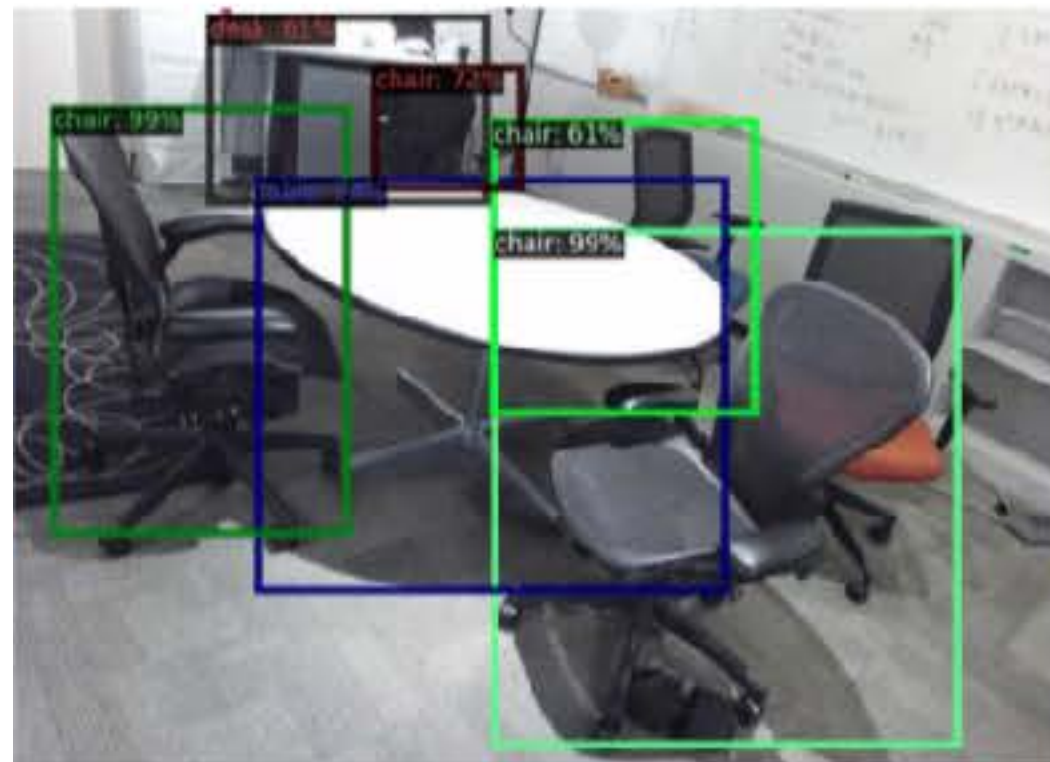
ImVoteNet: Boosting 3D Object Detection in Point Clouds with **Image Votes** [19]

Charles R. Qi*, Xinlei Chen*, Or Litany, Leonidas Guibas. CVPR 2020.

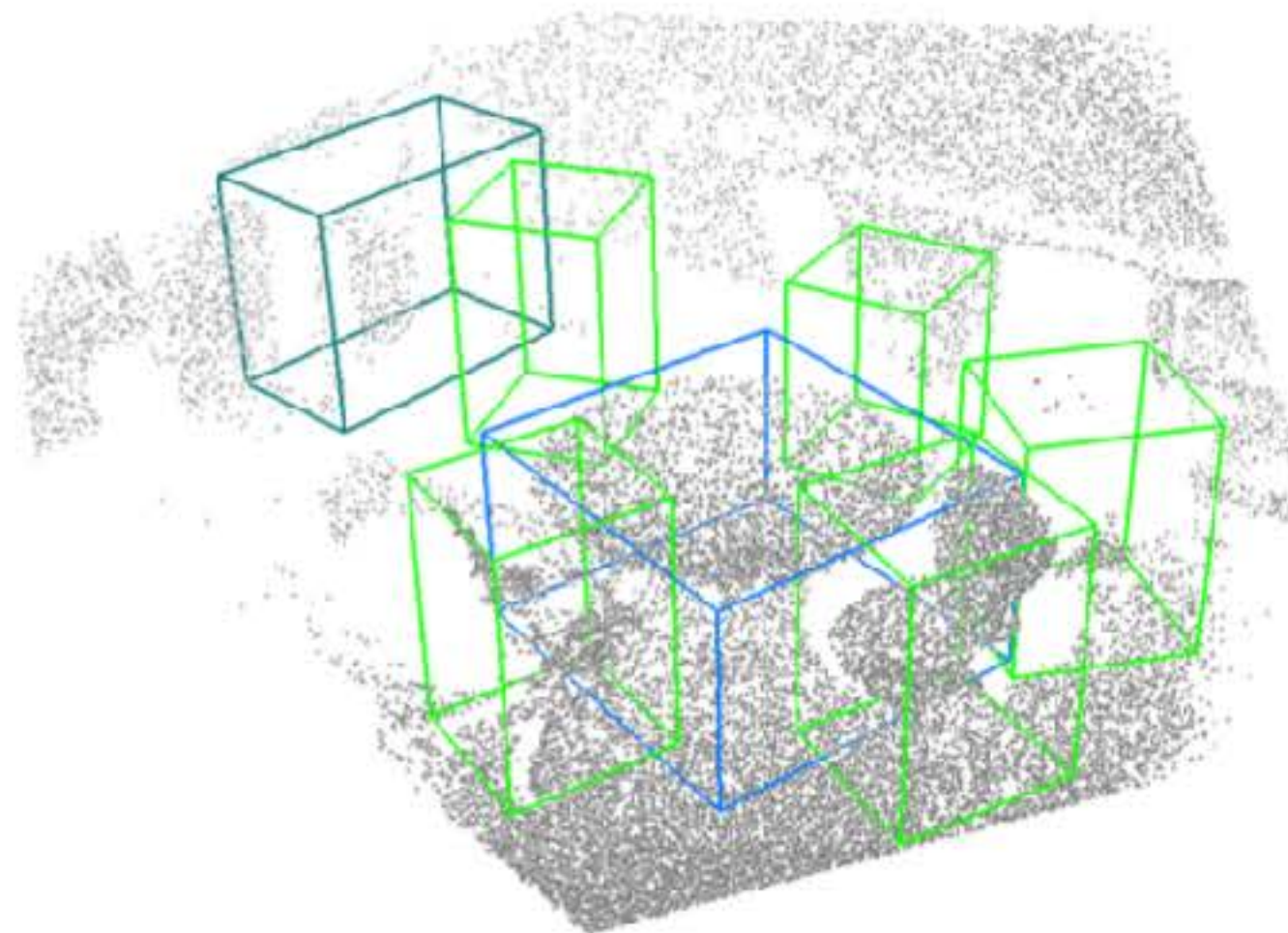
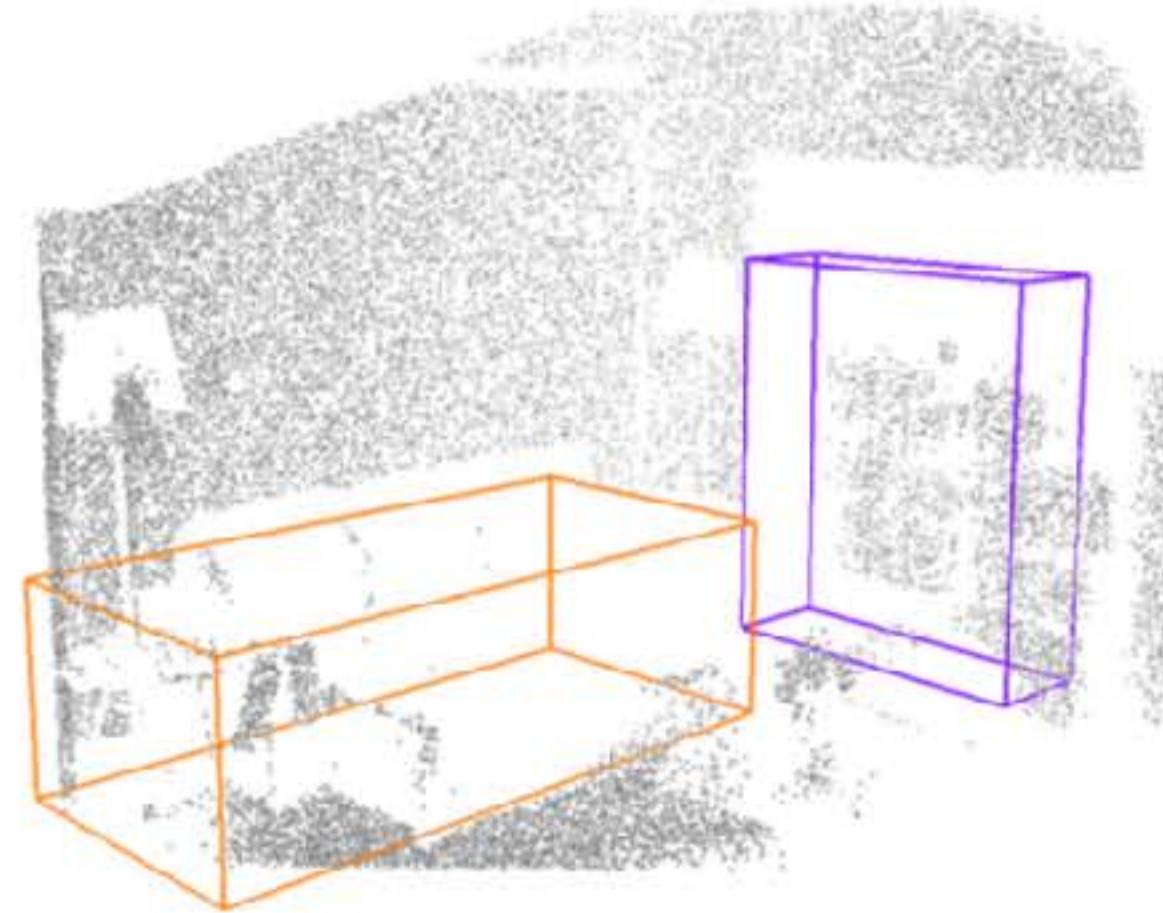


Results on SUN RGB-D

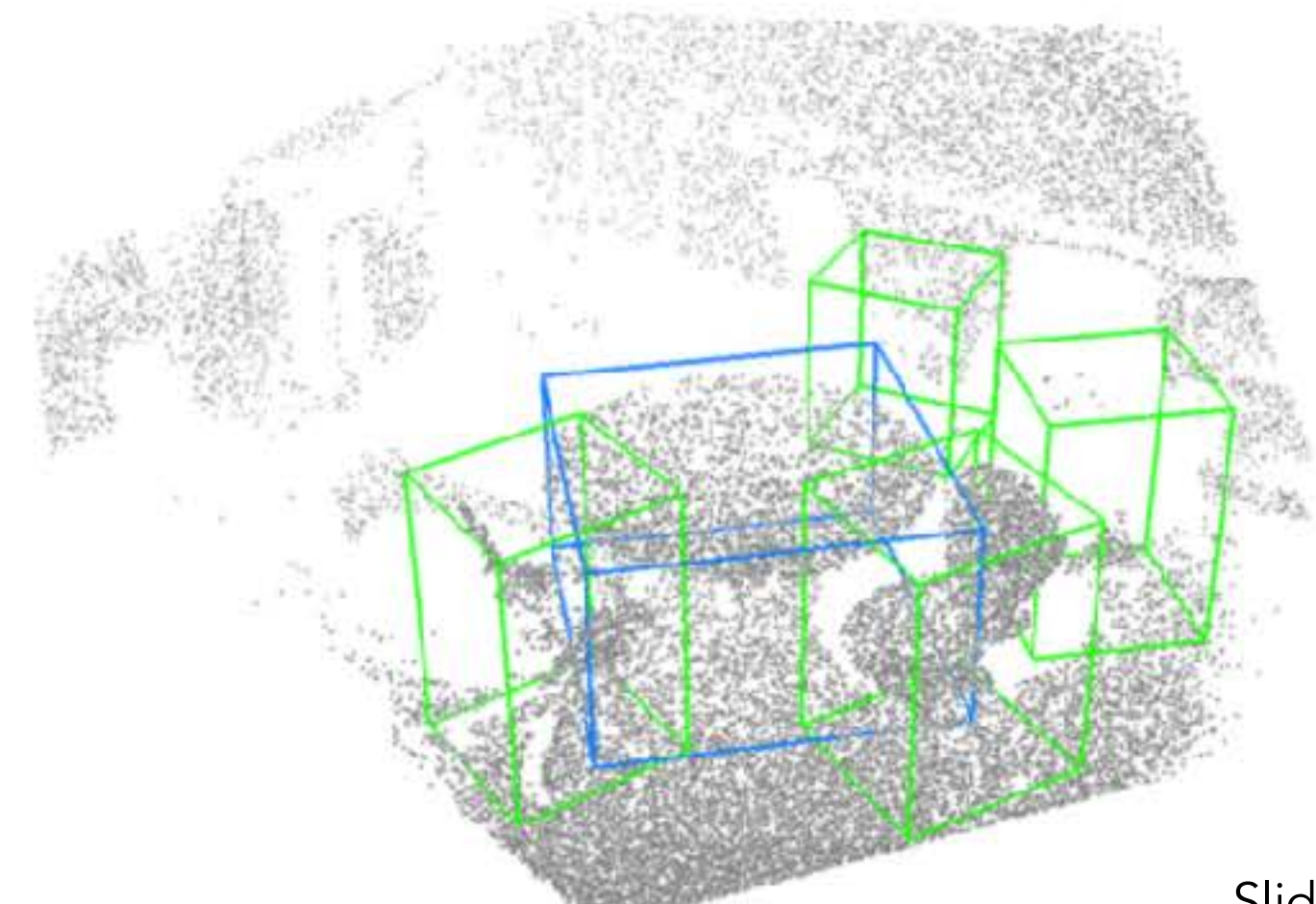
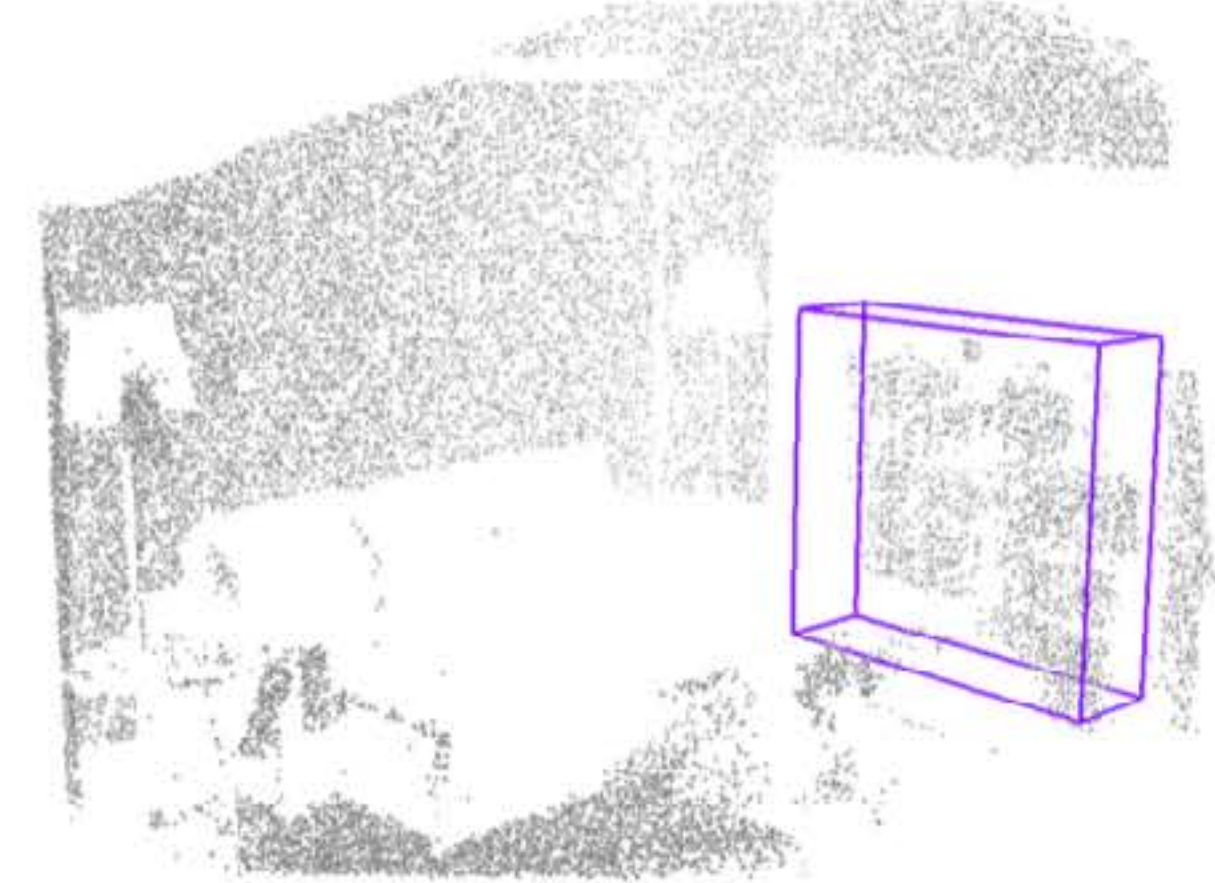
Ours 2D detection



ImVoteNet



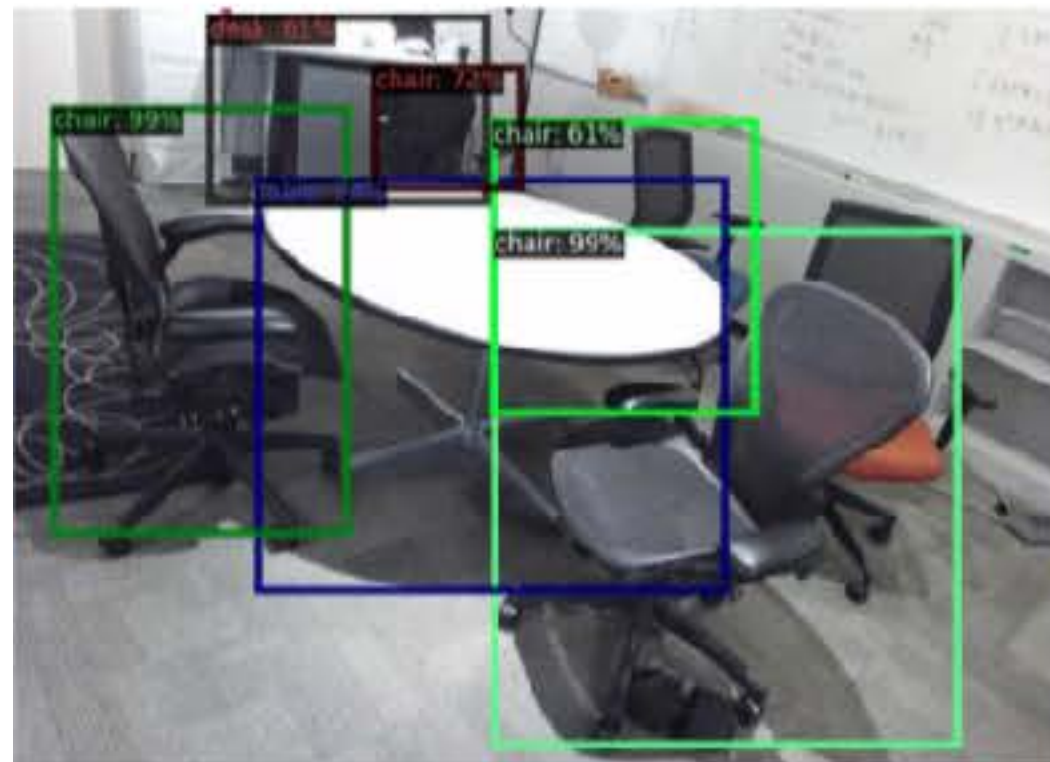
VoteNet



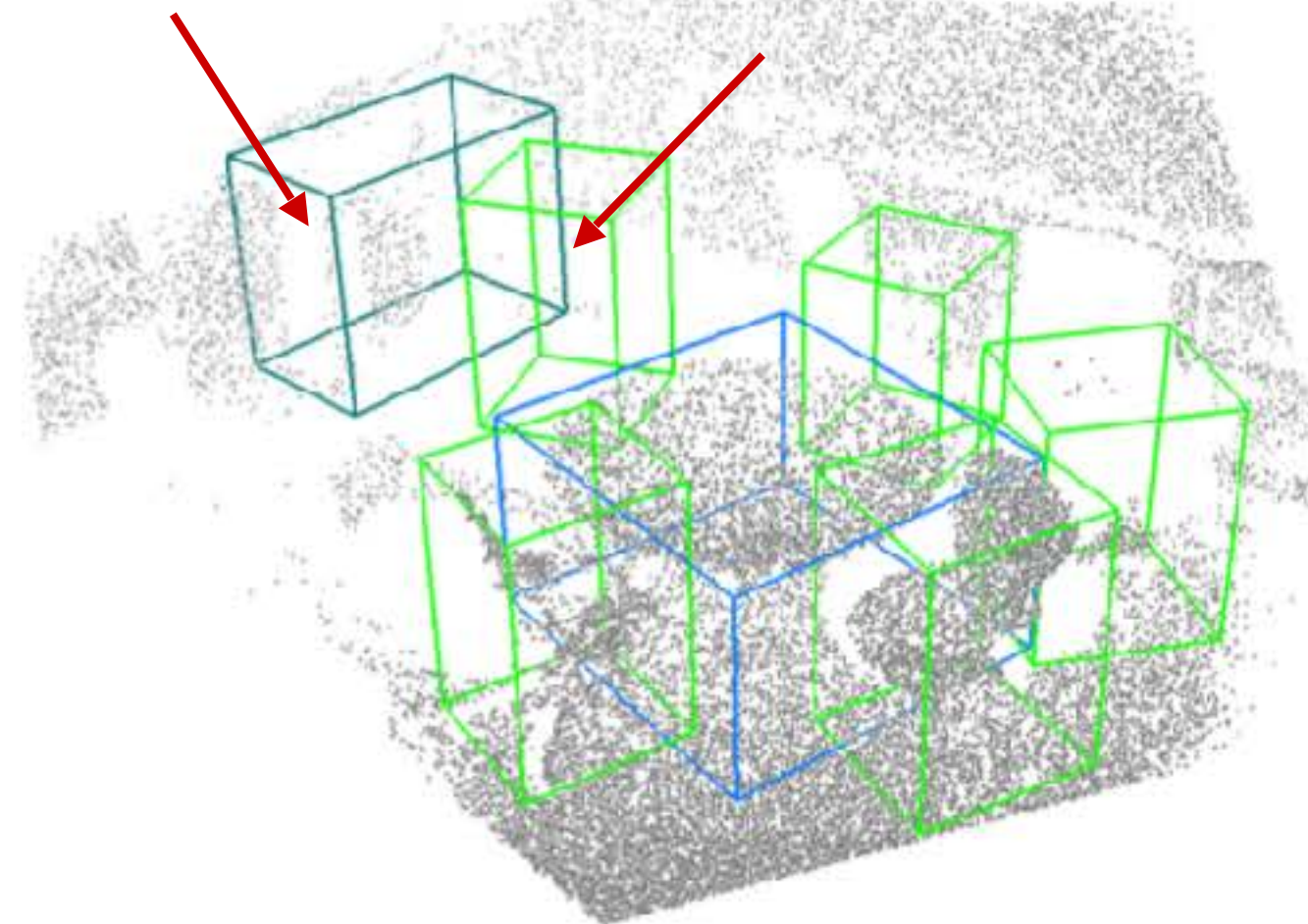
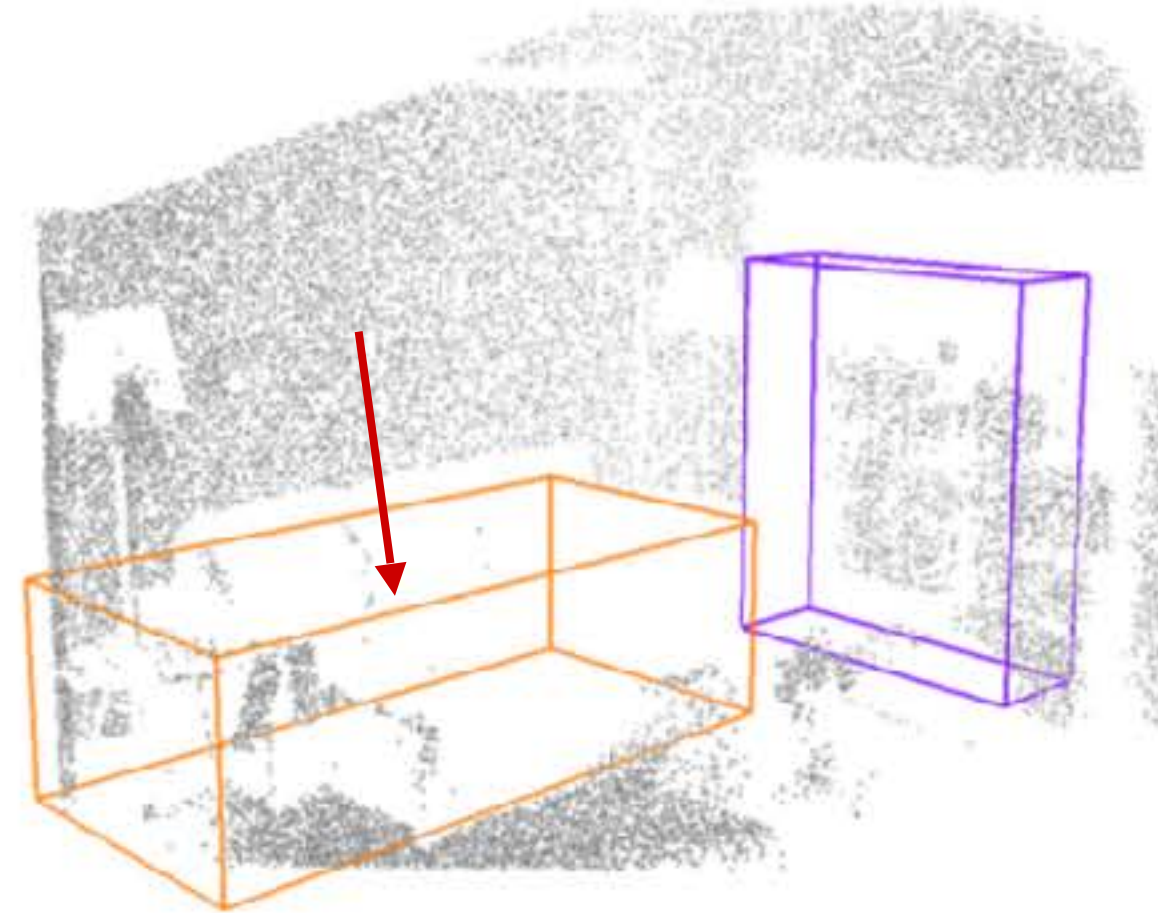
orange sofa purple bookshelf green chair blue table teal desk

Results on SUN RGB-D

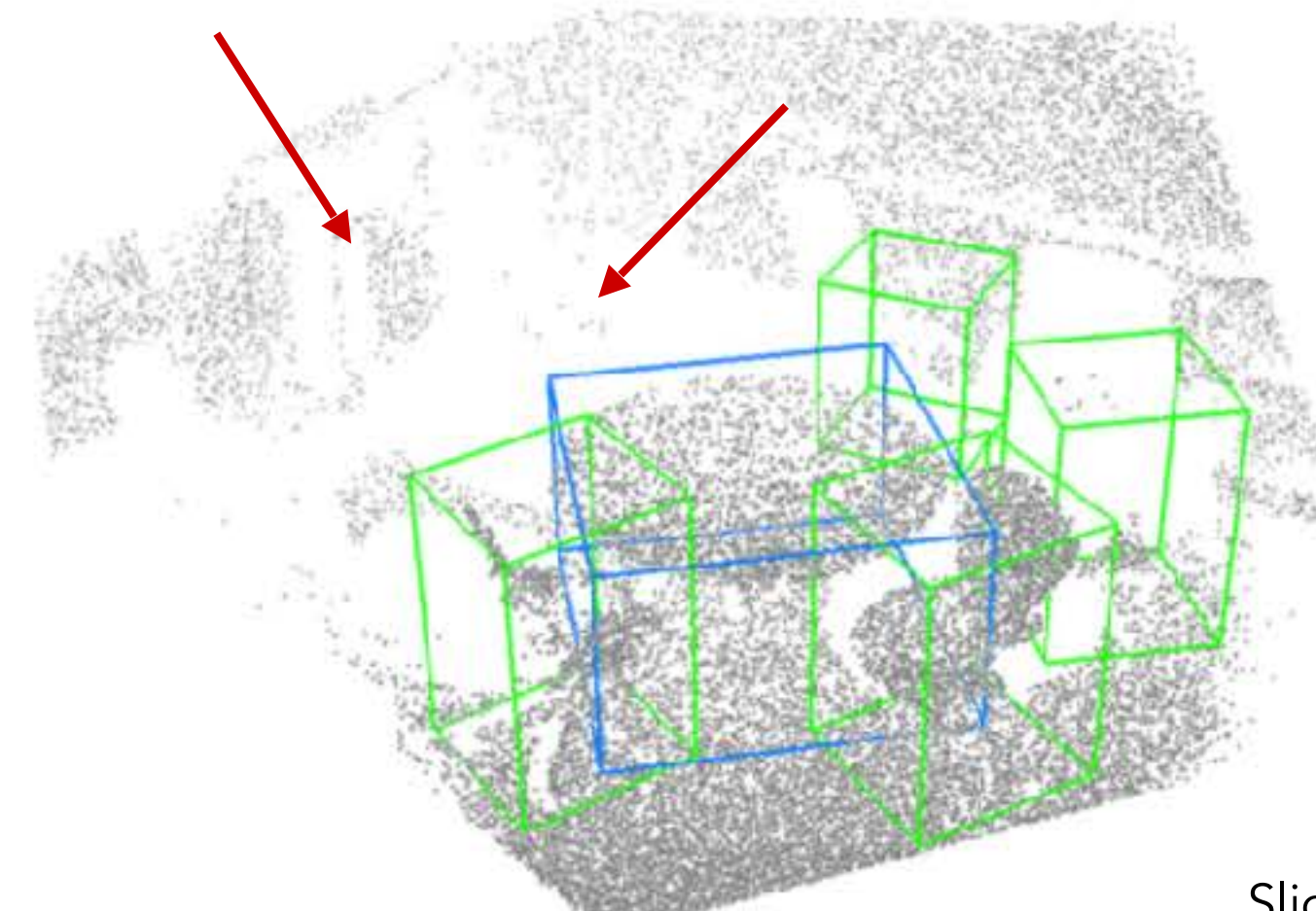
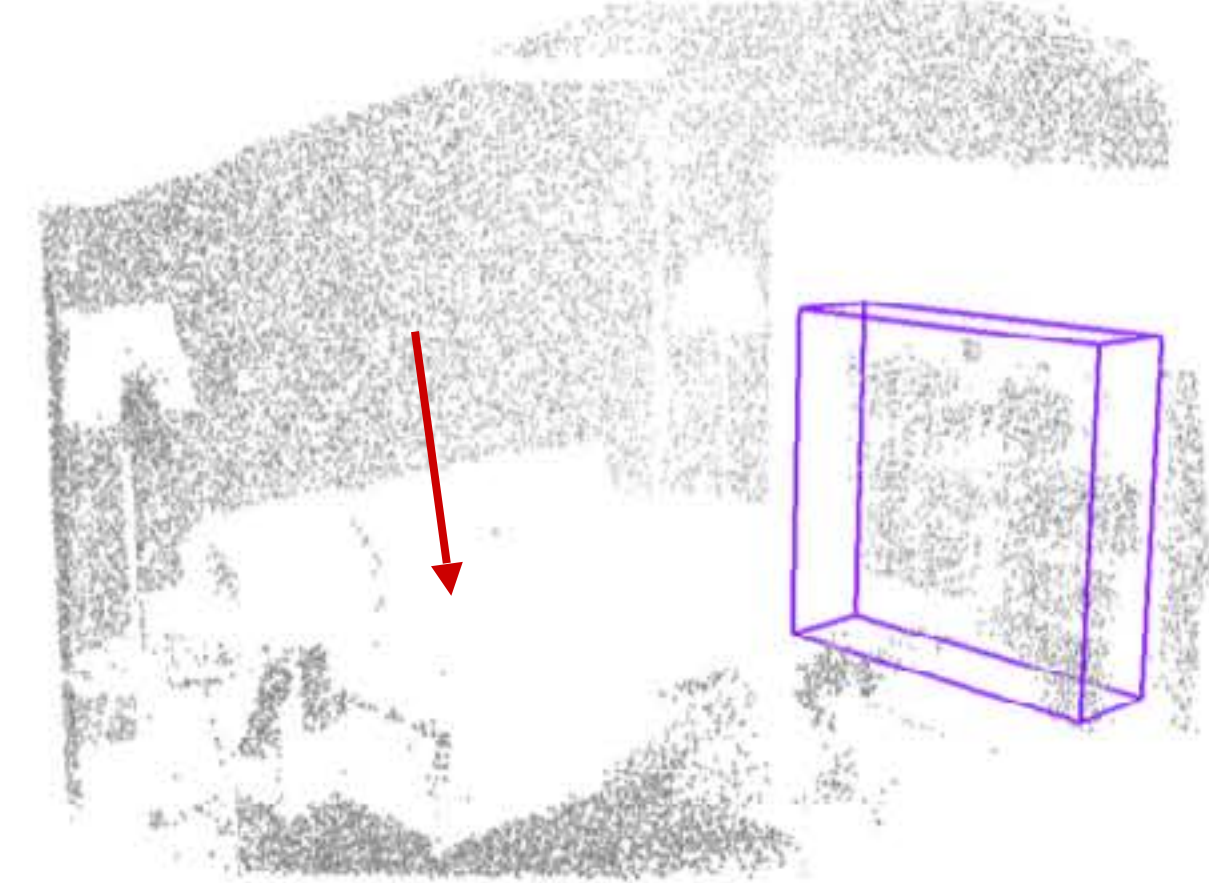
Ours 2D detection



ImVoteNet



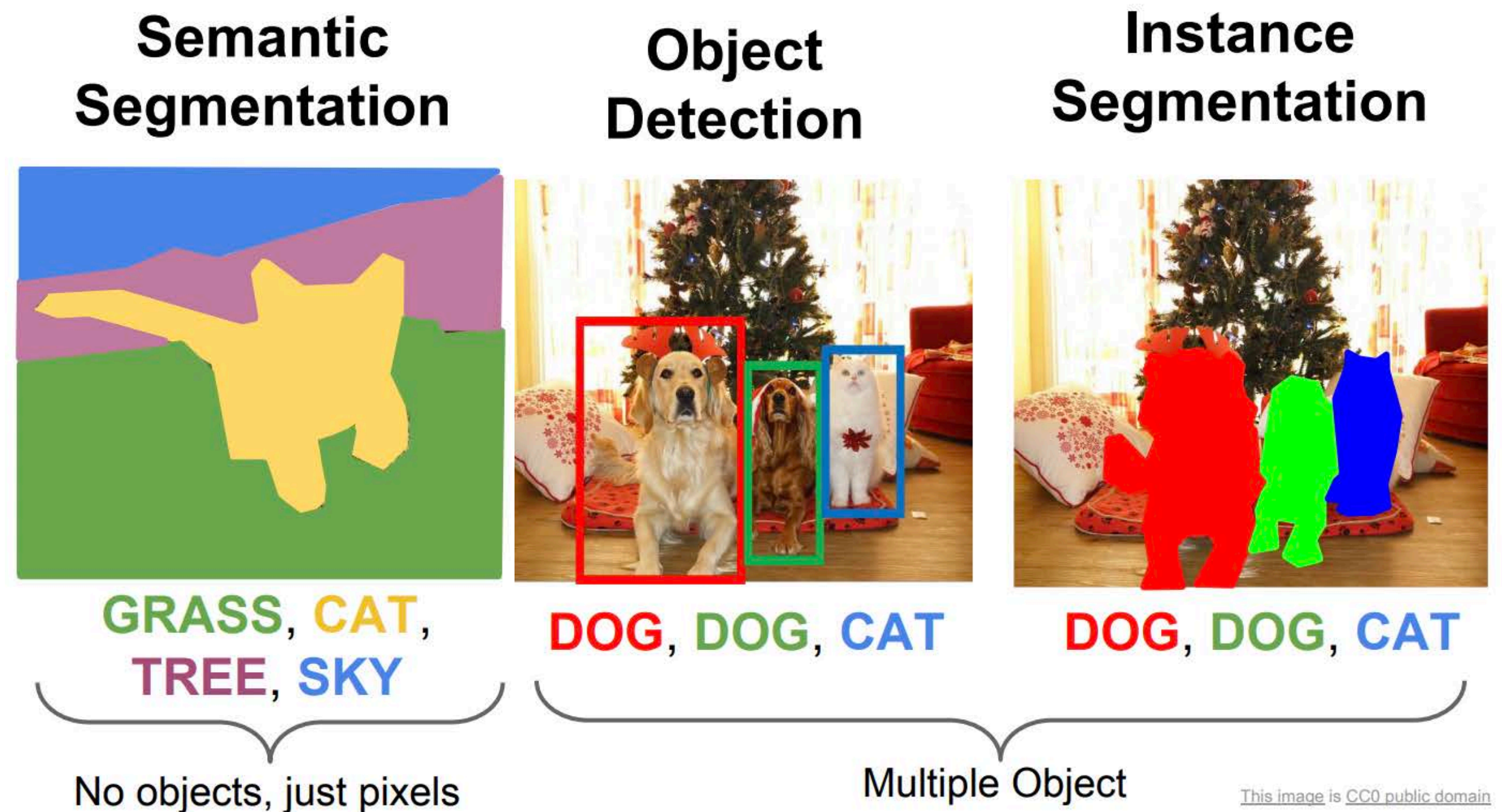
VoteNet



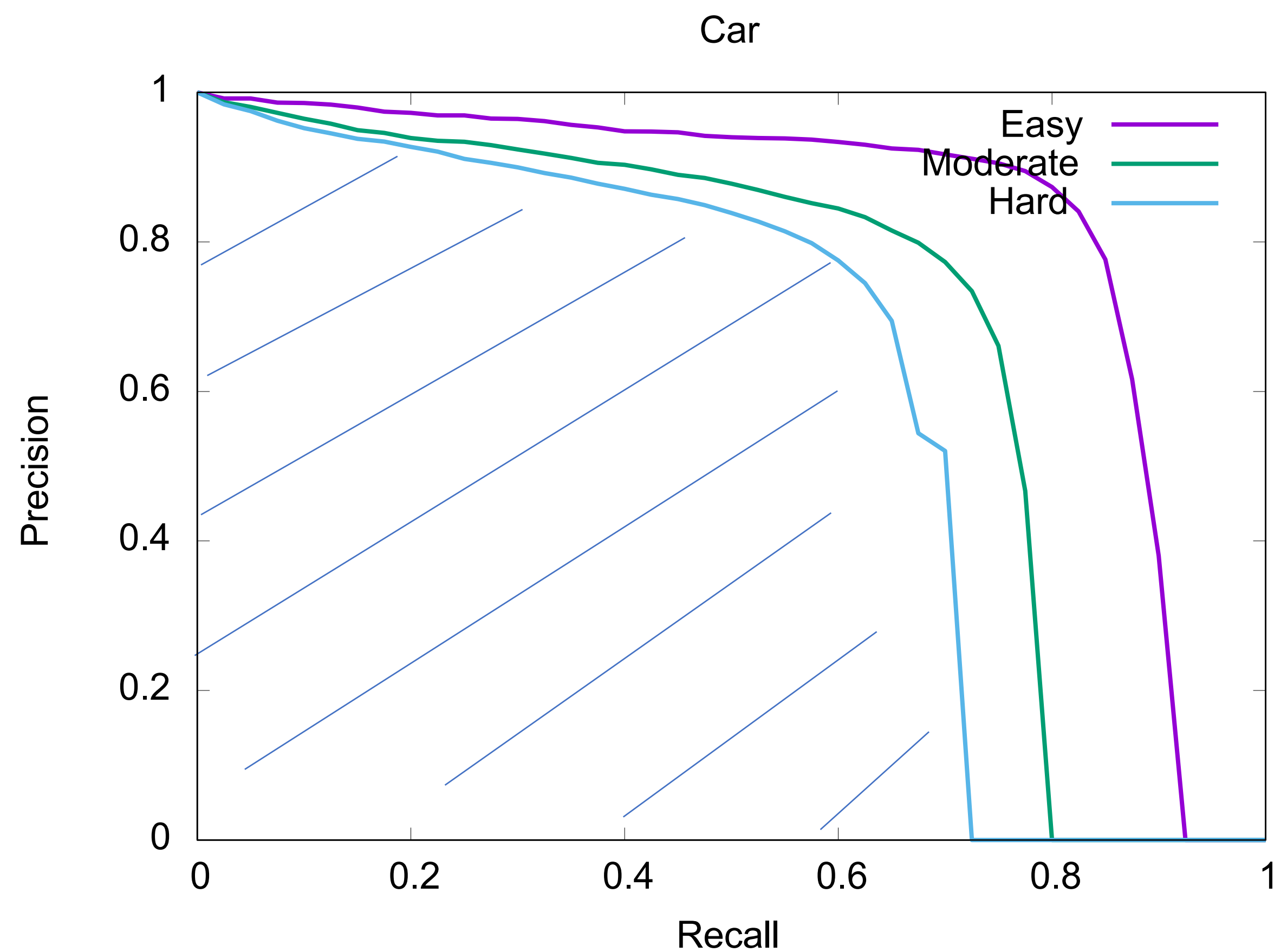
sofa bookshelf chair table desk

Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- **3D Instance Segmentation**
 - Top-Down Approaches
 - Bottom-Up Approaches



Evaluation Metric: Average Precision (AP) with a segmentation Intersection over Union (IoU) threshold

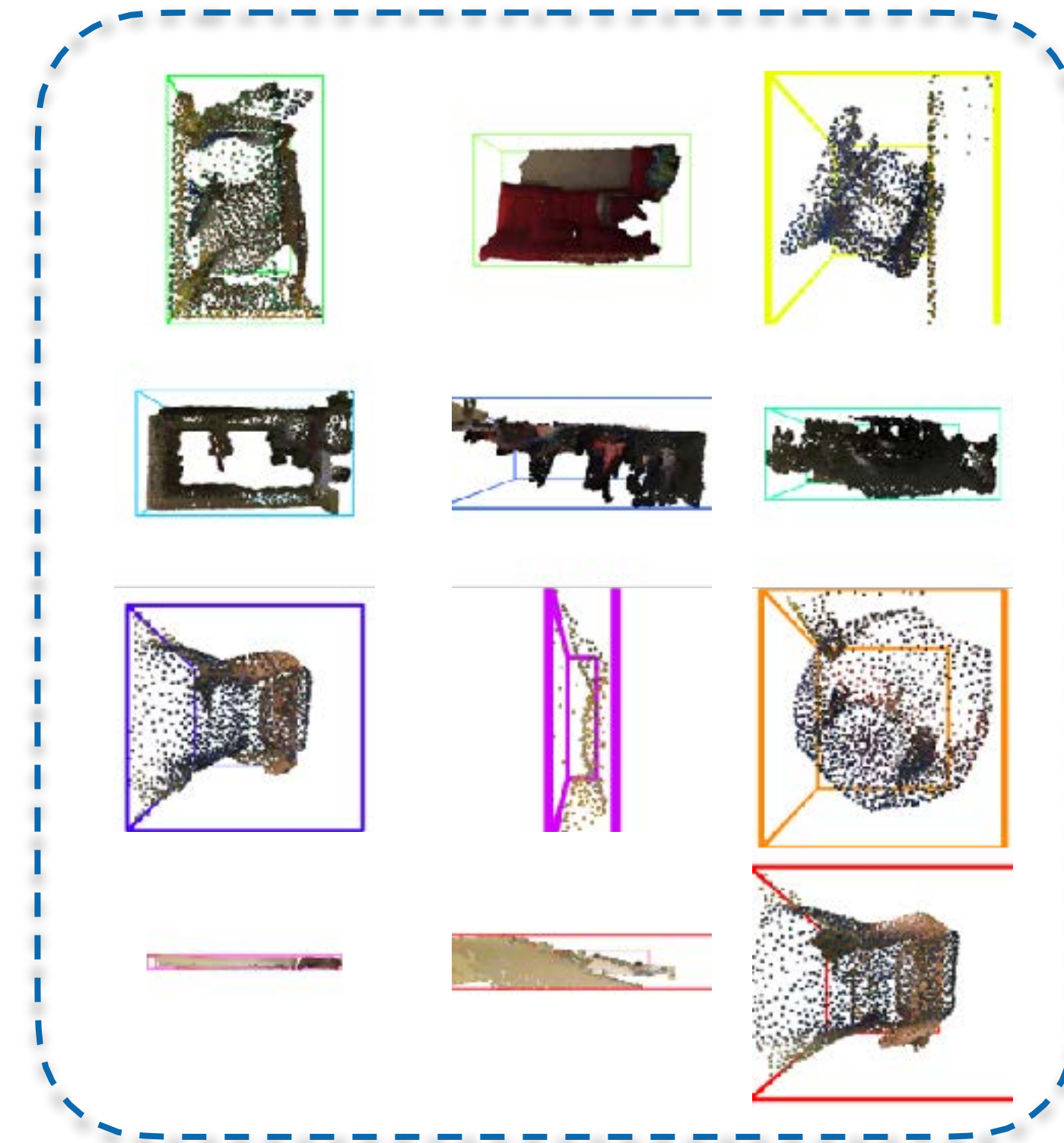
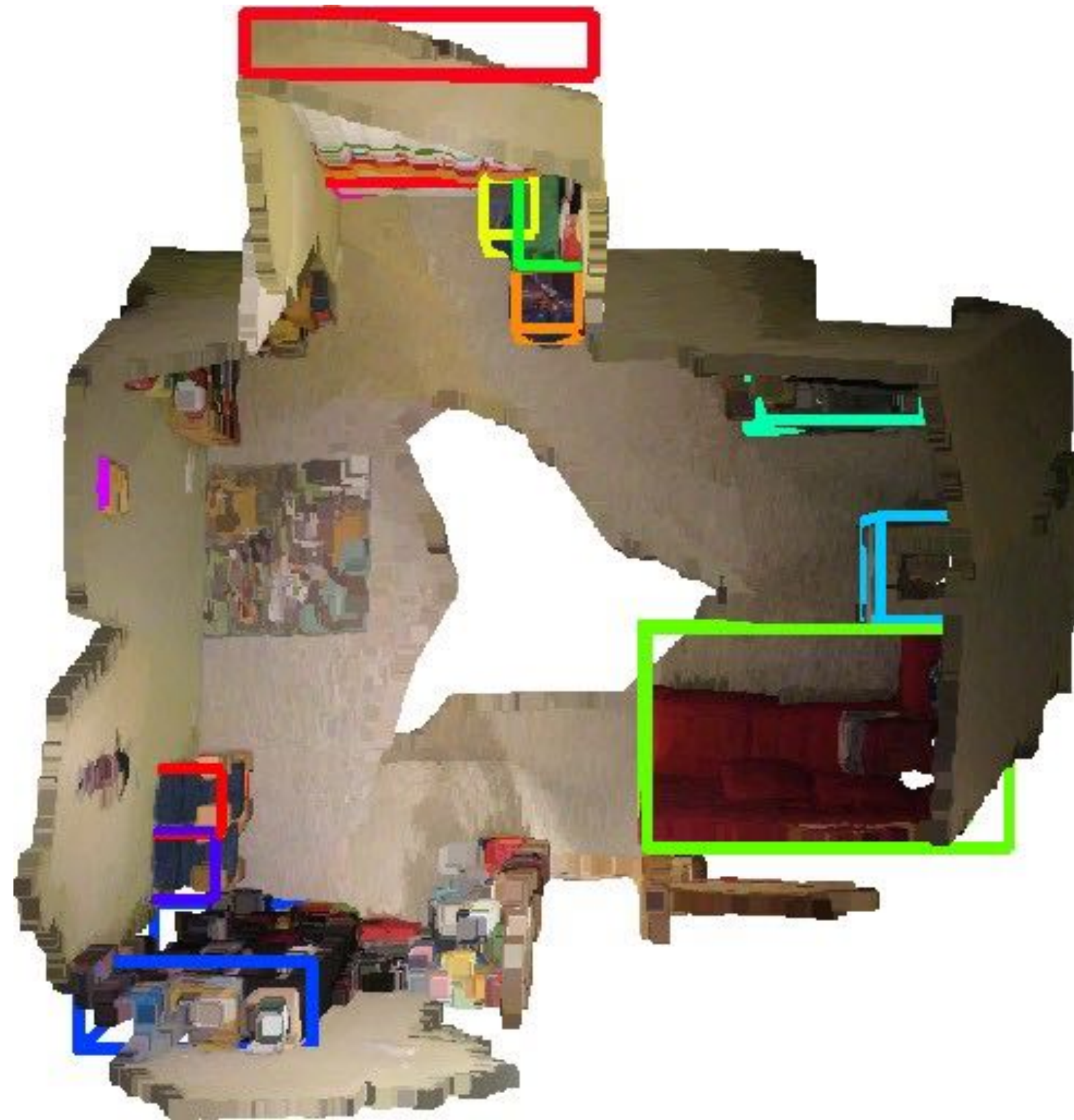


Outline

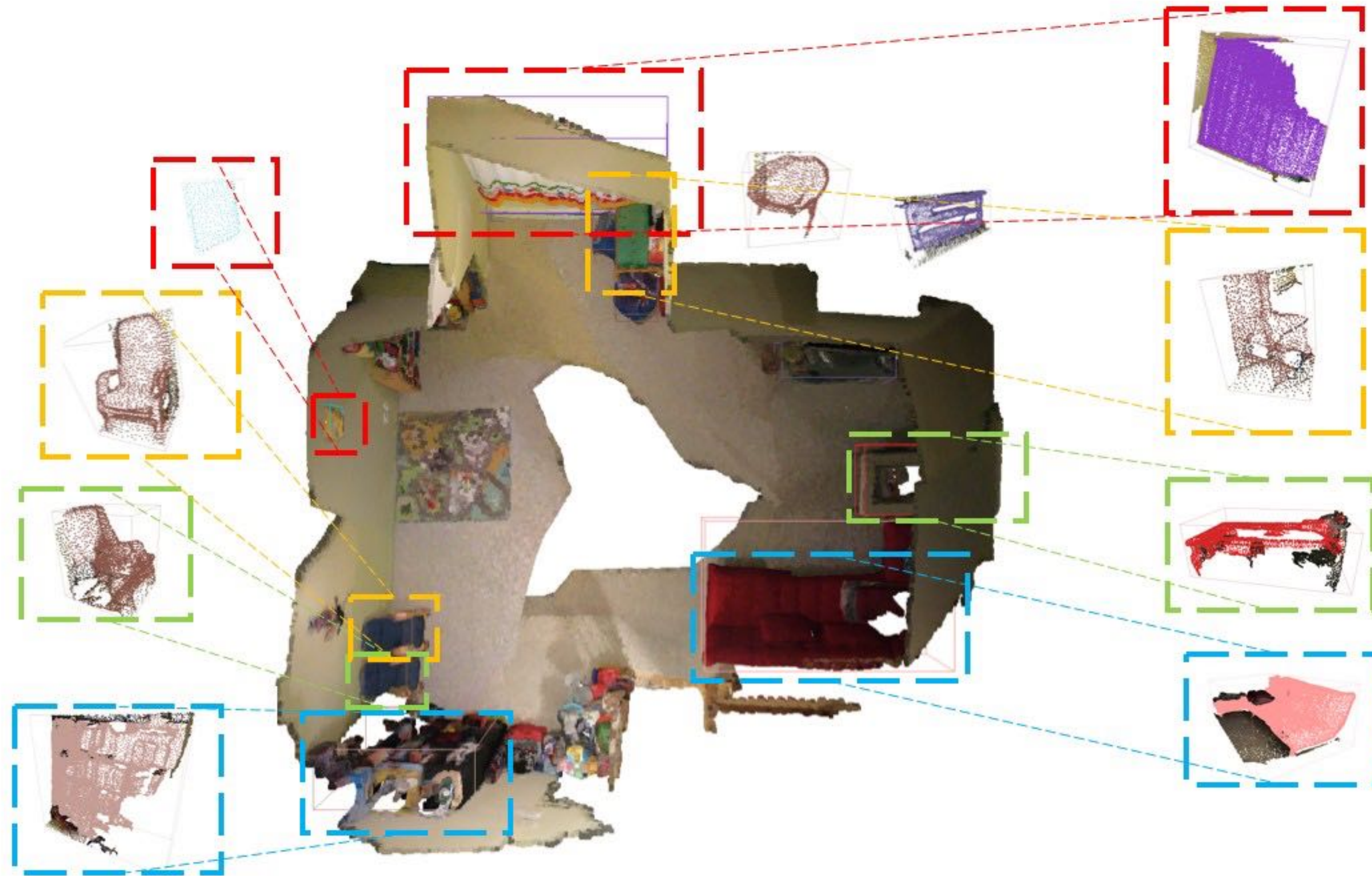
- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - *Top-Down Approaches*
 - Bottom-Up Approaches

First Step: Generate Proposals (e.g., Bounding Boxes)

Proposals



Second Step: Associate Points with Proposals

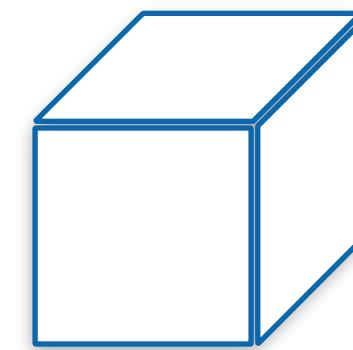


Two Key Questions:

- How to generate (instance) proposals?
- How to associate points with proposals?

First of all, what is a good proposal representation?

- Easy to parameterize and predict
- Easy to classify whether a point belongs to it
- Parameterization:
 - Primitive type
 - Parameters (position, rotation, ...)
- Common choices: 3D bounding box, spheres



Proposal Generation: Non-Learning

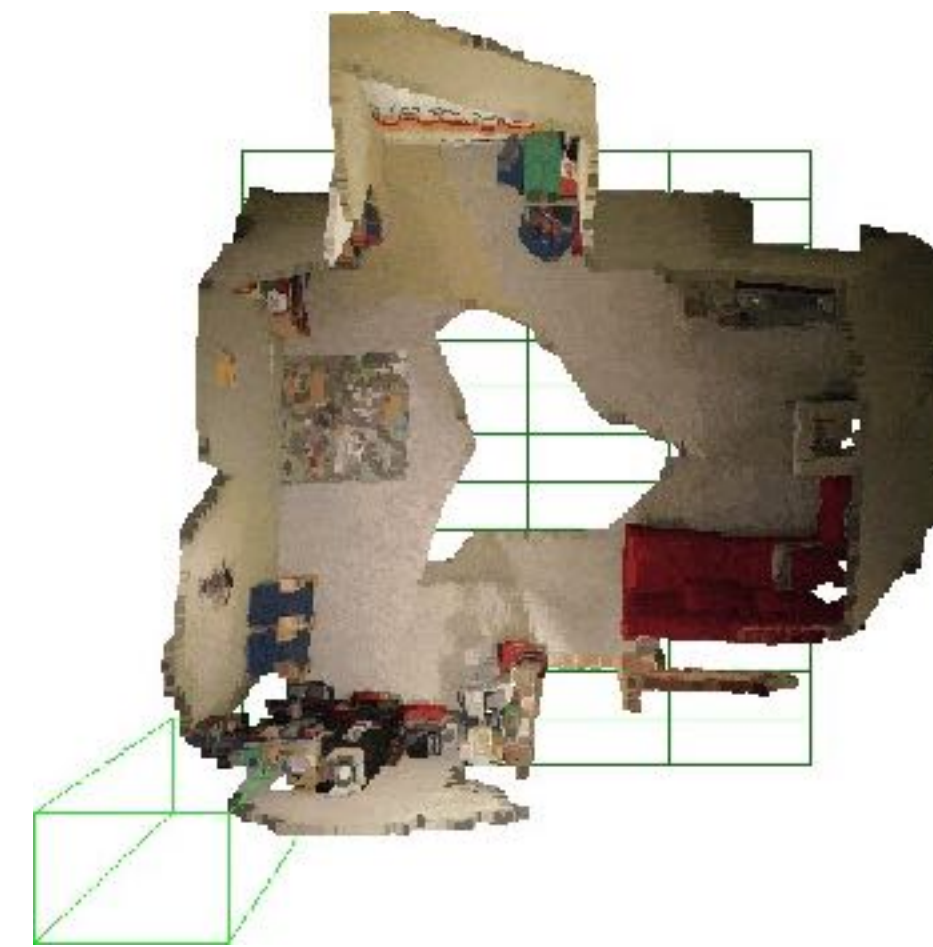
- **Sliding window:** The straightforward, heuristic method to generate proposals without learning
- Slide a (template) window over the input point cloud



stride=(0.5, 0.5)
size=(1.5, 1.5)



stride=(1.5, 1.5)
size=(1.5, 1.5)



stride=(1.5, 0.5)
size=(1.5, 1.0)

Proposal Generation: Learning-based

- To have a high recall, we need to densely slide a window
- However, too heavy burden for the association step

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

3D-BoNet Pipeline

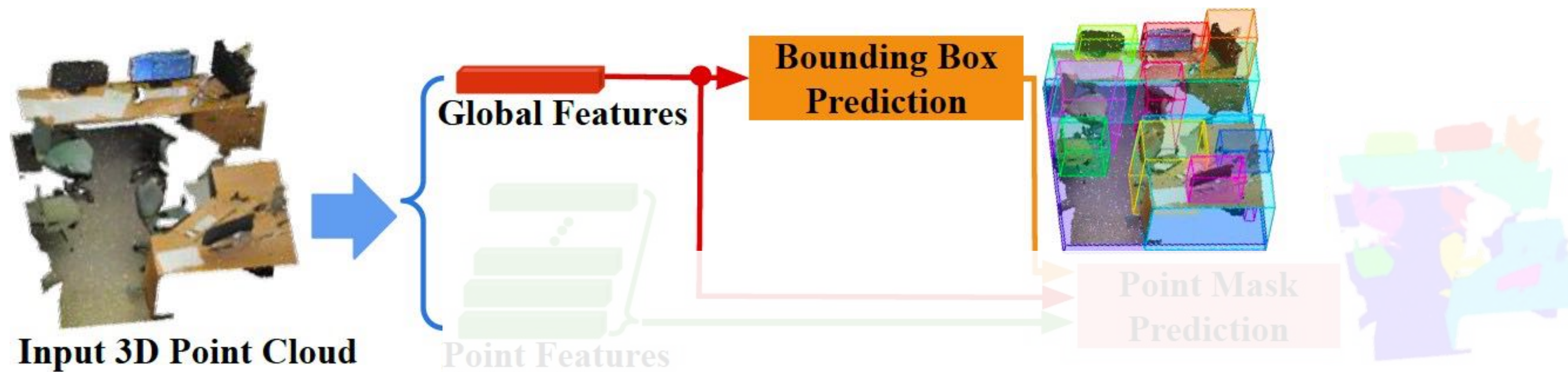


Figure 1: The 3D-BoNet framework for instance segmentation on 3D point clouds.

3D-BoNet Pipeline

“set prediction” task

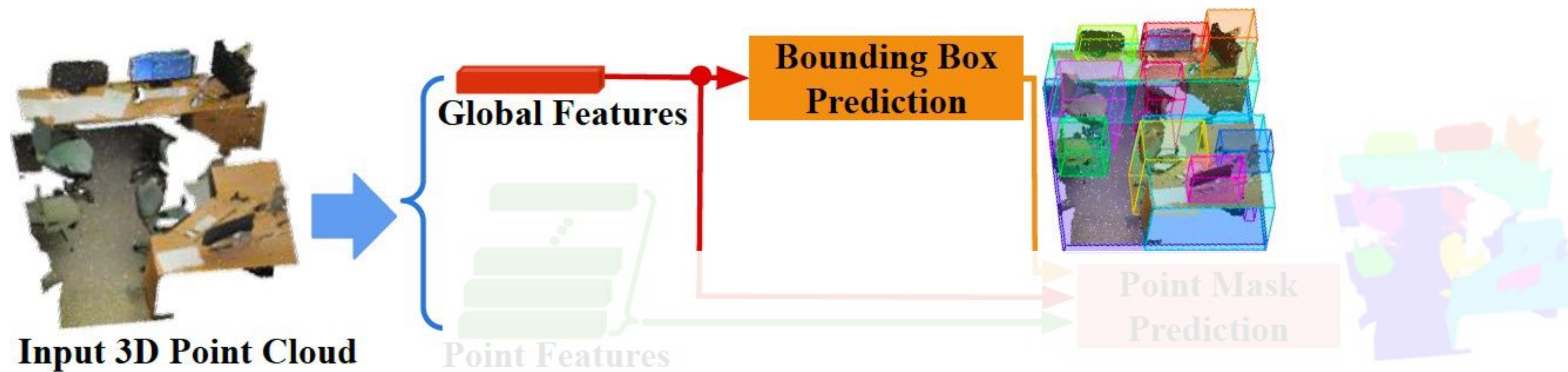
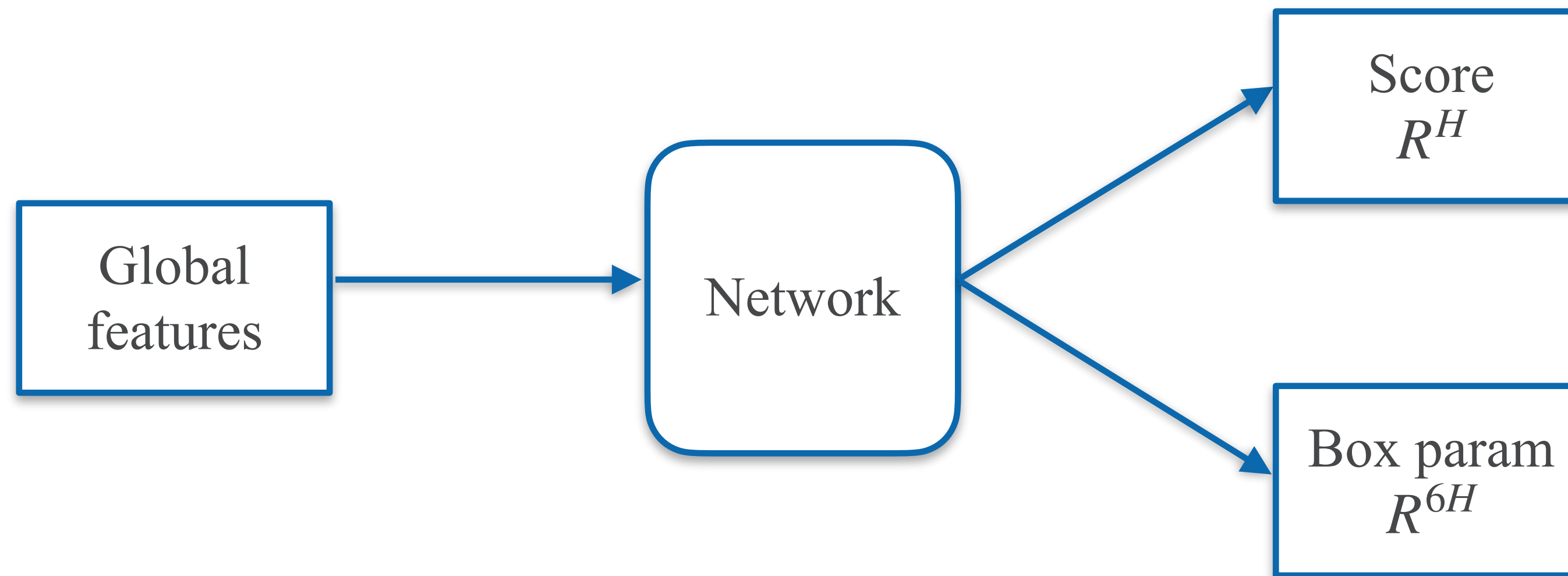


Figure 1: The 3D-BoNet framework for instance segmentation on 3D point clouds.

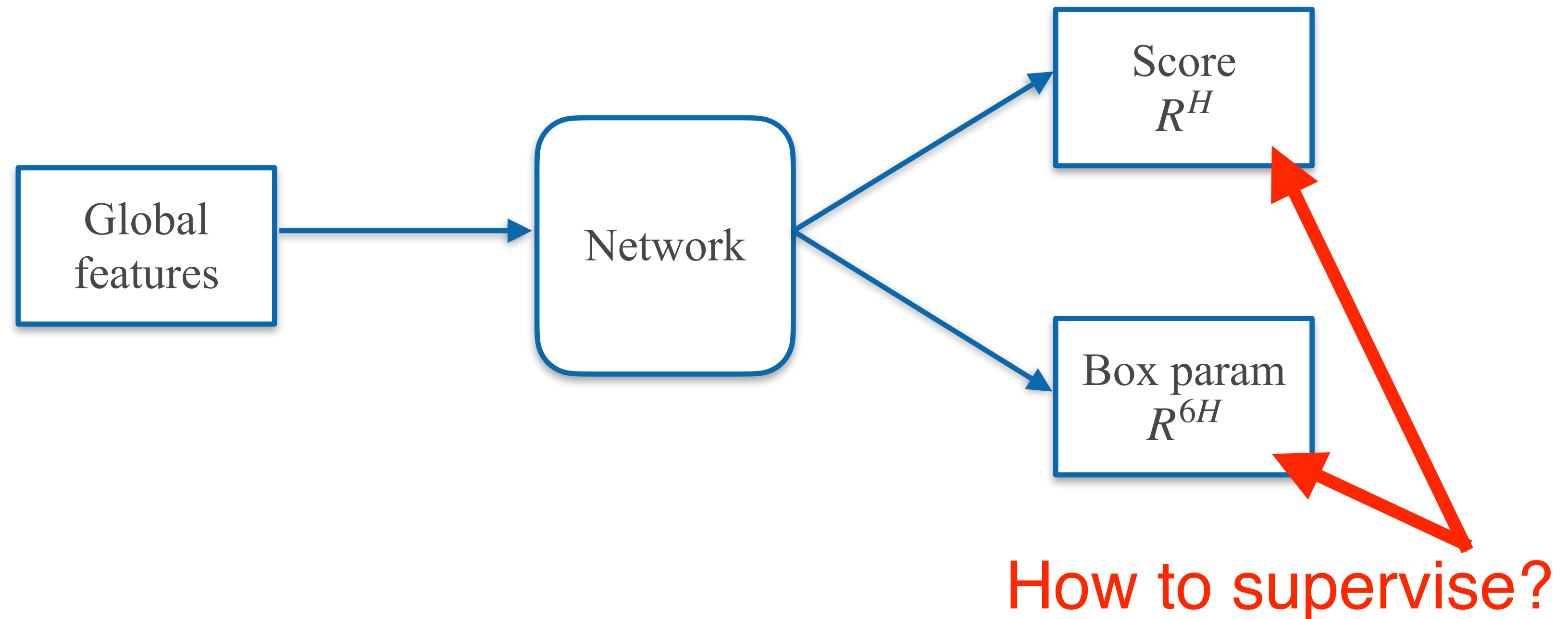
Bounding Box Prediction

- Bounding box parameterization:
 $\{x_{min}, y_{min}, z_{min}, x_{max}, y_{max}, z_{max}\}$
- Regress a predefined, fixed number (H) of bounding boxes



Bounding Box Prediction

- Bounding box parameterization:
 $\{x_{min}, y_{min}, z_{min}, x_{max}, y_{max}, z_{max}\}$
- Regress a predefined, fixed number (H) of bounding boxes

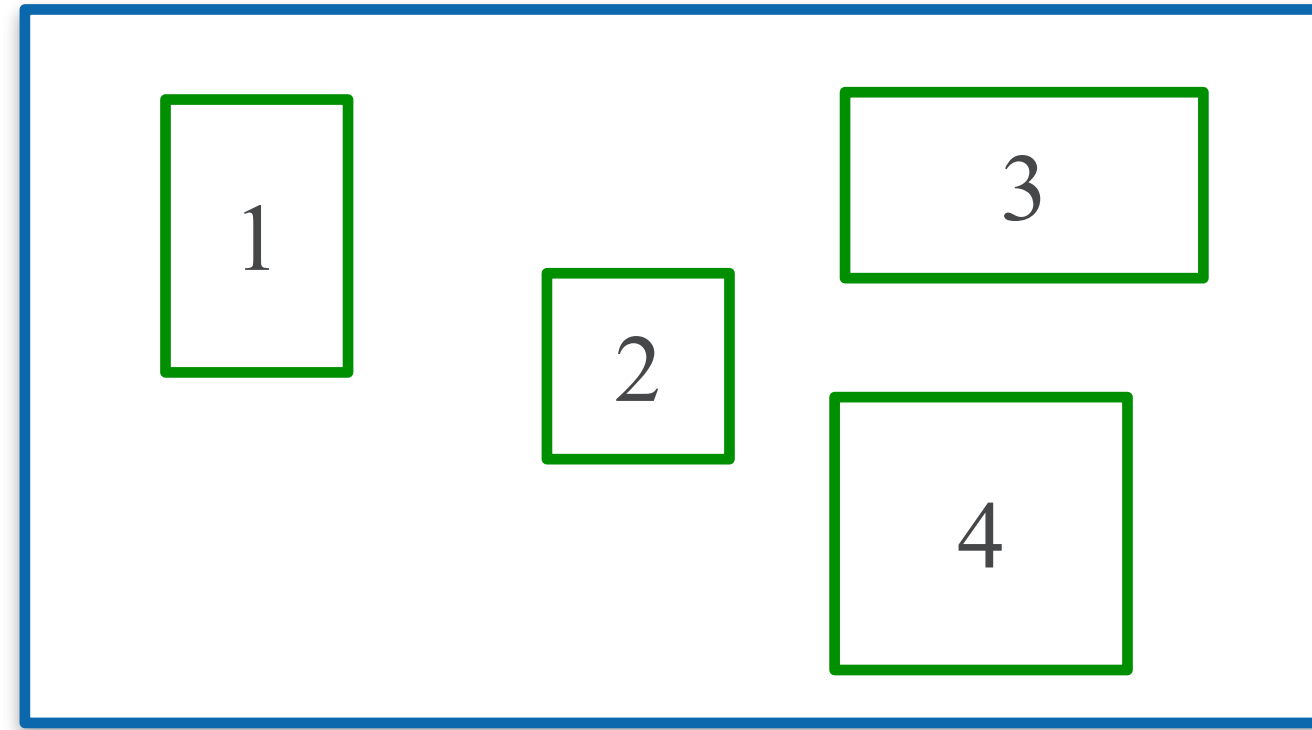


Loss: Bounding Box Association

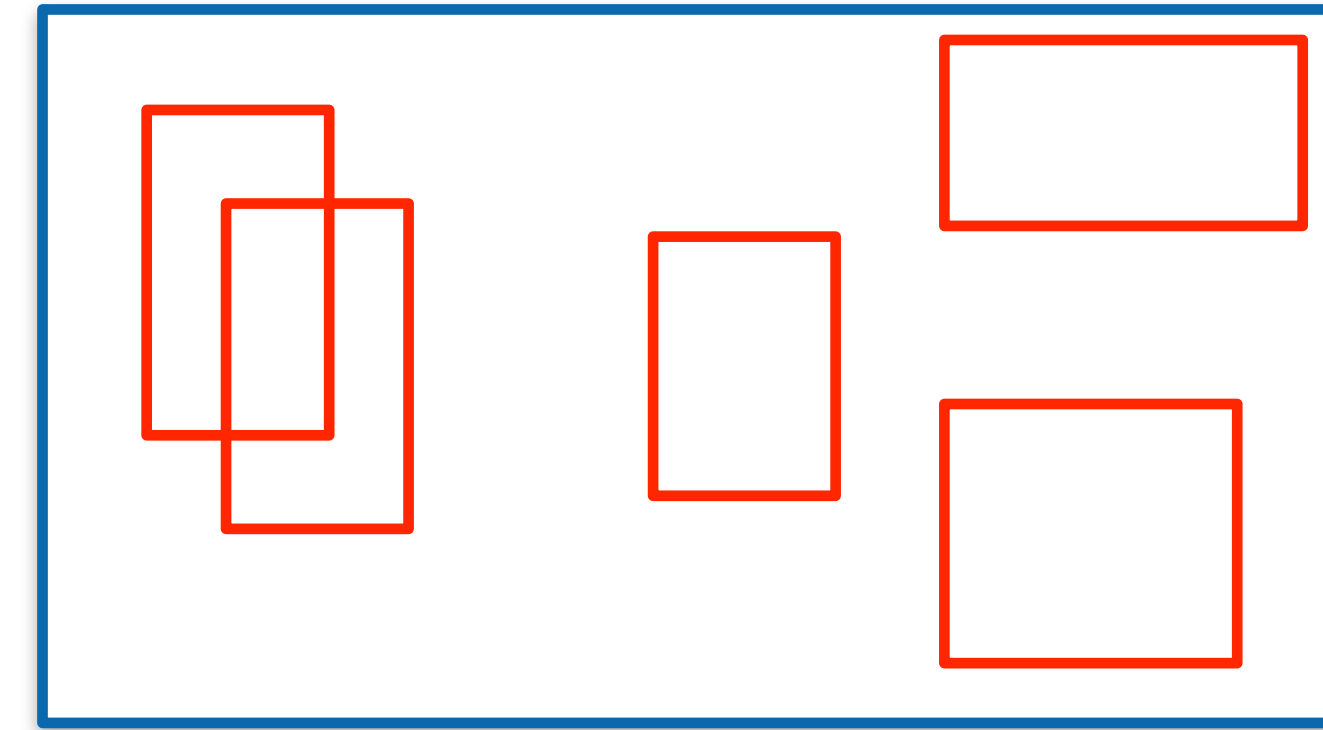
How to know the GT on-the-fly?

Find a match between the GT and predicted boxes

Optimal Association (2D case)

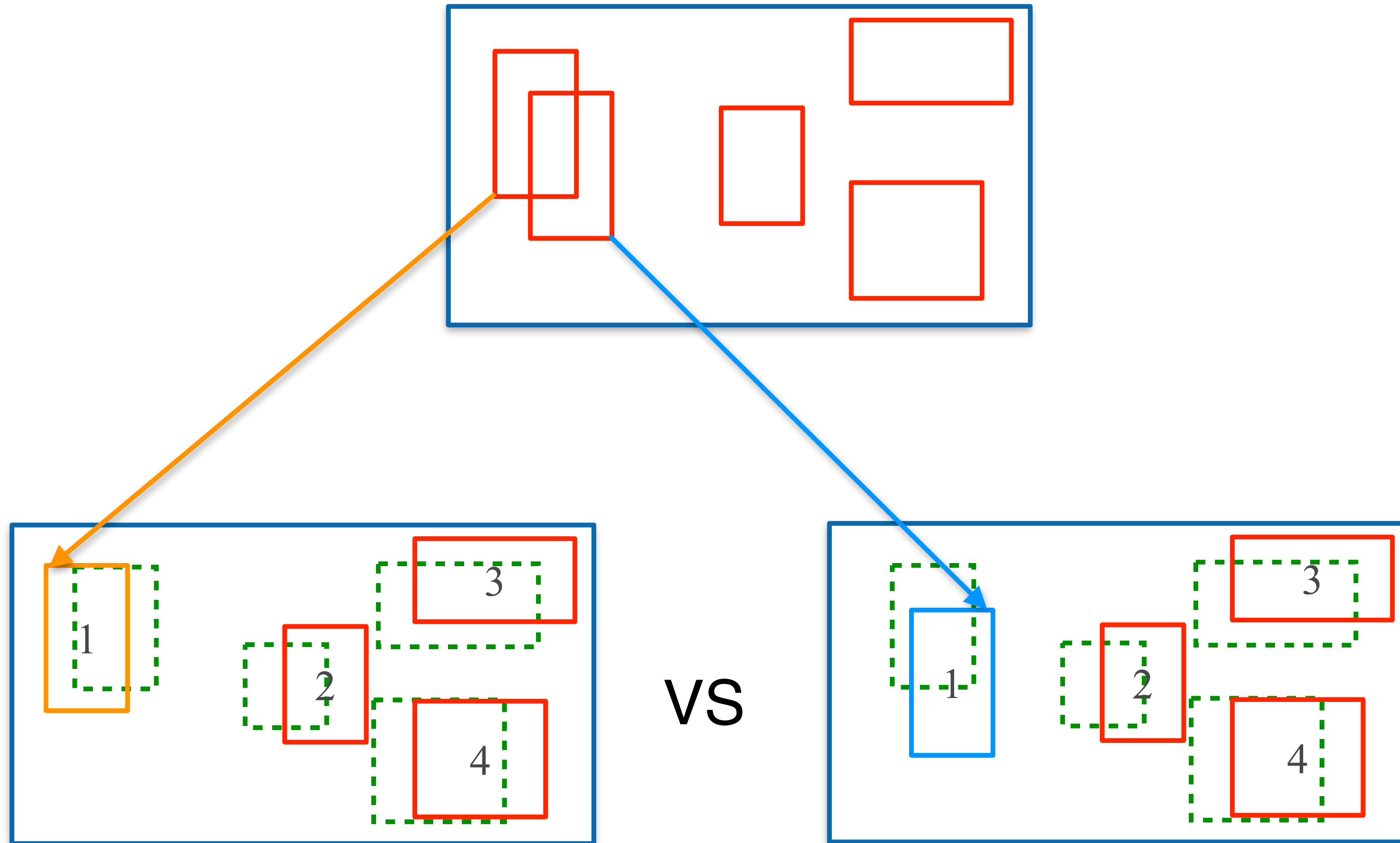


GT boxes



Prediction

Optimal Association (2D case)

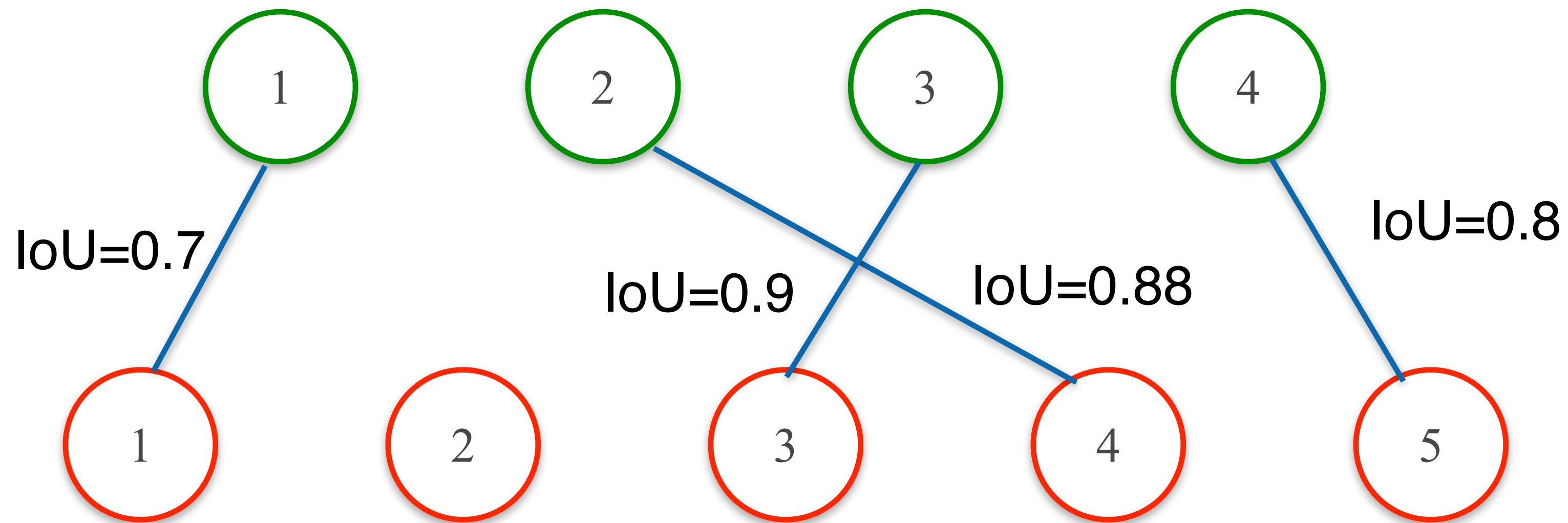


Matching 1

Matching 2

Optimal Association

- Objective: maximize the overall match gain
- Hungarian algorithm can solve this problem (similar to EMD)



The overall gain is $0.7 + 0.9 + 0.88 + 0.8$

- Gain $\Rightarrow$ cost, maximize $\Rightarrow$ minimize

Association Cost

- The cost (weight of bipartite graph) should evaluate the similarity between the predicted box and GT box (e.g., L_2 over b.box vertices offset)

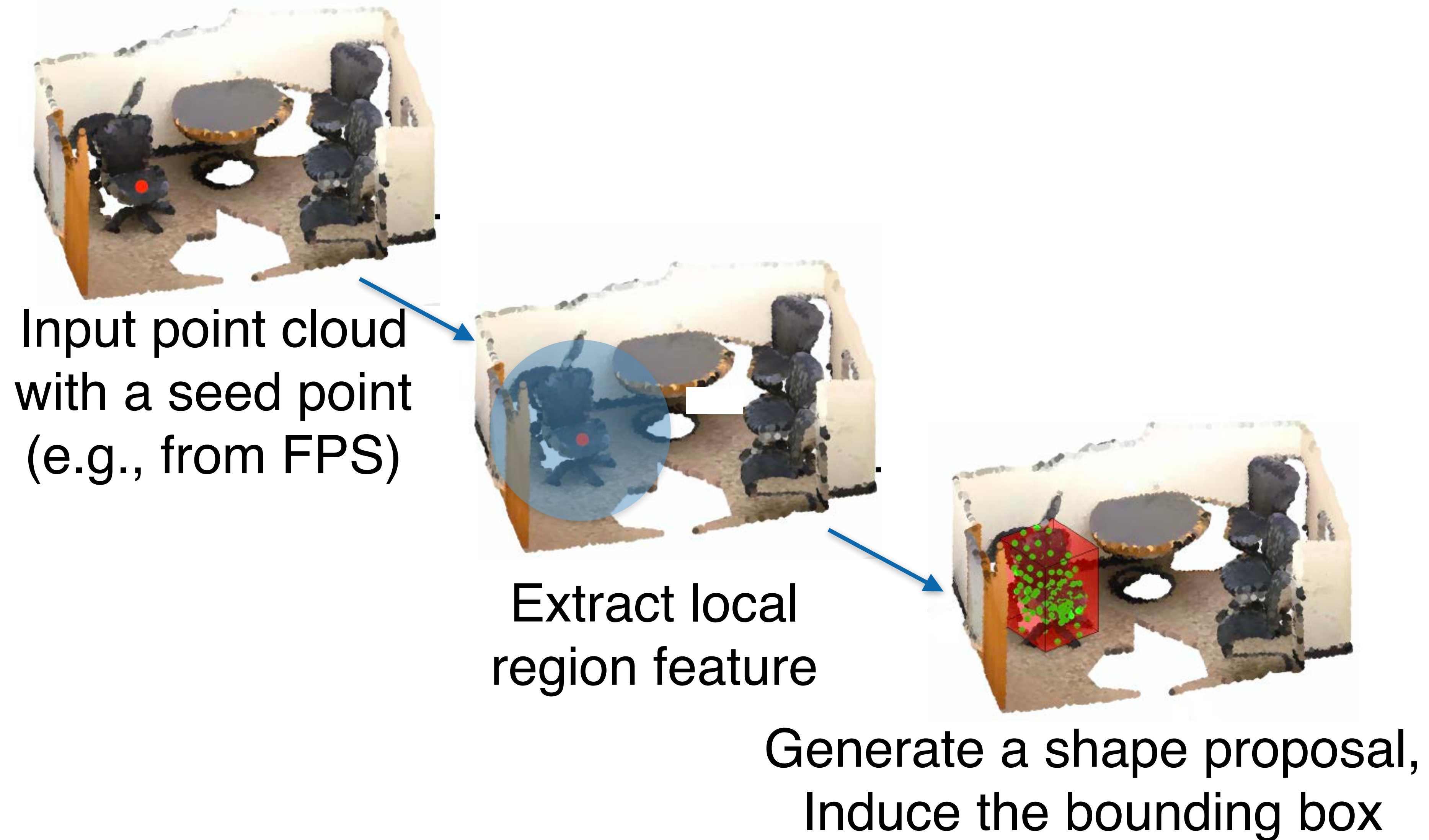
$$C_{i,j}^{ed} = \frac{1}{6} \sum (B_i - \bar{B}_j)^2$$

- Other criteria
 - Soft IoU
- The cost can be used as the loss directly

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

GSPN Pipeline



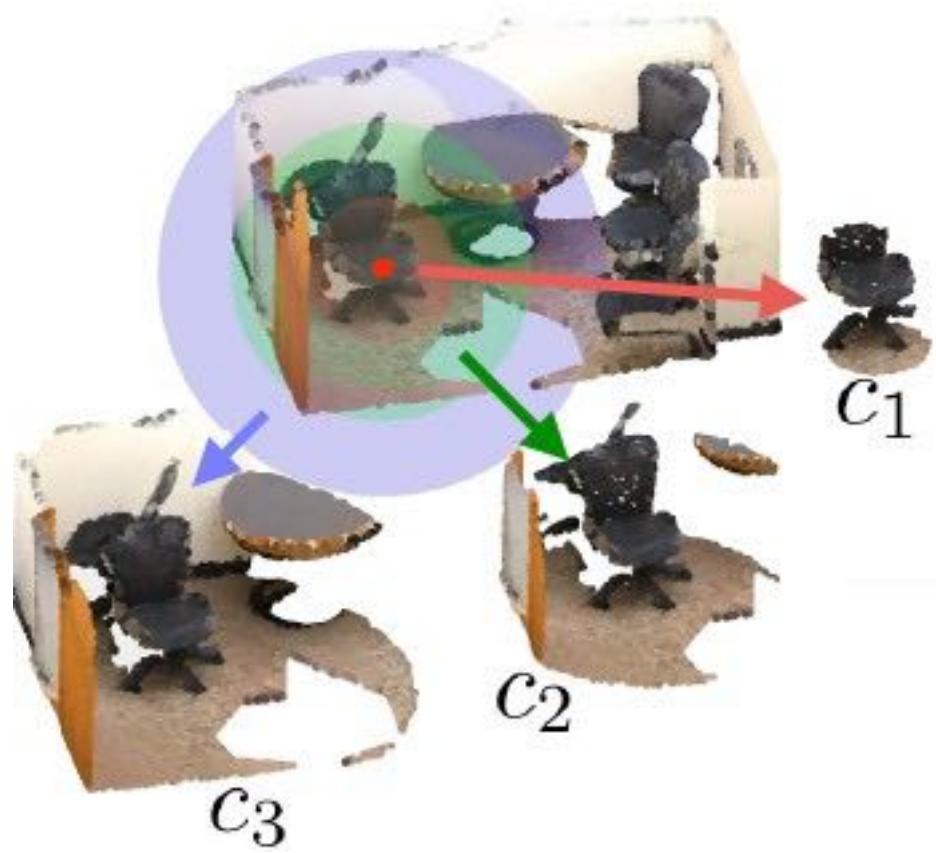
Point Cloud as Object Proposal

- Unlike primitive-based proposals, it is possible to **generate a point cloud** as a proposal (recall the single image to point cloud work)



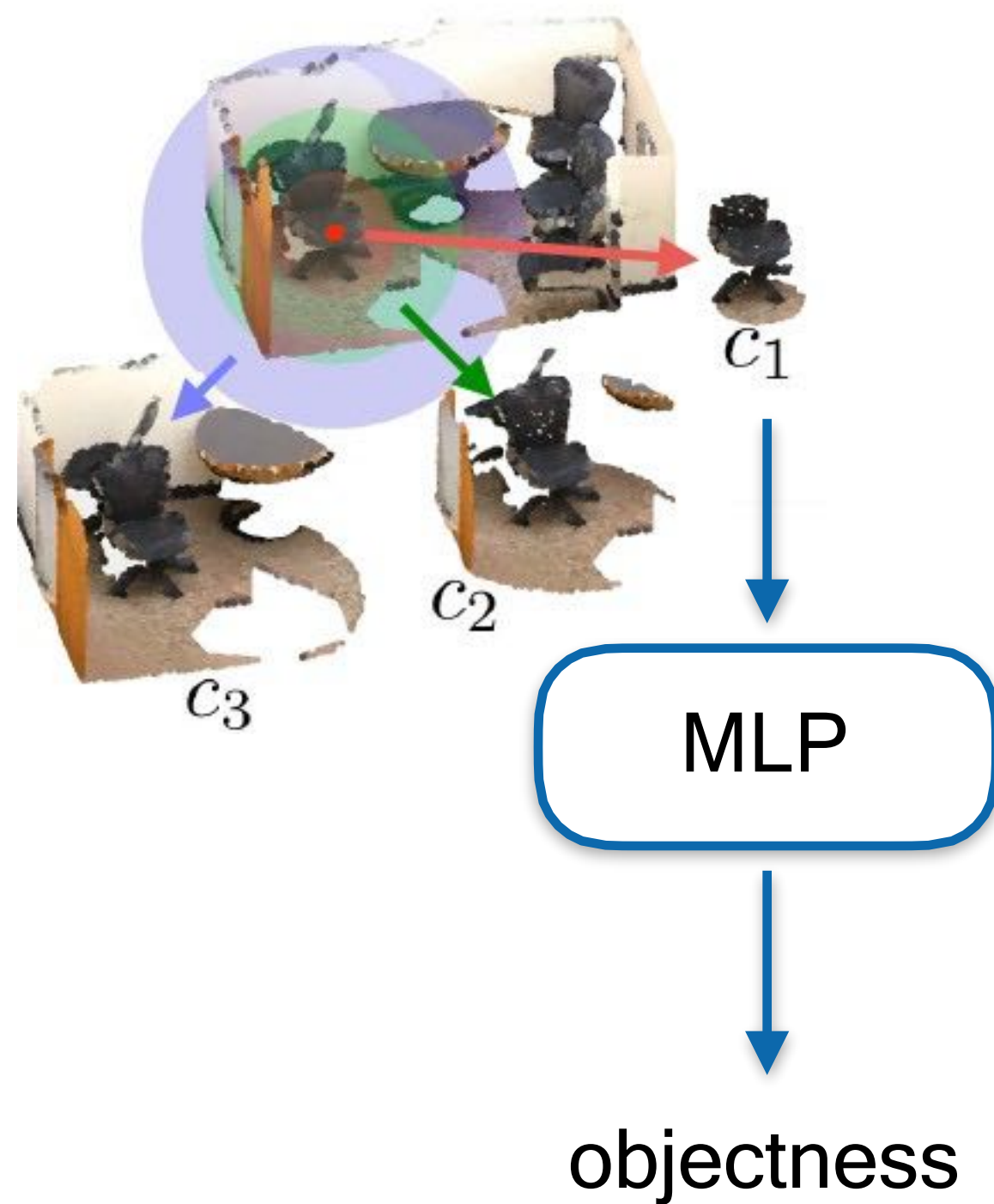
Generate Proposal as a Point Cloud

- Take a seed point and local context of different scales



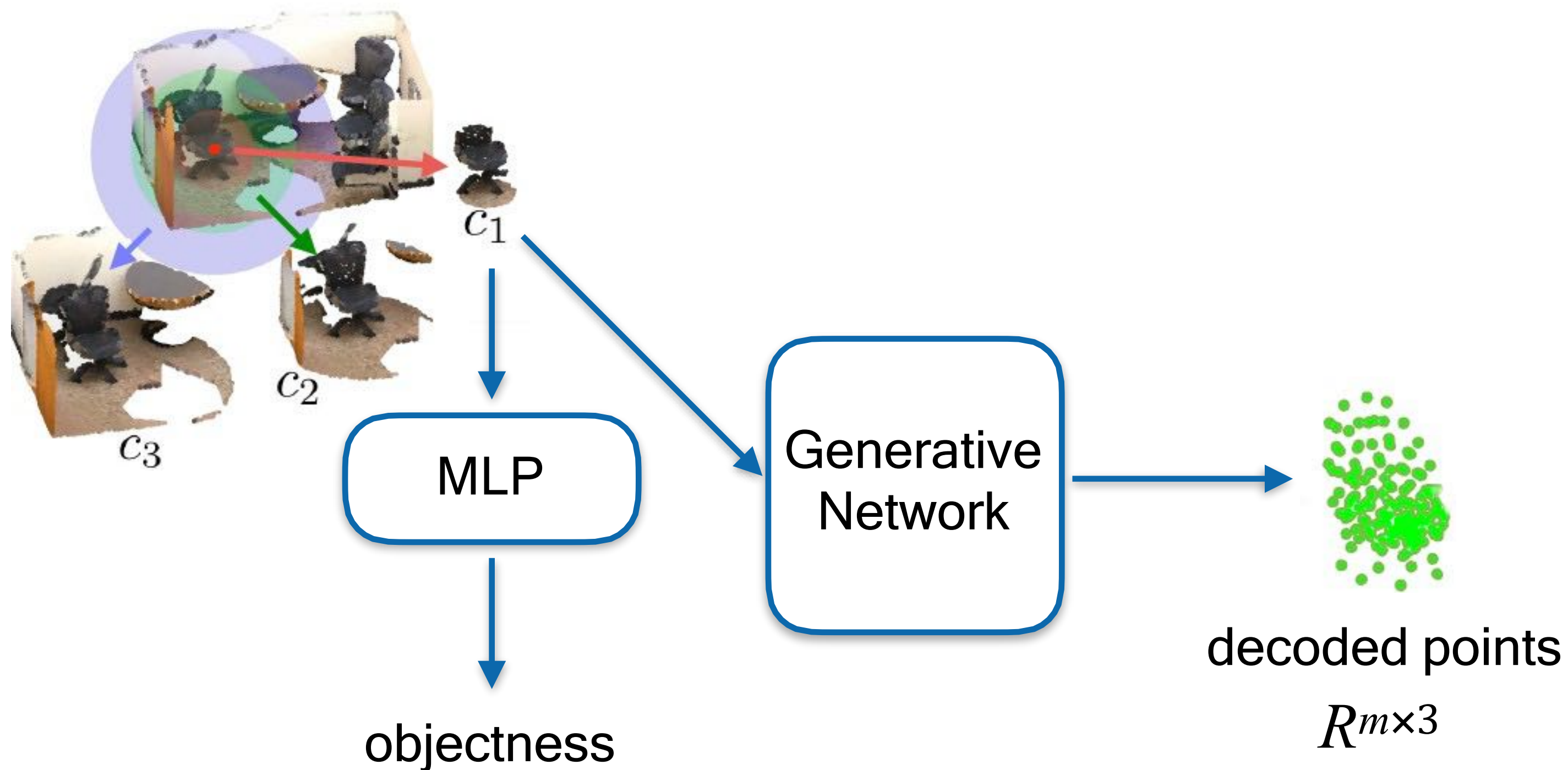
Generate Proposal as a Point Cloud

- Predict “objectness” (object v.s. non-object)



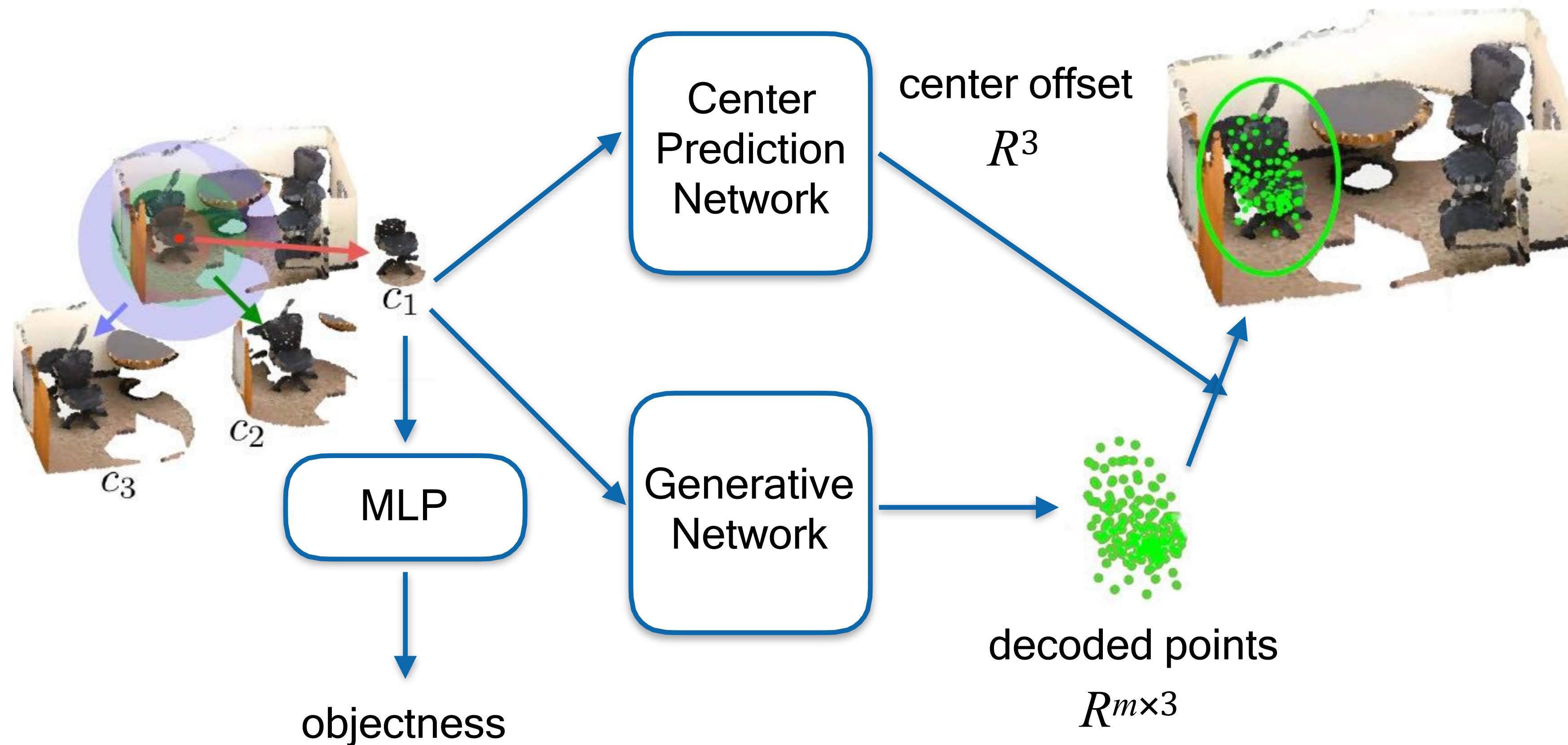
Generate Proposal as a Point Cloud

- Decode points, e.g., by a fully-connected network, as in single-image to point cloud work



Generate Proposal as a Point Cloud

- Predict a center offset from the seed point to the center of the instance



Losses for Point Cloud Proposals

- Only for positive proposals
 - Center prediction loss: huber loss (smooth l1)
 - Shape generation loss: chamfer distance
- For all the proposals
 - Objectness loss: cross-entropy

**How to associate points
with proposals?**

Basic Idea

- Given the proposal, predict a binary mask for each point whether the point belongs to the instance

Example: 3D-BoNet

- Steps:
 - Extract per-point features $\tilde{F}_l \in \mathbb{R}^{N \times D}$
 - Get instance-aware features $\hat{F}_l \in \mathbb{R}^{N \times (D+7)}$, e.g.,
 - point features (D dim)
 - bounding parameters (6 dim)
 - confidence (1 dim)
 - Predict point-wise mask $M_i \in \{0,1\}^N$

Point Label Generation and Loss

- Given the matched proposal and GT
 - For each proposal, we can induce a per-point binary mask given its corresponding GT



overall instance label



instance label for each proposal

Point Label Generation and Loss

- Given the matched proposal and GT
 - For each proposal, we can induce a per-point binary mask given its corresponding GT
 - We use a cross-entropy loss to do per-point binary classification



overall instance label



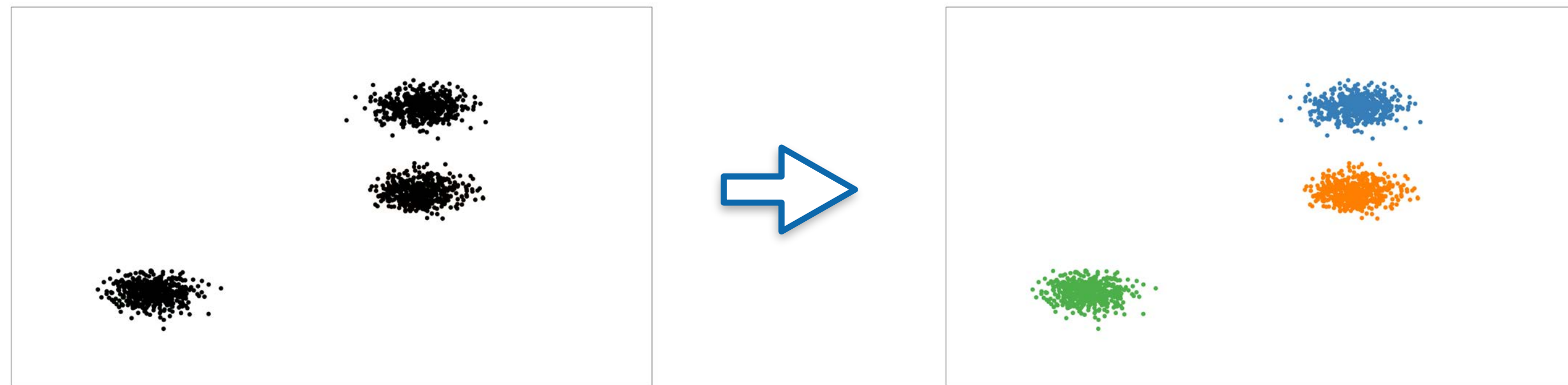
instance label for each proposal

Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - *Bottom-Up Approaches*

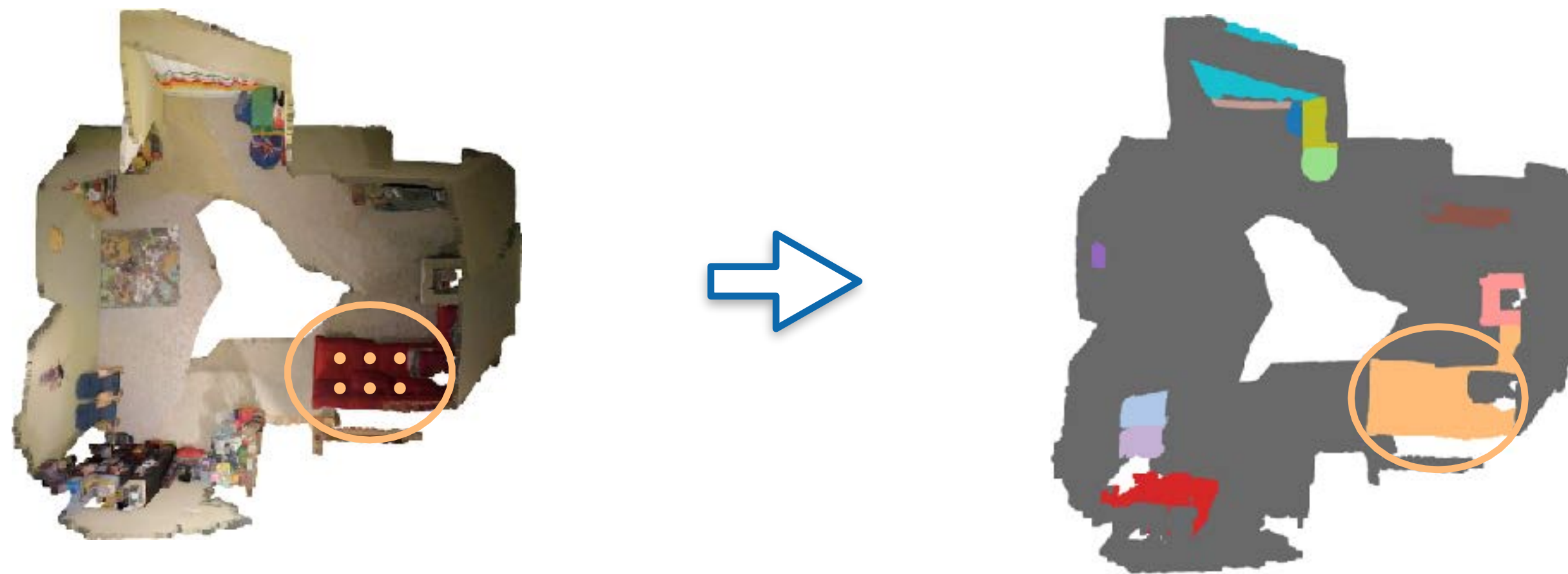
What is Bottom-up?

- A bottom-up approach is grouping the pieces of the points together to form an object.



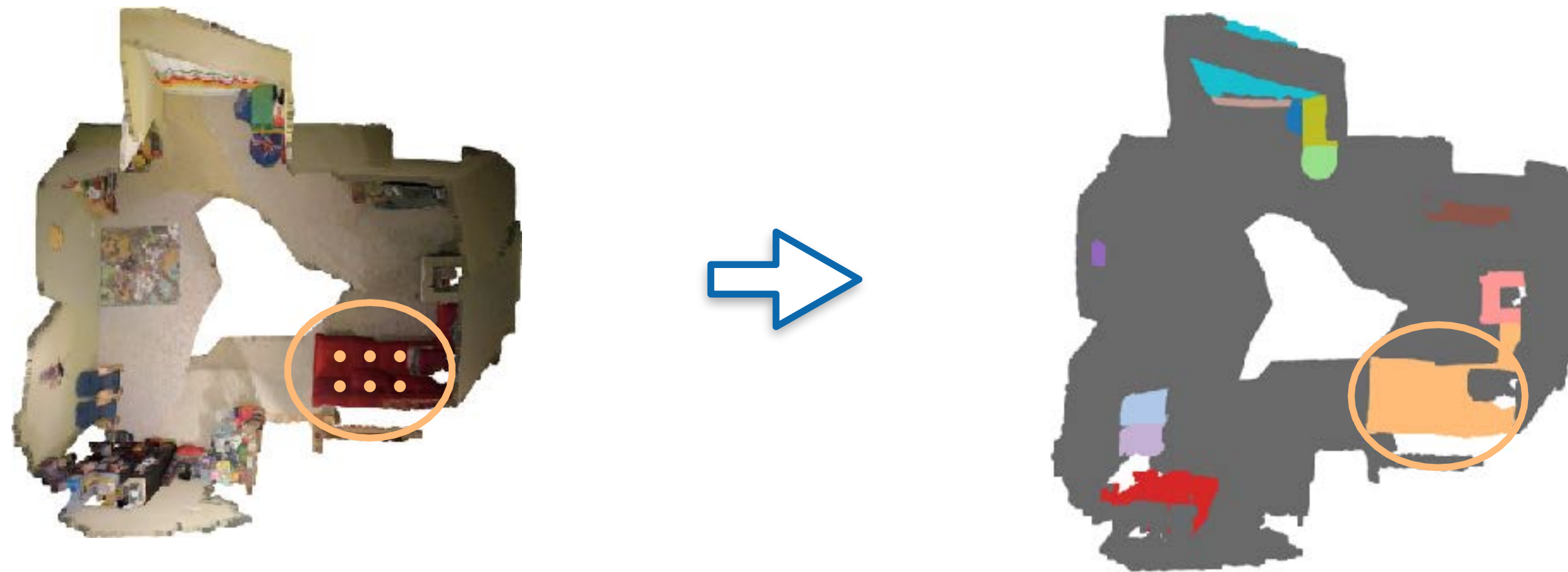
What is Bottom-up?

- A bottom-up approach is grouping the pieces of the points together to form an object.



What is Bottom-up?

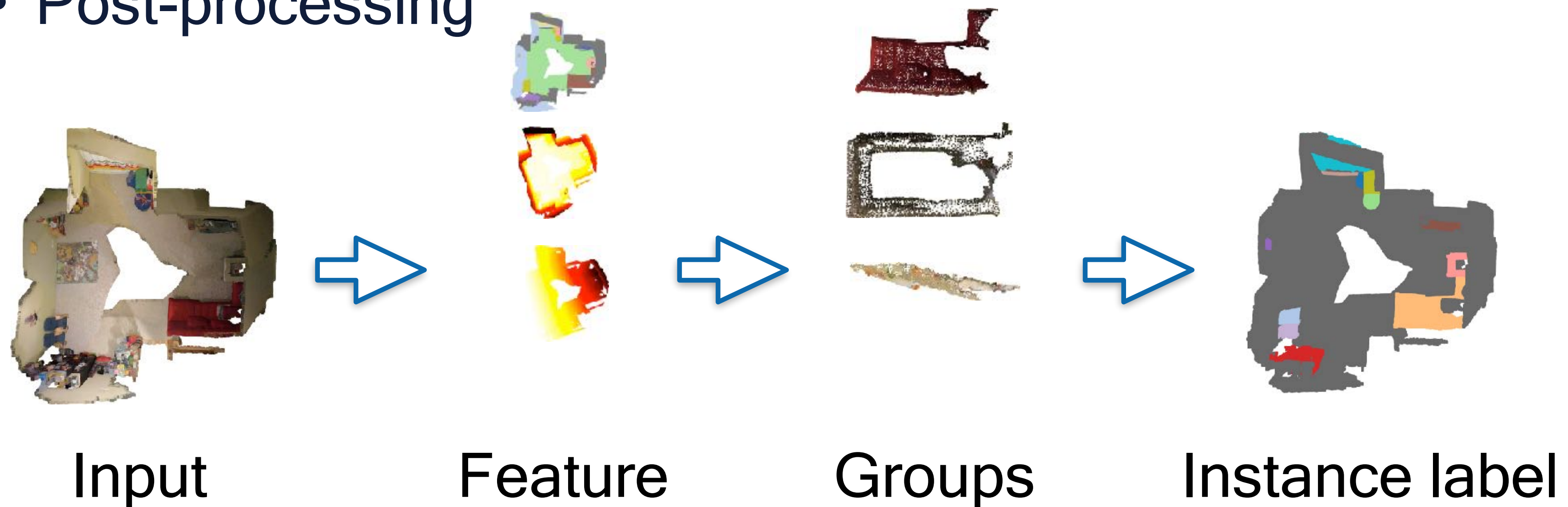
- A bottom-up approach is grouping the pieces of the points together to form an object.



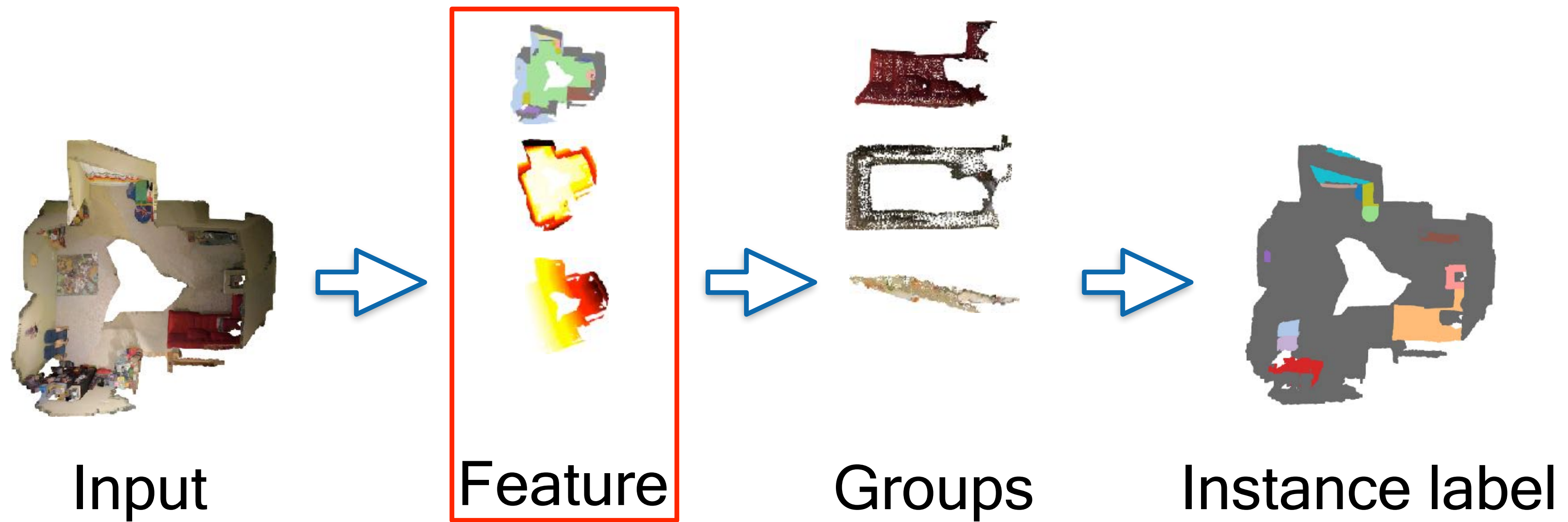
- In contrast, top-down: directly predict a proposal as object proxy and verify

Grouping-based Instance Segmentation

- Key Question: What points/fragments should be grouped?
 - Distance function
- Group procedure
 - Grouping/Clustering algorithm
- Post-processing

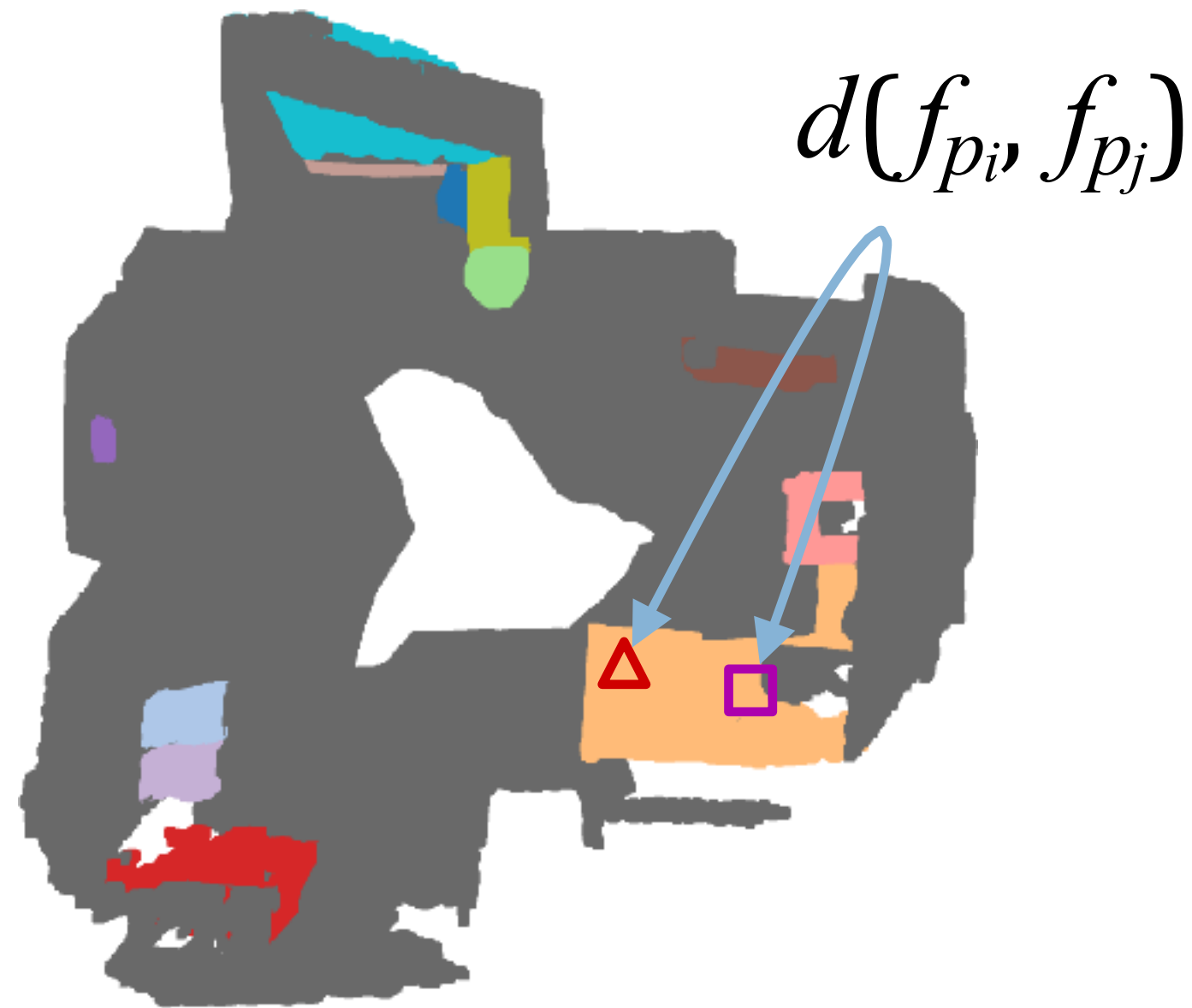


Grouping-based Instance Segmentation



Key Ideas

- Points in the same instance should be close in the **feature** space, such that clustering can be applied.



Distance in Feature Space

- Common choice: L_p -distance
 - e.g., L_1 -distance: $\|F_i - F_j\|_1$
- Potential features to consider:
 - Semantic features (about semantic label)
 - Spatial feature (about point location)
 - Instance feature (to distinguish instances)

Candidate I: Semantic Feature

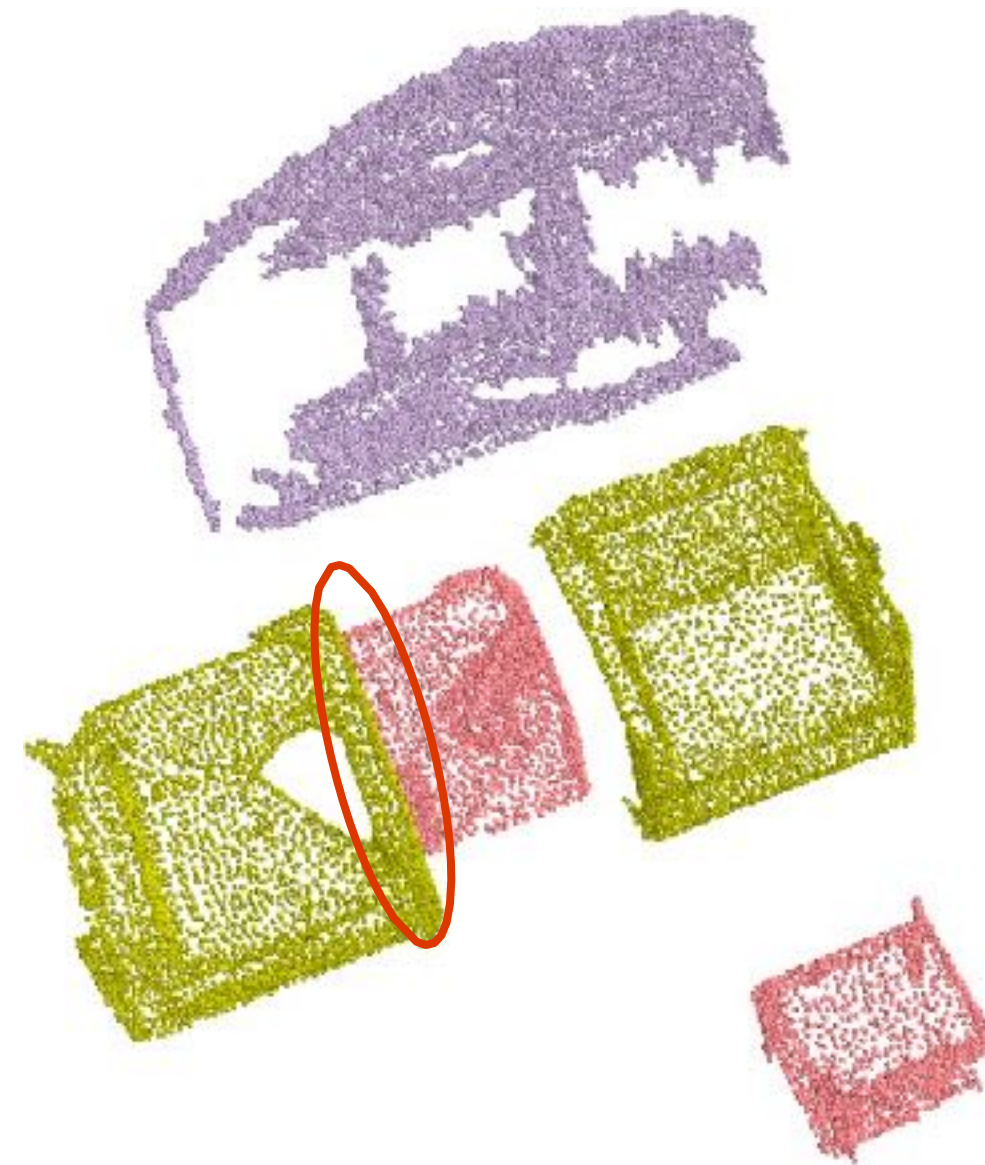
- Learn semantic feature for each point by point cloud segmentation loss.



MLAJiang, Li, et al. "Pointgroup: Dual-set point grouping for 3d instance segmentation." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.

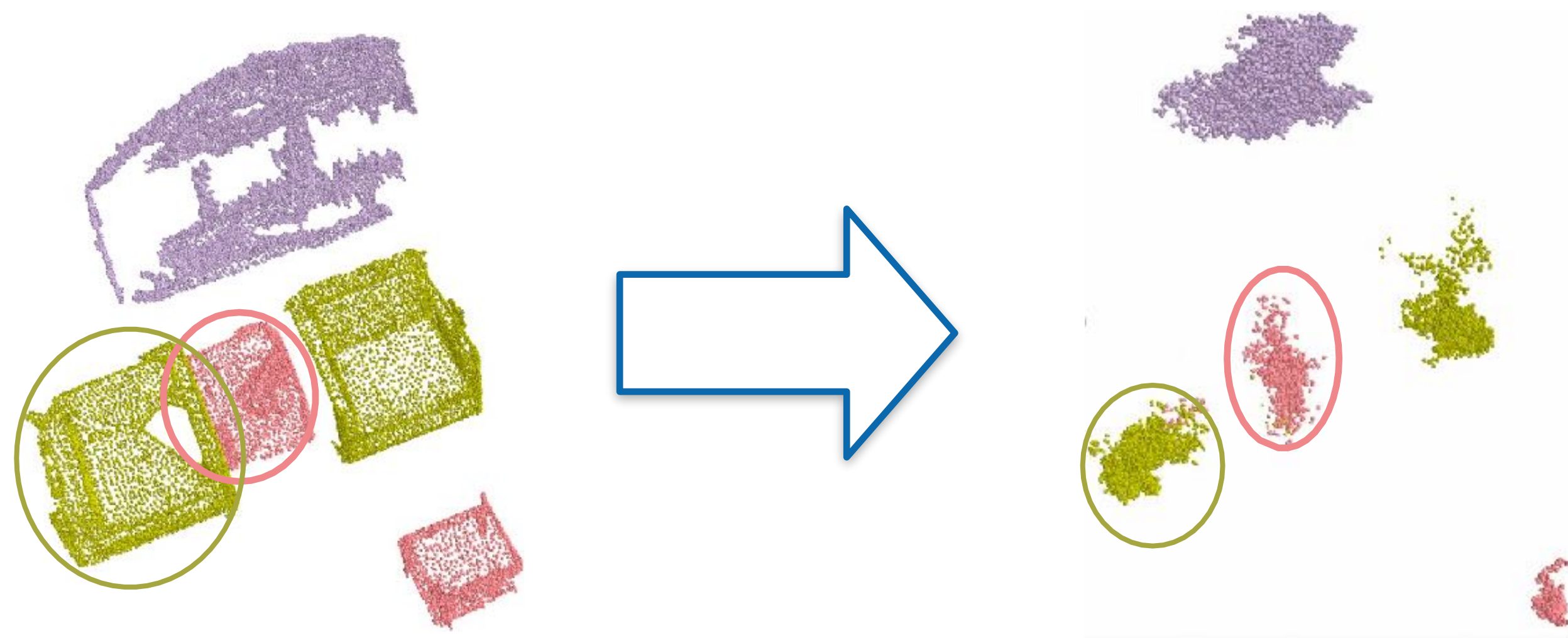
Candidate II: Spatial Feature

- Use 3D coordinates of points?
 - Reasonable, however,
 - Fails for points around object boundaries



Candidate II: Spatial Feature

- Learn to predict object center coordinates, and use the predicted object center as the spatial feature

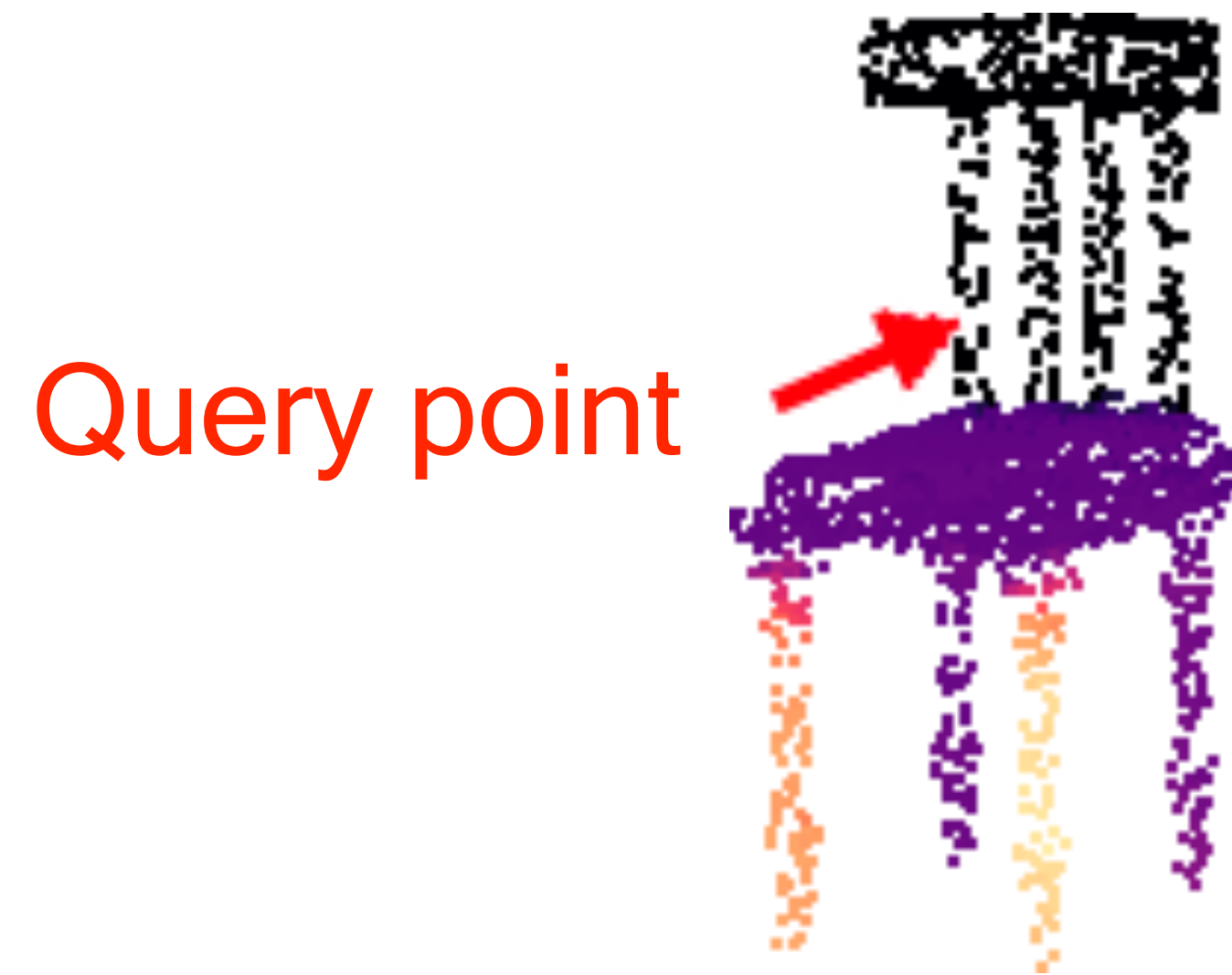


Predicted Object Centers

MLAJiang, Li, et al. "Pointgroup: Dual-set point grouping for 3d instance segmentation." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.

Candidate III: Instance Features

- Fundamentally, we hope that the feature can be powerful enough to distinguish different instances
- Why not directly design a loss to learn it?!



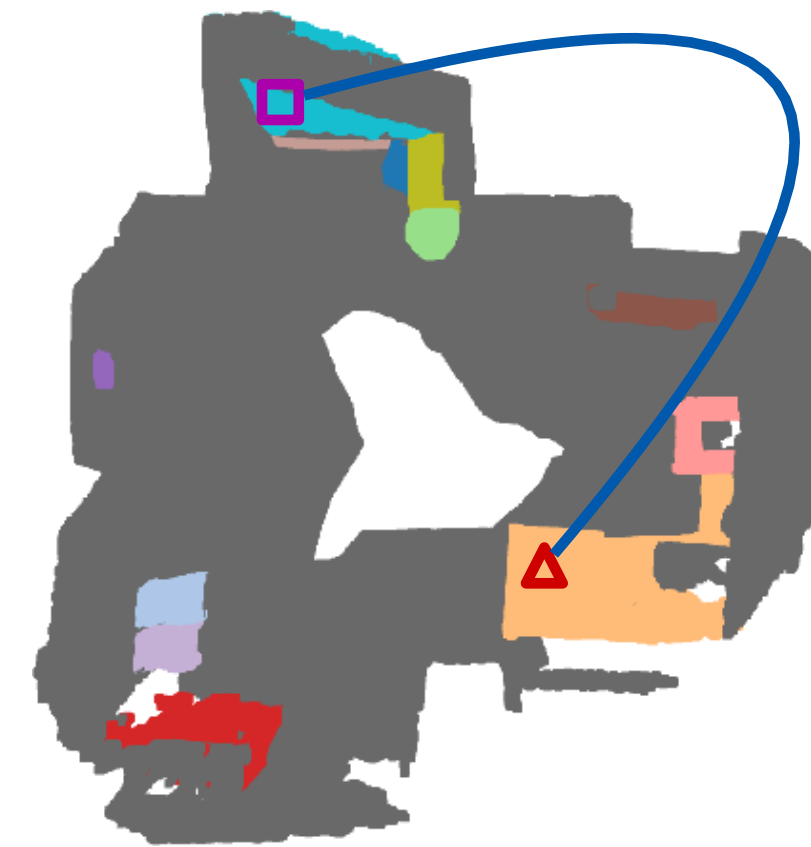
Color map of distances between the given point and rest points (darker means closer)

Contrastive Loss

- Build loss for each **pair of points** to train point features.



Semantic label



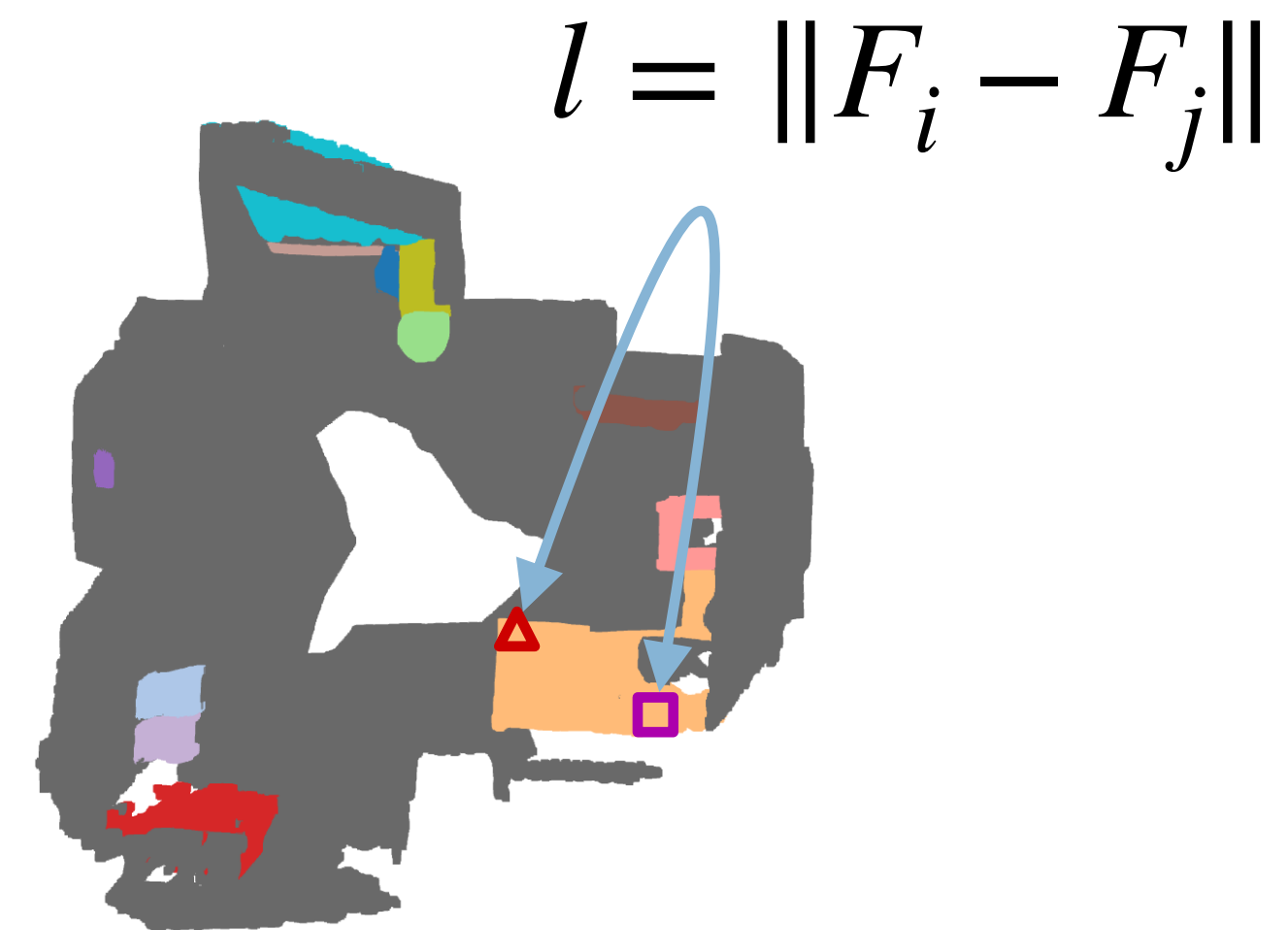
Instance label

Same Instance Case

- Point i and point j belongs to in the same instance.



Semantic label



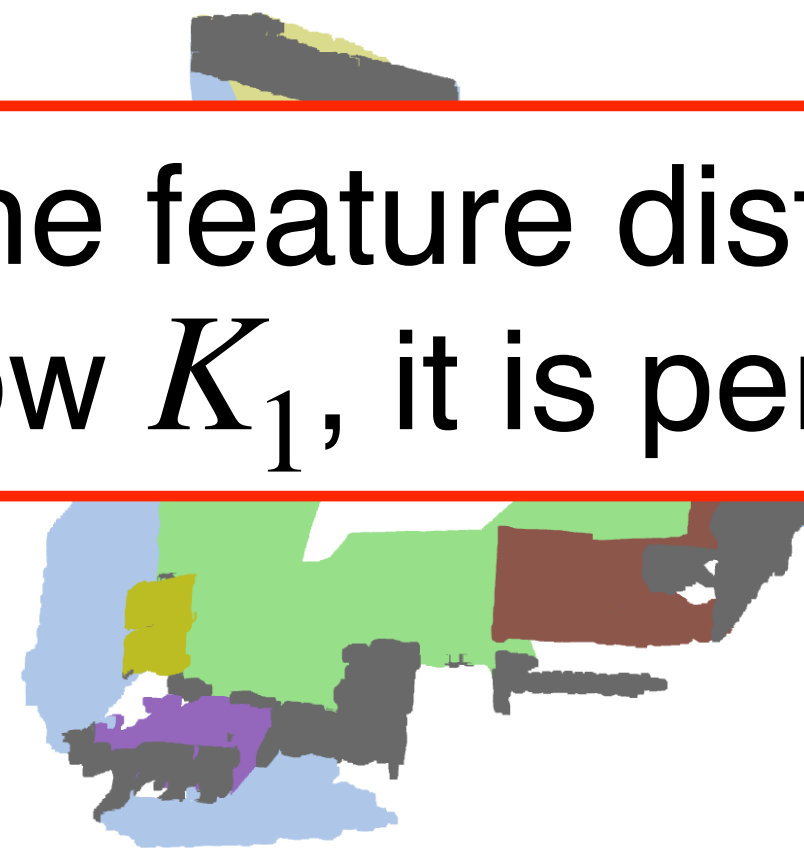
Instance label

Same Instance Case

- Point i and point j belongs to different instances with the same semantic label.

$$l(i, j) = \alpha \max(0, K_1 - \|F_i - F_j\|)$$

“If the feature distance is below K_1 , it is penalized”

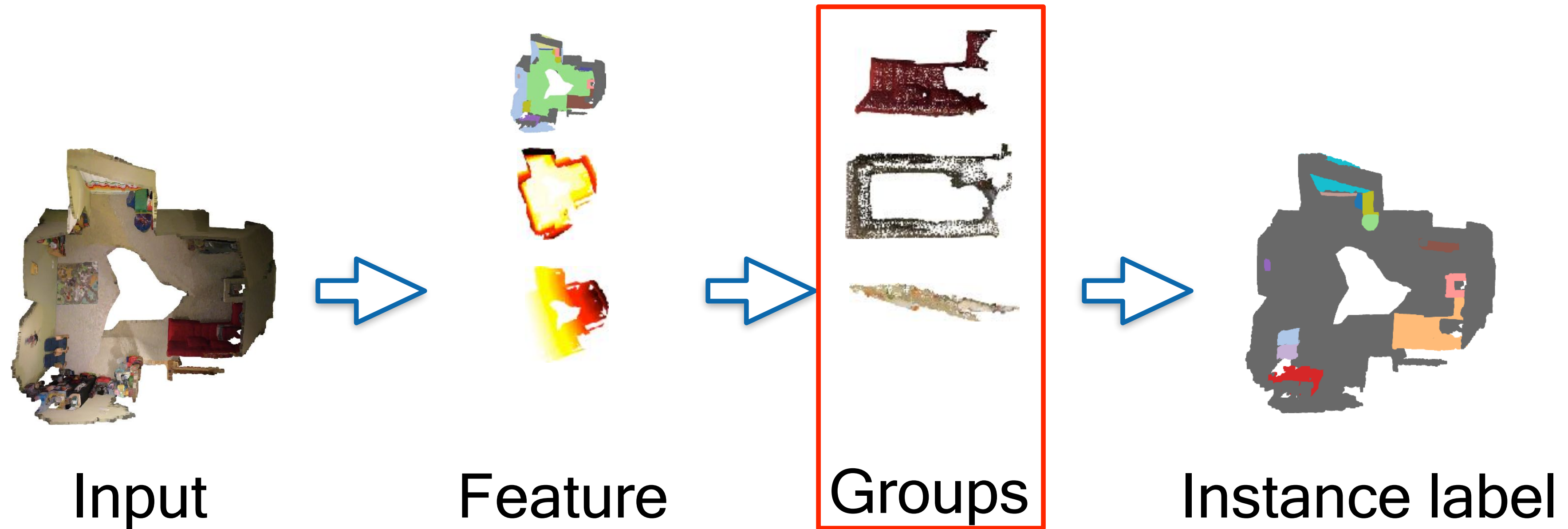


Semantic label



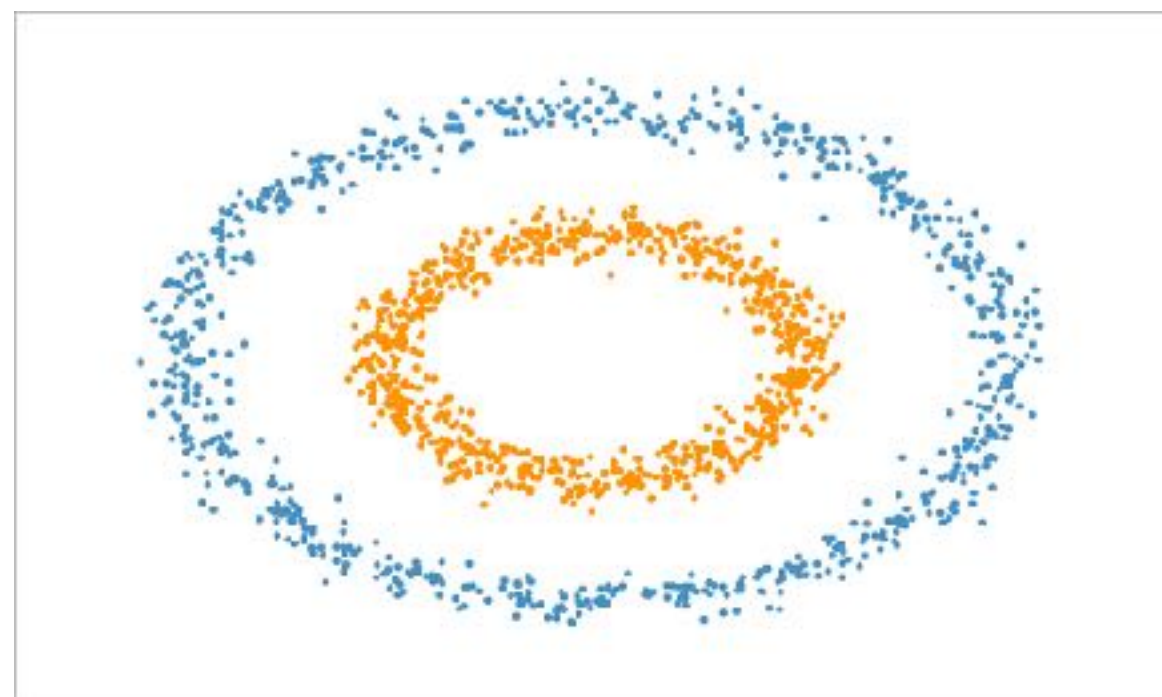
Instance label

Grouping-based Instance Segmentation

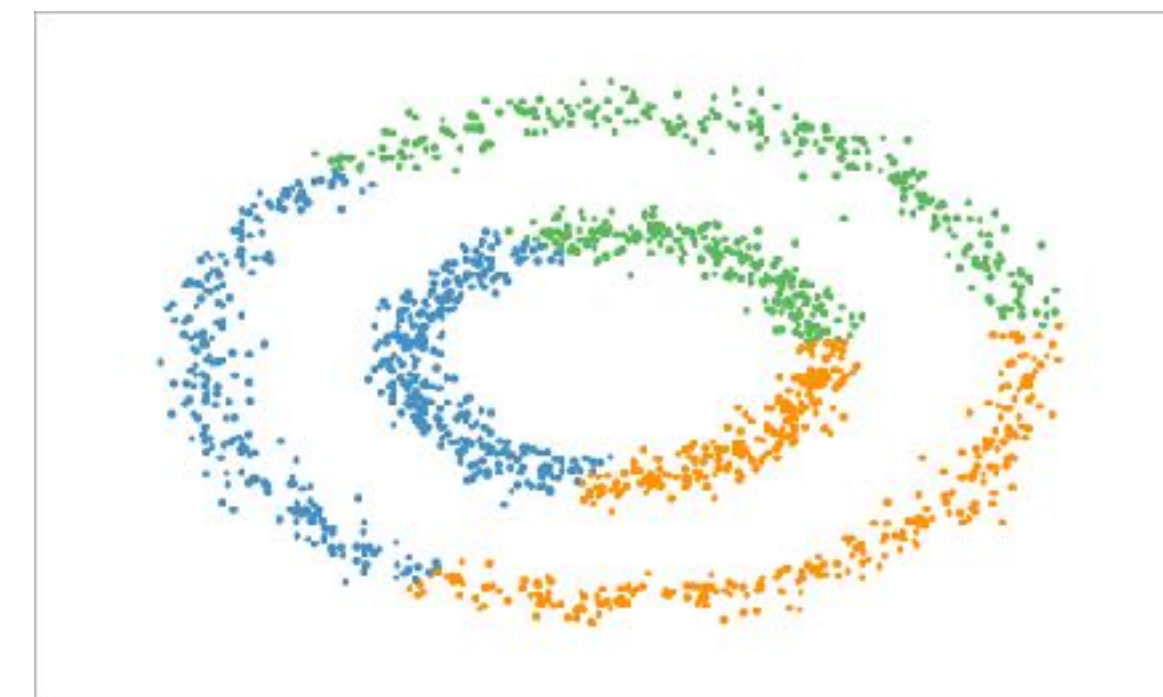


Grouping by Clustering Point Features

- Choose your favorable clustering algorithm
 - DBSCAN
 - Mean shift
 - ...



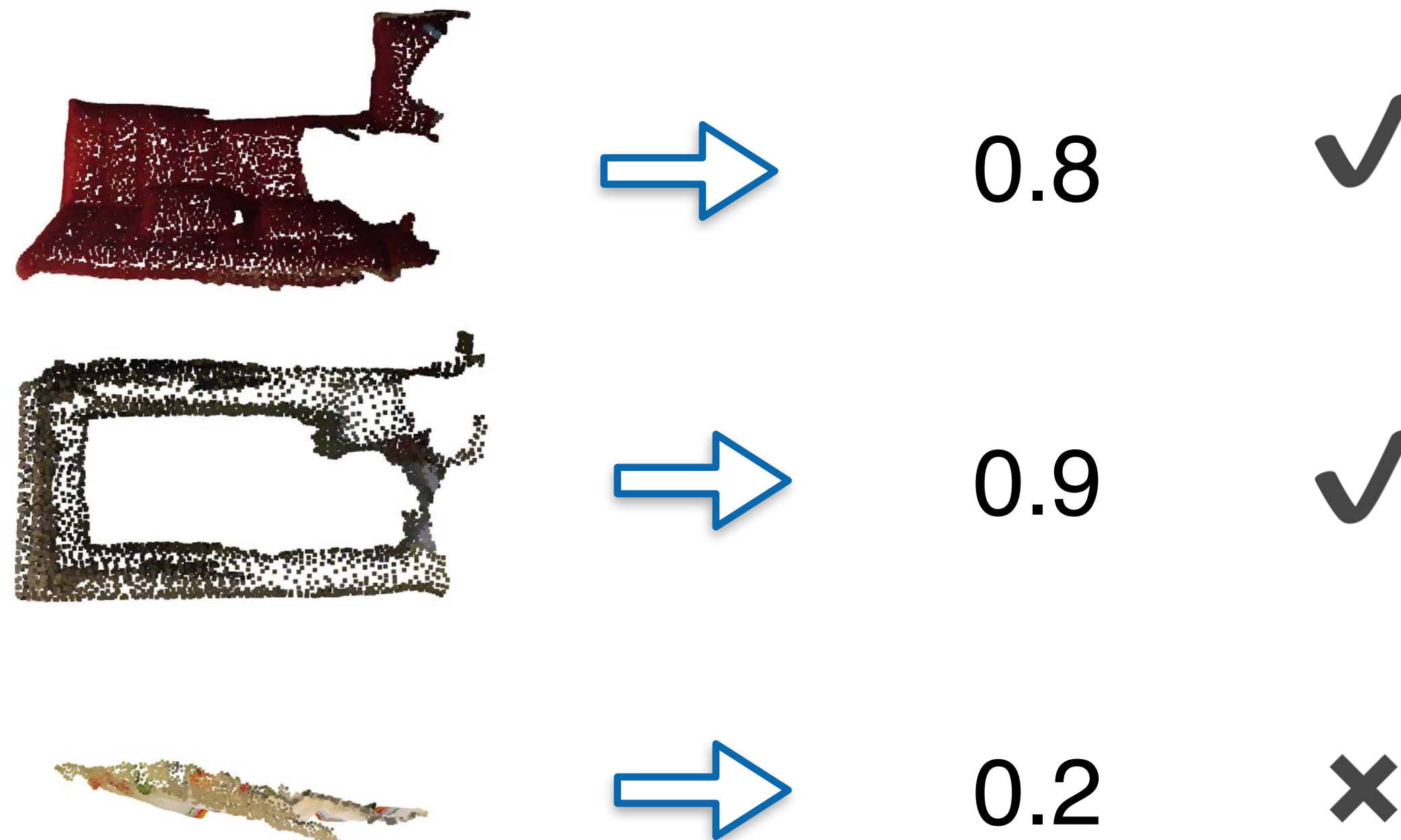
DBSCAN



Mean shift

Final Step: Post-Processing

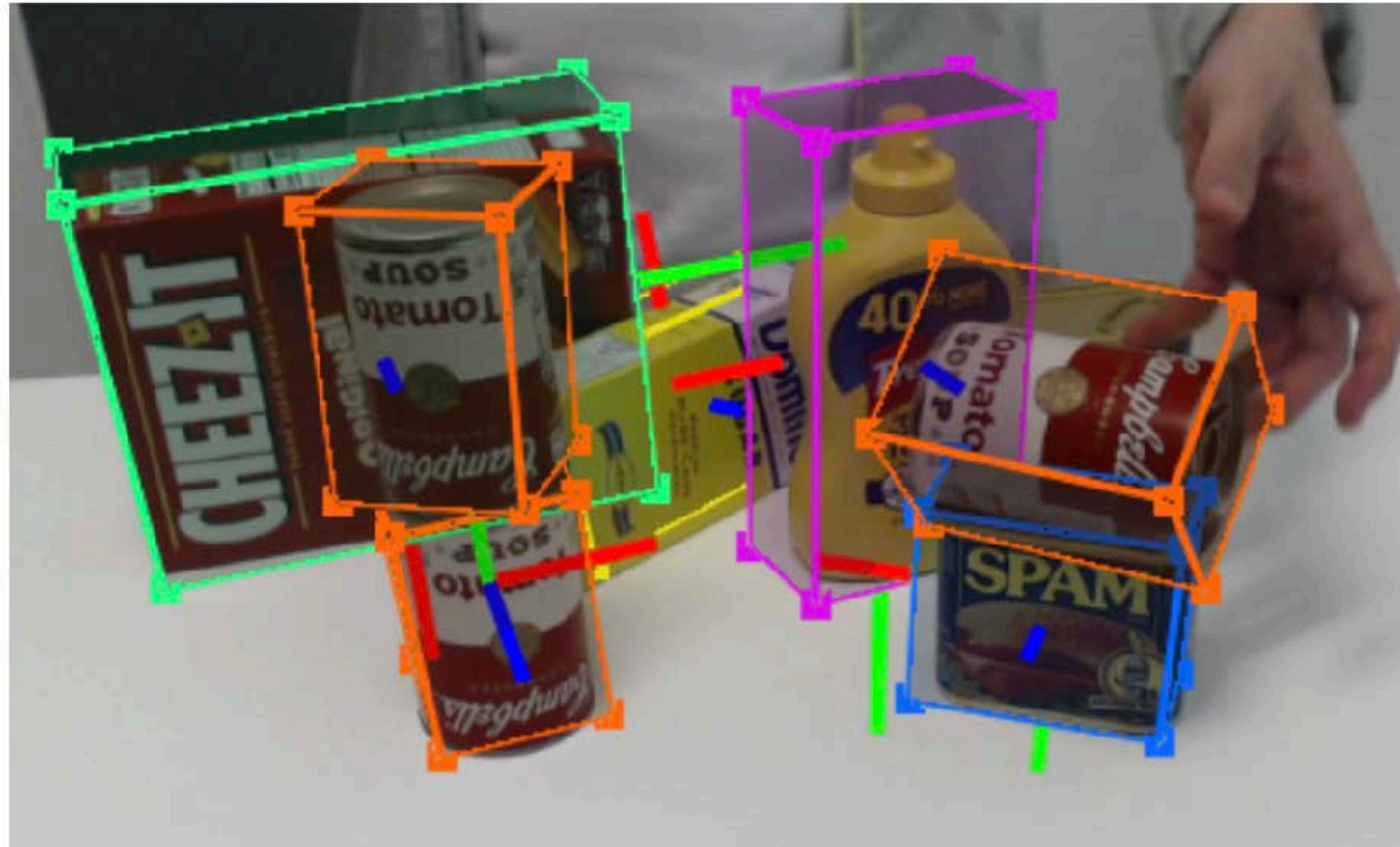
- May also be achieved by learning methods
- e.g., we use a network to predict a score which can represent the IoU between prediction and ground truth, and remove instances with low scores.



Summary

- 3D Object Detection
 - Background
 - The History and Development
 - ✦ Frustum PointNets — Leveraging image proposal for 3D detection
 - ✦ PointPillars — Projecting 3D into 2D with a strong assumption
 - ✦ VoteNet — Clustering based proposal
- 3D Instance Segmentation
 - Top-Down Approaches — Proposal generation + point association
 - Bottom-Up Approaches — Careful feature learning for grouping purposes

Next time



Pose Estimation

References

1. Object recognition in 3D scenes with occlusions and clutter by Hough voting Fourth Pacific-Rim Symposium on Image and Video Technology by Federico et al. IEEE, 2010.
2. Tutorial: Point Cloud Library: Three-Dimensional Object Recognition and 6 DOF Pose Estimation by Aldoma et al. IEEE Robotics & Automation Magazine 19.3 (2012): 80-91.
3. Object discovery in 3d scenes via shape analysis by Karpathy et al. *IEEE International Conference on Robotics and Automation*. IEEE, 2013.
4. Sliding shapes for 3d object detection in depth images by Song et al. *European conference on computer vision*. Springer, Cham, 2014.
5. Deep sliding shapes for amodal 3d object detection in rgb-d images by Song et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016.
6. Frustum pointnets for 3d object detection from rgb-d data by Qi et al. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018.
7. Pointpillars: Fast encoders for object detection from point clouds by Lang et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2019.
8. Deep hough voting for 3d object detection in point clouds by Qi et al. *Proceedings of the IEEE International Conference on Computer Vision*. 2019.
9. 3d bounding box estimation using deep learning and geometry by Mousavian et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017.
10. Pseudo-lidar from visual depth estimation: Bridging the gap in 3d object detection for autonomous driving by Wang et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2019.
11. Objects as points by Wang et al. 2019.
12. Volumetric and Multi-View CNNs for Object Classification on 3D Data by Qi et al. CVPR 2016.
13. Multi-view 3d object detection network for autonomous driving by Chen et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017.
14. Voxelnet: End-to-end learning for point cloud based 3d object detection by Zhou et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018.
15. Pointnet: 3d object proposal generation and detection from point cloud by Shi et al. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2019.
16. Std: Sparse-to-dense 3d object detector for point cloud by Yang et al. *Proceedings of the IEEE International Conference on Computer Vision*. 2019.
17. 3dssd: Point-based 3d single stage object detector by Yang et al. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.
18. Pv-rcnn: Point-voxel feature set abstraction for 3d object detection by Shi et al. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.
19. Imvotenet: Boosting 3d object detection in point clouds with image votes by Qi et al. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.